

JAXBase

Programming Guide



Fox-Pirate created by GROK Imaging

Jon Lee Walker
JLWalker.Dev@gmail.com
Copyright 2025 ©

The Team

Design

Jon Lee Walker

Documentation

Jon Lee Walker

Version 1 Development Team

Jon Lee Walker

Version 2 Development Team

Testers

If I have seen further, it is by standing on the shoulders of giants -- Isaac Newton

Acknowledgements

C. Wayne Ratliff & JPL

Ashton Tate

Dr. Dave Fulton, Bill Ferguson, and the FoxBase/FoxPro Teams

Nantucket Corporation

Borland dBase Team

Microsoft Visual FoxPro Team

StackOverflow.Com

Learn.Microsoft.Com

GROK

And finally

Helen Sue Walker

For her patience, support, and sense of humor.

Table of Contents

Forward.....	27
JAXBase Features	28
JAXBase Departures from XBase	29
Current Status	30
Release Goals	31
Features Not Supported in Version 1	32
Licensing, etc.	33
Command Reference.....	34
?	35
??	36
& (macro substitution).....	37
&&, *, NOTE	39
ACTIVATE	40
ADD	41
ADD CLASS	41
ADD OBJECT.....	43
ADD TABLE	44
ALTER TABLE	46
APARAMETERS.....	48
APPEND	49
APPEND	49
APPEND BLANK.....	50
APPEND FROM ARRAY	51
APPEND FROM	53
APPEND FROM JSON	56
ASSERT	58
AVERAGE	60
BEGIN TRANSACTION	62
BLANK	63

BROWSE	64
BUILD	66
BUILD APPLICATION	66
BUILD EXECUTABLE	67
BUILD PROJECT	68
CALCULATE	69
CANCEL	71
DO CASE ... CASE... OTHERWISE...ENDCASE	72
CD	73
CLEAR	74
CLEAR	74
CLEAR CLASS	76
CLEAR CONSOLE	77
CLEAR EVENTS	79
CLEAR ERRORS	80
CLEAR MACROS	81
CLEAR MEMORY	82
CLEAR PROGRAM	83
CLEAR TYPEAHEAD	84
CLOSE	85
CLOSE ALL	85
CLOSE ALTERNATE	86
CLOSE DATABASE	87
CLOSE DATASESSION	88
CLOSE INDEXES	89
CLOSE TABLES	90
COMPILE	91
CONTINUE	92
COPY FILE	93
COPY INDEXES	94

COPY STRUCTURE	95
COPY TO ARRAY	96
COPY TO	98
COUNT	101
CREATE CURSOR / TABLE	103
CREATE DATABASE	106
CREATE FORM	108
CREATE LABEL	109
CREATE MENU	110
CREATE PROJECT	111
CREATE QUERY	112
CREATE REPORT	113
DEACTIVATE	114
DEBUG	115
DEBUGOUT	116
DEFINE CLASS	117
DELETE	119
DELETE FILE	120
DIMENSION	121
DIRECTORY	123
DISPLAY DATABASE	124
DISPLAY FILES	125
DISPLAY MEMORY	126
DISPLAY OBJECTS	128
DISPLAY STATUS	129
DISPLAY STRUCTURE	131
DISPLAY TABLES	132
DO	134
DO Program	134
DO FORM	136

DO WHILE/ENDDO	138
DO CASE/CASE/OTHERWISE/ENDCASE	140
DO/UNTIL	142
DOEVENTS	144
DROP TABLE	145
EDIT	146
END	147
ERROR	148
EXIT	149
EXTERNAL	150
FOR/ENDFOR	151
FOREACH/ENDFOR	153
FUNCTION	154
GATHER	155
GETEXPR	157
GOTO	158
HELP	160
IF/ELSEIF/ELSE/ENDIF	161
IMPORT	163
INDEX	164
INSERT	167
KEYBOARD	169
LIST	170
LOCATE	171
LOCAL	173
LOOP	174
LPARAMETERS	175
LPROCEDURE	177
MD	179
MODIFY	180

MODIFY CLASS.....	180
MODIFY COMMAND	181
MODIFY FILE	182
MODIFY FORM.....	183
MODIFY LABEL	184
MODIFY MENU	185
MODIFY PROJECT	186
MODIFY QUERY	187
MODIFY REPORT	188
MODIFY STRUCTURE	189
MOUSE	190
NODEFAULT	191
ON.....	192
ON ESCAPE	192
ON KEY LABEL	193
ON SHUTDOWN	194
OPEN.....	195
PACK	197
PARAMETERS	198
PLAY	201
PRIVATE	202
PROCEDURE.....	204
PUBLIC	206
QUIT	207
RD	208
READ EVENTS	209
RECALL.....	210
REINDEX.....	211
RELEASE.....	212
RELEASE Command	212

RELEASE CLASSLIB	213
RELEASE CONSOLE.....	214
RELEASE PROCEDURE	215
REMOVE	216
REMOVE CLASS.....	216
REMOVE TABLE	217
RENAME	219
RENAME Command.....	219
RENAME CLASS.....	220
RENAME TABLE	221
REPLACE	222
REPLACE Command.....	222
REPLACE FROM.....	224
RESTORE	226
RESTORE FROM.....	226
RESTORE MACROS	228
RESUME.....	229
RETRY	230
RETURN	231
SAVE.....	232
SAVE MACROS	232
SAVE TO	233
SCAN / ENDSCAN	234
SCATTER.....	236
SEEK Command	238
SELECT Command.....	240
SET Commands	242
SET ALTERNATE Command	243
SET ASSERTS Command.....	245
SET AUTOINCERROR Command	246

SET BELL Command	247
SET BLOCKSIZE Command	248
SET BROWSEIME Command	249
SET CARRY Command	250
SET CENTURY Command.....	251
SET CLASSLIB Command	252
SET COLLATE Command	253
SET CONFIRM Command	254
SET CONSOLE Command.....	255
SET COVERAGE Command	257
SET CPCOMPILE Command.....	258
SET CPDIALOG Command.....	259
SET CURRENCY Command.....	260
SET CURSOR Command.....	261
SET DATABASE Command.....	262
SET DATASESSION Command.....	263
SET DATE Command.....	264
SET DEBUGOUT Command.....	265
SET DECIMALS Command	267
SET DEFAULT Command.....	268
SET DELETED Command	269
SET DEVELOPMENT Command.....	270
SET EPOCH Command	271
SET ENCRYPTION Command	272
SET ESCAPE Command	273
SET EVENTLIST Command	274
SET EVENTTRACKING Command	275
SET EXACT Command.....	276
SET EXCLUSIVE Command	277
SET FDOW Command.....	278

SET FIELDS Command.....	279
SET FILTER Command.....	280
SET FIXED Command.....	281
SET FULLPATH Command.....	282
SET FWEEK Command.....	283
SET HEADINGS Command.....	284
SET HELP Command	285
SET HOURS Command	286
SET INDEX Command.....	Error! Bookmark not defined.
SET LOCK Command	287
SET MACKEY Command.....	290
SET MEMOWIDTH Command.....	291
SET MESSAGE Command	292
SET MULTILOCKS Command.....	293
SET NAMING Command	294
SET NEAR Command.....	295
SET NOCPTRANS Command.....	296
SET NOTIFY Command	297
SET NULL Command	298
SET NULLDISPLAY Command.....	299
SET ODOMETER Command	300
SET ORDER Command	301
SET PATH Command.....	302
SET POINT Command.....	303
SET PROCEDURE Command	304
SET REFRESH Command.....	306
SET RELATION Command	307
SET REPROCESS Command	308
SET RESOURCE Command.....	309
SET SAFETY Command	310

SET SECONDS Command.....	311
SET SECURITY Command	312
SET SEPARATOR Command	313
SET SKIP Command.....	314
SET SPACE Command	315
SET SQLCONNECTION Command	316
SET SQLLOAD Command.....	Error! Bookmark not defined.
SET STEP Command	318
SET STRICTDATE Command	319
SET SYSMENU Command	320
SET TABLEPROMPT Command	321
SET TABLEVALIDATE Command	322
SET TALK Command	323
SET TEXTMERGE Command	324
SET TEXTDELIMITERS Command.....	325
SET TOPIC Command	326
SET TOPIC ID Command	327
SET TRBETWEEN Command	328
SET TYPEAHEAD Command.....	329
SKIP.....	330
SORT	332
STORE.....	334
SUM.....	335
SUSPEND	337
TEXT / ENDTEXT.....	338
THROW.....	340
TRY/CATCH/FINALLY/ENDTRY	341
UNLOCK	343
UPDATE	344
UPDATE FROM	345

USE	346
WAIT	349
WITH/ENDWITH	351
ZAP	352
Function Reference	354
\$	355
%	356
::	357
ABS()	358
AClass()	359
ACOPY()	360
ACOS()	362
ADATABASES()	363
ADBOBJECTS()	364
ADDBS()	366
ADDPROPERTY()	367
ADEL()	368
ADIR()	370
AELEMENT()	372
AERROR()	374
AEVENTS()	376
AFIELDS()	377
AFONT()	379
AINS()	380
AINSTANCE()	382
ALen()	383
ALIAS()	384
ALINES()	385
ALLTRIM()	387
ALOCFILE()	388

AMEMBERS()	390
ASC()	392
ASCAN()	395
ASELOBJ() – Under consideration	397
ASESSIONS()	398
ASIN()	399
ASORT()	400
ASTACKINFO()	402
ASUBSCRIPT()	404
AT()	406
AT_C()	408
ATC()	409
ATCC()	412
ATCLINE()	413
ATLINE()	415
ATN2()	417
ATOKENS()	418
AUSED()	420
BETWEEN()	422
BINDEVENT()	424
BINTOC()	425
BITAND()	427
BITCLEAR()	428
BITLSHIFT()	429
BITNOT()	430
BITOR()	431
BITRSHIFT()	432
BITSET()	433
BITTEST()	434
BITXOR()	435

BOF()	436
CANDIDATE()	437
CAPSLOCK()	438
CDOW()	439
CEILING().....	440
CHR().....	441
CHRTRAN()	442
CHRTRANC().....	443
CMONTH()	444
COMMIT()	445
COMPILE()	446
COS().....	447
CPCONVERT()	448
CPCURRENT()	449
CPDBF().....	450
CREATEBINARY()	451
CREATEOBJECT().....	453
CTOBIN().....	454
CTOD().....	456
CTOT().....	457
CURSORGETPROP()	458
CURSORSETPROP()	459
CURSORTOJSON().....	460
CURSORTOXML()	461
CURVAL()	462
DATE().....	464
DATETIME()	465
DAY().....	467
DBC().....	468
DBF()	469

DBGETPROP()	470
DBSETPROP().....	471
DBUSED()	472
DELETED()	473
DESCENDING()	474
DIFFERENCE().....	476
DIRECTORY().....	477
DISKSPACE()	479
DOY()	480
DYM().....	481
DODEFAULT().....	482
DOW().....	484
DRIVETYPE().....	486
DTOC().....	487
DTOR()	488
DTOS()	489
DTOT().....	490
EMPTY()	491
EOF()	493
ERROR().....	494
EVALUATE()	495
EVL()	496
EXECSCRIPT()	497
EXP().....	498
FCOUNT()	499
FDATE()	500
FIELD().....	501
FILE()	502
FILETOSTR()	503
FILTER().....	504

FLOCK()	505
FLOOR().....	507
FONTMETRIC()	508
FOR()	509
FORCEEXT()	511
FORCEPATH().....	512
FORCESTEM()	513
FOUND()	514
FSIZE()	515
FTIME()	517
FULLPATH()	518
FV().....	519
GETAUTOINCVALUE()	520
GETCP()	521
GETDATE().....	522
GETDIR()	524
GETENV()	526
GETFILE()	527
GETFIELDSTATE()	529
GETFONT().....	530
GETNEXTMODIFIED()	531
GETJSON() Function	535
GETPICT()	537
GETPRINTER()	539
GETTIME()	540
GETWORDCOUNT()	542
GETWORDNUM()	543
GETCURSORADAPTER()	545
GOMONTH()	546
GUID()	547

HEADER().....	548
HEXTOINT()	549
HOUR()	550
ICASE()	551
IDXCOLLATE()	552
IIF()	553
INDEXSEEK()	554
INLIST()	558
INPUTBOX().....	559
INSMODE()	561
INT().....	562
INTTOHEX()	563
ISALPHA()	564
ISBLANK()	565
ISDIGIT()	567
ISEXCLUSIVE()	568
ISFLOCKED().....	569
ISLEADBYTE().....	571
ISLOWER()	572
ISNULL()	574
ISODD()	575
ISPEN()	576
ISREADONLY().....	577
ISRLOCKED()	578
ISUPPER()	580
JSONTOCURSOR()	582
JSONTOOBJ()	583
JUSTDRIVE()	584
JUSTEXT().....	585
JUSTFNAME()	586

JUSTFULLPATH()	587
JUSTPATH()	588
JUSTSTEM()	589
KBBUFFER()	590
KEY()	591
LASTKEY()	593
LEFT()	594
LEFTC()	595
LEN()	596
LENC()	597
LIKE()	598
LIKEC()	600
LINENO()	601
LOADPICTURE()	602
LOCK()	604
LOG()	606
LOG10()	607
LOWER()	608
LTRIM()	609
MAX()	610
MDY()	611
MEMLINES()	614
MESSAGE()	615
MESSAGEBOX()	616
MIN()	618
MINUTE()	619
MLINE()	620
MOD()	621
MONTH()	622
NDX()	623

NORMALIZE()	626
NUMLOCK()	627
NVL()	628
OBJTOCLIENT()	629
OBJTOJSON()	631
OCCURS()	633
OLDVAL()	635
ON()	636
ORDER()	638
OS()	640
PADC() / PADL() / PADR()	641
PARAMETERS()	642
PAYMENT()	643
PEMSTATUS()	644
PI()	645
PIXELPOS()	646
PROGRAM()	648
PROPER()	649
PUTJSON()	651
PUTFILE()	653
PV()	654
QUARTER()	655
RAND()	656
RAT()	657
RATC()	659
RATLINE()	660
RECCOUNT()	662
RECNO()	663
RELATION()	664
REMOVEPROPERTY()	665

REPLICATE()	666
REQUERY()	667
RGB()	668
RIGHT()	670
RIGHTC()	671
RLOCK()	672
ROLLBACK()	674
ROUND()	675
RTOD()	676
RTRIM()	677
SAVEPICTURE()	678
SEC()	679
SECONDS()	680
SEEK()	681
SELECT()	683
SET()	684
SETFLDSTATE()	688
SIGN()	689
SIN()	690
SOUNDEX()	691
SPACE()	692
SQRT()	693
STR()	694
STRCONV()	695
STREXTRACT()	696
STRFORMAT()	698
STRTOFILE()	701
STRTRAN()	702
STUFF()	704
STUFFC()	706

SUBSTR()	707
SUBSTRC()	708
SYSID()	709
TAN()	710
TARGET()	711
TEXTMERGE()	712
TEXTPOS()	713
TIME()	715
TRANSFORM()	717
TRIM()	718
TTOC()	719
TTOD()	721
TOSEC()	722
TXNLEVEL()	723
TXTWIDTH()	724
TYPE()	726
UNBINDEVENTS()	728
UPDATED()	729
UPPER()	730
USED()	731
VAL()	732
VARTYPE()	733
WEEK()	735
XMLTOCURSOR()	736
YEAR()	737
Notes	738
JAXBase Environment Objects and Variables	740
JAXBase Environment Variables	741
_JAXFolder	741
_JAXHome	741

_JAXTemp	741
JAXBase Environment Objects and Arrays	742
_JAX	743
_CPUS	744
_DRIVES	745
_Header	746
_KEYBOARD	748
_MONITORS	749
_MOUSE	750
_OS	751
_PRINTERS	752
_REGEX	753
_TABLE	754
JAXBase Class Reference	756
_APPEND	757
_BROWSE	758
_EDIT	761
_ERROR	762
BARCODE	764
CHECKBOX	766
COLLECTION	767
COMBOBOX	768
COMMANDBUTTON	769
COMMANDGROUP	771
CONTAINER	772
CUSTOM	773
EDITBOX	775
EMPTY	777
FILE	778
FORM	779

FORMSET	782
FTP	783
GRID	784
HTTP	785
IMAGE	786
LABEL	787
LINE	Error! Bookmark not defined.
LISTBOX	789
MENU	790
OPTIONBUTTON	791
OPTIONGROUP	792
PAGE	Error! Bookmark not defined.
PAGEFRAME	793
PIPE	794
POP3	795
PRINTER	796
SHAPE	797
SMS	799
SMTP	800
SOUND	801
SPINNER	803
SQL	804
TEXTBOX	806
TCP	808
TIMER	809
UDP	810
VIDEO	811
JAXBase Concepts	814
Arrays, Variables, and Objects	815
Code Page Support	816

Data Sessions and Work Areas	818
Dates	819
DBF Fields	820
Debugging.....	821
Error Handling.....	822
Indexes	825
Keyboard Commands.....	828
Logging	829
Merging Text	832
Moving Data to and from a SQL Database	833
Reading and Writing Barcodes	834
Scope Clauses.....	836
Tables and Cursors.....	838
Using One or More a SQL Database	839
Using the Regular Expression Object	840
Appendixes.....	842
Appendix A – File Types.....	843
Appendix B – Table File Structure	844
32-bit Table Structure	844
32-bit Table Field Sub-Records	845
64-bit Table Structure	846
64-bit Table Field Sub-Records	847
Appendix C – JAXBase Field Types.....	849
APPENDIX D – JAXBase Data Types	850
Appendix E – System Variables	851
Appendix F – JAXBase Data Sessions	852
Appendix G – DEF Files	853
Appendix H – Collation Sequences	854
JAXBase FAQ	857

DRAFT

Forward

If you are knowledgeable in DBase or FoxPro, you will be quite familiar with JAXBase.

This is version 0.1 and is just a partial document release. Version 0.4 will be the first binary release where the user will have a basic IDE that allows them to compile a few different types of files (such as PRG and SCX) with the ability to do simple input/output and work with tables and indexes.

I have been asked why I am doing this. The quick answer is because I love the XBase language and detest the corporate decision to abandon it because it gave too much power to the developer and not enough to the corporate bottom line.

Further, it is my opinion that, when done right, the XBase language is one of the most powerful and easy to use languages ever developed for the desktop and is well within reach for a beginner to learn. Additionally, computers are about handling data, and the XBase language is all about handling data in clear and concise ways that are easy for many people to understand.

I was tempted to make an open-source version of Visual FoxPro, but I decided that I wanted to improve the language by eliminating the "last century thinking" and improve the language constructs that remain. Additionally, there are desperately needed features that I think a modern XBase dialect should have. You will also see a broad array of new features that allow you to do things not easily done in other dialects.

I don't know if anyone will be greatly interested in reviving the XBase language, but finding the answer to that question is my way of giving back to an industry that has given me so much.

XBase. It has been very, very good to me.

Jon Lee Walker
jwalker.dev@gmail.com

JAXBase Features

The JAXBase Project is an attempt to create a modern version of the venerable XBase language. As stated earlier, Version 0.4 will be the proof of concept and everything before Version 2.0 will be written in C# and .Net which is not the ideal platform for a cross-platform computer language.

Version 1.0 is will new features over legacy xBase dialects, including:

JSON support. Examples are SCATTER, GATHER, GETJSON(), JSONTTOOBJ(), OBJTOJSON(), and PUTJSON() functions.

APARAMTERS captures and accepts non-array parameters sent and stuffs them into a one-dimensional array.

New classes, including SQL, BARCODE, PRINTER, REPORT, LABEL, HTTP, and TCP.

Better backend SQL handling using the SQL class and JAXBase commands like:

```

CLOSE ALL
CLEAR ALL

* Open the TEST database and create the MyDB SQL object variable
OPEN DATABASE TO MyDB USING "server=127.0.0.1;uid=root;pwd=12345;database=test"

* Since there isn't an open table in this work area, JAXBase assumes
* you are accessing a table in the current SQL database.
* MyDB!MyTable explicitly tells JAXBase to go get a SQL Table.
SELECT * FROM MyTable TO CURSOR SqlResults

* Insert or append a record to the local cursor.
* Extra code may be needed, depending on the primary key.
INSERT INTO MyTable (FirstName, LastName, BirthDate) VALUES ("JOHN","SMITH",{2001/07/15})

* Update the SQL Table using the cursor in this work area
MyDB.UPDATE("MyTable")

IF MyDB.AError[1].ErrorNo>0
  ? "Error!"
ENDIF

* Close the connection
CLOSE DATABASE MyDB

```


JAXBase Departures from XBase

JAXBase is highly object oriented, and most legacy/MS-DOS commands and last-century paradigms have been removed.

The @ commands, color pairs, and color schemes no longer exist.

ON ERROR is gone. Use TRY/CATCH/FINALLY or the Error method in classes.

Menus and its related components are now a class instead of commands.

COLUMNS and other collections common to various classes have now been rolled up into the OBJECTS collection.

Most JAXBase table commands will be able to interact with a SQL database and its tables. Additionally, the SQL class provides you with rich features, allowing you to use the specific syntax for your preferred SQL engine.

JAXBase is designed to be a true cross platform language. Windows will be the first targeted operating system with Linux support coming in Version 1. Operating system specific features are not part of the language, but there will be ways to communicate with the operating system using external add-on libraries.

When the XBase language was introduced, micro-computers had one floppy disk capable of holding under three hundred Kbytes which meant that every byte was precious and thus abbreviated commands were allowed. Today, clarity is much more important and JAXBase is fairly limited in what it will allow for abbreviations. If the command can be abbreviated, there will be a section header called Abbreviation which details accepted abbreviations for the command.

Proper spacing with commands is required. An example of required spacing (where ^ = space)
`INSERT^INTO^Table^(field1,field2)^VALUES^(expr1,expr2)`

Expressions have fewer requirements for proper spacing since A=B.OR.C=D is easily parsed by a computer.

There is no local database container support.

Only simple indexes (IDX) are available for local tables and cursors.

Reports and labels are rendered using open-source office suites.

Current Status

2025-08-31 – Version 0.2

Version 0.1 is an unfinished document release to GitHub which can be considered a "Request for Comment" period. Version 0.2 will be a proposed Final Draft, and Version 0.3 will be the Final Draft for Version 1.0 of JAXBase.

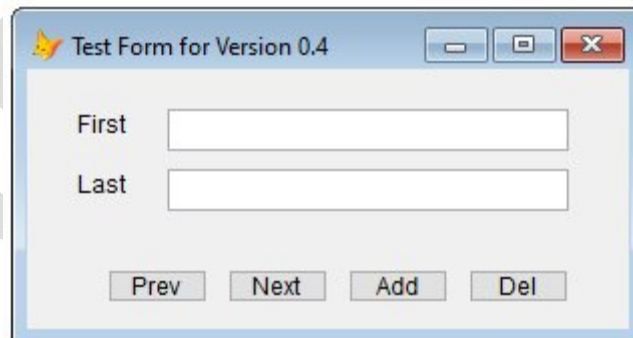
If you see a Status header, then that feature is expected to be released in a later version.

The first public binary release will be Version 0.4 for Windows. At present, JAXBase is written in C# and .Net and will remain in C# through Version 1. Linux versions will use MONO.

Form, TextBox, Label, and CommandButton are the only visual classes slated for Version 0.4 of JAXBase. Most visual classes will be available in Version 0.6, while the rest are expected to show up by the Version 1.0 release.

The Version 0.4 binary will be released when a simple form containing labels, text boxes, and buttons can be correctly displayed, and a user can interact with the form and save data to a table.

This is the form that is being used to test JAXBase Version 0.4, as displayed by Visual FoxPro.



The code for table and index handling is written, though it is still in testing. Tables are currently set for exclusive access only. Version 0.8 will introduce table/record locking and sharing.

Release Goals

Unfortunately, as a one-man-band development team, I cannot promise anything because "life happens". However, the goals for future releases are as follows.

Version 0.4

- Initial binary release capable of running a form and saving data to a DBF

Version 0.6

- Complete most visual classes and functions for Version 1
- Port to a Raspberry Pi 5 running Ubuntu Server for text-only testing
- Initial release of SQL backend support

Version 0.8

- Introduce DBF sharing and locking.
- Get development editors to the "good enough" level
- Further support for SQL backend

Version 1.0

- Most editors and tools will be written in JAXBase code with full source in the TOOLS folder
- Simplistic report and label designer
- Simplistic form, menu, class, and table editors
- DBF header, and record locking with multi-user support on tables
- Full documented support for MS SQL Server and MySQL databases

Version 1.1

- Error correction release

Version 1.2

- Further command and function development

Version 1.4

- Complete development for all commands, classes, and functions for Version 1

Version 2.0

- Move JAXBase to C++ or Rust and use a GUI framework like Qt
- Enhanced report and label designers
- Add support for PostgreSQL, Oracle, and other data engine back ends
- Integration with IIS and Apache Web Server
- Double-byte characters and advanced multi-Lingual support

Features Not Supported in Version 1

Double byte expressions, functions, and fields.

General and Blob fields.

Code pages are not supported.

Only MACHINE collation is currently supported.

Some JSON functions will not be available until Version 2.

XML support.

Direct multi-lingual support. I am working on ideas to make it easier to develop multi-lingual applications. These ideas are in the JAXBase: Muti-Lingual Applications document and input is always welcome.

There may be more, and it will be mentioned in the documentation. Look for the Status headers.

Licensing, etc.

For now, the JAXBase system is Copyrighted, but it is expected to be released under the GNU General Public License version 2 (GPL-2.0) with the first source release.

JAXBase is written in C# for Windows using .NET, and efforts by others willing to help make JAXBase run on Linux will be very welcome. Version 2.0 is expected to move to C++ or Rust.



Security Concerns



JAXBase is under rapid development and there is no intention or expectation of meeting modern security standards. Development and integration of data-at-rest security will begin after Version 1.0 is released. For that reason, Version 1 of JAXBase will be labeled as an educational and personal development platform.

Version 2 will be built to comply with modern security requirements from the ground up.

Mission of the JAXBase Project

The mission of the JAXBase Project is to encourage use of open-source software and the renewal of the XBase language as a powerful and modern solution, allowing the thousands of legacy XBase applications to once again be relevant to Windows and Linux users.

Our Goals

To create and encourage the use and development of secure, open-source software.

To create a cross-platform language that frees the developer and user from hardware/OS concerns, licensing fees, and use restrictions.

To create an ecosystem that encourages the development of multilingual support.

To allow users to create responsive applications that can run on something as small as the amazing Raspberry PI and have no limits on the hardware to which it can be scaled.

To encourage further development of the XBase ecosystem, supporting the use of popular back-end SQL database engines, including ones that are not open-source.

To encourage integration with other open-source projects.

Command Reference

Please be aware of the crucial differences between JAXBase and most XBase language dialects.

- 1) The popularity of the XBase language started with DBase 2, back when you had a few hundred kilobytes of storage on a floppy disk. Since every byte was considered precious, most commands could be abbreviated to four characters. For instance, GOTO BOTTOM could be written GO BOT. Modern computer systems offer multi-trillion-byte storage media and application sources can span dozens or even hundreds of megabytes of code, making clarity and readability vital. Thus, the decision was made to not allow unlimited command abbreviations in the JAXBase language.
- 2) A particular corporation made it difficult to write free text when using their operating system with their stock development languages. The JAXBase free text commands `?`, `??`, `DISPLAY`, `DIRECTORY`, along with others will only display to a text console window which is normally hidden unless you `SET CONSOLE ON`. Currently, only one console window (for the same reasons) is available.
- 3) In the typical XBase dialect, you reference reports, printers, and other objects through commands designed for those features and devices. JAXBase attempts to dispense with that paradigm and instead uses classes to create reports and labels. Similarly, printers, files, and other devices are accessed through JAXBase classes.
- 4) Because JAXBase is designed to be cross-platform, it does not directly support OLE and other Windows specific technologies that are not available in Linux. However, JAXBase will have a method of working with external libraries, providing direct support to your operating system of choice.

? ---

? [eExpression1] [, eExpression2...]

Remarks

Sends a carriage return followed by the comma delimited expression list to the console window, if open.

Parameters

eExpression1 [, eExpression2...]

Expressions to send to the console or alternate device.

Example

```
A=4  
B=7  
? "Hello"  
? A, B, B+A
```

Note

The ? command first sends a text line terminator (Windows characters 0x0A and 0x0D, Linux character 0x0D) before displaying any expressions. The expressions are separated by commas, and each comma adds a space to the displayed text.

If SET ALTERNATE or SET PRINT is turned on, then it will be sent to either or both of those two devices. If the console window is not open and ALTERNATE and PRINT is set off, the information is lost.

See Also

?? Command

Print Command

??

?? [eExpression1] [, eExpression2...]

Remarks

Sends the comma delimited expression list to the console window, if open.

Parameters

eExpression1 [, eExpression2...]

Expressions to send to the console or alternate device.

Example

```
A=4  
B=7  
? "Hello"  
?? A,B,B+A
```

Note

Unlike the ? command, the ?? command begins to display expressions at the current cursor location. The expressions are separated by commas, and each comma adds a space to the displayed text.

If SET ALTERNATE or SET PRINT is turned on, then it will be sent to either or both of those two devices. If the console window is not open, and ALTERNATE and PRINT are set off, the information is lost.

See Also

? Command

Print Command

& (macro substitution)

&var

Status

Slated for Version 1.0

Remarks

Macro substitution comes in two forms. The two types are & and () macros.

& Macro

This is the standard replacement where there is a & and variable name pair (Example &a1) and the value inside the variable is inserted into the code and then processed. An example would be:

```
* Demonstrate macro substitution
STORE "B" to A
B=1
C=&A+2 && C contains 3
```

When JAXBASE encounters a command with a & macro, it solves the value of the macro before executing the command.

A macro that is followed by a period will attach whatever follows to the value of the macro. For instance, the following prints out JAXBase.

```
a="JAX"
? "&a.Base"
```

Macro substitution typically is part of a

() Macro

This is another type of macro substitution and is faster but has limits. It can only be placed in the code where a literal is expected. These macros cannot be put into expressions or contain code for execution as they are simply meant to allow an expression to be placed where a literal is expected.

```
* Demonstrate literal substitution
A="Test.dbf"
USE (A)
```

Note

A macro can only be used to substitute a full line of code when that line of code is self-contained. In other words, you can only use a macro substitution for a line of code like an assignment or for calling a program or form.

A macro used inside a FOR or WHILE command is processed once and the value it represents is used during the execution of the loop, even if the value of the macro variable changes.

A multiline macro cannot be executed except through the EXECSCRIPT() function.

See Also

EXECSCRIPT() Function

DRAFT

&& | *

&& [programming information]
* [programming information]

Remarks

The && and * commands allow you to insert programming comments into the code, making the code easier to understand, or to give information that you want to place into the code. The comment information is stripped out of the compiled code.

While the * comment command must be the only command on the line, the && and // commands can appear anywhere, but at that point, the remainder of the statement becomes part of the comment and discarded during compilation.

Example

```
* HelloWorld.prg demonstration program
SET CONSOLE ON   && open the command window
? "Hello world"
```

ACTIVATE

ACTIVTE CONSOLE cConsole

Remarks

Activate a console

Abbreviation

ACT

Parameters

cConsole

Name of console to activate. If omitted, DEFAULT is used.

Note

Currently, JAXBase only supports the default console.

See Also

CREATE CONSOLE command

DEACTIVATE CONSOLE command

SET CONSOLE command

ADD

ADD CLASS

ADD CLASS *ClassName* [**OF** *ClassLibraryName1*] **to** *ClassLibraryName2* [**OVERWRITE**]

Status

Slated for Version 1.0

Remarks

Copy a class from one class library to another.

Parameters

ClassName

Name of class to be added to the VCX file.

OF ClassLibraryName1

If included, copies the class from the supplied VCX file.

TO ClassLibraryName2

VCX file name that will have the class written to it.

OVERWRITE

If included, it will overwrite any matching class name.

Note

Use ADD CLASS to add a class definition to an existing class library or copy from one class library to another. A class can only be added to a VCX file.

ADD CLASS will work at runtime or in the IDE, but the VCX needs to be compiled before it can be used.

If the class name already exists in *ClassLibraryName1* and **OVERWRITE** is not specified, an error is generated.

If you omit *OF ClassLibraryName1*, JAXBase will look for *ClassName* in all open class libraries. If the *ClassName* is not found, an error is generated.

An error message will be generated if you attempt to write to a class library that already has a class with the same name.

See Also

AClass() Function
AMembers() Function
CREATE CLASS Command
CREATE CLASSLIB Command
DEFINE CLASS Command
MODIFY CLASS Command
RELEASE CLASSLIB Command
REMOVE CLASS Command
RENAME CLASS Command
SAVE CLASS Command
SET CLASSLIB Command

DRAFT

ADD OBJECT

ADD OBJECT *ClassName* **AS Class** [**WITH** *cPropertyList*] [**PROTECTED**] [**HIDDEN**]

Status

Slated for Version 0.6

Remarks

Specifies the name of the class to add in a definition.

Parameters

ClassName

Character literal of name of the class to add to the definition.

AS Class

Indicate the class that *ClassName* is to be created as

WITH cPropertyList

Comma delimited list of properties to add or set in the class.

PROTECTED

Sets the new class as protected and can't be updated from outside the definition.

HIDDEN

Sets the new class so that it can only be seen from inside the definition.

Example

See DefineForm.prg in the EXAMPLES folder

Note

ADD OBJECT can only be used in a class definition.

See Also

DEFINE Command

ADD TABLE

ADD TABLE TableName [AS SQLTableName] [TO cDBName] [OVERWRITE] [NODATA]

Status

Slated for Version 0.6

Remarks

Specifies the name of the DBF table to copy to an open SQL Database.

Parameters

TableName

Name of DBF table to copy.

AS SQLTableName

If included, allows the renaming of the DBF table, else the table's filename stem will be used.

TO cDBName

Specifies the database connection to use, otherwise the current database connection is used.

OVERWRITE

If included, allows the overwriting of a SQL table if it exists.

NODATA

If included, only the table structure is copied to the SQL database.

Example

* Demonstrate

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

* Assumes a SQL Connection exists to a backend database

OPEN DATABASE TEST USING FILETOSTR("SQLConnect.dsn")

CREATE TABLE TEST1 (PKey C(18), Name C(30), Balance N(10,2))

* Send the structure to the SQL Server

ADD TABLE (TEST1) OVERWRITE NODATA

Note

The JAXBase table fields will be converted to corresponding SQL tables. For instance, memo fields will be converted to VarChar(MAX) in SQL Server databases and TEXT for MySQL databases. This command is used to quickly move a DBF into a SQL table. For better control of the process, see the SQL class.

See Also

CLOSE Commands

CREATE DATABASE Command

DISPLAY TABLES Command

OPEN DATABASE Command

REMOVE TABLE Command

SELECT DATABASE Command

SELECT DATASESSION Command

INDBC() Function

SQL Class

ALTER TABLE

ALTER TABLE TableName

ADD|ALTER COLUMN FieldName FieldType [(nFieldWidth [,nPrecision])]
[NULL | NOT NULL] [AUTOINC [NEXTVALUE nNextValue [STEP nStepValue]]]

Status

ALTER for DBF tables is slated for Version 0.6

NULL and AUTOINC support is slated for Version 2.0

Remarks

Use ALTER TABLE to change a field in a DBF table.

Abbreviation

ALT

Parameters

ADD COLUMN

Command used to add a column to a table.

ALTER COLUMN

Command used to alter a column in a table.

FieldName

Literal character name of field to add or alter.

FieldType

Literal character value indicating a JAXBase field type. See [DBF Fields](#)

nFieldWidth

Literal numeric value indicating width of the field.

nPrecision

Literal numeric value indicating of decimal points.

NULL | NOT NULL (Slated for Version 2, ignored for now)

AUTOINC, NEXTVALUE, STEP (Slated for Version 2, ignored for now)

Example

*** Demonstrate ALTER TABLE**

CLEAR ALL

CLOSE ALL

SET SAFETY OFF

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

CREATE TABLE Test1 (Name C(20))

USE

ALTER TABLE Test1 ALTER COLUMN Name C(30)

ALTER TABLE Test1 ADD COLUMN Age N(3)

USE Test1

DISPLAY STRUCTURE

Note

ALTER TABLE requires exclusive access to a table. If the table being altered is open, even with an alias, it will be closed.

If the table cannot be opened exclusively, an error is generated.

Use the SQL class EXEC method to alter a table in a SQL database.

See Also

ADD TABLE Command

AFIELDS() Function

CREATE TABLE

INDEX Command

MODIFY STRUCTURE Command

OPEN DATABASE Command

SQL Class

APARAMETERS

APARAMETERS ArrayName

Remarks

APARAMETERS places all parameter values passed to the routine into a local array specified as aVar. If a variable exists with that name, it becomes unavailable as the new local variable takes precedence. If no parameters were passed, then aVar will have one element holding a .null. value.

The APARAMETERS statement must be the first executable command in a routine, otherwise an error will be generated.

By default, parameters sent using DO / WITH commands are handle where Arrays and Objects are passed by reference and all others are sent by value. However, the APARMETERS command will generate an error if you attempt to send an array as a parameter.

Abbreviations

APAR

APARAMETER

Parameters

ArrayName

Name of the array variable to place parameters passed from the calling program.

Example

```
* Demonstrate APARAMETERS
Do Testing1 WITH 1,3,9,"A"
RETURN
```

```
PROCEDURE Testing1
APARAMETERS aTest
LIST MEMO LIKE aTest
RETURN
```

Note

If APAMETERS is used in the main program, then any parameters passed to it from outside the program, such as from a command prompt, are placed into the array by value.

See Also

ALen()

DIMENSION

DO FORM

DO program

LPARAMETERS

PARAMETERS

User Defined Functions

APPEND

APPEND

APPEND

Status

Slated for after Version 1.0

Remarks

Displays the append screen allowing you to directly apply data to the end of the currently open table or cursor in the work area, if it is not read-only. If no table or workarea is open, it will generate an error.

Example

*** Demonstrate APPEND**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

CREATE TABLE TEST (Name C(30))

APPEND

? RECCOUNT()

USE

See Also

APPEND BLANK

APPEND FROM ARRAY Command

BROWSE Command

EDIT Command

IMPORT Command

INSERT command

SELECT – SQL Command

APPEND BLANK

APPEND BLANK [IN nWorkArea | cTableAlias [SESSION nSessionID]]

Remarks

Appends a blank record to the end of the DBF table. If APPEND BLANK is applied to a SQL Table then an error is generated. The Append Window is not open with APPEND BLANK. You can alter the record with BROWSE, EDIT, or the REPLACE command.

Parameters

IN nWorkArea | cTableAlias

Causes ADD BLANK to create a record in a different work area or table

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE TEST1 (Name C(20))
```

```
SELECT 2
CREATE TABLE TEST2 (Name C(20))
```

```
SELECT 3
CREATE TABLE TEST3 (Name C(20))
```

```
* Add a blank record to TEST1
APPEND BLANK IN 1
```

```
* Add a blank record to TEST2
APPEND BLANK IN TEST2
```

```
CLOSE TABLES ALL
```

Note

If the table has a UNIQUE or CANDIDATE index open, you could generate an error if a blank key already exists.

See Also

APPEND

APPEND FROM ARRAY Command

BROWSE Command

EDIT Command

IMPORT Command

INSERT command

SELECT – SQL Command

APPEND FROM ARRAY

APPEND FROM ARRAY ArrayName [FOR lExpression]
 [FIELDS FieldList | FIELDS LIKE Skeleton | FIELDS EXCEPT]

Status

Slated for Version 0.6

Remarks

Append elements from an array into a table.

Parameters

ArrayName

Name of array containing data for the table.

FOR lExpression

Conditional expression that controls whether a record is appended from the array.

FIELDS FieldList

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DIMENSION aRecs[3]
aRecs[1]="TOM"
aRecs[2]="DICK"
aRecs[3]="MARY"
```

```
CREATE TABLE TEST1 (Name C(20), Age I)
APPEND FROM ARRAY aRecs
```

Note

Memo, General, and Blob fields are ignored in APPEND FROM ARRAY

When a table is open for shared use, APPEND FROM ARRAY locks the table header while records are being added.

If a one-dimensional array has more elements than fields, the remainder are ignored.

If a two-dimensional array has more columns than fields being appended in each row, the remaining elements in each row of the array are ignored.

If there are not enough columns in an array to fill all fields, the rest are left blank.

See Also

APPEND

APPEND BLANK

BROWSE Command

EDIT Command

IMPORT Command

INSERT command

SELECT – SQL Command

APPEND FROM

Appending from DBF or Cursor

**APPEND FROM cFile [FIELDS FieldList] [FOR lExpression]
[AS nCodePage]**

Appending from Plain Delimited Text File

**APPEND FROM cFile [FIELDS FieldList] [FOR lExpression] [AS nCodePage]
[DELIMITED [WITH Delimiter | WITH BLANK | WITH TAB | WITH CHARACTER Delimiter]]
[AS nCodePage]**

Appending from File of TYPE

**APPEND FROM cFile [FIELDS FieldList] [FOR lExpression]
[TYPE CSV| SDF | [XLS | XLSX | ODS [SHEET cSheetName]]
[AS nCodePage]**

Status

Slated for after Version 1.0

Remarks

Appends from an ALIAS, or closed DBF file if TYPE is omitted, otherwise appends from the TYPE of external file specified.

Parameters

cFile

Character literal of source file.

FIELDS FieldList

Character literal comma delimited list of fields to append into.

FOR lExpression

Logical expression filtering the records being appended.

DELIMITED

Delimited files are assumed as .txt unless otherwise specified. If DELIMITED WITH is omitted, then the default delimiter is a comma. Character fields are also delimited with double quotes.

DELIMITED WITH Delimiter

Allows specification of a different field character delimiter.

DELIMITED WITH BLANK

Specifies that fields are delimited with blanks instead of commas.

DELIMITED WITH TAB

Specifies that fields are delimited with tabs instead of commas.

DELIMITED WITH CHARACTER Delimiter

Specifies that fields are delimited with the included character literal.

TYPE

Character literal that specifies source file type. The following types are recognized.

Type	Description
CSV	Comma separated values. Like a DELIMITED file, the first row contains field names and is discarded.
SDF	System Data Format file. Fields are a fixed width format in a similar manner to a JAXBase table. Dates are in yyyyMMdd format and DateTime values are in yyyyMMddTHH:mm:ss format.
XLS	An older Excel spreadsheet file.
XLSX	A modern Excel spreadsheet file.
ODS	An Open Document Spreadsheet file.

SHEET cSheetName

Specifies the spreadsheet tab to append from.

AS CodePage (Version 2.0)

Specifies the Code Page of the source file.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE TEST_DELIMITED (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\import\PEOPLE_DELIMITED.TXT") DELIMITED
USE
```

```
CREATE TABLE TEST_TAB (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\import\PEOPLE_TAB.TXT") DELIMITED WITH TAB
USE
```

```
CREATE TABLE TEST_SDF (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\import\PEOPLE_SDF.TXT") TYPE SDF
USE
```

```
CREATE TABLE TEST_CSV (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\import\PEOPLE.CSV") TYPE CSV
USE
```

```
CREATE TABLE TEST_XLS (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\data\PEOPLE.XLS") TYPE XLS
USE
```

```
CREATE TABLE TEST_ODS (Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\data\PEOPLE_DELIMITED.ODS")
USE
```

Note

You cannot append from a table in a SQL database, but you can SELECT from a SQL table to a cursor and append to a DBF.

You cannot APPEND directly onto a SQL Table. You can SELECT an empty cursor (SELECT * FROM table WHERE 1=2), APPEND to that cursor, and then use the UPDATE FROM Command to copy the cursor into the SQL table.

If a table name is specified and there is no matching alias, then JAXBase will attempt to find the table in the path list if a path was not specified with the name.

Any fields that do not match the FIELDS FieldList parameter are ignored.

It is important to have good error checking in case if a rule is violated in a SQL table, or a CANDIDATE index is violated in a DBF, causing an error to be generated.

See Also

COPY FILE Command
COPY TO Command
EXPORT Command
GETCP() Function
IMPORT Command
SET DEFAULT TO Command
SET DELETED Command
SET PATH TO Command
TRY/CATCH/FINALLY/ENDTRY Command
UPDATE Method
SQL Class

APPEND FROM JSON

APPEND FROM JSON *cExpression* [**FOR** *lExpression*]
[FIELDS *FieldList* | **FIELDS LIKE** *Skeleton* | **FIELDS EXCEPT**]

Status

Slated for Version 0.6

Remarks

Appends one or more JSON records to a table.

Parameters

cExpression

Character expression containing a valid JASON string.

FOR *lExpression*

Conditional expression that controls whether a record is appended from the array.

FIELDS *FieldList*

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
cJSON="{[{Name:'John Smith', Age:20, ShoeSize:10},{Name:'Jane Doe', Age: 18, ShoeSize:6.5}]}"
CREATE TABLE TEST1 (Name C(20), Age I)
APPEND FROM JSON cJSON      && Adds two records to TEST1
```

Note

Each JSON record appends a record to the table.

When appending a record, any field names in the JSON string that match field names in the table are used. Extra JSON fields are ignored. Fields with no matching JSON fields are left empty.

The APPEND FROM JSON command will work with JAXBase tables and cursors. To update a SQL table, first append to a table or cursor and use the UPDATE Command.

See Also

GETJSON() Function
JSONTOOBJ() Function
OBJTOJSON() Function
PUTJSON() Function
UPDATE command

ASSERT

**ASSERT lExpression [MESSAGE cMessage
[TO CONSOLE cConsole | TO VAR cVariable | TO FILE cFilename] [NODIALOG]**

Remarks

If the lExpression resolves to true (.T.) then the message is sent to the debug screen and displayed to a dialog with Debug, Cancel, Ignore, and Ignore All buttons.

Selecting one of the buttons produces the following actions:

Debug – Suspends program execution and displays the Trace Window (Version 1.0)

Cancel – Cancels program Execution.

Ignore – Continues program execution.

Ignore All – Performs a SET ASSERT OFF and continues program execution.

Parameters

lExpression

Logical expression which, if evaluated to true (.T.), sends the optional cMessage to the dialog.

cMessage

Optional character expression containing the message to send if lExpression is True (.T.).

TO CONSOLE cConsole | VAR cVariable | FILE cFilename

Optionally send the assert message out to a console, a variable, or appends to a text file.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** Send the assert message to the file Asserts.txt**

ASSERT 1=1,MESSAGE "Equals test passed" TO Asserts.txt

Returns

Numeric

Note

An ASSERT brings up the Assert Dialog and requires user intervention, depending on the SET ASSERTS settings.

ASSERT always sends data out to the DEBUGOUT file if DEBUG is ON.

SET ASSERT defaults to ON in the IDE and OFF in the runtime.

If SET ALTERNATE is set to save to a file or printer, then the assert message will be written and what button was selected.

If SET ASSERTS has been set to ON NODIALOG, the ASSERT message will be sent to the active Console and, if ALTERNATE is ON to that device.

If there is nothing after TO VAR or TO FILE, an error will be generated.

TO FILE appends the information from the ASSERT to the specified file in the format of

```
ASSERT: <cMessage>  
RESPONSE: <nResponse>
```

See Also

SET ALTERNATE Command

SET ASSERTS Command

SET CONSOLE Command

SUSPEND Command

Trace Window

AVERAGE

```
AVERAGE ExprList [Scope] [FOR lExpr1] [WHILE lExpr2]
TO VarList | TO ARRAY ArrayName [ADDITIVE]
[IN nWorkArea | cAlias]
```

Remarks

Takes the averages of the ExprList and puts them into VarList or an array.

Abbreviation

AVER

Parameters

ExprList

Comma delimited list of expression to use when creating the averages to place into VarList.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

TO VarList

Character literal list of variable names that will receive the averages.

TO ArrayName [ADDITIVE]

Character literal of array name to create or append to if ADDITIVE is provided.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Example

```

* Demonstrate AVERAGE
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
USE (_JAXFolder+"data\polldata")

* Using a unique numeric literal as one of the expressions provides an
* identifier for each result row. If there are no matching records, all
* non-literal numeric expressions will return 0.00 as their result.
AVERAGE 1,age TO ARRAY aVoteAge for Precinct=203 AND Party="I"
AVERAGE 2,age to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="D"
AVERAGE 3,age to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="R"

DISPLAY MEMO LIKE aVoteAge

```

Note

When saving to an array, if the array does not exist then it is created. If it does exist, it is overwritten with the corresponding number of elements. If ADDITIVE keyword is included, then the array is appended with a new row. If the number of expressions does not match the existing array column count, an error is generated. If there are no results, the array is updated numeric literals and any non-literal expressions are set to 0.00.

ADDITIVE should not be in the first of a series of AVERAGE statements.

If saving to a variable list, values are returned even if there are no matching records, in which case 0.00 is filled in for any non-literal numeric expression.

See Also

CALCULATE Command
 COUNT Command
 SUM Command
 TOTAL Command
 JAXBase Concepts: Arrays, Variables, and Objects
 JAXBase Concepts: Scope Clauses

BEGIN TRANSACTION

BEGIN TRANSACTION

Status

Slated for Version 0.6

Remarks

Puts the current SQL Database into transaction mode, allowing you to ROLLBACK or COMMIT the changes. JAXBase tables are not affected by transactions.

Abbreviation

BEG

Note

The COMMIT() and ROLLBACK() functions allow you to manage the transaction directly. If you do not use them, then when the ENDTRANSACTION command is executed, a commit will be sent. If the commit fails, an error will be generated, and the transaction will be automatically rolled back.

The SQL Class can also be used to generate and manage transactions.

SQLTest.prg in the TOOLS folder demonstrates working with a SQL backend database.

See Also

COMMIT() Function

ROLLBACK() Function

ENDTRANSACTION Command

SQL Class

BLANK

BLANK [**FIELDS** FieldList] [**Scope**] [**FOR** lExpression1]
 [**WHILE** lExpression2] [**IN** nWorkArea | cTableAlias]

Status

Slated for Version 0.8

Remarks

Blanks all fields in a DBF if a field list is not specified.

Parameters

FIELDS

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Note

AUTOINC fields are never altered by the BLANK command.

If you execute BLANK with a SCOPE when a CANDIDATE index is open, you risk generating an error.

If a scope is not specified, then only the current record is affected.

See Also

APPEND Command

EMPTY() Function

ISBLANK() Function

REPLACE Command

INDEX Command

JAXBase Concepts: Scope Clauses

BROWSE

BROWSE [**FIELDS** FieldList] [**TITLE** cTitleText] [**NAME** cName]
 [Scope] [**FOR** lExpr]
 [**NOSHOW**] [**NOWAIT**] [**NOAPPEND**] [**NODELETE**]

Status

Slated for Version 1

Abbreviation

BROW

Remarks

JAXBase has a BrowseWindow class that is used to create a browse-window and can be manipulated like other objects.

PARAMETERS

FIELDS FieldList

An optional list of fields to display in the browse window.

TITLE cTitleText

Optional text to place into the caption of the browse window.

NAME cName

If declared, creates a public variable that holds the browse window object allowing the browse window class to be addressed.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr

Logical expression used to filter what records are used.

NOSHOW

Prevents the browser window from becoming visible and sets the AUTOONEXIT to false (.F.) so that the browser window stays in memory as long as the variable remains in scope. If a NAME was not specified, the NOSHOW clause is ignored.

NOWAIT

The NOWAIT clause will continue execution after the browser windows is displayed. The NOWAIT clause is meaningless if the NOSHOW clause is also used.

NOAPPEND

Prevents the user from appending to the table or cursor.

NODELETE

Prevents the user from marking records for deletion.

Example

A browse-window can be modified in code by affecting its properties as a JAXBrowseWindow object.

```
BROWSE NAME MyBrowser NOSHOW      && Create the MyBrowser object with Visible=.F.
```

```
MyBrowser.Columns[1].Header1="Name"
MyBrowser.Columns[1].Width=150
MyBrowser.Columns[2].Header1="Address"
MyBrowser.Columns[2].Width=150
MyBrowser.Height=100
MyBrowser.Width=250
MyBrowser.ReadOnly=.T.
MyBrowser.Show
```

Notes

Omitting the NAME clause creates a private variable using the browser's default name BrowseWindowX, where X is 1 and is incremented until a free name is found. When the private variable goes out of scope, the browser window will be removed from memory. Using the NAME clause creates a public variable allowing the browser to remain in memory until released or overwritten.

When the user hits ESCape or otherwise closes a browse window created with a BROWSE command, it automatically unloads from memory. The NAME parameter creates a private variable of that name (unless it already exists in which case it makes that variable a BrowseWindow object) and sets the AUTONEXIT property to a logical false. When AUTOEXIT is set to false, the object remains in memory until it is released, or the variable loses scope.

See Also

APPEND Command

EDIT Command

JAXBase Concepts: Scope Clauses

BUILD

BUILD APPLICATION

BUILD APPLICATION cProjectName [TO cApplicationName]

Status

Slated for Version 1.0

Remarks

Compiles and bundles all files included in a project into an APP which can be run by the JAXRun executable.

Parameters

cProjectName

Literal filename of the project file (PJX) to compile into an application.

cApplicationName

If supplied, creates an application using the cApplicationName literal.

Example

* Create TEST1.APP from Test.pjx
BUILD APPLICATION Test AS Test1

Note

If any errors are generated, a text file named the same as the application name but with an ERR extension is created. This will contain the log of the process and list any errors found.

See Also

BUILD EXE Command

BUILD PROJECT Command

CREATE PROJECT Command

BUILD EXECUTABLE

BUILD EXECUTABLE cProjectName [TO cApplicationName] [AS WINDOWS | LINUX]

Status

Slated for Version 2.0

Remarks

Compiles and bundles all files included in a project into an executable which can be run natively by the operating system.

Parameters

cProjectName

The name of the project file (PJX) to compile into an application.

cApplicationName

Normally, the application file stem is the same as the application file stem, but the TO clause allows you to specify a different application name.

Windows|Linux

Specify the type of executable for the desired target operating system.

Example

*** Create a Linux executable**

BUILD EXECUTABLE Test TO Test1 AS LINUX

Note

You can create an EXE for any supported operating system.

If any errors are generated, a text file named the same as the application name but with an ERR extension is created. This will contain the log of the process and list any errors found.

See Also

BUILD APPLICATION Command

BUILD PROJECT Command

CREATE PROJECT Command

BUILD PROJECT

BUILD PROJECT cProjectName

Status

Slated for Version 2.0

Remarks

Rebuilds a project, checking for missing references and updating the project's state.

Parameters

cProjectName

The name of the project file (PJX) to rebuild.

Example

BUILD PROJECT Test

Note

If any errors are generated, a text file named the same as the application name but with an ERR extension is created. This will contain the log of the process and list any errors found.

See Also

BUILD APPLICATION Command

BUILD EXECUTABLE Command

CREATE PROJECT Command

CALCULATE

```

CALCULATE eExpressionList [Scope]
    [FOR lExpression1]
    [WHILE lExpression2]
    [TO VarList | TO ARRAY ArrayName][ADDITIVE]
    [IN nWorkArea | cTableAlias]
  
```

Remarks

Takes the averages of the ExprList and puts them into VarList or an array.

Abbreviation

CALC

Parameters

ExprList

Comma delimited list of CALCULATE expressions to use when creating the averages to place into VarList. The list of valid CALCULATE expressions is as follows.

Expression	Result Type	Description
AVE(nExpression)	Numeric	Gets the average of the numeric expression
CNT()	Numeric	Counts the number of records selected
MAX(eExpression)	JAXBase data type	Returns the maximum value of the expression
MIN(eExpression)	JAXBase data type	Returns the minimum value of the expression
NPV(nExpr1, nExpr2[, nExpr3])	Numeric	Computes the net present value of the series
STD(nExpression)	Numeric	Computes the standard deviation
SUM(nExpression)	Numeric	Sums the values of the series
VAR(nExpression)	Numeric	Calculates the variance of the series

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

TO VarList

Character literal list of variable names that will receive the averages.

TO ArrayName [ADDITIVE]

Character literal of array name to create or append to if ADDITIVE is provided.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Example

*** Demonstrate CALCULATE**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\polldata")

*** Using the MIN or MAX of a unique numeric literal as one of the**

*** expressions will provide an identifier for each result row. If**

*** there are no matching records, all non-literal numeric**

*** expressions will return 0.00 as their result.**

CALCULATE MIN("I"), AVE(age), CNT(Party) TO ARRAY aVoteAge for Precinct=203 AND Party="I"

CALCULATE MIN("D"), AVE(age), CNT(Party) to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="D"

CALCULATE MIN("R"), AVE(age), CNT(Party) to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="R"

DISPLAY MEMO LIKE aVoteAge

Note

When saving to an array, if the array does not exist then it is created. If it does exist, it is overwritten with the corresponding number of elements. If ADDITIVE keyword is included, then the array is appended with a new row. If the number of expressions does not match the existing array column count, an error is generated. If there are no results, the array is updated with non-literal expressions set to 0.00.

ADDITIVE should not be in the first of a series of CALCULATE statements.

If saving to a variable list, values are returned even if there are no matching records, in which case 0.00 is filled in for any non-literal numeric expression.

See Also

AVERAGE Command

COUNT Command

SUM Command

TOTAL Command

JAXBase Concepts: Arrays, Variables, and Objects

JAXBase Concepts: Scope Clauses

CANCEL

CANCEL

Remarks

Cancels execution of the current program, releases all private and local variables, and clears code cache.

Note

If in the IDE, control is returned to the IDE command window. If it is running as an APP or EXE, control is turned back to the operating system.

See Also

SUSPEND

ASSERT

SET STEP

DRAFT

DO CASE ... CASE... OTHERWISE...ENDCASE

```
DO CASE
  CASE lExpression1
    [commands]
  [CASE lExpression2]
    [commands]
  ...
[OTHERWISE]
ENDCASE
```

Remarks

The CASE structure allows you to specify a block of code to run based on the first logical expression evaluating to True (.T.) or failing that executing the OTHERWISE code block.

Example

```
* User will enter a number and the appropriate
* case or otherwise block will execute
*
SET CONSOLE ON
A=INPUTBOX("Give me a number between 1 and 5")
I = VAL(A)
CASE
  CASE I=1
    ? "You entered 1"
  CASE I<3
    ? "You entered 2"

  CASE I<6
    ? "You entered ",I

  OTHERWISE
    ? "I said a number between 1 and 5!"
ENDCASE
```

Note

Only the commands following the first CASE clause that evaluates to True (.T.) are executed, or the commands following the DEFAULT if they all fail.

After the selected block of code is executed, control is passed to the command following the ENDCASE statement.

If no case expression evaluates to True (.T.) and there is no OTHERWISE command, then control is passed to the command following the ENDCASE.

See Also

FOR / ENDFOR
 FOREACH / ENDFOR
 IF / ENDIF
 DO / UNTIL
 DO WHILE / ENDDO

CD

CD cLiteral

Remarks

Changes the default folder to an already existing folder.

Parameters

cLiteral

A literal expression such as C:\TEMP

Example

* Demonstration of MD command
cFolder="C:\TEMP\JAXBase\Jobs"

* Prevent an error if the folder exists

```
TRY
MD (cFolder)
CATCH
ENDTRY
```

```
=strtofile("TEST", "test.txt")
```

* The following commands all do the same thing

```
CD C:\TEMP\JAXBase\Jobs
CD &cFolder
CD (cFolder)
```

* Check to make sure text.txt exists

```
DIR
```

The default folder is set to C:\TEMP\JAXBase\Jobs and the file listing is displayed in the default console, which will include the file test.txt.

Note

The folder expression must be literal expression, such as **CD C:\TEMP** or a macro expression. If the folder does not exist, an error will be raised.

Windows is not case sensitive for folder and file names, but other operating systems are. You have two choices in this matter. You can use the SET NAMING which will automatically set all file names and paths to either upper, lower, or proper case, or you can be consistent in your naming scheme.

See Also

ADIR() Function

MD Command

SET NAMING Command

Macros and Macro Expressions

CLEAR

CLEAR

CLEAR

Remarks

Clears the active CONSOLE window.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
? "TESTING"
=INKEY(3)
CLEAR
```

See Also

ACTIVATE CONSOLE Command
SET CONSOLE Command

CLEAR ALL

CLEAR ALL

Remarks

Releases all program variables, arrays, and objects, and classes. It also closes all editor windows, open tables, files, and any active projects.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
MODIFY FILE TEST.TXT NOWAIT
```

```
A=4  
B=7  
DISPLAY MEMORY
```

```
CLEAR ALL  
DISPLAY MEMORY
```

CLEAR CLASS

CLEAR CLASS [cClassName | ALL]

Status

Slated for Version 1

Remarks

Clears all references to the named class from memory.

Parameters

cClassName

Literal expression of the class to clear from memory.

ALL

Causes all class references and class libraries to be removed from memory.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
A=CREATEOBJECT("form")
```

```
DISPLAY MEMO
```

```
CLEAR CLASS FORM  
DISP MEMO
```

Note

Classes and class libraries opened with MODIFY CLASS are not affected by the CLEAR CLASS command.

If a class name is not provided, the command is ignored.

CLEAR CONSOLE

CLEAR CONSOLE [cConsoleName]

Remarks

Like the CLEAR command but specifies which console to clear. If the named console does not exist, an error is generated.

Parameters

cConsoleName

Literal name expression specifying the console to clear.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR CONSOLE
```

Note

If a console name is not provided, the DEFAULT console is cleared.

In Windows, only the DEFAULT console can be created. Version 2.0 is expected to address this shortcoming.

CLEAR CLASSLIB - !!!

CLEAR CLASSLIB [cClassLibraryName]

Status

Slated for Version 1

Remarks

Clears all references of the specified class library from memory and closes the class library file if it is open.

Parameters

cClassLibraryName

Literal file name of class library to open.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

Note

A VCX extension is assumed for the cClassLibrary literal.

If a name is not provided, an open dialog will prompt you for a class library to open.

CLIB_EDITOR.APP in the TOOLS folder is used to modify class libraries.

All user defined classes are either manually created in code or stored in class libraries.

CLEAR EVENTS

CLEAR EVENTS

Status

Slated for Version 1

Remarks

Clears the active READ EVENTS from memory, releasing control to the code after the READ EVENTS command.

Example

*** Demonstrate CLEAR EVENTS**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

aFrm=createobject("form")

aFrm.AddObject("TMR1","timer")

aFrm.TMR1.Enabled=.T. && Timer initializes when form is created

aFrm.TMR1.Interval=5000

aFrm.SetMethod("TIMER","CLEAR EVENTS")

aFrm.Show

READ EVENTS

CLEAR ERRORS

CLEAR ERRORS

Remarks

Clears all errors from the error stack.

DRAFT

CLEAR MACROS

CLEAR MACROS

Status

Slated for Version 1

Remarks

Clears all ON KEY LABEL macros from memory.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

ON KEY LABEL F5 ? "HELLO"

DISPLAY MACROS

CLEAR MACROS

DISPLAY MACROS

CLEAR MEMORY

CLEAR MEMORY

Status

Slated for Version 1

Remarks

Releases all program variables, arrays, macros, and objects from memory.

DRAFT

CLEAR PROGRAM

CLEAR PROGRAM [cProgramName]

Status

Slated for Version 1

Remarks

Clears the named program from the cache.

Parameters

cProgramName

Literal file name or file stem of program to clear from memory.

Note

If no program name is specified, it clears the program cache, including any cached classes.

CLEAR TYPEAHEAD

CLEAR TYPEAHEAD

Status

Slated for Version 1

Remarks

Clears the keyboard buffer.

DRAFT

CLOSE

CLOSE ALL

CLOSE ALL

Status

Slated for Version 1

Remarks

Close the alternate file and all open editors, projects, databases, tables, and indexes.

DRAFT

CLOSE ALTERNATE

CLOSE ALTERNATE

Status

Slated for Version 1

Remarks

Closes the ALTERNATE output device. If none specified, it is ignored.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET ALTERNATE TO TEST.TXT

SET ALTERNATE ON

DIRECTORY (JAXFOLDER+"TOOLS*.*)")

CLOSE ALTERNATE

Note

CLOSE ALTERNATE is the same as executing the commands

SET ALTERNATE OFF

SET ALTERNATE TO

CLOSE DATABASE

CLOSE DATABASE cDatabaseName | ALL

Status

Slated for Version 1

Remarks

Close the current active database.

Parameters

cDatabaseName

Name of database to close.

ALL

If ALL is used, then all active databases are closed.

Note

When a database is closed, all active transactions are abandoned and data is discarded.

If the cDatabaseName is not open, an error message is generated.

CLOSE DATASESSION

CLOSE DATASESSION nID | ALL

Status

Slated for Version 1

Remarks

Close the current data session, the indicated data session ID, or all user data sessions.

Parameters

nExpression

Numeric literal indicating which data session to close.

ALL

Close all active user data sessions.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** Initialize and close data session 2**

SELECT DATASESSION 2

A=2

CLOSE DATASESSION (A)

CLOSE INDEXES

CLOSE INDEXES

Status

Slated for Version 1

Remarks

Close all indexes that are open in the current workarea. If none are open, it is ignored.

DRAFT

CLOSE TABLES

CLOSE TABLES [ALL]

Status

Slated for Version 1

Remarks

Close all tables and cursors in the current data session.

Parameters

ALL

If ALL is specified, closes all tables and cursors in all data sessions. Open databases are not affected.

DRAFT

COMPILE

COMPILE *cFileName* **CODEPAGE** *nCodePage*

Remarks

Compiles source files and definitions into JAXBase runtime files.

Abbreviation

COMP

Parameters

cFileName

Literal filename of the file to compile. If no extension is provided, then PRG is assumed.

nCodePage

Specifies the code page to use when compiling.

Example

COMPILE TEST.*

Notes

The file extension must match what's being compiled if you are compiling a FORM, MENU, POPUP, BAR, or CLASS LIBRARY created with their respective builders, otherwise an error will be raised. The compile command uses the extension of the specified file to determine what type of file is being compiled. If you have multiple file stems with different file extensions and do not specify the type of file to compile, then all will be compiled.

If you use a wildcard in *cFileName* then all matching files will be compiled.

CODEPAGE is not accepted with DATABASE, FORM, CLASSLIB, LABEL, MENU, PROJECT, or REPORT as you are extracting from a table definition, creating an intermediary file, and finally compiling into a tokenized format. To set the global code page setting for development, use SET CPCOMPILE.

See Also

CREATE COMMMAND Command
 CREATE CLASS Command
 CREATE CLASSLIB Command
 CREATE FORM Command
 CREATE LABEL Command
 CREATE MENU Command
 CREATE PROJECT Command
 CREATE REPORT Command
 SET CPCOMPILE Command

CONTINUE

CONTINUE

Status

Slated for Version 0.6

Remarks

Sends control to the bottom of the looping structure.

Abbreviation

CONT

Example

```
SET TALK OFF
CLOSE DATABASES
USE customers  && Open Customer table

STORE 0 TO gnCount

LOCATE FOR ALLTRIM(UPPER(customers.country)) = 'CANADA'

* All records are displayed where the country is equal to CANADA.
DO WHILE FOUND()
    gnCount = gnCount + 1
    ? company,address,city,country
    CONTINUE
ENDDO

? 'Total companies: '+ LTRIM(STR(gnCount))
```

Notes

After a successful LOCATE, the CONTINUE command looks for the next match based on the same criteria as the LOCATE and set the FOUND(), EOF(), and RECNO() accordingly.

A CONTINUE that is issued after a failed LOCATE/CONTINUE set the record pointer past the end of the file and set FOUND() to .F. (false), EOF() to .T. (true), and RECNO() to record count + 1.

CONTINUE can be repeated endlessly without generating an error, using FOUND() will allow you to detect if it was successful.

See Also

EOF() Function
 FOUND() Function
 LOCATE Command
 SEEK Command

COPY FILE

COPY FILE cFileName1 TO cFileName2

Status

Slated for Version 1

Remarks

Copies a file.

Parameters

cFileName1

Literal filename which is the source.

cFileName2

Literal filename which is the destination.

Note

COPY FILE will make an exact duplicate of cFileName1 to cFileName2.

If cFileName already exists, it will be overwritten.

If a problem occurs during copy, and appropriate error is generated.

See Also

RENAME Command

COPY INDEXES

COPY INDEXES `cIDXList` | All [TO `cCDXFile`]

Status

Possible integration into for Version 2.0

Remarks

Used to copy one or more single index file into a compound index file.

DRAFT

COPY STRUCTURE

COPY STRUCTURE TO cTableTo [FIELDS FieldList] [DATABASE cDBName] [OVERWRITE]

Status

Slated for Version 1

Remarks

Copy the currently open table structure to a new DBF or SQL Table.

Parameters

cTableTo

Character literal of target table.

FIELDS FieldList

Optional comma delimited character field literals.

DATABASE cDBName

Optional character literal of database alias to send the new structure to create a SQL table.

OVERWRITE

Optional flag indicates if cTableTo exists, to overwrite it.

Example

```
* Demonstrate COPY STRUCTURE by copying
* the supplied polldata table to a file
* in the default directory.
```

```
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
COPY STRUCTURE TO poll2 OVERWRITE
USE poll2
LIST STRUCTURE
```

Note

If the destination table exists and OVERWRITE is not included, an error is generated.

JAXBase converts the fields in a specific manner, depending on the backend database engine and may not create the best structure for SQL use.

See Also

COPY TABLE Command

CREATE TABLE Command

SQL Class

COPY TO ARRAY

```
COPY TO ARRAY ArrayName
    [FIELDS cFieldList | FIELDS LIKE cSkeleton | FIELDS EXCEPT cSkeleton]
    [Scope] [FOR lExpression1] [WHILE lExpression2]
```

Status

Slated for Version 1

Remarks

Copy fields from records that fit the scope and logical expressions to an array.

Parameters

ArrayName

Name of array to create.

FIELDS FieldList

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
COPY TOP 10 TO ARRAY aTop10
DISPLAY MEMORY LIKE aTOP10
```

Note

If ArrayName does not exist, it will be created with the corresponding number of columns and rows. If the ArrayName exists, it will be overwritten. If no records fit the parameters, then the ArrayName is not created or altered.

The number of rows you can create in an array is subject to memory.

See Also

APPEND FROM ARRAY Command
DIMENSION Command
GATHER Command
STORE Command
JAXBase Concepts: Scope Clauses

DRAFT

COPY TO

Copy to a DBF

```
COPY TO cFile [FIELDS FieldList]
    [Scope] [FOR lExpression1] [WHERE lExpression2]
[AS nCodePage]
```

Copy to a Delimited File

```
COPY TO cFile [FIELDS FieldList]
    [Scope] [FOR lExpression1] [WHERE lExpression2]
[DELIMITED [WITH Delimiter | WITH BLANK | WITH TAB | WITH CHARACTER Delimiter]]
[AS nCodePage]
```

Copy to a File Type

```
COPY TO cFile [FIELDS FieldList]
    [Scope] [FOR lExpression1] [WHERE lExpression2]
[TYPE CSV| SDF | [XLS | XLSX | ODS]
[AS nCodePage]
```

Status

Slated for after Version 1.0

Remarks

Creates a new file from the contents of the currently selected file.

Parameters

cFile

Character literal of source file.

FIELDS FieldList

Character literal comma delimited list of fields to append into.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

DELIMITED

Delimited files are assumed as .txt unless otherwise specified. If DELIMITED WITH is omitted, then the default delimiter is a comma. Character fields are also delimited with double quotes.

DELIMITED WITH Delimiter

Allows specification of a different field character delimiter.

DELIMITED WITH BLANK

Specifies that fields are delimited with blanks instead of commas.

DELIMITED WITH TAB

Specifies that fields are delimited with tabs instead of commas.

DELIMITED WITH CHARACTER Delimiter

Specifies that fields are delimited with the included character literal.

TYPE

Character literal that specifies source file type. The following types are recognized.

Type	Description
CSV	Comma separated values. Like a DELIMITED file, the first row contains field names and is discarded.
SDF	System Data Format file. Fields are a fixed width format in a similar manner to a JAXBase table. Dates are in yyyyMMdd format and DateTime values are in yyyyMMddTHH:mm:ss format.
XLS	An older Excel spreadsheet file.
XLSX	A modern Excel spreadsheet file.
ODS	An Open Document Spreadsheet file.

AS CodePage (Version 2.0)

Specifies the Code Page of the source file.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
COPY TO TEST_XLS TYPE XLS FOR Precinct=203
```

Note

If an error occurs trying to write the file, an error is generated.

If the destination file exists, it will be overwritten.

Writing to a worksheet writes the data to the default sheet name.

A CSV file will have the JAXBase field names in the first row with character data delimited with quotes.

MEMO, GENERAL, or BLOB fields will not be written, but any valid expression that evaluates to less than 255 characters will be written. Expressions 255 characters or greater will generate an error.

See Also

APPEND FROM Command

COY FILE Command

EXPORT Command

IMPORT Command

JAXBase Concepts: Scope Clauses

DRAFT

COUNT

COUNT [SCOPE] [FOR lExpression1] [WHILE lExpression2] TO Var

Remarks

The COUNT command counts the number of matching records within the scope and returns the value to Var. If no records match, the variable is set to 0.00.

Parameters

Scope

Specifies a range of records to be scanned and only the records within the range are included. The valid scope clauses are: ALL, NEXT nRecords, RECORD nRecordNumber, REST, and TOP nRecords. The Scope allows you to limit the number of records processed.

FOR

A logical expression that must be evaluated to True (.T.) for the current record to be processed by the block of code. If the FOR command is not present, it is assumed to be True (.T.).

WHILE

A logical expression that must be evaluated to True (.T.) for processing to continue. If the expression evaluates to False (.F.) then control passes to the command after the ENDSCAN.

TO Var

Character literal representing the variable name to which the count is saved.

Example

```
* Demonstrate CALCULATE
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
USE (_JAXFolder+"data\polldata")

a=0
SCAN FOR PRECINCT=203
    a=a+1
ENDSCAN

? a,"records counted"
```

Note

Both the FOR and WHILE commands are evaluated each time the SCAN command is processed.

ALL is the default SCOPE value one is not provided.

When the ENDSCAN command is executed, the table that was current when entering the SCAN/ENDSCAN loop is reselected, and the record pointer advanced, before passing control back to the SCAN command.

See Also

TOTAL Command

JAXBase Concepts: Scope Clauses

DRAFT

CREATE CURSOR / TABLE

```
CREATE TABLE cFile from ARRAY ArrayName [AS JAX | VFP | FOX26]
```

```
CREATE TABLE cFile
  (cField cType [(nWidth [,nPrecision])])
  [, cField cType [(nWidth [,nPrecision])])
  [, cField...])
  [AS JAX | FOX26 | VFP9 | DBASE]
```

Slated for Version 2.0

```
CREATE TABLE cFile
  (cField cType [(nWidth [,nPrecision])] [NULL | NOTNULL] [ENCRYPTED] [NOCPTRANS]
  [AUTOINC [NEXTVALUE nNext [STEP nStep]])]
  [, cField cType [(nWidth [,nPrecision])] [NULL | NOTNULL] [ENCRYPTED] [NOCPTRANS]
  [, cField...]) [CODEPAGE nCodePage] [COLLATE cCollateSequence] [ENCRYPTED]
```

Remarks

Create a JAXBase DBF table.

Abbreviation

CREA CURS

CREA TABL

Parameters

cFile

Character literal name of the table to create.

cField

Character literal name of a field in the table.

cType

Character literal type code (see below).

See AFIELDS() Function.

nWidth

Numeric literal for width of the field.

nPrecision

Numeric literal for number of places to the right of the decimal point are supported (for numeric non-Integer fields).

NULL | NOT NULL (Version 2.0)

Indicates if the field can hold a null value.

ENCRYPTED (Version 2.0)

Indicates that encryption should be used when writing to the table. (Version 2.0)

NOCPTRANS (Version 2.0)

Indicates that the field should not allow Code Page translation. (Version 2.0)

AUTOINC [NEXTVALUE nNext [STEP nStep]] (Version 2.0)

The field is autoincrementing and by default has a nNext value of 1 and nStep value of 1 if nNext or nStep are omitted. AUTOINC values are integer only and can range from -2,147,483,647 to 2,147,483,647.

CODEPAGE nCodePage (Version 2.0)

Specifies the code page to use for this table. See Code Page Support

COLLATE cCollateSequence (Version 2.0)

Specifies the collation sequence other than the default of MACHINE.
Supporting International Applications

FROM ARRAY ArrayName

Array name holding field information in the following format.
See AFIELDS () Function

AS JAX | FOX26 | VFP9 – FOX26 and VFP9 Slated for Version 0.8

By default, all DBF files are created as JAX (JAXBase) tables and cannot be directly opened by other legacy dialects. AS provides a way to create a table that can be opened by the respective dialects.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE TEST1 (Name C(20), Age I)
DISPLAY STRUCTURE
```

Note

If a path is not specified in cFile then the table is placed in the default path.

The new table will be opened in the lowest available work area.

nStep cannot be a value under one.

A table can hold up to 254 fields.

Visual FoxPro 9 introduced several new features that can make a DBF file unusable by earlier versions. Use FOX26 if you are trying to create a table for an earlier version of FoxPro or VFP.

JAXBase does not convert legacy DBF files, rather it reads and writes to the best of its ability. Please back up legacy tables before using them in JAXBase or open them with NOUPDATE.

See Also

AFIELDS() Function

DISPLAY STRUCTURE Command

USE Command

Code Page Support

Supporting International Applications

CREATE DATABASE

CREATE DATABASE *cDBName* [**USING** *cConnection* | *oDBObject*]

Status

Slated for Version 1

Remarks

Create a new database to a SQL backend.

Abbreviation

CREA DATA

Parameters

cDBName

Character literal of the database name to create.

USING *cConnection* | *oDBObject*

cConnection is a string expression with a valid SQL Connection string to the desired server, while *oDBObject* is a SQL Class object.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
MyDB=CREATEOBJECT("sql")
```

```
* Correct the connection data
WITH MyDB
  .Driver="SQL Server"
  .Server="MyServer"
  .Port=1433    && Default SQL Server connection
  .USERID="sqladmin"
  .PASSWORD="BobIsAtHD-3167"
  .DATABASE="MASTER"
ENDWITH
```

```
CREATE DATABASE USING MyDB
```

Note

If any problem arises while creating the database, the appropriate error is generated.

The oDbObject does not have to be connected, but the server information must be valid. Successfully calling the Connect method in a SQL object does not affect using it in the CREATE DATABASE command. Only the SQL object's connection properties are used by the command to create a new connection.

See Also

CREATE TABLE Command
OPEN DATABASE Command
SQL Class
DBC() Function

DRAFT

CREATE FORM

CREATE FORM [cFormName] [AS cClassName FROM cClsLibrary]

Status

Slated for Version 1.0

Abbreviation

CREA FORM

Remarks

Opens the Form Editor and creates a new SCX file named after cFormName.

Parameters

cFormName

Character literal of the form name to create.

AS cClassName FROM cClassLibrary

Character literal of the class to copy from cClassLibrary.

Note

If the base class of cClassName is not a form, an error is generated.

JAXBase creates an SCX table very similar to the VFP SCX table. JAXBase can use a SCX from VFP, but once altered by JAXBase, that SCX will be incompatible with VFP.

Like VCX, MNX, RPX, and LBX files, JAXBase uses the SCX table to create an intermediate source file and will not use the SCX table during runtime.

See Also

COMPILE Command

DO FORM Command

MODIFY FORM Command

Form Designer

CREATE LABEL

CREATE LABEL cLabel [AS cClassName FROM cClsLibrary | cDEFFile]

Status

Slated for after Version 1.0

Abbreviation

CREA LABE

Remarks

Open the label editor and create a new LBX file.

Parameters

cLabel

Character literal name of the label LBX file to create.

cClassName

Character literal of the class to copy from cClassLibrary or DEF file.

cClassLibrary

Character literal name of the VCX class library to get the cClassName definition.

cDEFFile

Name of DEF file that holds cClassName

Note

If the cLabel file already exists, it is overwritten.

See Also

MODIFY LABEL Command

LABEL Class

Label Designer

CREATE MENU

CREATE MENU cMenu [AS cClassName FROM cClsLibrary | cDEFFile]

Status

Slated for after Version 1.0

Abbreviation

CREA MENU

Remarks

Open the menu editor and create a new MNX file.

Parameters

cMenu

Character literal name of the label MNX file to create.

cClassName

Character literal of the class to copy from cClassLibrary or DEF file.

cClassLibrary

Character literal name of the VCX class library to get the cClassName definition.

cDEFFile

Name of DEF file that holds cClassName

Note

If the cMenu file already exists, it is overwritten.

See Also

MODIFY MENU Command

MENU Class

Menu Designer

CREATE PROJECT

CREATE PROJECT cProject

Status

Slated for Version 1.0

Abbreviation

CREA PROJ

Remarks

Open the project editor and create a new PJX file.

Parameters

cProject

Character literal name of the label PJX file to create.

Note

If the cProject file already exists, it is overwritten.

See Also

MODIFY PROJECT Command

PROJECT Class

Project Designer

CREATE QUERY

CREATE QUERY cQuery

Status

Slated for after Version 1.0

Abbreviation

CREA QUER

Remarks

Open the Query editor and create a new QRX file.

Parameters

cQuery

Character literal name of the label QRX file to create.

Note

If the cQuery file already exists, it is overwritten.

See Also

MODIFY QUERY Command

QUERY Class

Query Designer

CREATE REPORT

CREATE REPORT **cReport** [**AS** **cClassName** **FROM** **cClsLibrary** | **cDEFFile**]

Status

Slated for after Version 1.0

Abbreviation

CREA REPO

Remarks

Open the report editor and create a new RPX file.

Parameters

cReport

Character literal name of the label RPX file to create.

cClassName

Character literal of the class to copy from cClassLibrary or DEF file.

cClassLibrary

Character literal name of the VCX class library to get the cClassName definition.

cDEFFile

Name of DEF file that holds cClassName

Note

If the cReport file already exists, it is overwritten.

See Also

MODIFY Report Command

REPORT Class

Re[p]rt Designer

DEACTIVATE

DEACTIVATE CONSOLE cConsole

Status

Slated for Version 1.0

Remarks

Deactivates the named console and frees memory used by the console. If DEFAULT is deactivated, it is simply hidden.

Abbreviation

DEAC

See Also

DEBUGOUT Command
SET STEP Command

DEBUG

DEBUG

Status

Slated for after Version 1.0

Remarks

Opens the JAXBase Debugger Window

See Also

DEBUGOUT Command

SET STEP Command

DRAFT

DEBUGOUT

DEBUGOUT [TO CONSOLE cConsole] eExpression1 [,eExpression2 ...]

Status

Slated for Version 1

Remarks

Send information out the active or designated console.

Parameters

cConsole

Name of console to send the expressions to.

eExpression1 [,eExpression2 ...]

Comma delimited list of expressions to send to the console.

Note

Versions below Version 2.0 will only have the DEFAULT console.

If the cConsole name is not open, an error will be generated.

If TO cConsole is not specified, the active console is used.

See Also

SET CONSOLE Command

ACTIVATE CONSOLE Command

DEFINE CLASS

```

DEFINE CLASS ClassName1 AS ParentClass
  [[PROTECTED | HIDDEN] PropertyName1, PropertyName2 ...]
  [[.]Object.]PropertyName = eExpression ...]
  [ADD OBJECT [PROTECTED] ObjectName AS ClassName2 [NOINIT] [WITH cPropertylist]]
  [[PROTECTED | HIDDEN] FUNCTION | PROCEDURE Name[_ACCESS |_ASSIGN]
    ([cParamName | cArrayName[]
      cStatements
  [ENDFUNC | ENDPROC]
ENDDDEFINE

```

Status

Slated for Version 1.0

Remarks

Define a class using JAXBase source code instead of the class editor.

Abbreviation

DEF

Example

```

* Program: LoadForm.prg
thisForm = CREATEOBJECT("MyForm")
thisForm.show()

DEFINE CLASS MyForm AS FORM
  Height=600
  Width=800
  Caption="My First Form"
  WindowType=1 // Modal

  ADD OBJECT btnOK AS Button with caption="\<OK",left=700,top=640
  btnOK.ADDCODE("click")="wait window this.name timeout 5"
ENDDDEFINE

```

Note

The order of precedence for loading a user defined class definition is:

- DEFINE CLASS definitions in current PRG
- Class libraries opened using SET CLASSLIB command
- DEFINE CLASS definitions in modules opened with SET PROCEDURE
- DEF file where the stem of file name is same as the class name

You can use the JAXBase ADDCODE(“method/event name”) method to assign a method or event with JAXBase source code. The code is compiled and then placed into the method or event during runtime and will be executed the next time that method or event is called. If code already exists in that method or event, an error will be raised.

When loaded, a class definition is put into a cache and executed when the class is referenced, creating an object as defined. The CREATEOBJECT command is used to convert the class definition into an object.

ClassDemo.prg in the SAMPLES folder shows how to define and use classes.

See Also

ADD Command
CREATEOBJECT() Function

DELETE

DELETE [**Scope**] [**FOR** **lExpression1**] [**WHILE** **lExpression2**] [**IN** **nWorkArea** | **cAlias**]

Remarks

Marks the specified records in the indicated work area or alias as deleted.

Abbreviation

DEL

Parameters

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpression1

Logical expression used to filter what records are used.

WHILE lExpression2

Logical expression limiting the records searched to while the expression is true.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE Test1 (Name C(20))
INSERT INTO TEST1 (NAME) VALUES ("JOHN SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JANE SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JOHN DOE")
INSERT INTO TEST1 (NAME) VALUES ("JANE DOE")
```

```
DELETE ALL
COUNT TO A FOR DELETED()
? A,"deleted records"
```

See Also

COUNT Command
 CREATE TABLE Command
 DELETE Command
 DELETED() Function
 INSERT Command
 JAXBase Concepts: Scope Clauses

DELETE FILE

DELETE FILE cFileName

Status

Slated for Version 1.0

Remarks

Deletes the indicated file from storage.

Abbreviation

DEL FILE

Parameters

cFileName

Character literal file name to delete.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
STRTOFIL("HELLO!", "test.txt", 0) && Overwrite if it exists
DELETE FILE test.txt
```

```
IF FILE("test.txt")
? "DELETE FAILED"
ENDIF
```

Note

If the file does not exist, the command is ignored, otherwise if there is a problem with deleting the file, an error is generated.

You may use wildcards in the name to delete more than one file at a time.

See Also

ADIR() Function

DIR / DIRECTORY Command

CREATE FILE Command

MODIFY FILE Command

DIMENSION

DIMENSION ArrayName1(nRows1 [,nColumns1]) [AS cType]
[,ArrayName2(nRows1 [,nColumns1])[As cType]...]

Remarks

Create one or more arrays with optional type protection.

Abbreviation

DIM

Parameters

ArrayName1

nRows1 [, nColumns1]

Numeric expressions that define how many rows and columns with which to initialize the array.

AS cType - Slated for Version 0.6

You may specify that all elements of an array to be of a type. The allowed data types are as follows.

Code	Full Name	Compatible Value Types
B	Double	Double, Float, Integer, Numeric, Currency
C	Character	Character, VarChar, VarBinary, Memo, General, or Blob
D	Date	Date or DateTime
F	Float	Double, Float, Integer, Numeric, Currency
I	Integer	Double, Float, Integer, Numeric, Currency
L	Logical	Logical, numeric 1 (.T.) and 0 (.F.)
N	Numeric	Double, Float, Integer, Numeric, Currency
O	Object	Any JAXBase Class Object
T	DateTime	Date or DateTime
V	VarChar	Character, VarChar, VarBinary, Memo, General, Blob
Y	Currency	Double, Float, Integer, Numeric, Currency

Example

```
* Demonstrate DIMENSION
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DIMENSION aArray1[2,2] AS Double, aArray2[2,2] AS DATE, aArray3[2,2]
DISPLAY MEMORY LIKE aArray?
```

Note

All array elements are initialized to false(.F.) unless AS cType is used, in which case the empty value for that type is created.

Arrays set AS cTYPE that receive compatible data types will convert as needed, otherwise an error will be generated.

An array that is not set AS cType can hold any data type in each array element.

Arrays passed as parameters via DO / WITH are passed by reference unless they are part of an expression or you are sending one element of the array, in which case that parameter is sent by value.

See Also

APARAMETERS Command
LOCAL Command
LPARAMETERS Command
PRIVATE Command
PARAMETERS Command
PUBLIC Command

DIRECTORY

DIRECTORY [cSkeleton] [TO PRINTER [PROMPT] | TO FILE cFileName [ADDITIVE]]

Remarks

List the directory contents to the active console, printer, or to a text file.

Abbreviation

DIR

Parameters

cSkeleton

Character literal of filename with or without wild card characters.

TO PRINTER [PROMPT] (Slated for Version 1.0)

Send the directory information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is append to the file, otherwise it is overwritten.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

DIR (_JAXHome+"\*.*")
```

Note

You can add path information to the cSkeleton and cFileName. If cSkeleton contains spaces, you must make it into an expression, such as ("c:\my text folder*.txt")

The wildcards ? and * can be used in cSkeleton. If no files match cSkeleton, then a message indicating no files will be sent to the output device.

If an error occurs while trying to list the directory information, such as media, network, or printer error then an error will be generated.

See Also

SET DEFAULT Command

ADIR() Function

DISPLAY DATABASE

DISPLAY DATABASE [nDatabase] [TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCONSOLE]

Status

Slated for Version 1.0

Remarks

Displays a connected SQL database to console, printer, or filename.

Parameters

TO PRINTER [PROMPT]

Sends the directory information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is appended to the file, otherwise it is overwritten.

NOCONSOLE

Prevents information going to a printer or file from also displaying to the active console.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** Assumes a SQL Connection exists to a backend database**

OPEN DATABASE TEST USING FILETOSTR("SQLConnect.dsn")

DISPLAY DATABASE TO FILE dbinfo.txt NOCONSOLE

Note

If displaying to console, then the display pauses after a screen full and you must press a key or click a mouse button to continue. Pressing the ESC or CTRL+C key cancels the operation.

If displaying to a printer or file and NOCONSOLE is not included, the information will also go to the active console (if one exists), but the display will not pause.

See Also

DISPLAY Commands

LIST Commands

DISPLAY FILES

DISPLAY FILES [cSkeleton] [TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]]

Status

Slated for Version 1

Remarks

Display the directory contents to the active console, printer, or to a text file.

Parameters

cSkeleton

Character literal of filename with or without wild card characters.

TO PRINTER [PROMPT]

Sends the directory information out to the active printer or to a chosen printer if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is appended to the file, otherwise it is overwritten.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DISPLAY FILES (_JAXHome+"\"*.*")
```

Note

If the output device is a console, then the directory is paused after every screen-full and you must press a key or click a mouse button to continue. If the ESC key is pressed, output stops.

You can add path information to the cSkeleton and cFileName.

The ? and * wildcards can be used in cSkeleton, but not cFileName.

If no files match cSkeleton, then a message indicating no files will be sent to the output device.

If an error occurs while trying to list the directory information, a JAXBase error generated.

See Also

ADIR() Function

DISPLAY MEMORY

**DISPLAY MEMORY [LIKE cSkeleton]
[TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCONSOLE] [EXTENDED]**

Status

Slated for Version 1

Remarks

Displays all user JAXBase or user defined variables, arrays, and objects.

Parameters

LIKE cSkeleton

Specifies a variable skeleton containing wild cards is used to match the variables to display.

TO PRINTER [PROMPT]

Sends the variable information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the variable information out to the cFileName defined with a character literal. If ADDITIVE is included, the information is append to the file, otherwise it is overwritten.

[EXTENDED]

Normally, an object will just be listed as an object. Extended will also list the object properties at the time.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DISPLAY MEMO LIKE _JAX* TO FILE JAXInfo.txt EXTENDED
```

Note

The JAXBase environment variables will not display unless you use a cSkeleton such * or as _*.

If the output device is a console, then the information display is paused after every screen-full and you must press a key or click a mouse button to continue. If the ESC key is pressed, output stops.

See Also

CREATEOBJECT() Function

DIMENSION Command

DISPLAY Commands

LIST Commands

DRAFT

DISPLAY OBJECTS

DISPLAY OBJECTS [**LIKE** cObjectSkeleton]
 [TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCOSOLE]

Status

Slated for Version 1.0

Remarks

Displays all active user defined objects in memory to a device.

Parameters

LIKE cSkeleton

Specifies a variable skeleton containing wild cards is used to match the variables to display.

TO PRINTER [PROMPT]

Sends the variable information out to the active printer or to a chosen printer if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the variable information out to the cFileName defined with a character literal. If ADDITIVE is included, the information is append to the file, otherwise it is overwritten.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DISPLAY OBJECTS LIKE *
```

Note

Normally DISPLAY OBJECTS will not display JAXBase environmental objects, but If you use a cSkeleton with a wildcard that would include the JAXBase environment objects, then they will also be displayed.

Each object will also have its properties at the time listed to the desired device.

If the output device is a console, then the directory is paused after every screen-full and you must press a key or click a mouse button to continue. If the ESC key is pressed, output stops.

See Also

CREATEOBJECT()

DISPLAY Commands

LIST Commands

DISPLAY STATUS

DISPLAY STATUS [TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCONSOLE]

Status

Slated for Version 1.0

Remarks

Displays the JAXBase status to console, printer, or file.

Parameters

TO PRINTER [PROMPT]

Sends the directory information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is appended to the file, otherwise it is overwritten.

NOCONSOLE

Prevents information going to a printer or file from also displaying to the active console.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

DISPLAY STATUS

Note

If displaying to console, then the display pauses after a screen full and you must press a key or click a mouse button to continue. Pressing the ESC or CTRL+C key cancels the operation.

If displaying to a printer or file and NOCONSOLE is not included, the information will also go to the active console (if one exists), but the display will not pause.

DISPLAY STATUS can be very helpful when debugging a problem in your code.

The DISPLAY STATUS command lists out the following information about the JAXBase Environment.

Data Environment

- Open data sessions
 - o Open tables or cursor names/aliases
 - Open indexes for each table
- Open table information
 - o Open index information for each table/Cursor
- Active SQL Objects
- Active Database Connections

Program Environment

- Current File/Procedure being executed
- Current Line and Source line (if available)
- _JAX class properties
- _CPU class properties
- _Drives array
- _Monitors array
- _PRINTERS information
- Current data session
- Current work area
- Current SET Command settings
- Active keyboard macros
- User defined variables, arrays, and objects

See Also

DISPLAY Commands
LIST Commands

DISPLAY STRUCTURE

DISPLAY STRUCTURE [IN nWorkArea | cAlias]
 [TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCONSOLE]

Status

Slated for Version 1.0

Remarks

Displays the structure of an open JAXBase DBF or cursor.

Parameters

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

TO PRINTER [PROMPT]

Sends the directory information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is append to the file, otherwise it is overwritten.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
DISPLAY STRUCTURE
```

Note

If displaying to console, then the display pauses after a screen full and you must press a key or click a mouse button to continue. Pressing the ESC or CTRL+C key cancels the operation.

If displaying to a printer or file and NOCONSOLE is not included, the information will also go to the active console (if one exists), but the display will not pause.

See Also

AFIELDS() Function
 DISPLAY Commands
 LIST Commands

DISPLAY TABLES

**DISPLAY TABLES [DATABASE [cDbSkeleton]] [SESSION nID]
[TO PRINTER [PROMPT] | TO FILE FileName [ADDITIVE]] [NOCONSOLE]**

Status

Slated for Version 1.0

Remarks

Displays all open tables in a data session or tables in a database.

Parameters

DATABASE [cDbSkeleton]

If included, displays the tables any open database matching cDBSkeleton. If cDbName is not included, lists the tables in the current database.

DATASESSION nID

If included, displays the tables for the data session, otherwise just the tables in the current data session.

TO PRINTER [PROMPT]

Sends the directory information out to the active printer or to a printer you choose if PROMPT is included.

TO FILE cFileName [ADDITIVE]

Sends the directory information out to the cFileName defined with a character literal. If ADDITIVE is included, the directory is append to the file, otherwise it is overwritten.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
SELECT 1
USE (_JAXFolder+"data\phonebook")
```

DISPLAY TABLES

Note

If displaying to console, then the display pauses after a screen full and you must press a key or click a mouse button to continue. Pressing the ESC or CTRL+C key cancels the operation.

If displaying to a printer or file and NOCONSOLE is not included, the information will also go to the active console (if one exists), but the display will not pause.

If a database is not open and DATABASE is included without a cDbSkeleton, an error message will be generated. If cDbSkeleton does not contain a wildcard (* or ?) and there is no match, an error message will be generated.

See Also

ASESSIONS() Funtion

DISPLAY Commands

LIST Commands

OPEN DATABASE Command

USE Command

DO

DO Program

DO cProgram1 | cProcedure [IN cProgram2] [WITH ParameterList]

Remarks

Executes a JAXBase program or procedure.

Parameters

cProgram1

Specifies the name of the program to execute.

cProcedure

Specifies the name of the procedure to execute.

IN cProgram2

Executes a procedure in the program file specified with ProgramName2. When the file is located, the procedure is executed. If the program file cannot be located or the procedure cannot be located in the program file, an error is generated.

WITH ParameterList

Specifies parameters to pass to the program or procedure.

Note

If you do not include an extension with cProgram1, JAXBase looks for and executes versions of the program in the following order: EXE, APP, JXP, PRG.

If a program was called by another program, then when execution is complete, control is returned to the calling program. If called from the IDE command window, control is passed to the command window, otherwise control is passed back to the operating system.

JAXBase looks for the cProcedure in the currently executing program. If the procedure is not located there, JAXBase then looks for the procedure in the procedure files opened with SET PROCEDURE. You can include the IN ProgramName2 clause to tell JAXBase to look for the procedure in a file you specify.

The parameters listed can be expressions, memory variables, literals, fields, or user-defined functions. By default, arrays and objects are sent by reference. You can pass an array or object by value by placing it in parentheses. An array element for a parameter is passed by value unless the element contains an object, which is passed by reference.

See Also

CANCEL Command

COMPILE Command

APARAMETERS Command

LPARAMETERS Command

PARAMETERS Command

QUIT Command

RETURN Command

DRAFT

DO FORM

DO FORM cForm [NAME cVar [PRIVATE|LOCAL]] [WITH eParameterList] [TO cVar] [NOSHOW]

Status

Slated for Version 1.0

Remarks

Loads and displays a form.

Parameters

cForm

Specifies the name of the form or form set to run.

NAME cVar [PRIVATE]

Specifies a variable or array element with which you can reference the form or form set. associated with the form. The NAME variable is created as a global variable unless PRIVATE is issued.

WITH eParameterList

Specifies the parameters passed to the form or form set. If a form set is run, the parameters are passed to the form set's Init method if the form set's WindowType property is set to ModeLess (0) or Modal (1). The parameters are passed to the Load method if the form set's WindowType property is set to Read (2) or ReadModal (3).

TO cVar

Specifies a variable to hold a value returned from the form. If the variable doesn't exist, JAXBase automatically creates it. Use the RETURN command in the Unload event procedure of the form to specify the return value. If you do not include a return value, the default value of true (.T.) is returned. If you use TO, the WindowType property of the form must be set to 1 (Modal). If the form Init event procedure returns .F., preventing the form from being instantiated, the Unload event procedure will not return a value to VarName.

NOSHOW

Specifies that the form's Show method isn't called when the form is run. When you include NOSHOW and run the form, the form is not visible until the form's Visible property is set to true (.T.) or the form's Show method is called.

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

DO FORM TEST1
RETURN

DEFINE CLASS TEST1 AS FORM
    .Caption="Empty Form - Click close icon to exit -->"
    .HEIGHT=200
    .LEFT=100
    .TOP=100
    .WIDTH=300
    .WindowType=1&& MODAL
ENDDDEFINE

```

Note

DO FORM executes the Show method for the form or form set.

If you specify a NAME variable that doesn't exist, JAXBase creates it. If you specify an array element, the array must exist before you issue DO FORM. If the variable or array element you specify already exists, its contents of that element are overwritten.

If you omit the NAME clause, JAXBase creates an object type variable with the same name as the form or form set file. Include PRIVATE to set the scope of the format as a private variable. The form is released when the variable loses scope. If you don't include LINKED, the variable is set as a global variable and will stay in memory until the form is closed, the variable is released, or the program ends.

If the form property RELEASEONEXIT=.F., then the form object will not be released when the form is closed.

DO WHILE/ENDDO

```

DO WHILE lExpression
    Commands
    [LOOP]
    [EXIT]
ENDDO

```

Parameters

lExpression

Specifies a logical expression whose value determines whether the commands between DO WHILE and ENDDO are executed. If lExpression evaluates to true (.T.), the set of commands are executed, otherwise control is passed to the first command following the ENDDO.

Commands

Specifies the set of JAXBase commands to be executed if lExpression evaluates to true (.T.).

LOOP

Returns program control directly back to DO WHILE. The LOOP command can be placed anywhere between DO WHILE and ENDDO.

EXIT

Transfers program control from within the DO WHILE loop to the first command following ENDDO. EXIT can be located anywhere between DO WHILE and ENDDO.

ENDDO

Indicates the end of the DO WHILE and transfers control back to the DO WHILE command.

Example

```

* Demonstrate DO WHILE/ENDDO
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
USE (_JAXFolder+"data\polldata")

LOCATE FOR Precinct=203
DO WHILE Precinct=203 AND NOT EOF()
    ? Name, Age, Party
    SKIP
ENDDO

```

Note

Commands between DO WHILE and ENDDO are executed for as long as the logical expression lExpression remains true (.T.). Each DO WHILE statement must have a corresponding ENDDO statement.

Comments can be placed after DO WHILE and ENDDO on the same line. The comments are ignored during program compilation and execution.

See Also

IF/ELSEIF/ELSE/ENDIF
DO CASE/CASE/OTHERWISE/ENDCASE
DO/UNTIL
FOR/ENDFOR
FOREACH/ENDFOR

DRAFT

DO CASE/CASE/OTHERWISE/ENDCASE

```

DO CASE
  CASE lExpression1
    [Commands]
  [CASE lExpression2
    [Commands]]
  ...
  [CASE lExpressionN
    [Commands]]
  [OTHERWISE
    [Commands]]
ENDCASE

```

Remarks

Executes the first set of commands whose conditional expression evaluates to true (.T.).

Parameters

CASE lExpression1 Commands...

When the first true (.T.) CASE expression is encountered, the set of commands following it is executed. Execution of the set of commands continues until the next CASE or ENDCASE is reached.

Execution then resumes with the first command following ENDCASE. If a CASE expression is false (.F.), the set of commands following it up to the next CASE clause is ignored. Only one set of commands is executed. These are the first commands whose CASE expression evaluates to true (.T.).

Any succeeding true (.T.) CASE expressions are ignored.

OTHERWISE Commands

If all of the CASE expressions evaluate to false (.F.), OTHERWISE determines if an additional set of commands is executed.

If you include OTHERWISE, the commands following OTHERWISE are executed and execution skips to the first command following ENDCASE.

If you omit OTHERWISE, execution skips to the first command following ENDCASE.

Note

DO CASE is used to execute a set of JAXBase commands based on the value of a logical expression. When DO CASE is executed, each logical expressions is evaluated and the first one that evaluates to true (.T.) will have its command block executed.

Comments can be placed after DO CASE and ENDCASE on the same line. The comments are ignored during program compilation and execution.

Example

```
CLEAR ALL  
CLOSE ALL
```

```
STORE CMONTH( DATE( ) ) TO month  && The month today
```

```
DO CASE  && Begins loop
```

```
    CASE INLIST(month, 'January', 'February', 'March')  
        STORE 'First Quarter Earnings' TO rpt_title
```

```
    CASE INLIST(month, 'April', 'May', 'June')  
        STORE 'Second Quarter Earnings' TO rpt_title
```

```
    CASE INLIST(month, 'July', 'August', 'September')  
        STORE 'Third Quarter Earnings' TO rpt_title
```

```
    OTHERWISE  
        STORE 'Fourth Quarter Earnings' TO rpt_title
```

```
ENDCASE  && Ends loop
```

```
WAIT WINDOW rpt_title NOWAIT
```

See Also

IF/ENDIF Command

FOR/ENDFOR Command

FOREACH/ENDFOR Command

DO/UNTIL Command

DO WHILE/ENDDO Command

DO/UNTIL

DO

Commands

[EXIT]

[LOOP]

UNTIL lExpression

Remarks

Executes a block of code at least once and continues to execute the block of code until the lExpression resolves as True (.T.)

Parameters

lExpression

Specifies a logical expression whose value determines whether the commands between DO and UNTIL are executed. While lExpression evaluates to true (.T.), the set of commands are executed.

Commands

Specifies the set of commands to be executed as long as lExpression evaluates to true (.T.).

LOOP

Sends program control directly to UNTIL. LOOP can be placed anywhere between DO and UNTIL.

EXIT

Transfers program control from within the DO loop to the first command following UNTIL. EXIT can be placed anywhere between DO and UNTIL.

Example

In the following example, the program reads in a text file and prints out each line until a period is found. A DO/UNTIL is most useful when you need to perform a process at least once, or where you may need to duplicate code outside of a while loop to finish processing left over information.

```
CLEAR ALL
```

```
CLOSE ALL
```

```
I=1
```

```
A=FILETOSTR("test.txt")
```

```
DO
```

```
    B=MLINE(A, I)
```

```
    ? B
```

```
UNTIL "." $ B
```

Note

The code between DO and UNTIL will be executed once and then the UNTIL is evaluated to determine if control is passed back to the DO command.

See Also

DO WHILE/ENDDO Command

FOR/ENDFOR Command

FOREACH/ENDFOR Command

SCAN/ENDSCAN Command

DRAFT

DOEVENTS

DOEVENTS

Status

Slated for after Version 1.0

Abbreviation

DOEV

Remarks

Causes the GUI to refresh.

Note

When changes to a visual object occur in rapid succession, adding DOEVENTS to the code forces the visual object to be updated.

See Also

CREATE FORM Command
CREATEOBJECT() Function

DROP TABLE

DROP cTable [FROM cDbName]

Status

Slated for Version 1.0

Remarks

Drops a table from a SQL database

Parameters

cTable

Character literal indicating the name of the SQL table to drop.

FROM cDbName

Character literal indicating the name of SQL database that the table is being dropped from.

Note

If the FROM clause is omitted, the currently open database.

When using FROM cDbName, you must have the database open either using the OPEN DATABASE command or creating a connection using the SQL class.

See Also

CREATE TABLE Command

OPEN DATABASE Command

SQL Class

EDIT

EDIT [TO oVar] [NOSHOW]

Status

Slated for after Version 1.0

Remarks

Brings up the Edit form for the table or cursor open in the current work area.

Parameters

TO oVar

You can assign the edit form to a variable name to open more than one edit form.

NOSHOW

Creates the edit form but does not show it.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
EDIT
```

Note

When the EDIT form is created, the current environment variables are assigned, and many are read-only. EDIT starts with the current record. If you reach the end of the file or click on NEW then a blank record is appended. If hit the ESC key, the current records changes are discarded.

If you created a blank record or made alterations to the current record and click on the CLOSE button, the record is saved. If you edit a record and move to another row, the changes are saved.

Errors generated by the EDIT form will be displayed in a dialog, allowing you to fix them before continuing.

The EDIT window has properties that can be altered by code.

See Also

APPEND Command
BROWSE Command

END

ENDCASE

ENDDEFINE / ENDDEF

ENDDO

ENDFOR

ENDFUNCTION / ENDFUNC

ENDPROCEDURE / ENDPROC

ENDSCAN

ENDTRANSACTION / ENDTRAN

ENDTRY

DRAFT

ERROR

ERROR [nExpression]

Status

Slated for Version 1.0

Remarks

Forces the generation of the specified JAXBase error.

Parameters

nExpression

Numeric expression used to generate an error. Default is 9000.

Example

```
* Demonstrate ERROR command
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ON ERROR ? "Error",ERROR()
ERROR 10
```

Note

If an error handler is currently set, JAXBase will attempt to execute it.

See Also

ON ERROR
Error Method

EXIT

EXIT

Remarks

The EXIT command transfers control to the command immediately after the current FOR, WHILE, SCAN, DO UNTIL, or TRY/CATCH/ENDTRY/FINALLY structures.

Example

```
* Demonstrate EXIT
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

FOR i = 1 TO 17
  IF i < 10
    ? i
  ELSE
    ? "EXIT!"
    EXIT
 ENDIF
ENDFOR
```

See Also

DO WHILE/ENDDO Command
FOR EACH/ENDFOR Command
FOR/ENDFOR Command
SCAN/ENDSCAN Command
TRY/CATCH/ENDTRY/FINALLY Command

EXTERNAL

EXTERNAL ARRAY cArrayList

EXTERNAL FILE | PROGRAM | CLASS | CLASSLIB | FORM | LABEL | QUERY cName

Status

Slated for Version 2.0

Remarks

During compilation, JAXBase looks for references to arrays and files in the code. The EXTRNAL command overrides missing references to various files or arrays in the program where they are referenced.

Parameters

ARRAY cArrayList

List of comma delimited character literals indicating arrays that are initialized externally to the current program, procedure, method, or event.

FILE | PROGRAM | CLASS | CLASSLIB | FORM | LABEL | QUERY cName

Indicates the type of eternal file and name.

Example

```
* Demonstrate EXTERNAL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
EXTERNAL FORM (_JAXFolder+"samples\testform.scx")
DO FORM TEST
```

Note

If a listed parameter name matches a the name of an unknown array, it is assumed that the parameter will be an array.

See Also

BUILD APPLICATION Command
 BUILD EXECUTABLE Command
 BUILD PROJECT Command

FOR/ENDFOR

```
FOR cVar = nStart TO nEnd [STEP nStep]
    [EXIT]
    [LOOP]
ENDFOR
```

Status

Slated for Version 1.0

Remarks

The FOR/ENDFOR loop allows you to execute the same code until the numeric end expression is reached.

Parameters

cVar

Character literal of the variable to create or overwrite for incrementing through the FOR loop

nStart

Numeric expression of the starting value.

nEnd

Numeric expression of the ending value.

nStep

Numeric expression of the incremental value. A positive value increments nStart up while a negative value decrements nStart down.

Example

```
* Demonstrate FOR LOOP
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
FOR I = 10 to 1 STEP -1
    IF I=2
        LOOP
    ENDIF
```

```
    ? I
ENDFOR
```

Note

When the for loop is first executed, the numeric starting expression is placed into the variable and the numeric end expression along with the numeric step value (if missing the value is 1) are recorded.

If the step value is positive, then as long as the variable's value is not greater than the end expression's original value, looping will continue.

If the step value is negative, then as long as the variable's value is not less than the end expression's original value, looping will continue.

If the step value is zero (0) then looping will continue unless an EXIT command is encountered.

If an EXIT command is executed, the loop is exited and the code following the ENFOR is executed.

If a LOOP command is executed, control is passed to the ENDFOR.

When the ENFOR is executed, control is passed to the FOR statement. The variable will be incremented by the numeric step value. If the variable's value is out of range for the numeric end expression, then control is passed to the command following the ENDFOR, otherwise the loop's code is executed again.

See Also

DO/UNTIL

DO WHILE/ENDDO

FOREACH/ENDFOR

SCAN/ENDSCAN

FOREACH/ENDFOR

Status

Slated for Version 2.0

DRAFT

FUNCTION

FUNCTION *cName*

Status

Slated for Version 1

Abbreviation

FUNC

Remarks

Marks the start of a user defined function.

Parameters

cName

Character literal name of the function it will be referenced as.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DO TEST1 WITH 9
RETURN
```

```
FUNCTION TEST1
PARAMETERS nVal
? nVal
RETURN
```

Note

FUNCTION is included for compatibility with other dialects. FUNCTION is equivalent to PROCEDURE in JAXBase.

See Also

PROCEDURE Command
Methods and Events

GATHER

GATHER FROM ARRAY cArray | **MEMVAR** | **NAME** cObject | **JSON** cExpression
 [FIELDS FieldList | **FIELDS LIKE** Skeleton | **FIELDS EXCEPT** Skeleton]
 [MEMO][BLANK]

Status

Slated for Version 0.8

Remarks

Moves information from memory to the current record.

Abbreviation

GATH

Parameters

FROM ARRAY cArrayName

If a FIELDS expression exists, works from Column 1 to column x filling in the fields of the record.

If there are no FIELDS expression, starts with the first field and moves towards the last field, filling them in with information from the columns of the array, except for MEMO, GENERAL, and BLOB fields are always skipped.

MEMVAR

Memory variables that match the fields in the table are used to update the current record.

NAME cObject

The object's properties are used to find matching field names to update.

JSON cExpression

Places matching field information from the JSON object in to the record.

FIELDS FieldList

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

MEMO

Updates memo, general, and blob fields if there are matching memory references. MEMO will not work with FROM ARRAY.

BLANK

If there is a FIELDS expression, blanks out all values for the matching fields. Otherwise blanks out the fields with matching names in memory.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
cInfo="{Name: 'JOHN SMITH', Age: 30}, {Name: JANE DOE", Age: 28}"
```

```
CREATE TABLE TEST1 (Name C(20), Age I)
GATHER FROM JSON cInfo
BROWSE
```

Note

You must have ARRAY, MEMVAR, NAME, or JSON after FROM to specify how to save the requested fields.

When gathering from an array, If there are too few columns, the rest of the fields are ignored, and if there are too many columns, the rest of the columns are ignored.

If there is more than one row in the array, the extra rows are ignored.

If a JSON string has more than one entry, only the first entry is used to is update the current record.

See Also

REPLACE Command
SCATTER Command

GETEXPR

Status

Slated for Version 2.0

DRAFT

GOTO

GOTO TOP | BOTTOM | nExpression [IN nWorkArea | cAlias [SESSION nSessionID]]

Remarks

Moves the table or cursor record pointer to the indicated physical record.

Parameters

TOP

Literal indicating the first record of the table or cursor.

BOTTOM

Literal indicating the last record of the table or cursor.

nExpression

Numeric value indicating where to set the record pointer.

IN nWorkArea | cAlias

Table or alias indicating which table or cursor to use.

SESSION nSessionID

Numeric expression indicating which data session to use.

Example

*** Demonstrate GOTO**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\polldata")

GOTO 15

BROWSE

Note

If nExpression is less than 1 or greater than the number of records in the table, an error is generated.

Issuing TOP moves the record pointer to the top of the table.

Issuing BOTTOM moves the record pointer to the bottom of the table.

If there is an active index, TOP and BOTTOM move to the respective top and bottom of the sorted table.

If there is no table open in the indicated work area or the alias is not found, an error is generated.

See Also

SET INDEX Command

SKIP Command

USE Command

HELP

Status

Slated for Version 2.0

DRAFT

IF/ELSEIF/ELSE/ENDIF

```
IF lExpression
    [Command block]
[ELSEIF lExpression
    Command block]
[ELSE
    Command block]
ENDIF
```

Remarks

The IF/ELSEIF/ELSE/ENDIF structure allows for rapid decision making on what code to execute.

Parameters

lExpression

Expression to check. If it evaluates to true (.T.) then the command block for the IF is executed.

ELSEIF lExpression

Another IF expression to check. If it evaluates to true (.T.) then the command block for that ELSEIF is executed.

ELSE

If the IF expression and any ELSEIF expressions evaluate to false (.F.), then the ELSE block, if it exists, is executed.

Command Block

One or more JAXBase commands to execute.

Example

```
* Demonstrate IF statement
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

IF 1=2
    ? "That's not right!"
ELSEIF 2=3
    ? "That's still not right"
ELSE
    ? "Everything is great!"
ENDIF
```

Note

ELSEIF allows you to change:

```
IF Expression
ELSE
IF Expression
ELSE
    IF Expression
    ENDIF
ENDIF
ELSE
ENDIF
```

To something that looks like:

```
IF lExpression
ELSEIF lExpression
ELSEIF lExpression
ELSE
ENDIF
```

See Also

CASE/OTHERWISE/ENDCASE
DO/WHILE
DO/UNTIL
FOR/ENDFOR
FOREACH/ENDFOR

IMPORT

```
IMPORT FROM FileName  
    [TYPE] XLS | XL5 | XLSX | CALC [SHEET cSheetName]  
[FOR lExpr] [AS nCodePage]
```

Status

Slated for Version 2.0

DRAFT

INDEX

INDEX ON eExpression TO cFile

**[COLLATE cCollateSequence] [FOR lExpression]
[ASCENDING | DESCENDING] [UNIQUE | CANDIDATE] [NOCASE] [REGISTER]**

Status

Version 2.0 or later

INDEX ON eExpression TAG cTag [BINARY]

**[COLLATE cCollateSequence] [OF CDXFileName] [FOR lExpression]
[ASCENDING | DESCENDING] [UNIQUE | CANDIDATE] [NOCASE]**

Remarks

Creates an index file (IDX)

Parameters

ON eExpression

Creates an index based on the given expression.

TO cFile

Saves the index using the character literal expression as the file stem.

COLLATE cCollateSequence (Version 2.0)

Character literal expression indicating

FOR lExpression (Version 1.0)

ASCENDING | DESCENDING

Indicates whether the index is in Ascending (default) or Descending order.

UNIQUE | CANDIDATE (Version 0.8)

The UNIQUE keyword will create an index that only inserts the first record that matches the key, while the CANDIDATE keyword prevents duplicate keys from being inserted into the table. If a IDX index is used, it must be open whenever a record is added or a key field is altered, otherwise the index will not correctly deal with new or changed keys.

When a PACK statement is run against a table, all open indexes are automatically reindexed. If a record is marked for deletion, a SEARCH will always locate it if matched, but a LOCATE statement will not.

NOCASE (Version 0.8)

Traditional XBase languages have typically been incompatible with how a SQL database works in a few respects when dealing with indexes. One major incompatibility is that a XBase index tends to be case sensitive while SQL indexes tend to be case insensitive for character data. The NOCASE

parameter tells JAXBase to build an index that is case insensitive. If the eExpression is not character based (CHAR, VARCHAR, VARBINARY, VARCHAR (Binary)) then an error is raised.

When a SEEK is performed on an index that has been created with the NOCASE parameter, the key search is done on a case insensitive basis.

REGISTER

Register this index with the JAXBase table.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

USE (_JAXFolder+"data\phonebook")
INDEX ON name to phonebook_name_nocase NOCASE
? SEEK("smith"), LastName && Displays ".T. Smith"
```

Note

When an index is created, any other open indexes remain open, but after the indexing is completed, the newly created index is now the controlling index.

When a simple index (IDX File) command is executed, if the IDXFileName already exists, it will overwrite the old file. If you run the index command against a CDX file with a similar name, it too will overwrite the old named index.

You cannot index a memo, general, blob, or JASON field unless you take some sort of subset of the data from the field, such as the following being used against a memo field named TextData.

```
INDEX ON MLINE(TextData,1) TO TextData_Line1
INDEX ON JSON(TextData,"Name") TO TextData_Name
```

All index expressions are limited to 254 characters.

Registered index names are saved to the disk in the format of DBFFFileName_IndexName.idx and are automatically opened when a table is opened. If a registered index cannot be opened, the appropriate error will be generated (1601 – 1604).

Registered indexes will only close when the table is closed.

A registered index can be referenced either by the full file name or just the name given in the INDEX command. If a index called NAME is registered with a table called Company, the index file will be called COMPANY_NAME.IDX and if the SET ORDER command for that index can be as follows.

```
SET INDEX TO NAME
SET INDEX TO COMPANY_NAME
```

See Also

CANDIDATE() Function
UNIQUE() Function
COPY INDEXES Command
DESCENDING() Function
ASSQL()
FOR() Function
INDEXSEEK() Function
KEY() Function
MDX() Function
NDX() Function
ORDER() Function
SET INDEX Command
SORT Command

DRAFT

INSERT

```
INSERT INTO cTable (FieldName1[, FieldName2, ...]) VALUES (eExpression1[, eExpression2, ...])
```

Slated for Version 2.0

```
INSERT INTO cTable [(FieldName1[, FieldName2, ...])]
    SELECT SELECTClauses [UNION UnionClause SELECT SELECTClauses ...]
```

Remarks

Insert a record into a DBF table or SQL table.

Parameters

cTable

Literal character name of DBF or SQL table to insert a record.

FieldName1[, FieldName2, ...]

List of character literals specifying the field names to insert into.

eExpression1[, eExpression2...]

List of expressions specifying the values to place into the fields.

SELECT SELECTClauses [UNION UnionClause SELECT SELECTClauses ...]

Sub queries will be explained in greater detail in an upcoming release.

Example

```
* Demonstrate INSERT
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE MyTable (FirstName C(30), LastName C(30), BirthDate D)
INSERT INTO MyTable (FirstName, LastName, BirthDate) VALUES ("JOHN", "SMITH", {2001/07/15})
INSERT INTO MyTable (FirstName, LastName, BirthDate) VALUES ("JANE", "SMITH", {2004/01/27})
INSERT INTO MyTable (FirstName, LastName, BirthDate) VALUES ("JACK", "SMITH", {2025/12/30})
BROWSE
```

USE

Note

Field and value counts must match if you include the VALUES clause. If there is a difference between the number of each, an error is generated.

Each value data type must be compatible with field types, otherwise an error is generated.

An inserted record is appended to the end of a table and therefore acts much like the APPEND Command.

If there are multiple rows in the array or JSON object, then a separate record will be inserted for each row.

See Also

APPEND Command

GATHER Command

SCATTER Command

KEYBOARD

KEYBOARD cKeyValue [PLAIN] [CLEAR]

Status

Slated for Version 1.0

Remarks

Insert keystrokes into the keyboard buffer.

Abbreviation

KEYB

Parameters

cKeyValue

Character expression that will be placed into the keyboard buffer.

PLAIN

If a key in the expression has an ON KEY LABEL definition, the definition is not triggered and the key's value is placed into the keyboard buffer instead.

CLEAR

Clears all keys from the keyboard buffer

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
KEYBOARD "DIRECTORY"+CHR(13)
```

Note

KEYBOARD is used to push keystrokes into the keyboard buffer to produce automation.

See Also

MOUSE Command

CLEAR TYPEAHEAD Command

LIST

List commands are exactly like Display commands, but where display commands will pause at the end of every screen of information displayed, list commands will display all the information without stopping until it is done.

See the Display command for more information.

DRAFT

LOCATE

```
LOCATE [FOR lExpression1] [Scope] [WHILE lExpression2]
      [IN nExpr | cAlias [SESSION nDataSession]]
```

Remarks

Starting at the top of the file, sequentially searching for the first record that matches the logical expressions. Works only on the table in the current work area.

Abbreviation

LOC

Parameters

FOR lExpression

Expression is used to locate the record that matches the logical expression. Expressions are case sensitive unless you take steps to make it case insensitive.

SCOPE

Specifies the range of records to use. Only records that fall within the scope are located. Scope clauses: ALL, NEXT nRecords, RECORD nRecordNo, and REST. The default scope is ALL.

WHILE lExpression

LOCATE will continue to search as long as this condition is true.

IN nExpr | cAlias [OF nDataSession] (Version 0.6)

LOCATE can optionally look in a different data session and work area.

Example

```
SET TALK OFF
CLOSE DATABASES
USE customers  && Open Customer table

STORE 0 TO gnCount

LOCATE FOR ALLTRIM(UPPER(customers.country)) = 'CANADA'

DO WHILE FOUND( )
    gnCount = gnCount + 1
    ? company,address,city,country
    CONTINUE
ENDDO

? 'Total companies: ' + LTRIM(STR(gnCount))
```

Note

The table does not require an index. If an index is active, it will search the records in the order indicated by the index.

LOCATE without a FOR expression will start at the first logical record of the table.

If a match is not found, the record position is placed after the end of the table and RECNO() returns the number of records+1, FOUND() returns .F. (false), EOF() returns .T. (true). You can issue a CONTINUE command after a successful LOCATE to attempt to find the next matching record.

If you issue a CONTINUE after a failed LOCATE, FOUND() will return false (.F.), EOF will return true (.T.), and the record pointer will be set past the last logical record.

LOCATE is specific to the current work area if the IN clause is not included.

If the IN clause is included, LOCATE will change the work area during the execution of the command which may affect the FOR and WHILE expressions, but current data session and work area is in control after the LOCATE executes.

See Also

CONTINUE Command

EOF() Function

FOUND() Function

INDEXSEEK() Function

RECNO() Function

SEEK Command

SEEK() Function

JAXBase Concepts: Scope Clauses

LOCAL

LOCAL var1 [,var2 [,var3...]]

Remarks

The LOCAL command creates a local variable or array whose scope is the currently executing code module. All variable elements are set to .F. upon initialization.

Parameters

Variable list containing variables or array declarations.

Example

```
LOCAL a,b,c[3]
DISPLAY memory like *
```

```
A  LOC L .F.
B  LOC L .F.
C  LOC A
    (1) L .F.
    (2) L .F.
    (3) L . F.
```

Note

Some xBase languages use the LOCAL command as a flag setting and can have wildcards in the variable name so that if a matching variable is created, it is marked as local. JAXBase expects you to specify the exact variable name.

If the variable is a private variable that was created in the current scope, an error will be thrown.

See Also

PRIVATE
PUBLIC

LOOP

LOOP

Remarks

Transfers control to the end of the FOR, WHILE, SCAN, or UNTIL structure.

Example

```
* Demonstrate LOOP
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

FOR i = 1 TO 17
  IF i = 5
    LOOP
  ENDIF

  ? i
ENDFOR
```

See Also

DO WHILE/ENDDO Command
FOR EACH/ENDFOR Command
FOR/ENDFOR Command
SCAN/ENDSCAN Command
TRY/CATCH/ENDTRY/FINALLY Command

LPARAMETERS

LPARAMETERS *var1 [As Type1][, var2 [As Type2]]...*

Remarks

Allows information to be received from a calling routine and places those values into the list of LOCAL variables. If the calling routine sends fewer values than the LPARAMETERS statement allows, the remaining variables are set to .F. (false). If the calling routine sends more values than are listed in the LPARAMETERS statement, an error is raised.

Abbreviation

LPAR

LPARAMETER

Parameters

Var

Variable name to use in this routine

As Type (Slated for Version 1.0)

Allows for strong typing and if the value passed is not of a compatible type or a value is later assigned that is not of a compatible type, an error is raised. By Type only refers to JAXBase field types (B,C,D,F,I,L,N,T) and standard object classes.

Values from General, Blob, Memo, JSON, VarChar (V) and Binary VarChar fields are considered Character (C) types.

Date values can be sent to DateTime and the time value will be set to 00:00:00 while sending a DateTime value to a Date type will have the time value dropped.

Any number type (B,F,I, or N) can be sent to any other number type with the appropriate conversion made if the value is not out of range (such as a Double value sent to an Integer that is outside of the -2147483647 to 2147483647 range would raise an error).

Using the EMPTY class will allow any object to be passed. Attempting to send an array or other data type value to an object type will raise an error.

The ARRAY type will accept any array. If a field type value or object is sent, the results will be that value placed into an array, by value, with 1 element. If an array is sent, it is passed by reference.

Example

```
* Demonstrate LPARAMETER
CLEAR ALL
CLOSE ALL
SET TALK OFF
```

```
DO TESTREF WITH 1,2,3
RETURN
```

```
PROCEDURE TESTREF
LPARAMETERS AA1, BB1, CC1
DO TESTLOCAL
RETURN
```

```
* Try to print out a variable from the TESTREF procedure
PROCEDURE TESTLOCAL
TRY
    ? "BB1=",BB1
CATCH
    ?? "BB1 is not available"
ENDTRY
RETURN
```

Note

The LPARAMETERS statement creates the local variables from the list based on the information sent by the calling routine. If a value, array or object is sent, the associated parameter variable becomes a copy of that value, array or object.

Passing by Reference may not be available before Version 1.0

By default, DO/WITH passes objects and arrays by reference unless the object is part of an expression or a single array element is passed. All other calls, such as a function call or object call, are passed by value.

See Also

APARAMETERS Command
PARAMETERS Command
DO/WITH Command
JAXBase Field Types
UDF Calls
Object Calls

LPROCEDURE

LPROCEDURE cName

Status

Slated for Version 1.0

Remarks

Creates a procedure that is local to the object or program file to which it is associated and makes it so that procedure cannot be called by code outside of the object or program file.

Abbreviation

LPROC

Parameters

Character literal used as the procedure name.

Example

```
* -----
* Parent.prg
* -----

DO child.prg

PROCEDURE Test1
? "Test 1 routine executed"
RETURN
ENDPROCEDURE

LPROCEDURE Test2
? "Test 2 routine executed"
RETURN
ENDPROCEDURE

* -----
* Child.prg
* -----

TRY
    Do Test1
CATCH
    ? "Failed to call Test1 in Parent.prg"
ENDTRY

TRY
    Do Test2
CATCH
    ? "Failed to call Test2 in Parent.prg"
ENDTRY
RETURN
```

Note

A LPROCEDURE code block, like PROCEDURE is terminated with ENDRPROCEDURE, but JAXBase does not require the ENDRPROCEDURE command as it is only included for readability.

See Also

PROCEDURE Command

DO Command

User Defined Functions

DRAFT

MD

MD cDirectory

Status

Slated for Version 0.6

Remarks

Creates a directory if it does not exist.

Parameters

cDirectory

A literal character expression defining the directory to create, such as C:\TEMP

Example

* Demonstration of MD command

MD C:\TEMP

cFolder="C:\TEMP\JAXBase\Jobs"

MD (cFolder)

Note

The folder expression must be literal expression, such as **MD C:\TEMP** or an expression inside parenthesis, such as **MD (cFolder)**, otherwise an error will be raised. If the folder already exists, the command continues as normal.

If the folder cannot be created, an error will be generated.

See Also

ADIR() Function

CD Command

DIRECTORY/DIR Commands

MODIFY

MODIFY CLASS

Status

Slated for after Version 1.0

Abbreviation

MODI CLAS

Remarks

Brings up the Class Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY COMMAND

MODIFY COMMAND cFile

Abbreviation

MODI COMM

Remarks

Brings up the Program Editor application

Parameters

cFile

Character expression indicating command file (prg) to modify.

Note

The default MODIFY COMMAND editor is built into the JAXBase IDE.

MODIFY FILE

MODIFY FILE cFile

Abbreviation

MODI FILE

Remarks

Brings up the File Editor application

Parameters

cFile

Character expression indicating command file to modify.

Note

The default MODIFY COMMAND editor is built into the JAXBase IDE.

MODIFY FORM

MODIFY FORM cForm

Status

Slated for Version 1.0

Abbreviation

MODI FORM

Remarks

Brings up the Form Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY LABEL

MODIFY LABEL cLabel

Status

Slated for after Version 1.0

Abbreviation

MODI LAB

Remarks

Brings up the Label Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY MENU

MODIFY MENU cMenu

Status

Slated for after Version 1.0

Abbreviation

MODI MENU

Remarks

Brings up the Menu Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY PROJECT

MODIFY PROJECT cProject

Status

Slated for Version 1.0

Abbreviation

MODI PROJ

Remarks

Brings up the Project Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY QUERY

MODIFY QUERY

Status

Slated for after Version 1.0

Abbreviation

MODI QUER

Remarks

Brings up the Query Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY REPORT

MODIFY REPORT cReport

Status

Slated for Version 1.0

Abbreviation

MODI REP

Remarks

Brings up the Report Editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MODIFY STRUCTURE

MODIFY STRUCTURE

Status

Slated for Version 1.0

Abbreviation

MODI STRU

Remarks

Brings up the Table Structure editor application

Note

The project and source code for this editor can be found in the TOOLS folder

MOUSE

MOUSE [TO xPos, yPos] [MOVE xPixels, yPixels] {CLICK LEFT | MIDDLE | RIGHT}

Status

Slated for after Version 1.0

Remarks

The mouse command is used to move the mouse cursor around in order to create automation.

Parameters

TO xPos, yPos

Moves the mouse cursor to the absolute x,y position.

MOVE xPixels, yPixels

Moves the mouse cursor the number of pixels along the x and y axis.

CLICK LEFT | MIDDLE | RIGHT

Simulates the clicking of the Left, Middle, or Right mouse key.

Example

*** Demonstrate MOUSE command**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

aFRM=CREATEOBJECT("FORM")

aFRM.WindowType=1 && Modal

aFRM.LEFT=100

aFRM.TOP=100

aFRM.SHOW

*** Move onto the form then to the top right corner**

**MOUSE TO aFRM.LEFT+20, aFRM.TOP+30 TO aFRM.LEFT+aFRM.WIDTH, aFROMTOP
INKEY(3)**

*** Move down and left into the close icon and click to close the form**

MOUSE MOVE -5,-5 CLICK LEFT

Note

Creating automation in a JAX application is a straightforward task using the KEYBOARD and MOUSE commands.

You can use the TO, MOVE, and CLICK parameters in the same statement.

See Also

KEYBOARD Command

NODEFAULT

NODEFAULT

Status

Slated for Version 1

Remarks

Abbreviation

NODEF

DRAFT

ON

ON ESCAPE

ON ESCAPE Command

Status

Slated for Version 1.0

Remarks

Executes a JAXBase Command when the ESC key is pressed.

Parameters

Command

Any valid single-line JAXBase command

Example

```
ON ESCAPE ? "You hit the ESC Key!"
```

See Also

The ON ESCAPE command cannot execute any multi-line command, such as DO WHILE/ENDDO.

The most common command executed by an ON ESCAPE command is DO Program

Issuing ON ESCAPE with no parameters disables the command.

ON KEY LABEL

ON KEY LABEL <key> Command

Status

Slated for Version 1.0

Remarks

Executes a JAXBase Command when the a designated key is pressed.

Parameters

<key>

A key label as described in [Using Keyboard Commands](#)

Command

Any valid single-line JAXBase command

Example

ON KEY LABEL ALT+F5 ? "You hit the ALT F5 Key!"

See Also

The ON ESCAPE command cannot execute any multi-line command, cush as DO WHILE/ENDDO.

The most common command executed by an ON KEY LABEL command is DO Program

Issuing ON KEY LABEL <key> with no command disables that label command.

ON SHUTDOWN

ON SHUTDOWN Command

Status

Slated for Version 1.0

Remarks

Executes a JAXBase Command when a program is being terminated.

Parameters

Command

Any valid single-line JAXBase command

Example

ON SHUTDOWN DO CleanUp.prg

See Also

The ON SHUTDOWN command cannot execute any multi-line command, such as DO WHILE/ENDDO.

The most common command executed by an ON SHUTDOWN command is DO Program

Issuing ON SHUTDOWN with no parameters disables the command.

OPEN

OPEN DATABASE *AliasName* [**USING** *cExpr*] [**NOUPDATE**]

Status

Slated for Version 0.6

Remarks

Opens a backend SQL database connection for use in JAXBase.

Parameters

AliasName

Indicates that the database should be referenced using the provided alias name. All database and table alias names must be unique.

USING

The connection string expression following the USING command is used to open the SQL connection. If there is no USING command, then the system relies on a previous SET SQLCONNECTION command for connection information, but if SQLCONNECTION information is null or blank, an error is generated.

NOUPDATE

Marks a database connection as read-only and will not allow changes to be sent back through that connection.

Example

```
* OPEN THE DATABASE
USE DATABASE JAXCorp

* GET THE OFFICERS TABLE
JC.EXEC("SELECT * FROM OFFICERS", "results")

* UPDATE THE JOHN SMITH RECORD
IF USED("results")
    SELECT results
    LOCATE FOR NAME="JOHN SMITH"
    IF FOUND()
        REPLACE POSITION WITH "CEO"
ENDIF

* UPDATE THE OFFICERS TABLE AND
UPDATE TABLE
ENDIF

* CLOSE THE DATABASE CONNECTION
CLOSE DATABASE JAXCorp
```

Note

To access the data in a database table, you will first need to use a SELECT statement and place the information into a local table or cursor.

If you save to a table, any changes to the database would need to be done manually standard SQL commands, such as UPDATE, INSERT, or DELETE.

If you save the results of a SELECT query to a cursor, you can use the UPDATE TABLE command to push changes back to the database in bulk. Using the UPDATE TABLE command against cursors created using JOIN, UNION, or sub-queries will raise an error. Only field names that exist in the table will be used in an UPDATE TABLE command. Issuing a REFRESH TABLE command will refresh one or more records from the database table (erasing any changes made to the refreshed records).

You can find several programs demonstrating how to use a SQL backend in the EXAMPLES folder.

See Also

CREATE DATABASE
CREATE TABLE
CREATE VIEW
DELETE DATABASE
OPEN DATABASE
REFRESH TABLE
UPDATE TABLE
DATABASE()
Database Class
Table Class
SQL Class

PACK

PACK [MEMO | DBF] [IN nWorkarea | cAlias]

Remarks

Removes all records from a DBF file that are marked for deletion.

Parameters

MEMO | DBF

MEMO only packs the memo file while DBF causes the DBF and MEMO files to be packed and any open indexes to be reindexed.

IN nWorkArea | cAlias

The PACK command will be run against the table in nWorkArea or with the alias name of cAlias

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE Test1 (NAME c(20))
INSERT INTO TEST1 (NAME) VALUES ("JOHN SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JANE SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JOHN DOE")
INSERT INTO TEST1 (NAME) VALUES ("JANE DOE")
? RECCOUNT(),"records found"
```

```
GOTO 2
DELETE REST
PACK
? RECCOUNT(),"records found"
```

Note

PACK will not work against a table or cursor that is set as READONLY.

If the MEMO keyword is used against a table that has no Memo file, the command is ignored.

If there is no open table in nWorkArea or the cAlias name is not found, an error is generated.

If work area or alias is omitted, the current work area is used.

See Also

DELETE Command
 RECALL Command
 REINDEX Command
 SET DELETED Command

PARAMETERS

LPARAMETERS var1 [As Type1][, var2 [As Type2]...

Remarks

Allows information to be received from a calling routine and places those values into the list of PRIVATE variables. If the calling routine sends fewer values than the PARAMETERS statement allows, the remaining variables are set to .F. (false). If the calling routine sends more values than are listed in the PARAMETERS statement, an error is raised.

Abbreviation

PARA
PARAMETER

Parameters

Var

Variable name to use in this routine

As Type (Slated for Version 1.0)

Allows for strong typing and if the value passed is not of a compatible type or a value is later assigned that is not of a compatible type, an error is raised. By Type only refers to JAXBase field types (B,C,D,F,I,L,N,T) and standard object classes.

Values from General, Blob, Memo, JSON, VarChar (V) and Binary VarChar fields are considered Character (C) types.

Date values can be sent to DateTime and the time value will be set to 00:00:00 while sending a DateTime value to a Date type will have the time value dropped.

Any number type (B,F,I, or N) can be sent to any other number type with the appropriate conversion made if the value is not out of range (such as a Double value sent to an Integer that is outside of the - 2147483647 to 2147483647 range would raise an error).

Using the EMPTY class will allow any object to be passed. Attempting to send an array or field type value to an object type will raise an error.

The ARRAY type will accept any array. If a field type value or object is sent, the results will be that value placed into an array, by value, with 1 element. If an array is sent, it is passed by reference.

Example

* Demonstrate PARAMETERS and passing by value and reference

```
CLEAR ALL
CLOSE ALL
SET TALK OFF
PRIVATE A[2], B, C
DO ResetVars
```

* Passing A and C by reference

```
DO TESTREF WITH A,B,C
? "Test 1 Results"
? REPLICATE("-",20)
DISPLAY MEMOMORY EXTENDED
INKEY(5)
```

* C is passed by value because it is an expression

```
DO TESTREF WITH A,B,(C)
? "Test 2 Results"
? REPLICATE("-",20)
DISPLAY MEMOMORY EXTENDED
INKEY(5)
```

* Only passed by value

```
=TESTREF(A,B,C)
? "Test 3 Results"
? REPLICATE("-",20)
DISPLAY MEMOMORY EXTENDED
INKEY(5)
```

```
RETURN
```

```
PROCEDURE TESTREF
PARAMETERS AA1 AS ARRAY, BB1 AS N, CC1 AS EMPTY
AA[1]=7
BB1=3
CC1.NAME="NewName"
```

```
DO TESTLOCAL
RETURN
```

* Try to print out a variable from the TESTREF procedure

```
PROCEDURE TESTLOCAL
TRY
? "BB1=",BB1
CATCH
?? "BB1 is not available"
ENDTRY
RETURN
```

* Reset the variables for testing

```
PROCEDURE ResetVars
A[1]=1
A[2]=2
B=7.01
C=CREATEOBJECT("FORM")
RETURN
```

Note

The LPARAMETERS statement creates the local variables from the list based on the information sent by the calling routine. If a value, array or object is sent, the associated parameter variable becomes a copy of that value, array or object.

Passing by Reference may not be available before Version 1.0

By default, DO/WITH passes objects and arrays by reference unless the object is part of an expression or a single array element is passed. All other calls, such as a function call or object call, are passed by value.

See Also

APARAMETERS Command
PARAMETERS Command
DO/WITH Command
JAXBase Field Types
UDF Calls
Object Calls

PLAY

```
PLAY cExpression1 [, cExpression2 ...] [TIME nDelay)
PLAY MACRO cName [TIME nDelay)
```

Status

Slated for after Version 1.0

Remarks

Send keystrokes to the keyboard buffer with optional delays between each expression or keystroke.

Parameters

cExpression1 [, cExpression2 ...]

Comma delimited character expressions containing keys you wish to push to the keyboard buffer.

MACRO cName

Character literal of the name of the keyboard macro to execute.

TIME nDelay

Numeric expression specifying the delay, in milliseconds, between each MACRO or each line terminated with character 13 (carriage return) . The carriage return is not sent with the keystrokes.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
PLAY "DIRECTORY", "{RETURN}", "CLEAR", "{RETURN}" TIMEOUT 2000
```

Note

This command is different than other dialects.

{RETURN} simulates the ENTER/RETURN key on the keyboard. For more information, please see the chapter o.

See Also

CLEAR TYPEAHEAD Command

INKEY() Function

KEYBOARD Command

MOUSE Command

PRINT

PRINT eExpression1 [, eExpression2 ...]

Remarks

Writes one or more expressions to the active console followed by an end of line character(s).

Parameters

cExpression1 [, cExpression2 ...]

Comma delimited character expressions containing keys you wish to send to the console.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? "HELLO "
PRINT "WORLD!"
```

Note

Using the ? command causes an end of line character to be sent before the expressions that follow. While this is expected in XBase languages, many programming languages send the end of line character(s) after. The PRINT command makes it easier to translate output logic from other languages.

See Also

? Command
?? Command

PRIVATE

PRIVATE var1 [,var2 [,var3...]]

Remarks

The PRIVATE command creates a variable or array whose scope is the currently executing code module and any code modules after that point. All variable elements are set to .F. upon initialization

Parameters

Variable list containing variables or array declarations.

Example

```
LOCAL a,b,c[3]
DISPLAY memory like *
```

```
A  LOC L .F.
B  LOC L .F.
C  LOC A
    (1) L .F.
    (2) L .F.
    (3) L . F.
```

Note

Some xBase languages use the LOCAL command as a flag setting and can have wildcards in the variable name so that if a matching variable is created, it is marked as local. JAXBase expects you to specify the exact variable name.

If the variable is a private variable that was created in the current scope, an error will be thrown.

See Also

LOCAL
PUBLIC

PROCEDURE

LPROCEDURE cName

Remarks

Creates a procedure for a program file to which it is associated.

Abbreviation

PROC

Parameters

Character literal used as the procedure name.

Example

```
* -----
* Parent.prg
* -----

DO child.prg

PROCEDURE Test1
? "Test 1 routine executed"
RETURN
ENDPROCEDURE

LPROCEDURE Test2
? "Test 2 routine executed"
RETURN
ENDPROCEDURE

* -----
* Child.prg
* -----

TRY
    Do Test1
CATCH
    ? "Failed to call Test1 in Parent.prg"
ENDTRY

TRY
    Do Test2
CATCH
    ? "Failed to call Test2 in Parent.prg"
ENDTRY
RETURN
```


Note

A PROCEDURE code block, like LPROCEDURE is terminated with ENDRPROCEDURE, but JAXBase does not require the ENDRPROCEDURE command as it is only included for readability.

See Also

PROCEDURE Command

DO Command

User Defined Functions

DRAFT

PUBLIC

PUBLIC var1 [,var2 [,var3..]]

Remarks

The PUBLIC command creates a variable or array whose scope is the entire application. All variable elements are set to False (.F.) upon initialization.

Parameters

Variable list containing variables or array declarations.

Example

```
LOCAL a,b,c[3]
DISPLAY memory like *
```

```
A  LOC L .F.
B  LOC L .F.
C  LOC A
    (1) L .F.
    (2) L .F.
    (3) L . F.
```

Note

Some xBase languages use the LOCAL command as a flag setting and can have wildcards in the variable name so that if a matching variable is created, it is marked as local. JAXBase expects you to specify the exact variable name.

If the variable is a private variable that was created in the current scope, an error will be thrown.

See Also

LOCAL
PRIVATE

QUIT

Quit [nStatus]

Remarks

Terminates the executing program, and if in the IDE it will cause JAXBase to terminate as well.

Parameters

nStatus

Numeric expression containing the status to return to the operating system.

Example

*** Demonstrate QUIT**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

QUIT

See Also

CANCEL

SUSPEND

RD

RD cDirectory

Status

Slated for Version 0.6

Remarks

Removes the indicated directory.

Parameters

cDirectory

Character literal indicating the directory to remove.

Note

If any reason the directory cannot be removed, an error is generated.

See Also

ADIR() Function

DIRECTORY/DIR Commands

MD Command

READ EVENTS

DO EVENTS

Status

Slated for Version 1.0

Remarks

Causes JAXBase to start event processing.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** Call a nonmodal form and wait for EXIT button**

DO FORM (_JAXFolder+"examples\nonmodal.scx")

READ EVENTS

Note

When READ EVENTS is issued in a program, execution pauses at that point and event processing begins.

Only object methods and events and the code called from them are executed during event processing.

When CLEAR EVENTS is executed, event processing ends and the program continues with the command following READ EVENTS.

Only one READ EVENTS can be active at one time. If another READ EVENTS is encountered during event processing, it is ignored.

See Also

CLEAR Commands

DOEVENTS Command

RECALL

RECALL [**Scope**] [**FOR** **lExpression1**] [**WHILE** **lExpression2**] [**IN** **nWorkArea** | **cAlias**)

Remarks

Removes the deletion mark from the specified records in the indicated work area or alias.

Parameters

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpression1

Logical expression used to filter what records are used.

WHILE lExpression2

Logical expression limiting the records searched to while the expression is true.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE Test1 (Name C(20))
INSERT INTO TEST1 (NAME) VALUES ("JOHN SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JANE SMITH")
INSERT INTO TEST1 (NAME) VALUES ("JOHN DOE")
INSERT INTO TEST1 (NAME) VALUES ("JANE DOE")
```

```
DELETE ALL
COUNT TO A FOR DELETED()
? A,"deleted records"
```

```
GOTO TOP
RECALL
COUNT TO A FOR DELETED()
? A,"deleted records"
```

See Also

COUNT Command
 CREATE TABLE Command
 DELETE Command
 DELETED() Function
 INSERT Command
 JAXBase Concepts: Scope Clauses

REINDEX

REINDEX

Status

Slated for Version 1.0

Abbreviation

REIND

Remarks

Causes all open indexes to be recreated for the current work area.

Note

Indexes can become outdated. REINDEX is used to bring them up to date.

If UNIQUE, CANDIDATE, or DESCENDING keywords were used in the original indexing, they will carry over to the new index.

The table must be open in EXCLUSIVE mode in order to issue the REINDEX command.

See Also

INDEX Command

SET EXCLUSIVE Command

SET INDEX Command

USE Command

RELEASE

RELEASE Command

RELEASE [cVarList] [ALL]

Status

Slated for Version 1.0

Remarks

Releases the specified variables from memory, freeing up resources.

Parameters

cVarList

Comma delimited character literals specifying the names of variables to release from memory.

ALL

Releases all variables from memory

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
A=5
B=3
C=7
LIST MEMORY
```

```
?
RELEASE C,D
LIST MEMORY
```

Note

If a variable name in the list does not exist, it is ignored.

See Also

LOCAL Command
PRIVATE Command
PUBLIC Command

RELEASE CLASSLIB

RELEASE CLASSLIB [cLibName1 [, cLibName2...]] [ALL]

Status

Slated for after Version 1.0

Remarks

Removes the indicated class library from memory and closes the VCX.

Parameters

cLibName

Name of class library to release.

ALL

Closes all open class libraries and removes them from memory.

RELEASE CONSOLE

RELEASE CONSOLE [cConsole1 [, cConsole2...]] [ALL]

Status

Slated for after Version 0.6

Remarks

Removes the indicated class library from memory and closes the VCX.

Parameters

cLibName

Name of class library to release.

ALL

Closes all open class libraries and removes them from memory.

RELEASE PROCEDURE

RELEASE PPROCEDURE [cProc1 [,cProc2]] [ALL]

Status

Slated for after Version 0.6

Remarks

Removes the indicated procedure file from memory.

Parameters

cProc1 [,cProc2]

List of procedure files to release.

ALL

Closes all open procedure files and removes them from memory.

REMOVE

REMOVE CLASS

REMOVE CLASS cClass of cLibrary

Status

Slated for after Version 1.0

Remarks

Removes a class from a class library file (.vcx).

Parameters

cClass

Name of class to remove.

cLibrary

Name of library to remove the class from.

REMOVE INDEX

REMOVE INDEX *cIndex*

Remarks

Removes an index from the auto-open register off a JAXBase table.

Status

Slated for Version 1.0

DRAFT

REMOVE TABLE

REMOVE TABLE cTable

Status

Slated for Version 1.0

Remarks

Removes a table from an open database.

Parameters

cTable

Name of table to remove from current database. To remove a table from a different database open in the current data session, use the Database!Table format.

RENAME

RENAME Command

RENAME cFileSource TO cFileDestination

Status

Slated for Version 1.0

Remarks

Change the name of a file

Parameters

cFileSource

Name of the file to change with extension.

cFileDestination

Name of the new file name with extension.

Note

If the cFileDestination is in a different path location than cFileSource, the file is effectively moved.

RENAME CLASS

RENAME CLASS cClass of cLibrary to cNewClass

Status

Slated for after Version 1.0

Remarks

Changes the name of a class definition in a class library file (.vcx).

Parameters

cClass

Name of class to change.

cLibrary

Class library in which the class is located.

cNewClass

New name of class.

RENAME TABLE

RENAME TABLE cTableSource TO cTableDestination

Status

Slated for Version 1.0

Remarks

Renames and/or moves a table and memo field file.

Parameters

cTableSource

Character literal specifying the name of the table.

cTableDestination

Character literal specifying the new name of the table.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TEST1 (Name C(20), Info M)
USE
```

```
RENAME TEST1 TO TEST1_NEW
USE TEST1_NEW
```

Note

If the destination table name exists, an error is generated.

If the destination table name has a path that is different from the source table, the name is moved to the new path.

If a memo file for the table exists, will also be renamed and/or moved.

REPLACE

REPLACE Command

```
REPLACE cField1 with eExpression1[, cField2 with eExpression2...]
      [SCOPE] [FOR lExpression1] [WHILE lExpression2]
      [IN nWorkArea | cAlias]
```

Remarks

Updates records in a table.

Abbreviation

REPL

Parameters

cField1 with eExpression1[, cField2 with eExpression2...]

Specifies the field name and the expression to update its value with.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

IN nWorkArea | cAlias

Specifies the work area or table alias the replace statement affects

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
CREATE TABLE TEST1 (Name C(20), Age I)
APPEND BLANK
REPLACE Name WITH 'John Smith',;
      Age WITH 30
```

```
APPEND BLANK
REPLACE Name WITH 'John Dpe',;
      Age WITH 33
```

Note

It is much faster to put numerous cField WITH eExpression clauses into the command, separated by commas, than to have a separate REPLACE command for each.

If you have multiple cField WITH eExpression clauses in the command, they must all be grouped together.

The data type of the expression must be compatible with the data type of the field.

If there is no table open in nWork area, or cAlias is not found, an error will be generated.

See Also

APPEND Command

GATHER Command

INSERT Command

SCATTER Command

JAXBase Concepts: Scope Clauses

REPLACE FROM

```
REPLACE FROM ArrayName | JSONVar [FIELDS fieldlist]
      [SCOPE] [FOR lExpression1] [WHILE lExpression2]
      [IN nWorkArea | cAlias]
```

Status

Slated for Version 0.6

Remarks

Updates the fields from an array or JSON string.

Parameters

ArrayName

An array containing values to be used in the REPLACE statement. A two-dimensional array is treated as a single dimensional array.

JSONVar

A character variable containing a JSON string that may contain one or more rows, but only the first JSON row is used.

FIELDS FieldList

An optional list of fields to update using the contents of the array.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

Example

```
* Demonstrate REPLACE FROM
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DIMENSION aRec[3]
aRec[1]='John Smith'
aRec[2]=30
aRec[3]='R'
```

```
CREATE TABLE TEST1 (Name C(20), Age I)
APPEND BLANK
REPLACE FROM ARRAY
```

```
APPEND BLANK
JSONStr=[{Name:"Jane Doe", Age:28, Balance:147.18}]
REPLACE FROM JSONStr
```

Note

If the array has more elements than the list of fields to replace, the extra elements are ignored. If a two-dimensional array contains 6 elements and the REPLACE statement has four fields, then the first four elements in the two-dimensional array are used.

If the number of fields to replace is greater than the number of elements in the array, the extra fields are not updated.

If an array element data type is not compatible with the data type of the corresponding field, an error is generated.

Arrays are limited to available memory.

Only matching field names are used when updating the fields in the REPLACE statement.

See Also

GATHER Command
 INSERT Command
 SCATTER Command
 JAXBase Concepts: Scope Clauses

RESTORE

RESTORE FROM

RESTORE FROM cFile [ALL LIKE cSkeleton | EXCEPT cSkeleton] [ADDITIVE]

Status

Slated for after Version 1.0

Remarks

Retrieves variables and arrays saved in a variable file and places them in memory.

Parameters

cFile

Name of variable file name to restore to memory.

ALL LIKE Skeleton

Specifies that all variables and arrays that match Skeleton will be saved.

ALL EXCEPT Skeleton

Specifies that all variables and arrays except those that match Skeleton will be saved.

ADDITIVE

If present, the variables in the file are added to what is already in memory.

Example

```
* Demonstrate RESTORE FROM
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
A=10
SAVE TO TEST1
A=1
LIST MEMORY
RESTORE FROM TEST1
LIST MEMORY
```

Note

If a variable being restored has the same name and scope as one that exists, it overwrites it.

Public and private variables in the file are restored as private variables during runtime.

Public variables are restored as public variables if restored in the IDE command window.

Local variables are always restored as local variables.

If ADDITIVE is omitted, all variables in memory are removed before the variable file is restored.

See Also

DIMENSION Command

LOCAL Command

PUBLIC Command

PRIVATE Command

SAVE TO Command

JAXBase Concepts: Scope Clauses

RESTORE MACROS

RESTORE MACROS FROM cFile [ALL LIKE cSkeleton | EXCEPT cSkeleton] [ADDITIVE]

Status

Slated for after Version 1.0

Remarks

Restores all macros from a macro file.

Parameters

cFile

Name of macro file used to restore saved macros.

ALL LIKE Skeleton

Specifies that all variables and arrays that match Skeleton will be saved.

ALL EXCEPT Skeleton

Specifies that all variables and arrays except those that match Skeleton will be saved.

Example

```
* Demonstrate RESTORE MACROS
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ON KEY LABEL F5 ? "HELLO"
SAVE MACROS TO lcMacro
ON KEY LABEL F5
RESTORE MACROS FROM lcMacro
KEYBOARD "{F5}"
```

Note

If the file extension is not supplied, .fky is assumed.

If the file does not contain valid macro definitions, an error is generated.

Only the macro definitions found in the file will be altered.

See Also

CLEAR Commands

KEYBOARD Command

ON KEY LABEL Command

PLAY Command

RESTORE MACROS Command

RESUME

RESUME

Status

Slated for after Version 1.0

Remarks

Resumes execution after a SUSPEND.

Note

A suspended program will RESUME where execution was suspended.

SUSPEND and RESUME are valuable debugging tools that allow you to create break points in a program running in the IDE.

SUSPEND and RESUME are ignored when running outside of the IDE.

See Also

CANCEL Command

SUSPEND Command

RETRY

RETRY

Status

Slated for after Version 1.0

Remarks

Returns control to the calling program from an error routine.

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Note

Unlike RETURN which returns control to statement after the call to the procedure, RETRY returns control to where the line that tripped the error handler to try processing that line again.

RETRY is very useful for creating error handlers for routines that lock files or records.

See Also

ON ERROR Command
RETURN Command
Error Method

RETURN

RETURN [eExpression]

Remarks

Returns a value to the calling statement.

Parameters

eExpression

Expression to return to the calling statement.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? TEST1(1,2)
RETURN
```

```
PROCEDURE TEST1
PARAMETERS A,B
RETURN A+B
```

Note

If there is no eExpression to return, the true (.T.) value is returned by default.

See Also

DO Command
User Defined Functions

SAVE

SAVE MACROS

SAVE MACROS TO cFile [ALL LIKE cSkeleton | EXCEPT cSkeleton]

Status

Slated for after Version 1.0

Remarks

Saves all ON KEY LABEL macros to the specified file.

Parameters

cFile

Character literal name file that the macros will be saved.

ALL LIKE Skeleton

Specifies that all variables and arrays that match Skeleton will be saved.

ALL EXCEPT Skeleton

Specifies that all variables and arrays except those that match Skeleton will be saved.

Example

*** Demonstrate SAVE MACROS**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

ON KEY LABEL F5 ? "HELLO"

SAVE MACROS TO lcMacro

ON KEY LABEL F5

RESTORE MACROS FROM lcMacro

KEYBOARD "{F5}"

Note

If the file extension is not supplied, .fky is used.

If a path is not supplied, the default path is used.

See Also

CLEAR Commands

KEYBOARD Command

ON KEY LABEL Command

PLAY Command

RESTORE MACROS Command

SAVE TO

SAVE TO cFile [ALL LIKE Skeleton | ALL EXCEPT Skeleton]

Status

Slated for after Version 1.0

Remarks

Stores current variables and arrays to a variable file (.mem).

Parameters

cFile

Character literal indicating name of file to save.

ALL LIKE Skeleton

Specifies that all variables and arrays that match Skeleton will be saved.

ALL EXCEPT Skeleton

Specifies that all variables and arrays except those that match Skeleton will be saved.

Example

```
* Demonstrate SAVE TO
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
A=10
SAVE TO TEST1
A=1
LIST MEMORY
RESTORE FROM TEST1
LIST MEMORY
```

Note

Use the RESTORE FROM command to place saved variables back into memory.

You cannot save objects to a variable file.

See Also

LOCAL Command

PRIVATE Command

PUBLIC Command

RESTORE FROM Command

SCAN / ENDSCAN

```
SCAN [Scope] [FOR lExpression1] [WHILE lExpression2]
    [Commands]
    [LOOP]
    [EXIT]
ENDSCAN
```

Remarks

The SCAN command starts at the top of the table in the current work area and executes the commands between it and the ENDSCAN, as long as the FOR and WHILE expressions, if present, evaluate to True (.T.). When the ENDSCAN is reached, the record pointer advances one record and as long as the FOR and WHILE statements allow and the end of the file is not reached, the process continues to loop through the commands.

Parameters

Scope

Specifies a range of records to be scanned and only the records within the range are included. The valid scope clauses are: ALL, NEXT nRecords, RECORD nRecordNumber, REST, and TOP nRecords. The Scope allows you to limit the number of records processed.

FOR

A logical expression that must be evaluated to True (.T.) for the current record to be processed by the block of code. If the FOR command is not present, it is assumed to be True (.T.).

WHILE

A logical expression that must be evaluated to True (.T.) for processing to continue. If the expression evaluates to False (.F.) then control passes to the command after the ENDSCAN.

Example

```
* Demonstrate CALCULATE
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
USE (_JAXFolder+"data\polldata")
```

```
a=0
SCAN FOR PRECINCT=203
    a=a+1
ENDSCAN
```

```
? a, "records counted"
```

Note

Both the FOR and WHILE commands are evaluated each time the SCAN command is processed.

ALL is the default SCOPE value one is not provided.

When the ENDSCAN command is executed, the table that was current when entering the SCAN/ENDSCAN loop is reselected, and the record pointer advanced, before passing control back to the SCAN command.

See Also

DO CASE/ENDCASE Command

DO/UNTIL Command

DO WHILE/ENDDO Command

FOR EACH/ENDFOR Command

FOR/ENDFOR Command

JAXBase Concepts: Scope Clauses

SCATTER

SCATTER TO ARRAY ArrayName | MEMVAR | NAME ObjectName | JSON varName
 [FIELDS FieldNameList | FIELDS LIKE Skeleton | FIELDS EXCEPT Skeleton]
 [ADDITIVE] | [MEMO] | [BLANK]

Status

Slated for Version 0.8

Remarks

Moves information from the current record to memory.

Abbreviation

SCAT

Parameters

TO ARRAY cArrayName

Copies specified fields to an array. If the ADDITIVE keyword exists, an error is generated.

MEMVAR

Memory variables are created to hold the fields in the current record. If the memory variables already exist, the are overwritten.

NAME cObject

The current fields are moved to an object based on the EMPTY class.

JSON cExpression

Places field information into a JSON string.

FIELDS FieldList

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

MEMO

Memo, general, and blob fields are usually ignored with SCATTER unless the MEMO keyword is included. Arrays cannot be used with the MEMO keyword.

BLANK

Creates or updates the memory objects with empty values instead of field values.

ADDITIVE

Adds the fields to the memory objects. If the memory variable or object exists, it is overwritten. If it does not exist, it is created. You cannot use ADDITIVE with arrays.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
SCATTER NAME oInfo
SCATTER JSON cInfo
```

```
DISPLAY MEMORY LIKE ?Info
```

Note

You must have ARRAY, MEMVAR, NAME, or JSON after TO to specify where to save the requested fields.

When scattering to an array, if the ARRAY does not exist, it is created with the corresponding columns. If the array name does exist, it is overwritten.

You can only SCATTER one row at a time.

See Also

REPLACE Command
GATHER Command

SEEK Command

SEEK eExpression

[ORDER nIndexNumber | cIndexName] [ASCENDING | DESCENDING]
[IN nWorkArea | cAlias] [SESSION nSession]

Status

Slated for Version 0.6

Remarks

SEEK uses an index to search for the expression in the indicated work area or table alias.

Parameters

eExpression

Expression matching the controlling index key.

ORDER nIndexNumber | cIndexName

Numeric or character literal indicating the index number from the current index list or index name.

ASCENDING | DESCENDING

ASCENDING is the default sort order, DESCENDING sets the sort order in revers.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

SESSION nSession

Numeric data session ID

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
SEEK 203 ORDER PoolData_Precinct
BROWSE
```

Note

If the indicated cIndexName is not in the index list, an attempt will be made to find it in the JAXBase path list and open it. Failing that, an error is generated.

If the ORDER parameter is omitted, the current controlling index is used.

If the IN parameter is omitted, the current work area is used.

If there is no table open in the specified work area or the alias is not found, an error is generated.

See Also

INDEX Command

SEEK() Function

USE Command

DRAFT

SELECT Command

SELECT nExpr | cExpr SESSION nExpr

Remarks

Selects the requested workarea or data session. Both work areas and data sessions are represented with a positive integer value.

Abbreviation

SEL

Parameters

cExpr

Specifies a work area alias if using the letters A through J, which represent work areas 1 through 10, otherwise it looks for an open table with the specified alias name. If not found, it will throw an error. If you want to use a character variable or expression, then you will need to surround it with parentheses.

* Two ways to select the workarea containing a table with an alias of MYTABLE
SELECT MyTable && Alias literal

A="MYTABLE"
SELECT (A) && Alias expression
SELECT (i+1) && Work area ID expression

SESSION nExpr

When included, directs the selection of a data session by numeric ID. If omitted, the numeric work area ID in the current data session is selected.

nExpr

Selects the work area based on the positive integer number. If using an expression like **SELECT 10** then work area 10 is selected, but if you use a math expression or variable, then you need to surround it with parenthesis indicating it is an expression to be computed.

* Two ways to select work area 4
SELECT 4 && Integer literal

A=4
SELECT (a) && Integer expression

Note

Traditional xBase systems limit the number of open work areas in a data session to 32767, but JAXBase limits it to available memory, with a maximum ID of 9007199254740992. JAXBase does not initialize a work area or data session until it is selected.

When you select or otherwise move to a work area or data session, it is initialized and takes up memory space. The following code snippet demonstrates the issue:

```
* Initializes 255 work areas needlessly takes
* up memory space for 255 work areas
FOR I=1 to 255
    SELECT (i)
ENDFOR
```

When a table or cursor is closed, most of the memory in use is released, but the initializing information remains. There is no way to release that memory in data session 1 without ending the application or executing a CLEAR ALL or CLOSE ALL command.

Data sessions greater than 1 can be CLOSED, releasing the memory it consumed. When you close a data session, all open databases and tables in that data session are closed. Issuing CLOSE ALL or CLOSE DATASESSION 1 closes all database connections, tables, and cursors, but the data session will remain open.

Forms that have a private data session will open the lowest available data session ID. Manually closing a form's data session will potentially generate numerous errors. Care should be taken to keep track of the data session IDs that you create with SELECT DATASESSION.

See Also

ADATASESSIONS() Function

ALIAS() Function

AWORKAREAS() Function

CLOSE Command

CLOSE DATASESSION Command

SELECT() Function

SET Commands

Remarks

The SET command is used to set properties, features, or values in the JAXBase system. The following examples will explain each set command.

With all SET commands, no value provided when ON or OFF is expected is the equivalent of sending OFF.

Status

The documentation for the SET commands is in progress. Unfinished SET commands will have a status entry.

Note

This following SET command pages started as a compilation of all known SET commands in Visual FoxPro and dBase. Some SET commands will not make it into the JAXBase language.

If a Status heading says "Slated for Version 1.0" then it is still under consideration.

The WITH clause must always be the last clause in the command as it loads the remainder of the command line to the WITH clause as a literal. If more than one option is selected, they must be comma delimited.

SET ALTERNATE Command

```
SET ALTERNATE ON|OFF
SET ALTERNATE TO [<FileName> [WITH [ADDITIVE] [, STAMP]]
SET ALTERNATE TO [<PrinterName>] [PROMPT] [WITH [BOLD][,ITALICS][,STAMP]]
```

Status

Full support by Version 1.0

Remarks

Directs console or printer output created with the ?, ??, DISPLAY, or LIST commands to a text file.

Parameters

ON|OFF

If set on, the console or printer output will be saved to the indicated path and filename.

TO [<FileName>]

Direct output to a file with an extension of TXT if not provided. If no FileName is given, then the current printer or file is closed for output, and any more output will be lost until a new file or printer is provided.

TO [<PrinterName>] [PROMPT]

Set the printer to the provided printer object name, or optionally prompt the user for which printer to direct output. If nothing is provided after TO, the current printer or file is closed for output, and any more output will be lost until a new file or printer is provided.

WITH

Set with the following comma delimited options.

ADDITIVE

Causes an existing file to be appended too rather than erased.

STAMP

Writes a timestamp to the file in the format Alternate printing set at HH:MM on YYYY-MM-DD.

BOLD

When used with a printer, it causes output to print in bold.

ITALICS

When used with a printer, it causes output to print in italics.

Example

```
* Demonstrate SET ALTERNATE
CLEAR ALL
CLOSE ALL
SET DEFAULT to c:\temp
SET ALTERNATE TO test WITH STAMP && Create c:\temp\test.txt
SET ALTERNATE ON
? "This is a test"
SET ALTERNATE TO && clear file name and SET ALTERNATE OFF

* Look at the file that was created
MODIFY FILE test.txt
```

Notes

When setting a file name, if a path is not provided, it will add the default path. If there is no file extension, it will add .txt to the end. If an illegal path\filename is given or the file cannot be created, an error will be raised.

You can only direct output to one ALTERNATE device at a time.

See Also

? Command
?? Command
PRINT Command
FILETOSTR() Function
STRTOFILE() Function

SET ASSERTS Command

SET ASSERTS ON|OFF [WITH NODIALOG]

Remarks

Specifies if ASSERT commands are evaluated or ignored.

Parameters

ON

Specifies that assert commands are evaluated.

OFF

Specifies assert commands are ignored (default).

WITH NODIALOG

Specifies that if assert commands are evaluated, that the Assert Dialog is not displayed.

Example

```
SET ASSERTS      && default is OFF
SET ASSERTS ON   && ASSERT commands will now be active

* Activate asserts with NODIALOG option
SET ASSERTS ON WITH NODIALOG

SET ASSERTS ON   && Resets assert command to use dialog
```

Note

If ASSERTS are set ON and NODIALOG then if TALK is set on, the message is still sent to the active console, but execution continues. This feature is designed for use in automated testing.

To reset ASSERTS to ON with DIALOG, omit the WITH NODIALOG clause.

See Also

ASSERT Command

SET INPUT Command

SET AUTOINCERROR Command

Remarks

Specifies whether attempts to update values in a field using autoincrementing generate an error or fail silently and proceed.

Status

Slated for Version 1.0

DRAFT

SET BELL Command

Remarks

Turns the computer bell on or off and sets the bell attributes.

Status

Slated for Version 0.6

DRAFT

SET BLOCKSIZE Command

Remarks

Specifies how JAXBase allocates disk space for the storage of memo fields.

Status

Slated for Version 0.8

DRAFT

SET BROWSEIME Command

Specifies if the Input Method Editor is opened when you navigate to a text box in a Browse window.

Status

Slated for Version 1.0

DRAFT

SET CARRY Command

Determines whether JAXBase carries data forward from the current record to a new record created with INSERT, APPEND, and BROWSE.

Status

Slated for Version 1.0

DRAFT

SET CENTURY Command

SET CENTURY [ON|OFF] [TO nCentury] [WITH nRollover]

Remarks

Use SET CENTURY to specify how date variables and functions are displayed in JAXBase. Issuing SET CENTURY TO without any additional arguments restores the default century to the current century's digits (20) and displays all 4 digits of the year.

Parameters

ON

(Default) Specifies a four-digit year in a format that includes 10 characters (including date delimiters).

OFF

Specifies a two-digit year in a format that includes eight characters and assumes the current century for date calculations. This setting is not recommended.

TO nCentury

A number from 1 to 99 that specifies the current century. When a date has a two digit year, nCentury determines in which century the year occurs.

WITH nRollOver

A numeric value between 0 and 99 which indicates that if the year value is that or above, it is the current century and below is the next century.

Note

SET CENTURY OFF always implies dates in the current century. However, the SET CENTURY TO syntax takes precedence over this setting.

The value of SET CENTURY TO is scoped to the application.

Setting Century to no value assumes the current century's digits (20).

See Also

DATE() Function

DATETIME() Function

YEAR() Function

SET CLASSLIB Command

Opens a visual class library (.vcx) containing class definitions.

Status

Slated for Version 1.0

DRAFT

SET COLLATE Command

Specifies a collation sequence for character fields in subsequent indexing and sorting operations.

Status

Slated for Version 2.0

DRAFT

SET CONFIRM Command

Remarks

Specifies whether the user can exit a text box by typing past the last character in the text box.

Status

Slated for Version 0.6

DRAFT

SET CONSOLE Command

```
SET CONSOLE [cName] [SIZE <nCols>,<nRows>]
  [IN nMonitor] [AT <nLeft>,<nTop>] [TO [ACTIVE | INACTIVE] [HIDDEN | VISIBLE]]
  [WITH DEBUG]
```



Warning: This command departs from normal XBase behavior.

A console in JAXBase is a text-based area which can be resized and affected by various JAXBase commands that work with a console (ex: ?, ??, CLEAR, etc.).

In Version 2.0, a console in a text-based environment (such as a Linux Server) will create a text window to the specified size.

Status

Full implementation slated for Version 2.0

Remarks

Enables or disables the console window and provides control over its size and position. At startup, the system creates a console window with the name DEFAULT. This console is set to 80 columns by 25 rows, positioned at the top left of the screen, and is not visible, and set to NODEBUG. In version 1.0, there is only one named console allowed.

Parameters

cName

If not provided, assumes the DEFAULT console. In Version 1.0, all names are ignored, and only the DEFAULT console is affected.

SIZE <nCols>,<nRows> - Slated for Version 2.0

Resizes the console window as specified by the numeric column and row parameters. Initially opens at 80 columns by 25 rows. Minimum values allowed are 40 columns by 4 rows. Using 0,0 for SIZE sets the console to the full size of the monitor.

IN nMonitor – Slated for Version 2.0

Places the console in the specified monitor.

AT <nLeft>,<nTop>

Positions the console window on the desktop to the graphical position (in pixels) based on the numeric left and top parameters. 0,0 positions it at the upper left of the desktop. The left and top values will be converted automatically in a text-based system. If IN nMonitor is included, confines the position to that monitor, otherwise positions onto the monitor display area.

TO [ACTIVE | INACTIVE] [HIDDEN|VISIBLE]

Comma delimited options to activate or deactivate the named monitor and hide it or make it visible.

WITH DEBUG

Allow debug information to be displayed to this console if DEBUG or TALK settings are set ON.

Example

Note

In the IDE, the default is to create the DEFAULT console at startup is to set it to SIZE 80,26, position it AT 0,0 and set it TO ACTIVE, VISIBLE and WITH DEBUG.

See Also

RELEASE CONSOLE

SET COVERAGE Command

Turns code coverage on or off or specifies a text file to which code coverage information is directed.

Status

Slated for Version 2.0

DRAFT

SET CPCOMPILE Command

Specifies the code page for compiled programs.

Status

Slated for Version 2.0

DRAFT

SET CPDIALOG Command

Specifies whether the Code Page dialog box is displayed when a table is opened.

Status

Slated for Version 2.0

DRAFT

SET CURRENCY Command

Defines the currency symbol and specifies its position in the display of numeric, currency, float, and double expressions.

Status

Slated for Version 2.0

DRAFT

SET CURSOR Command

Determines whether the insertion point is displayed when JAXBase waits for input.

Status

Slated for Version 1.0

DRAFT

SET DATABASE Command

Specifies the current database.

Status

Slated for Version 0.8

DRAFT

SET DATASESSION Command

Activates the specified data session.

Status

Slated for Version 0.8

DRAFT

SET DATE Command

Specifies the format for the display of Date and DateTime expressions.

Status

Slated for Version 1.0

DRAFT

SET DEBUG Command

SET DEBUG ON | OFF

Remarks

Turns debugging on or off.

Notes

If debugging is set on and DEBUGOUT is empty, then all debugging messages are sent to the active console. If there is no active console, the debug output is lost.

DRAFT

SET DEBUGOUT Command

Directs debugging output to a File.

Status

Slated for Version 1.0

DRAFT

SET DECIMALS Command

Specifies the number of decimal places displayed in numeric expressions.

Status

Slated for Version 1.0

DRAFT

SET DEFAULT Command

SET DEFAULT TO cPath

Remarks

Sets the default path of the application to the path specified. At startup, the default path is the user's document folder. Changing the DEFAULTPATH entry of the JAXUSER.INI file will change the default folder at startup.

SET DEFAULT TO does the same thing as the CD Command.

DRAFT

SET DELETED Command

SET DELETED ON | OFF [IN nWorkArea | cAlias]

Remarks

Tells JAXBase whether it should ignore records mark for deletion.

Parameters

ON

Deleted records are ignored.

OFF

Deleted records are not ignored.

IN nWorkArea | cAlias – Slated for Version 2.0

Note

The default is OFF. When set ON, all records marked for deletion in tables or cursors are ignored when using any command except the following explicit commands:

GOTO nRecord Command

INDEX Command

RECALL Command

SEEK Command

INDEXSEEK() Function

SEEK() Function

If the IN clause is included, the DELETED settings are changed just for that work area or alias while the current table or cursor is in use. If a SET DELETED is issued without the IN after that point, then the specific workarea | alias settings are overwritten.

SET DEVELOPMENT Command

SET DEVELOPMENT ON|OFF

Status

Slated for Version 1.0

Remarks

When set ON, forces JAXBase to compare the compiled program dates with the date of the source and if less than the source, automatically recompile.

DRAFT

SET EPOCH Command

SET EPOCH TO dDate | tDate

Status

Slated for Version 1.0

Remarks

The Epoch is the starting point in time that the application uses for various functions related to time. The epoch for JAXBase is Midnight of January, 1, 0001.

DRAFT

SET ENCRYPTION Command

Status

Slated for Version 1.2

Remarks

Set the encryption type used for the application. Setting up encryption provides security for the application while it runs, known as data-in-motion. Not only will the data in the application be more secure, but, by default, any communications with outside applications will require a secure connection.

DRAFT

SET ESCAPE Command

SET ESCAPE ON|OFF

Status

Slated for Version 1.0

Remarks

If set ON, the application can be interrupted by pressing the Escape key. When running from the IDE, the default is ON, but the default is OFF in the runtime package.

DRAFT

SET EVENTLIST Command

Remarks

Specifies events to track in the Debug Output window or in a file specified with SET EVENTTRACKING.

Status

Slated for Version 2.0

DRAFT

SET EVENTTRACKING Command

Remarks

Turns event tracking on or off or specifies a text file to which event tracking information is directed.

Status

Slated for Version 2.0

DRAFT

SET EXACT Command

SET EXACT ON|OFF

Remarks

Specifies the rules JAXBase uses when comparing two strings of different lengths.

Status

Slated for Version 0.8

DRAFT

SET EXCLUSIVE Command

SET EXCLUSIVE ON|OFF

Remarks

Specifies whether JAXBase opens table files for exclusive or shared use on a network.

Status

Slated for Version 0.8

DRAFT

SET FDOW Command

Specifies the first day of the week.

Status

Slated for Version 1.0

DRAFT

SET FIELDS Command

Specifies which fields in a table can be accessed.

Status

Slated for Version 1.0

DRAFT

SET FILTER Command

Remarks

Specifies a condition that records in the current table must meet to be accessible.

Status

Slated for Version 0.8

DRAFT

SET FIXED Command

SET FIXED TO nDecimals

Specifies if the number of decimal places used in the display of numeric data is fixed.

Status

Slated for Version 1.0

DRAFT

SET FULLPATH Command

SET FULLPATH ON|OFF

Remarks

Specifies if DBF() and NDX() return the path in a file name.

Status

Slated for Version 0.6

DRAFT

SET FWEEK Command

Specifies the requirements for the first week of the year.

Status

Slated for Version 1.0

DRAFT

SET HEADINGS Command

SET HEADINGS ON|OFF

Remarks

Determines whether column headings are displayed for fields.

Status

Slated for Version 1.0

DRAFT

SET HELP Command

Enables or disables Microsoft JAXBase online Help or specifies a Help file.

Status

Slated for Version 1.0

DRAFT

SET HOURS Command

Sets the system clock to a 12- or 24-hour time format.

Status

Slated for Version 1.0

DRAFT

SET ID Command

SET ID TO SYSTEM | GUID

Remarks

Controls what the SYSID() returns and what the ID in a class object holds.

Parameters

SYSTEM

SYSID() returns a 10-character string which is unique to the session.

GUID

SYSID() returns a 36-character GUID string which is universally unique.

Note

SET KBMINPUT Command

SET KBMINPUT TO cFile

Remarks

Causes JAXBase to ignore most keyboard and mouse activity and sets a text file as the input source for keyboard and mouse activity while a program is running.

Status

Slated for Version 1.0

Note

The file specified by KBMINPUT is only used when a program is running. Otherwise, the keyboard and mouse work as normal.

When KBMINPUT is in use, attempting to SET ASSERTS ON will set to NODIALOG instead.

When a file is set in SET INPUT while a program is running, the following keys are accepted from the keyboard. Pressing one of the numeric keys will cause the next statement to be executed before the delay value is reset. Execution delays work best in conjunction with the Trace window.

Key	Action
Escape	Brings up the Assert Dialog.
Space Bar	Pauses and restarts execution when the trace window is displayed.
0	Turns off execution delay.
1	Sets a 0.1 second delay between statements.
2	Sets a 0.2 second delay between statements.
3	Sets a 0.5 second delay between statements.
4	Sets a 1.0 second delay between statements when the trace window is displayed.
5	Sets a 1.5 second delay between statements when the trace window is displayed.
6	Sets a 2.0 second delay between statements when the trace window is displayed.
7	Sets a 5 second delay between statements when the trace window is displayed.
8	Sets a 10 second delay between statements when the trace window is displayed.
9	Sets a 15 second delay between statements when the trace window is displayed.

See Also

Automated Testing

SET LOCK Command

Remarks

Enables or disables automatic file locking in certain commands.

Status

Slated for Version 0.8

DRAFT

SET MACKEY Command

Remarks

Specifies a key or key combination that displays the Macro Key Definition dialog box.

Status

Slated for Version 2.0

DRAFT

SET MEMOWIDTH Command

Remarks

Specifies the displayed width of memo fields and character expressions.

Status

Slated for Version 1.0

DRAFT

SET MESSAGE Command

Defines a message for display in the main JAXBase window or in the graphical status bar, or specifies the location of messages for user-defined menu bars and menu commands.

Status

Slated for Version 1.0

DRAFT

SET MULTILOCKS Command

Remarks

Determines whether you can lock multiple records using LOCK() or RLOCK().

Status

Slated for Version 0.8

DRAFT

SET NAMING Command

SET NAMING TO [NATURAL | LOWER | UPPPER | PROPER] [ALL]

Remarks

When a JAXBase command that contains a database, or table name is executed, SET NAMING will alter the name to fit the requested setting.

Parameters

NATURAL

Makes no changes (default).

LOWER

Filenames are set to lower case.

UPPER

File names are set to upper case.

PROPER

File names are set to proper case. The first letter is upper case and any letter following a dash, underscore, or space are also set to upper case.

ALL - Version 1.0

Fix case of paths that are not recognized system paths. Examples would be the paths loaded and verified at runtime initiation, such as those paths found in the Environment Variables (_JAXWork, _JAXTemp, _JAXFolder) and verified paths defined in various Environment objects and arrays.

Notes

Omitting a parameter sets file and folder naming to NATURAL.

When you SET NAMING, omitting the parameter ALL will set folder references back to NATURAL.

While Windows is not case specific with file names and paths, Linux and other operating systems are. Using the SET NAMING can make it easier to move between operating systems as it allows you to avoid case mistakes on folders, files, and tables that are created by the application.

SET NEAR Command

SET NEAR ON|OFF

Remarks

Determines where the record pointer is positioned after FIND or SEEK unsuccessfully searches for a record.

Status

Slated for Version 0.8

DRAFT

SET NOCPTRANS Command

Remarks

Prevents translation to a different code page for selected fields in an open table.

Status

Slated for Version 2.0

DRAFT

SET NOTIFY Command

Enables or disables the display of certain system messages.

Status

Slated for Version 1.0

DRAFT

SET NULL Command

Remarks

Determines how null values are supported by the ALTER TABLE, CREATE TABLE and INSERT - SQL commands.

Status

Slated for Version 2.0

DRAFT

SET NULLDISPLAY Command

Remarks

Specifies the text displayed for null values.

Status

Slated for Version 2.0

DRAFT

SET ODOMETER Command

Remarks

Specifies the reporting interval of the record counter for commands that process records.

Status

Slated for Version 0.8

DRAFT

SET ORDER Command

SET ORDER TO [cIndex | nIndex] [WITH ASCENDING | DESCENDING]

Remarks

Designates a controlling index file or tag for a table.

Status

Slated for Version 0.8

DRAFT

SET PATH Command

SET PATH TO [cPath1 [,cPath2...]]

Specifies a path for file searches.

Status

Slated for Version 1.0

DRAFT

SET POINT Command

Determines the decimal point character used in the display of numeric and currency expressions.

Status

Slated for Version 1.0

DRAFT

SET PRIMARY TO

SET PRIMARY TO cIndex

Status

Slated for Version 1.0

Remarks

Registers an index in a JAXBase table as the controlling index when the current table is opened.

Parameters

cIndex

Name of index to set as primary.

Notes

If the index is already registered, it is set as the primary. If the index is not registered and is related to the currently open JAXBase table, it is registered, the index file name is updated, and made primary in the registry.

You can add an unregistered index to a JAXBase table using the SET PRIMARY TO command and then resetting the primary to the original index.

Registered index file names are in the format TABLEName_INDEXName.IDX and can be referenced either by their index name, their full name, or by their fully qualified file name.

When a registered index is set as primary, it is simply the last index opened and thus the controlling index.

See Also

INDEX Command

SET INDEX TO Command

PRIMARY() Function

SET PROCEDURE Command

SET PROCEDURE TO cfile [WITH ADDITIVE]

Opens a procedure file.

Status

Slated for Version 1.0

DRAFT

SET REFRESH Command

Determines whether and how often a Browse window is updated with changes made to records by other users on the network.

Status

Slated for Version 1.0

DRAFT

SET RELATION Command

Establishes a relationship between two open tables.

Status

Slated for Version 1.0

DRAFT

SET REPROCESS Command

Remarks

Specifies how many times and for how long JAXBase attempts to lock a file or record after an unsuccessful locking attempt.

Status

Slated for Version 0.8

DRAFT

SET RESOURCE Command

Updates or specifies a resource file.

Status

Slated for Version 1.0

DRAFT

SET SAFETY Command

Determines whether JAXBase displays a dialog box before overwriting an existing file, or whether table or field rules, default values, and error messages are evaluated when changes are made in the Table Designer or with ALTER TABLE.

Status

Slated for Version 1.0

DRAFT

SET SECONDS Command

Specifies whether seconds are displayed in the time portion of a DateTime value.

Status

Slated for Version 1.0

DRAFT

SET SECURITY Command

Remarks

Security at rest will become a concern starting with versions after JAXBase 1.0 and the SET SECURITY command will allow you to set levels of data encryption including NONE, 128, and 256 with yet-to-be-released options. More secure encryption options will, of course, slow down processing.

Status

Partial support slated for Version 1

Full support slated for Version 2.0

DRAFT

SET SEPARATOR Command

Specifies the character used to separate each group of three digits to the left of the decimal point in displaying numeric and currency expressions.

Status

Slated for Version 1.0

DRAFT

SET SKIP Command

Creates a one-to-many relationship among tables.

Status

Slated for Version 2.0

DRAFT

SET SPACE Command

Remarks

Determines whether a space is displayed between fields or expressions when you use the ? or ?? command.

Status

Slated for Version 0.6

DRAFT

SET SQLCONNECTION Command

SET SQLCONNECTION TO sqlObject | cConnectionString

Status

Slated for Version 0.8

Remarks

Specifies a connection to a backend SQL engine that will be used to create a default connection string. A SQL Object is created using the command `CREATEOBJECT("SQLCONNECTION")` which will create a JAXBase class that can be used to create a SQL connection.

Parameters

sqlObject

Variable name holding initialized SQL class.

cConnectionString

A connection string used to create a SQL class object.

Example

```
* CREATE THE SQL OBJECT
MyDB=CREATEOBJECT("sql")
```

```
* Just fill in the connection data
WITH MyDB
  .Driver="MYSQL"
  .Server="MyServer"
  .USERID="admin"
  .PASSWORD="BobIsAtHD-3167"
  .DATABASE="JAXCorp"
ENDWITH
```

```
* SET THE DEFAULT CONNECTION
SET SQLCONNECTION TO MyDB
```

```
SET SQLCONNECTION TO "Driver=MySQL,Server=MyServer,UID=admin,PW=BobIsAtHD-3167,Database=JAX"
```

```
* OPEN THE DATABASE AND BROWSE THE OFFICERS TABLE
OPEN DATABASE JAXCorp
SELECT * FROM OFFICERS
? RECCOUNT()
CLOSE DATABASE ALL
```


Notes

You can use JSON functions to easily save or load object properties.

A SQL Class object can be used by the OPEN DATABASE command even if the SQL object is already connected to the database. The only requirement is that the name used by the OPEN DATABASE command be a different name than the variable name holding the SQL object.

If a SQL Connection String is provided, it is parsed by JAXBase and used to create a SQL connection object. Variations on parameters (example: UID = User ID = UserID) are considered and if a parameter fails to parse, an error is generated.

See Also

CREATE DATABASE
JSONTOOBJ()
OBJTOJSON()
GETJSONFIELD()
PUTJSONFIELD()

SET STEP Command

SET STEP ON|OFF

Opens or closes the Trace window.

Status

Slated for Version 1.0

DRAFT

SET STRICTDATE Command

Specifies if ambiguous Date and DateTime constants generate errors.

Status

Slated for Version 1.0

DRAFT

SET SYSMENU Command

Enables or disables the JAXBase system menu bar during program execution and allows you to reconfigure it.

Status

Slated for Version 1.0

DRAFT

SET TABLEPROMPT Command

Enables or disables file open dialog from appearing when table cannot be located during execution of a data command such as SELECT - SQL Command.

Status

Slated for Version 1.0

DRAFT

SET TABLEVALIDATE Command

Specifies the level of table validation to perform.

Status

Slated for Version 1.0

DRAFT

SET TALK Command

SET TALK [ON | OFF] [TO cConsole]

Remarks

When TALK is set ON, most commands return a status or other form of response indicating what was done. These responses are sent to the active console and to the Alternate file if set and ON.

TALK is set ON by default in the IDE and OFF in the runtime package.

DRAFT

SET TEXTMERGE Command

Enables or disables the evaluation of fields, variables, array elements, functions, or expressions that are surrounded by text-merge delimiters, and lets you specify text-merge output.

Status

Slated for Version 1.0

DRAFT

SET TEXTDELIMITERS Command

Specifies the text-merge delimiters.

Status

Slated for Version 1.0

DRAFT

SET TOPIC Command

Specifies the Help topic or topics to open when you invoke the JAXBase Help system.

Status

Slated for Version 1.0

DRAFT

SET TOPIC ID Command

Specifies the Help topic to display when you invoke the JAXBase Help system. The Help topic is based on the topic's context ID.

Status

Slated for Version 1.0

DRAFT

SET TRBETWEEN Command

Enables or disables tracing between breakpoints in the Trace Window.

Status

Slated for Version 1.0

DRAFT

SET TYPEAHEAD Command

SET TYPEAHEAD TO nCharacters

Specifies the maximum number of characters that can be stored in the type-ahead buffer.

Status

Slated for Version 1.0

DRAFT

SKIP

SKIP [*nExpr* | **TOP** | **BOTTOM**] [**IN** *nWorkArea* | *cAlias* [**SESSION** *nSessionID*]]

Remarks

Advances the record pointer.

Parameters

nExpr

Numeric expression that indicates how many records to move down towards the bottom of the table if the value is positive, and how many records to move up towards the top of the table if the value is negative.

TOP

Set the record pointer to the logical top of the file. An active index may change the order of the table, and the logical top is the first record pointed to by the index.

BOTTOM

Set the record pointer to the logical bottom of the file. An active index may change the order of the table, and the logical bottom is the last record pointed to by the index.

IN *nWorkArea* | *cAlias*

Advances the record pointer in the indicated work area or alias. 0 is the current work area.

SESSION *nSessionID*

Indicates which session the work area or alias is in. 0 indicates the current session.

Example

* Demonstrate SKIP command

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\polldata")

? RECNO()

SKIP 3

? RECNO()

Note

If SKIP is issued with no expression, the default is to move the record pointer one row down. If the SKIP command has a positive numeric expression, then the record pointer is moved down that many records. If negative, the record pointer moves up toward the beginning of the file that many records.

An expression, TOP, or BOTTOM must be included if the IN clause is included.

If the record pointer tries to go past the beginning of the table order, then BOF() will return True (.T.) and if it attempts to go past the end of the table order, EOF() will return True (.T.). If skip is issued with a positive value after the record pointer has reached the end of the table and EOF() is set to true, then an error is generated. The same is true for attempting to skip with a negative value when the pointer is already at the top of the file and BOF() is set to true.

See Also

GOTO Command
USE Command

SORT

```

SORT TO cTable ON FieldName1 [/A | /D] [/C]
    [, FieldName2 [/D] [/C] ...]
    [Scope] [FOR lExpression1]
    [WHILE lExpression2]
    [FIELDS FieldList | FIELDS LIKE Skeleton | FIELDS EXCEPT Skeleton]
  
```

Status

Slated for after Version 1.0

Remarks

Sorts the current table or cursor in the specified manner and stores the results to a table.

Parameters

TO cTable

Character literal specifying the name of the JAXBase table to save the results.

ON FieldName1

Specifies a field to use as a sort key.

/D [/C]

Modifies the sort. /D indicates that the field is to be sorted in descending order. /C indicates that the field is to be sorted as case insensitive. If the field is not a character field, /C is ignored.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

FIELDS FieldList

Specifies the fields to populate from the array.

FIELDS LIKE

Specifies a field skeleton where matching fields are populated from the array.

FIELDS EXCEPT

Specifies fields to skip when populating from an array

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
USE (_JAXFolder+"data\polldata")  
SORT TO P203 ON Party /D, Precinct FOR Precinct=203  
USE P203  
BROWSE
```

Note

You cannot sort on a MEMO, GENERAL, or BLOB field.

FieldName cannot be an expression. It must be a field name.

The SORT command is quite slow compared to using indexes.

See Also

BROWSE

JAXBase Concepts: Using Indexes

JAXBase Concepts: Scope Clauses

STORE

STORE eExpression1 [,eExpression2...] **TO** cVar1 [, cVar2...]

Remarks

Assigns a value to a variable or property, or a list of values to a list of properties.

Parameters

eExpression1 [, eExpression2...]

One or more expressions that evaluate to JAXBase data types, objects, or .NULL.

TO cVar1 [, cVar2...]

One or more character literal names of variables to which the expressions are assigned.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
STORE 0,1,2 TO A,B,C
? A,B,C
```

Note

There must be the same number of expressions as variable names, or an error will be generated.

If you assign a value to an array instead of an array element, then that value is assigned to all elements of the array.

See Also

Arrays, Variables, and Objects

SUM

```
SUM [ExprList] [Scope] [FOR lExpr1] [WHILE lExpr2]
    [TO VarList | TO ARRAY ArrayName][ADDITIVE]
    [IN nWorkArea | cTableAlias]
```

Remarks

Takes the averages of the ExprList and puts them into VarList or an array.

Parameters

ExprList

Comma delimited list of expression to use when creating the averages to place into VarList.

Scope

Specifies the range of records to use. Only records that fall within the scope are used. Scope clauses are: ALL, NEXT nRecords, RECORD nRecordNo, REST, and TOP nRecords. The default scope is ALL.

FOR lExpr1

Logical expression used to filter what records are used.

WHILE lExpr2

Logical expression limiting the records used in AVERAGE while the expression is true.

TO VarList

Character literal list of variable names that will receive the averages.

TO ArrayName [ADDITIVE]

Character literal of array name to create or append to if ADDITIVE is provided.

IN nWorkArea | cAlias

Numeric work area literal or character literal for alias name of table to use.

Example

```
* Demonstrate AVERAGE
```

```
CLEAR ALL
```

```
CLOSE ALL
```

```
SET CONSOLE ON
```

```
ACTIVATE CONSOLE
```

```
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
```

```
* If there are no matching records, all non-literal numeric
```

```
* expressions will return 0.00 as their result.
```

```
SUM age TO ARRAY aVoteAge for Precinct=203 AND Party="I"
```

```
SUM age to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="D"
```

```
SUM age to ARRAY aVoteAge ADDITIVE for Precinct=203 AND Party="R"
```

```
DISPLAY MEMO LIKE aVoteAge
```

Note

When saving to an array, if the array does not exist then it is created. If it does exist, it is overwritten with the corresponding number of elements. If ADDITIVE keyword is included, then the array is appended with a new row. If the number of expressions does not match the existing array column count, an error is generated. If there are no results, the array is updated numeric literals and any non-literal expressions are set to 0.00.

ADDITIVE should not be in the first of a series of SUM statements.

If saving to a variable list, values are returned even if there are no matching records, in which case 0.00 is filled in for any non-literal numeric expression.

See Also

AVERAGE Command

CALCULATE Command

COUNT Command

TOTAL Command

JAXBase Concepts: Arrays, Variables, and Objects

JAXBase Concepts: Scope Clauses

SUSPEND

SUSPEND

Status

Slated for after Version 1.0

Remarks

If in the IDE, causes the program to suspend and brings up the Debug Window.

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Note

If the program is not running in the IDE, then the SUSPEND command is ignored.

See Also

ASSERT Command
CANCEL Command
RESUME Command
QUIT Command
Debug Window

TEXT / ENDTEXT

TEXT [to cVar [ADDITIVE][TEXTMERGE][NOSHOW][PRETEXT eExpression]]
 Merge Text
ENDTEXT

Status

Slated for Version 1.0

Remarks

Parameters

Merge Text

Text lines that are used to create the resulting text merge.

TO cVar

Character literal of variable name to store the resulting merged text.

ADDITIVE

If ADDITIVE is included, the resulting merged text is added to the contents of cVar, otherwise it is overwritten.

TEXTMERGE

The TEXTMERGE parameter causes the evaluation of merge fields in the merge text.

NOSHOW

By default, when the TEXT command is executed, the results are sent to the current output device (typically the console). The NOSHOW parameter prevents that from happening.

PRETEXT eExpression

If eExpression is a character expression, insert it before each line of the results.

If eExpression is a numerical expression, use the following table to understand its purpose.

Value (Additive)	Description
1	Eliminate spaces before each line
2	Eliminate tabs before each line
4	Eliminate carriage returns
8	Eliminate line feeds

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
```

```
TEXT TO cText TEXTMERGE NOSHOW
Dear <<Name>,
```

```
We have you listed as <<IsParty()>> and would like to remind you that
you live in voting precinct <<Precinct>>. Your voting location is located
<<VotingLoc>>.
```

```
Thank you,
```

```
Jack B Nimble
Office of Voter Registration
ENDTEXT
```

```
? cText
USE
RETURN
```

```
PROCEDURE IsParty
RETURN ICASE(Party="R", "Republican", Party="D", "Democrate", "Independent")
```

Note

The text merge delimiters are set using the SET TEXTMERGE DELIMITERS command.

When a textmerge field is evaluated, the results are trimmed of leading and trailing spaces.

If a textmerge field cannot be evaluated, an error is generated.

You have two ways of sending the results of the text command to a device.

You can capture the results in a variable using the TO parameter and sending the value to a file, printer, or console.

You can direct console output to an alternate file or printer.

While textmerge can be used to create letters, there are more modern methods discussed at Merging Text.

See Also

SET MEMOWIDTH Command

SET TEXTMERGE Command

SET TEXTMERGE DELIMITERS Command

THROW

THROW nError [, cMessage]

Status

Slated for Version 1.0

Remarks

Allows you to create a custom error message.

Parameters

nError

Numeric expression between 0 and 999 or 7000 and 7999.

cMessage

Character expression containing the user defined message.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
TRY
    THROW 0, "TEST ERROR"
CATCH TO loErr
    ? loErr.ErrorNo, loErr.Message
ENDTRY
```

Note

If the nError is less than 1000 then 7000 is added to it and the result is a user defined error between 7000 and 7999. If nError is not less than 1000 or between 7000 and 7999, a "Value or index is out of range" error is generated.

The original values in nError and cMessage are written out to the log file, if it's active.

See Also

ON ERROR Command

TRY/CATCH/FINALLY/ENDTRY Command

ERROR Event

TRY/CATCH/FINALLY/ENDTRY

TRY

Command block

CATCH [TO cErrObj] [WHEN lExpression]

Command block

FINALLY

Command block

ENDTRY

Remarks

Creates a structured error handling system to handle errors for a block of code.

Parameters

TRY

Beginning of the TRY/CATCH/FINALLY/ENDTRY structure.

CATCH

Beginning of the CATCH block which is executed when an error occurs in the command block following the TRY statement.

TO cErrObj

Creates an object variable named after the character literal name that contains the following properties.

Property	Data Type	Description
ErrorNo	Numeric	Error number
LineNo	Numeric	Line where error occurred
Message	Character	Error Message
Procedure	Character	Procedure where error occurred (may be catching an error from a UDF)
StackLevel	Numeric	Program level where error occurred. Especially used when performing recursion.

WHEN lExpression

The WHEN clause allows you to have several CATCH statements, each of which will handle different error conditions.

FINALLY

Start of finally block which is usually cleanup code to close resources opened in the TRY block.

ENDTRY

End of TRY/CATCH/FINALLY/ENDTRY structure.

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

TRY
  FOR i = 3 to -1
    DO TEST1 WITH i
  ENDFOR
CATCH TO loErr
  ? "Error", TRANSFORM(loErr.ErrorNo) "@ line", TRANSFORM(loErr.LineNo), "in", loErr.Procedure
ENDCATCH
RETURN

* Will thrown an error when passed -1
PROCEDURE TEST1
PARAMETERS tnIdx
? SUBSTR("ABCDEFG",tnIdx,1)
RETURN

```

Note

The CATCH block only executes if an error occurs which is passed to the CATCH command.

The FINALLY code block only executes if the TRY or CATCH block successfully executes and a QUIT or CANCEL command is not executed in either command block.

If an error occurs in the CATCH block, an error is generated that needs to be caught TRY/CATCH/FINALLY/ENDTRY in the CATCH block, an ON ERROR command, or an Error event. Failing that, the program will terminate.

A program, UDF, or method/event that was called from the TRY/CATCH/FINALLY/ENDTRY structure that generates an error and does not have its own error handler, sends the error to the CATCH block.

The WHEN clause can access the cErrObj created in the TO clause. During an error, each clause is evaluated and the first catch statement evaluating to True (.T.) is executed. If you use one or more CATCH WHEN statements, then there should be just a CATCH after to catch any unhandled error conditions.

See Also

ON ERROR Command
Error Handling

UNLOCK

UNLOCK [**RECORD** nRecord] [**IN** nWorkArea | cAlias] [**ALL**]

Status

Slated for Version 0.8

Remarks

Releases a locked record, multiple records, a file lock on a table, or all locks on all open tables.

Parameters

RECORD nRecord

Numeric expression specifying record to unlock.

IN nWorkArea | cAlias

Numeric work area literal or character literal of alias name of table to unlock.

ALL

Releases all locks on all tables.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata")
? LOCK(0)
? RLOCK(5)
? ISFLOCKED()
UNLOCK ALL
? ISFLOCKED()
```

Note

Specifying record 0 releases a header lock, otherwise releases a record lock for a specific record.

Record locks can only be released by the user that creates the locks.

See Also

FLOCK() Function

LOCK() Function

RLOCK() Function

UPDATE

UPDATE cSQLTable

```
SET Column_Name1 = eExpression1 [, Column_Name2 = eExpression2 ...]  
[FROM [FORCE] Table_List_Item [[, ...] | [JOIN [ Table_List_Item]]]  
WHERE FilterCondition
```

Status

Slated for Version 2.0

DRAFT

UPDATE FROM

UPDATE FROM nWorkArea | cAlias TO SQLTable

Status

Slated for Version 1

Remarks

Updates a SQL Table using records from the indicated alias or work area.

Parameters

FROM nWorkArea | cAlias

Numeric work area literal or character literal for alias name of an open table or cursor. 0 is the current work area.

TO SQLTable

Character literal specifying the name of the SQL table to update.

Example

- * Assumes a SQL Database has been opened and
- * that it contains a table that has the same
- * field names and compatible types as the
- * TEST_CSV table created here.

```
CREATE TABLE TEST_CSV (PKEY C(10), Name C(30), BirthDate D, Sex C(1), Height N(4,1))
APPEND FROM (_JAXFolder+"samples\data\PEOPLE.CSV") TYPE CSV
```

- * Insert code to create valid Primary Key values for PKEY
- * in each record. If the primary key matches an existing
- * primary key, that record's fields will be overwritten
- * with the fields from the table. If the primary key
- * does not match, then the record is appended to the table.

- * Update the SQL Table

```
UPDATE FROM 0 TO MySQLTable
```

Note

The FROM parameter is required.

The FROM parameter of 0 means the current work area.

TO SQLTable assumes the current database. To specify the database, use the Database!Table naming format.

See Also

APPEND FROM Command

OPEN DATABASE Command

CREATE TABLE/CURSOR Command

Moving Data To and From a SQL Database

USE

OPEN A DBF

```
USE [[cDatabase!]cTable] [IN nWorkArea [SESSION nSessionID]] [ALIAS cAlias]
    [INDEX cIndex [ASCENDING | DESCENDING][, cIndex...]]
    [AGAIN] [EXCLUSIVE|SHARED] [NOUPDATE]
```

RETRIEVE DATA FROM A SQL TABLE (Slated for Version 1.0)

```
USE cDbName!Table [IN nWorkArea] [ALIAS cAlias] [WHERE lExpression] [NOUPDATE]
```

Status

ALIAS is slated for Version 0.6

INDEX, SHARED are slated for Version 0.8

AGAIN, NOUPDATE are slated for Version 1.0

Remarks

The first command opens a local DBF table while the second command retrieves records from a SQL database table.

Parameters

cDatabase!

Including a character literal for a database followed by an exclamation point indicates that the table to use is in a currently open SQL connection using that name.

cTable

Character literal of table to open.

IN nWorkArea

Numeric work area literal of table to use. 0 is current.

ALIAS cAlias

Character literal of alias to use for table or cursor.

SESSION nSessionID

Numeric session ID. 0 is current.

INDEX cIndex [ASCENDING | DESCENDING] [, cIndex...]

Character literal of one or more indexes to use and optionally set each to ASCENDING (default) or DESCENDING.

WHERE

Slated for Version 1.0

AGAIN

A DBF cannot be opened more than once unless it is SHARED and you use the AGAIN keyword on the subsequent USE commands.

EXCLUSIVE

Open the table for exclusive access. Required if SET EXCLUSIVE is OFF and you do not wish to share the table.

SHARED

Open the table for shared access. Required if SET EXCLUSIVE is ON and you wish to share the table.

NOUPDATE

Opens the table for READONLY access.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\polldata") ALIAS Polls
? ALIAS()
? DBF()
? RECNO()
? RECCOUNT()
```

Note

If you omit the WHERE command when retrieving records from a SQL table, you automatically load the number of records set by SET SQLLOAD which has a default of 100.

ASCENDING and DESCENDING can be abbreviated as ASC and DES.

NOUPDATE will only set the table to read only for that USE statement. If you have the table open under other multiple aliases, they are not affected.

You can open any number of indexes, and you can also open additional indexes using the SET INDEX command.

Indexes are assumed to be in the same folder as the table. If you provide a path to the indexes, it will use that path. If you don't supply a path and the indexes are not found, JAXBase will search the path list until it finds a match.

JAXBase does not convert legacy DBF files, rather it reads and writes to the best of its ability. Please back up legacy tables before using them in JAXBase or open them with NOUPDATE.

See Also

CREATE Table Command
OPEN DATABASE Command
SELECT Command
Data Sessions and Work Areas
SQL Class

WAIT

WAIT [**cExpression**] [**AT row,col**] [**TO cVar**] [**TIMEOUT nSeconds**] [**CLEAR**] [**NOWAIT**]

Status

Full functionality slated for Version 1.0

Remarks

Displays a message window in the upper right corner of the monitor.

Parameters

cExpression

Character expression of message to display.

AT row,col

The row and column to place the message.

TO cVar

Character literal of variable name to return the integer value of the key press. See INKEY() for values.

TIMEOUT nSeconds

nSeconds is a numeric expression indicating how many seconds to wait before clearing the message.

CLEAR

Removes the window from the display area.

NOWAIT

Display the message and continue execution. The message is displayed until WAIT CLEAR is or CLEAR command is issued.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
WAIT WINDOW "Press a key" to lnKey
? lnKey
```

Note

Omitting the TIMEOUT clause is the same as TIMEOUT 0, which keeps the window displayed until a key is pressed or a CLEAR command is issued..

If TO cVAR is included, the NOWAIT clause is ignored, if present.

See Also

CLEAR Commands

DRAFT

WITH/ENDWITH

WITH oVar
 Command block
ENDWITH

Status

Slated for Version 1

Remarks

Specifies what object to use with properties in a block of code.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
WITH _MOUSE
? .XPos
? .YPos
ENDWITH
```

Note

WITH allows you a short-cut to working with an object by assuming that any .cName is a property that belongs to that object. If the .cName literal expression is not a property of the object in the WITH command, an error is generated.

See Also

ADD CLASS Command
CREATE CLASS Command
CREATEOBJECT() Command

ZAP

ZAP [IN nWorkArea | cAlias [SESSION nSessionID]]

Remarks

Removes all records from a table and causes all open indexes to be reindexed.

Parameters

IN nWorkArea | cAlias

Indicates which work area or alias to use. 0 is the current work area.

Session nSessionID

Indicates which session the work area or alias is located.

Note

ZAP is the same thing as DELETE ALL followed by PACK, but ZAP is faster.

A table must be used in EXCLUSIVE mode to ZAP, otherwise an error is generated.

You cannot ZAP a cursor that is in READONLY mode.

Any open indexes will be recreated,

See Also

DELETE Command

PACK Command

DRAFT

Function Reference

DRAFT

\$

cExpr1 \$ cExpr2

Remarks

Provides a quick way to see if a string is in another string.

Parameters

cExpr1

Character expression that will be searched for in cExpr2

cExpr2

Character expression searched

Example

? "A" \$ "ABC"

? "Al" \$ "Jay Allen"

Return Value

Logical

See Also

Like() Function

%

nDividend \$ nDivisor

Remarks

Returns the modulus; the remainder of the nDividend divided by the nDivisor.

Parameters

nDividend

The numeric expression to divide.

nDivisor

The numeric expression of how many times to divide nDividend.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? 51 % 7 && Returns 2

? 11 % 10 && Returns 1

? 15.7 % 5 && Returns 0.7

Return Value

Numeric

Note

The MOD() function and % operator produce identical results.

See Also

MOD() Function

::

cClass::cMethodStatus

Slated for Version 2.0

Remarks

The scope resolution operator is used to execute a parent method, skipping all code between the current subclass method and the specified parent method.

DRAFT

ABS()

ABS(nExpr)

Remarks

Returns the absolute value of a numeric expression.

Parameters

nExpr

Numeric expression that will be converted to its absolute value.

Example

? ABS(-1) && Returns 1

Return Value

Numeric

See Also

INT() Function

ROUND() Function

SIGN() Function

ISODD() Function

AClass()

AClass(ArrayName, ClassObject)

Status

Slated for Version 1.0

Remarks

Returns an array containing the hierarchy of the class names.

Parameters

ArrayName

The name of the array to place the results. If the array name does not exist, it is created. If the name exists, it is overwritten to the correct size. The array will be returned as a one-dimensional array.

ClassObject

The Object you wish to interrogate.

Example

* Create two custom classes and get the
* names of the class hierarchy

CLEAR

lnCount=AClass(laNames,"MyTextBox")

DISPLAY MEMORY LIKE mytextbox

DEFINE CLASS MYTXBOX AS TEXTBOX

ENDDDEFINE

DEFINE CLASS MYTEXTBOX AS MYTXBOX

ENDDDEFINE

Return Value

Numeric

Note

Returns 0 if the array cannot be created, otherwise the number of elements in the array.

See Also

ADD CLASS Command

AMEMBERS() Function

CREATE CLASS Command

CREATE OBJECT Command

DEFINE CLASS Command

ACOPY()

ACOPY(SourceArray, TargetArray [,nFirstSourceElement [,nCount [,nFirstTargetElement]]])

Status

Slated for Version 0.6

Remarks

Copies elements from one array to another.

Parameters

SourceArray

The array to copy from.

TargetArray

The array to copy to.

nFirstSourceElement

Which element to start with in the source array.

nCount

Number of elements to copy.

nFirstTargetElement

The starting element in the target array where the copy will be placed.

Example

```

* Test the ACOPY() function
nPCount=APRINTERS(aPrnt)
DIMENSION aNewList[20]

IF PCount>0
  IF pCount>20
    =ACOPY(aPrnt,aNewList,1,20)
  ELSE
    =ACOPY(aPrnt,aNewList)
  ENDIF
ENDIF

DISPLAY MEMO LIKE aNewList

```

Return Value

Numeric

Note

When copying from an array to an existing array, the target array's size is not altered, and you need to make sure that there is enough room for the copy.

The function copies from one array to the other as if they are one-dimensional, meaning it looks at the number of elements in the array and not the number of rows and columns.

If the array does not exist, it will be created. If the destination is not an array or large enough to accept the number of elements, an error will be generated.

See Also

ADEL() Function

AELEMENT() Function

AINS() Function

ASCAN() Function

ASORT() Function

How JAXBase Deals with Arrays

DRAFT

ACOS()

ACOS(nExpression)

Remarks

Returns the arc cosine of the numeric expression.

Parameters

nExpression

The numeric expression to use whose arc cosign is returned. The value of nExpression can start at -1 and go as high as +1. The value returned ranges from zero to PI (3.14159265)

Example

```
? ACOS(.5)           && Returns 1.05  
? RTOD(ACOS(.5)) && Returns 60
```

Return Value

Numeric

Note

If the nExpression is out of range, an error will be generated.

See Also

ASIN() Function

ATAN() Function

ATN2() Function

ACOS() Function

DTOR() Function

RTOD() Function

SIN() Function

TAN() Function

SET DECIMALS Command

ADATABASES()

ADATABASES(ArrayName)

Status

Slated for Version 0.6

Remarks

Returns a two dimensional array containing the names of open databases in column one, the Server\Instance name in column two, and the database version in column three.

Parameters

ArrayName

Name of the array to place the results.

Example

```
* Get a list of all open databases  
nCount=ADATABASES(aDBList)  
DISPLAY MEMORY LIKE aDBList
```

Return Value

Numeric

Note

If the array does not exist, it is created. If the name exists, it is overwritten and the corresponding number of rows added. If no databases are open, the array is not create and if the name exists, it is not altered.

See Also

CREATE DATABASE Command
DISPLAY DATABASE Command
LIST DATABASE Command
OPEN DATABASE Command
SQL Class

ADBOBJECTS()

ADBOBJECTS(ArrayName, cSetting)

Status

Slated for Version 0.6

Remarks

Places the names of tables or views in the current database to an array.

Parameters

ArrayName

Specifies the name of the array to place the results. If the array name does not exist, it will be created. If it does exist, it will be overwritten, and an array of the correct size will be created.

A one dimensional array is created for VIEW and TABLE settings. A two dimensional array is created for INDEX containing , for each index found, the table, the name of the index, the type of index, and the expression used to create the index.

cSetting

Character expression "index","table" or "view" indicating which to return.

Example

```
* Interrogate the JAXCorp database that you installed
* onto your MySQL or SQL Server database server using
* the CreatedSNFile program in the TOOLS folder.
CLEAR ALL
CLOSE ALL
CLEAR

SET NAMING TO LOWER
cConnect=DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx"))
OPEN DATABASE JAXBase USING cConnect
nCount=ADBOBJECTS(gaTables,"table")
DISPLAY MEMORY LIKE gaTables      && List of all tables
```

Return Value

Numeric

Note

ADBOBJECTS() will return information for the currently open database. If there is no database open, then an error will be generated.

This function allows you to directly interrogate the database and retrieve information regarding the data definition in the dialect of the database engine. Notably when using the INDEX setting, you will receive the expression used to create the index using the syntax of that database engine. MySQL and

SQL Server expressions may differ, so care must be taken when writing code to move a table from one server to the other.

A database opened by creating a SQL Class is addressable by the SET DATABASE command using the name property of the SQL Connection. As an example:

```
* Open a database using a SQL Class
oSQL1=CREATEOBJECT("SQL")
WITH oSQL1
    .Name="SQLDB1"
    .LoadDSN(DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx")))
    .Connect()
ENDWITH

IF oSQL1.Connected()=false
    ? "Error opening JAXCorp"
ENDIF

* Open the JAXSample database using a dsnx file
cConnect=DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXSample.dsnx"))
OPEN DATABASE JAX1 USING cConnect

? DBC()    && Displays JAXSAMPLE

SELECT DATABASE (oSQL1.Name)
? DBC()    && Displays JAXCORP
```

See Also

- ADATABASES() Function
- CREATE Command
- CREATE DATABASE Command
- CREATE TABLE - SQL Command
- CREATEOBJECT() Function
- DBC() Function
- DISPLAY DATABASE Command
- INDBC() Function
- LIST DATABASE Command
- OPEN DATABASE Command
- SET DATABASE Command
- SQL Class

ADDBS()

ADDBS(cPath)

Remarks

Adds a backslash to a path if it doesn't already have one.

Parameters

cPath

A string expression that evaluates to a path.

Example

? ADDBS("C:\WINDOWS") && Returns C:\WINDOWS\

Return Value

Character

Note

ADDBS() does no error checking, so if you send an invalid path, you'll get the invalid path back.

ADDBS() is different from JUSTFULLPATH() which returns the path with a backslash, because if you send "c:\temp" to ADDBS() you'll get "c:\temp" back where JUSTFULLPATH() would return "c:\".

See Also

DEFAULTTEXT() Function

FILE() Function

FORCEEXT() Function

FORCEPATH() Function

JUSTDRIVE() Function

JUSTEXT() Function

JUSTFULLPATH() Function

JUSTFNAME() Function

JUSTPATH() Function

JUSTSTEM() Function

ADDPROPERTY()

ADDPROPERTY(oObjectName, cPropertyName, [eExpression])

Remarks

Adds a property to an existing object at run time.

Parameters

oObjectName

Object variable containing the class that you want to which you want to add a property.

cPropertyName

The property name you wish to add as a string expression.

eExpression

The optional value you wish to assign to the new property.

Example

```
* Demonstrate ADDPROPERTY()
USE (_JAXFolder+"data\phonebook")
SCATTER NAME oEntry
ADDPROPERTY(oEntry, "NewProperty", "HI!")
? oEntry.NewProperty           && Returns HI!
```

Return Value

Logical True (.T.) means the property was successfully added.

Note

If you are adding an array to an already existing array property, the old property is overwritten as an array with the new dimensions and values. If you attempt to overwrite an array property with a non-array value, all elements in that array will be set to that value.

All new properties are added as visible, read/write, and can change data types.

You cannot use ADDPROPERTY() to overwrite a read-only property (like BASECLASS) and you cannot use it to change the data type of a native property (like trying to change NAME from character to an integer value).

ADDPROPERTY() will return a false (.F.) value if you attempt to update a hidden or protected property.

See Also

FOREACH

SCATTER Command

REMOVEPROPERTY() Function

AddProperty Method

ADEL()

ADEL(ArrayName, nElementNumber [,1|2])

Remarks

Deletes an element from a one-dimensional array or a row or column from a two-dimensional array but does not change the size of the array.

Parameters

ArrayName

Specifies the array to alter.

nElementNumber

Specifies which element, row, or column to remove.

1|2

With a two-dimensional array, the value of 1 will delete the row while 2 will delete the specified column.

Example

```
* Demonstrate ADEL()
DIMENSION aMyArray[5,5]
FOR i=1 to 5
  FOR j=1 to 5
    aMyArray(i,j)=(i-1)*5+j
  ENDFOR
ENDFOR

* Displays 25 element values ranging from 1 to 25
DISPLAY MEMORY LIKE aMyArray

ADEL(aMyArray,1,1)
ADEL(aMyArray,5,2)

* Displays 25 element values but row 5 is filled with
* a false value (.F.) as is column 5
* Values 1-5 plus 10, 15, 20, and 25 are now missing
DISPLAY MEMORY LIKE aMyArray
```

Return Value

Numeric

Note

If successful, returns a numeric value of 1, otherwise returns 0.

ADEL does not change the size of the array. The trailing elements are moved toward the front. In a one-dimensional array, the trailing elements move up and last element is set to a value of false (.F.).

In a two-dimensional array, deleting a column causes the trailing columns move left and the last column is filled with a value of false (.F.) while deleting a row causes the trailing rows to move up and the last row is filled with a value of false (.F.).

See Also

ACOPY() Function
ADIR() Function
AELEMENT() Function
AFIELDS() Function
AINS() Function
ALEN() Function
ASCAN() Function
ASORT() Function
ASUBSCRIPT() Function
DIMENSION Command

ADIR()

ADIR(ArrayName [,cFileSkeleton, [,cAttribute, [,nFlag]])

Remarks

Puts information about files in the default path, or the specified location, and returns the number of rows returned in the array.

Parameters

ArrayName

Name of array to create.

cFileSkeleton

A character expression of a valid path and file name. The file name can consist of wildcards.

cAttribute

String expression containing any combination of the letters D, H, and S where D returns subdirectory names, H returns hidden file names, and S returns System file names. By default, directory names, hidden and system files are not returned.

nFlag

Numeric expression indicating case of filenames returned.

nFlag	Description
0	(default) Returns the filename in original case
1	Returns the filename in uppercase
2	Returns the filename in lowercase
10	Returns the full path and filename in original case
11	Returns the full path and filename in upper case
12	Returns the full path and filename in lower case

Example

*** Demonstrate the ADIR() function**

CLEAR ALL

CLOSE ALL

CLEAR

ADIR(aMyDir,_JAXFolder+"data*.dbf")

*** Displays file name, size, date last modified, time last modified, attributes**

DISPLAY MEMO LIKE aMyDir

Returns

Numeric

Note

If the array name does not exist, it is created. If the name exists, it is overwritten, and the corresponding rows and columns are created.

If the path is invalid or unavailable, an error is generated.

If there are no matching files or folder entries, then the array is not created.

See Also

ADEL() Function
AELEMENT() Function
AFIELDS() Function
AINS() Function
ALEN() Function
ANETRESOURCES() Function
ASCAN() Function
ASORT() Function
ASUBSCRIPT() Function
DIMENSION Command
DIR or DIRECTORY Command

AELEMENT()

AELEMENT(ArrayName, nRowSubscript [, nColumnSubscript])

Remarks

Returns the element number of an array from the provided subscript information.

Parameters

Arrayname

Name of the array whose element you want returned.

nRowSubscript

A numeric expression indicating the row of a two-dimensional array or the element of a one-dimensional array.

nColumnSubscript

A numeric expression indicating the column of a two-dimensional array.

Example

* Demonstrate the AELEMENT() function

DIMENSION aTest1[3,7]

? AELEMENT(aTest1,2,3) && Returns 10

? AELEMENT(aTest1,3,7) && Returns 21

Returns

Numeric

Note

AELEMENT() with invalid subscript references generates an error.

AELEMENT() with a missing column on a two-dimensional array will generate an error.

AELEMENT() on a one-dimensional array will return the same number as the nRowSubscript.

Attempting to use AELEMENT against a non-array variable or property will generate an error.

All arrays are stored as one-dimensional arrays in JAXBase. A two-dimensional array can be addressed as a two-dimensional array or as a one-dimensional array. Elements in a two-dimensional array start at 1,1 and move right across the row, then down a row and at column 1. As an example, a two-dimensional array named aTest with 2 columns and 2 rows is arranged in memory as:

Array aTest

Row	Col	Element
1	1	1
1	2	2
2	1	3
2	2	4

Thus, the above two-dimensional array can be represented as `aTest[3]` or `aTest[2,1]` to get the same element value.

See Also

`ADEL()` Function

`ADIR()` Function

`AFIELDS()` Function

`AINS()` Function

`ALEN()` Function

`ASCAN()` Function

`ASORT()` Function

`ASUBSCRIPT()` Function

`DIMENSION` Command

`DISPLAY MEMORY` Command

How JAXBase Deals with Arrays

AERROR()

AERROR(ArrayName)

Status

Slated for Version 0.6

Remarks

Puts one or more rows of error information from the error stack into the specified ArrayName and returns the number of rows in the array.

Parameters

ArrayName

Name of the array to create with information from the error stack.

Example

* Test the AERROR() function

TRY

 DIMENSION aTest[3]

 aTest[4]=7

CATCH

 AERROR(aErrInfo)

 DISPLAY MEMORY LINE aErrInfo && Displays the captured error information

ENDTRY

Returns

Numeric

Note

When a JAXBase error occurs, the array contains one row.

Element	Description
1	Numeric. The error number returned, which is identical to ERROR().
2	Character. The text of the error message, identical to MESSAGE().
3	Empty character string unless there is an error parameter which is then returned.
4	Returns the work area number.
5	Returns the data session number.
6	Character value of the program where the error occurred.
7	Numeric value of the line where the error occurred.

When an error occurs in an ODBC call, the array contains one or more rows for each ODBC error.

Element	Description
1	Numeric. The error number returned, which is identical to ERROR().
2	Character. The text of the error message, identical to MESSAGE().
3	Character. The message related to the ODBC error.
4	Character. The current ODBC SQL state.
5	Numeric. The error number from the ODBC source.
6	Character. The name of the SQL handle.
7	Character. The name of the database.

The following table represents a function that is slated for Version 2

When a communication error occurs, the array contains one or more rows for each error returned.

Element	Description
1	Numeric. The error number returned, which is identical to ERROR().
2	Character. The text of the error message, identical to MESSAGE().
3	Character. The error text returned from the called program.
4	Character. The name of the program called.
5	Character. The empty string or additional error information from the called program.
6	Character. The address of the called program (such as IP address).
7	Character. The empty string or other connection information related to the address.

See Also

ERROR Command

ERROR() Function

MESSAGE() Function

ON ERROR Command

OPEN DATABASE Command

JAXBase Error Messages

JAX Classes:

BARCODE, FTP, HTTP, IPC, PIP, POP3, PRINTER, SMPT, SMS, SQL, TCP, and UDP

AEVENTS()

Remarks

Returns an array containing existing event bindings between objects.

Status

Slated for Version 2

DRAFT

AFIELDS()

AFIELDS(ArrayName [,nWorkArea | cAlias])

Remarks

Creates an array containing the structure of the table and returns the number of rows created.

Parameters

ArrayName

Name of the array to create and store the structure.

nWorkArea

A numeric expression for the workarea.

cAlias

A character expression for an open table or cursor alias.

Example

* Demonstrate the AFIELDS() function

CLOSE ALL

CLEAR ALL

USE (_JAXFolder+"data\phonebook")

AFIELDS(aStructure)

DISPLAY MEMORY LIKE aStructure

&& Returns field structure

Returns

Numeric

Note

The array will contain the following information for each JAXBase field.

Column	Data Type	Description
1	Character	Field Name
2	Character	Field Type – See JAXBase Field Types
3	Numeric	Field Width
4	Numeric	Precision (number of decimal places)
5	Logical	.F.
6	Logical	.F.
7	Character	Empty String
8	Character	Empty String
9	Character	Empty String
10	Character	Empty String
11	Character	Empty String
12	Character	Empty String
13	Character	Empty String

14	Character	Empty String
15	Character	Empty String
16	Character	Empty String
17	Numeric	NextValue for autoincrementing (Slated for Version 2)
18	Numeric	Step for autoincrementing (Slated for Version 2)

You can retrieve a SQL table's structure in JAXBase format or the native SQL format by using the `AFIELDS()` method in the SQL class.

See Also

`ALLEN()` Function

`ALTER TABLE` - SQL Command

`COPY STRUCTURE EXTENDED` Command

`CREATE` Command

`DIMENSION` Command

SQL Class

JAXBase Field Types

AFONT()

AFONT(ArrayName [, cFontName])

Status

Slated for Version 0.6

Remarks

Places available font information into an array.

Parameters

ArrayName

Name of array to place the font information into.

cFontName

Specifies the name of the font for which information is placed into the array.

Example

*** Demonstrate the AFont() function**

CLOSE ALL

CLEAR ALL

? AFont(afInfo)

LIST MEMO LIKE afInfo

Note

Due to differences in Windows and Linux, the AFont() function simply retrieves font information with name as the only filter.

If the cFontName parameter is not passed, all available font information is loaded to the array.

See Also

TXTWDITH() Function

AINS()

AINS(ArrayName, nElementNumber [, 2])

Remarks

Inserts an element into a one-dimensional array, or a row or column into a two-dimensional array.

Parameters

ArrayName

Name of element that will be modified.

nElementNumber

Specifies where the new element, row, or column is inserted into the array.

1|2

The value 1 specifies that a row is to be inserted while 2 specifies that a column will be inserted into a two-dimensional array.

Example

```
DIMENSION aMyArray[5,4]
FOR i=1 to 4
  FOR j=1 to 4
    aMyArray(i,j)=(i-1)*4+j
  ENDFOR
ENDFOR
AINS(aMyArray,2)
AINS(aMyArray,2,2)
```

The following is displayed:

```
AMYARRAY   Priv   A test
(  1,  1)   N  1   (  1.00000000)
(  1,  2)   L  .F.   (
(  1,  3)   N  2   (  2.00000000)
(  1,  4)   N  3   (  3.00000000)
(  2,  1)   L  .F.
(  2,  2)   L  .F.
(  2,  3)   L  .F.
(  2,  4)   L  .F.
(  3,  1)   N  5   (  5.00000000)
(  3,  2)   L  .F.
(  3,  3)   N  6   (  6.00000000)
(  3,  4)   N  7   (  7.00000000)
(  4,  1)   N  9   (  9.00000000)
(  4,  2)   L  .F.
(  4,  3)   N 10   ( 10.00000000)
(  4,  4)   N 11   ( 11.00000000)_
```

Another way of looking at the results is as follows:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

BECOMES

1	.F.	2	3
.F.	.F.	.F.	.F.
5	.F.	6	7
9	.F.	10	11

Returns

Numeric: 1 for a successful insert.

Note

Specifying the third parameter with a one-dimensional array will generate an error.

If there is no third parameter when inserting into a two-dimensional array, then a row is inserted.

The array size is not increased. In a one-dimensional array, the cells starting with nElementNumber are moved toward the end and a value of false (.F.) is inserted at that location. With a two dimensional array, the rows are pushed down and columns are pushed to the right starting at the nElementNumber row or column. That row or column is then filled with the value false (.F.).

See Also

ACOPY() Function

ADEL() Function

ADIR() Function

AELEMENT() Function

ALEN() Function

ASUBSCRIPT() Function

DIMENSION Command

AINSTANCE()

AINSTANCE(ArrayName, cClassName)

Status

Slated for Version 0.6

Remarks

Places instances of a class into a variable array and returns the number of elements created.

Parameters

ArrayName

The name of the array to which the instances will be written.

cClassName

The name of the class to search.

Example

```
CLEAR ALL
goObj1=CREATEOBJECT('TEXTBOX')
goObj2=CREATEOBJECT('TEXTBOX')

? AINSTANCE(gaTest, 'textbox')    && Returns 2
DISPLAY MEMORY LIKE gaTest        && Displays the two references
```

Returns

Numeric

Note

If the array name already exists and there are references returned, the array overwrites what's there and is given the corresponding number of elements. If the array name does not exist, it creates it.

Class elements assigned to arrays using CREATEOBJECT(), NEWOBJECT(), or linked to variables, such as in the DO FORM command are recognized.

See Also

ADD CLASS Command
 AMEMBERS() Function
 CREATE CLASS Command
 CREATE CLASSLIB Command
 CREATEOBJECT() Function
 DEFINE CLASS Command
 DO FORM

ALEN()

ALEN(ArrayName [,nArrayAttribute])

Remarks

Returns the number of elements, rows, or columns in an array.

Parameters

ArrayName

Name of the array you wish to check.

nArrayAttributes

Numeric expression that is used to determine if ALEN() returns the number of elements, rows, or columns.

Value	Description
0	Returns the number of elements. This is the default and the same as if you did not specify an attribute.
1	Returns the number of rows or elements.
2	Returns the number of columns.

Example

```
DIMENSION aTest[5,7]  
? ALEN(aTest)    && Returns 35  
? ALEN(aTest,1)  && Returns 5  
? ALEN(aTest,2)  && Returns 7
```

Returns

Numeric

Note

If you only provide the array name, ALEN() returns the number of elements. Specifying the nArrayAttribute of 2 on a one-dimensional array returns 0.

See Also

ADEL() Function

ADIR() Function

AELEMENT() Function

AINS() Function

ASUBSCRIPT() Function

DIMENSION Command

ALIAS()

ALIAS([nWorkArea | cAlias])

Remarks

Returns the table alias of the current or specified work area.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cAlias

Character expression indicating the alias to check.

Example

Note

If a numeric workarea value is provided, it will check the indicated workarea in the current data session and return the alias of any open table or cursor, otherwise an empty string is returned.

If the character alias is provided, it will act similar to the USED() function and check to see if that table alias is open in the current data session, returning it if found, otherwise an empty string is returned.

See Also

DBF() Function

SELECT() Function

USE Command

SQL Class

Alias Property

ALINES()

ALINES(ArrayName, cExpression [, nFlags][, cParseChar1 [, cParseChar2 [, ...]]])

Remarks

Copies lines in a character expression or memo field into an array.

Parameters

ArrayName

Name of the array to create that holds the individual lines parsed from cExpression.

cExpression

The character expression or field to parse.

nFlags

Numeric expression alters how the ALINES() function works and are described by the following table.

Bit	Additive Value	Description
0	1	(Default) Removes leading and trailing spaces and any trailing zeros (0), parse is case-sensitive, all elements are included, even if empty, does not include the parse characters in the results, does not include the last line if empty.
1	2	Include the last element in the array even if it is empty.
2	4	Do not include empty elements.
3	8	Specifies case-insensitive parse character searches.
4	16	Include the parsing characters in the results.

[, cParseChar1 [, cParseChar2 [, ...]]]

Specifies one or more parse characters used to break the data into separate lines. If no parse characters are included, then CHR(13) is the default.

Example

```

* A simple demonstration of ALINES()
SET CONSOLE ON  && Open the default console window
ACTIVATE CONSOLE && Activate the default console window
CLEAR ALL
CLEAR
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
    +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."

? ALINES(aGettysburg1,lcString)                && Displays 6

? ALINES(aGettysburg2,lcString,0,',','.')      && Displays 10

SET ALTERNATE TO GETTY.TXT
SET ALTERNATE ON
LIST MEMORY LIKE aGettysburg1
?
?
LIST MEMORY LIKE aGettysburg2
SET ALTERNATE OFF
SET ALTERNATE TO

```

Returns

Numeric

Note

If the array name does not exist, it will be created. If it does exist, it will be overwritten and given the corresponding number of elements.

In Windows, most text files terminate lines with the CHR(10)+CHR(13) character pair. If you do not want to include the CHR(10) in the results, you should use the TRANCHAR() command to first remove the unwanted characters.

See Also

MEMLINES() Function
 ATLINE() Function
 CHRTRAN() Function
 SET ALTERNATE Command
 SET MEMOWIDTH Command
 STRTRAN() Function
 How JAXBase Deals with Arrays

ALLTRIM()

ALLTRIM(cExpression, [, nFlags][, cParseChar1 [, cParseChar2 [, ...]]])

Remarks

Removes leading and trailing characters matching default or provided parameters.

Parameters

cExpression

Character or Binary Character expression that is to be trimmed.

nFlags – May be omitted

0 – (default) Case sensitive comparison of parse characters.

1 – Case insensitive comparison of parse characters.

2 – Remove only the first leading and trailing of each parse expression (case sensitive)

3 – Remove only the first leading and trailing of each parse expression (case insensitive)

cParseChars1 [, cParseChars2 [, ...]]

Optional parameters specifying one or more-character strings to remove from the beginning and end of cExpression.

Example

```
SET CONSOLE ON
ACTIVATE CONSOLE
SET MEMO WIDTH TO 0 && Turns off memo width tracking
CLEAR ALL
CLEAR
```

```
aStr="aAbBcCHELLO THERE!aABbCc"
```

```
? ALLTRIM(aStr,"A","a","B","b","c")
? ALLTRIM(aStr,"A","a","B","b","c",1)
? ALLTRIM(aStr,"A","a","B","b","c",2)
? ALLTRIM(aStr,"A","a","B","b","c",3)
```

Returns

Character

Note

If no parse characters are provided, then both CHR(0) and CHR(32) are assumed.

See Also

ALINES() Function

ATLINE() Function

MEMLINES() Function

SET MEMOWIDTH Command

JAXBase Character Functions

ALOCFILE()

ALOCFILE(ArrayName, cFileName, lRecurse)

Remarks

Attempts to locate the file and return the absolute path with the file name(s) in an array.

Parameters

ArrayName

Name of array to create containing any files that match cFileName.

cFileName

Character expression with a file name.

lRecurse

Logical expression indicating that ALOCFILE() should recurse sub-folders.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ALOCFILE(aFiles, "*. *")
DISPLAY MEMORY LIKE aFiles
```

Return Value

Numeric

Note

ALOCFILE() returns the number of elements created corresponding to the number of files found.

If cFileName does not have an absolute path, then ALOCFILE() will attempt to search based on what was given and use the JAXBase path list to continue the search.

Care must be taken if setting lRecurse is true (.T.) as this could cause ALOCFILE() to search a large number of folders and files, slowing down processing.

If ArrayName does not exist, it will be created. If it exists, it will be overwritten, and the number of elements will correspond to the number of files found. If no results were found, the ArrayName is not created or modified.

See Also

ADIR() Function

FILE() Function

GETFILE() Function

GETPICTURE() Function

PUTFILE() Function

SET PATH Command

DRAFT

AMEMBERS()

AMEMBERS(ArrayName, oObjectName [, nArrayContentsID] [, cFlags])

Remarks

Places the names of properties, procedures, and member objects for an object into an array.

Parameters

ArrayName

Name of the array that will hold the names of the member properties, procedures, or objects.

oObjectName

Specifies the object you are interested in. It can be expression that evaluates to an object.

nArrayContentsID

Specifies what to place into the array based on the following table.

Value	Description
0	Place just the property names into the array. Omitting this parameter is the same as 0.
1	Place the properties and methods of the object along with member objects. The array will be two-dimensional with the first column holding the name. The second column will hold "Property", "Event", "Method", or "Object".
2	Returns a two-dimensional array with the name of all child objects in column one, the class name in column two, and base class name in column three.

cFlags

Modifies what is returned in the array based on the following flags.

Flag	Description
F	Add an extra column that lists the following attributes for each entry: P – Protected H – Hidden G – Public N – Native to the object U – User defined C – Changed since INIT I – Inherited R – Read Only

Example

```
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
lcForm=CREATEOBJECT("FORM")
```

```
AMEMBERS(lcArray, lcForm, 1)    && Just return property names
DISPLAY MEMORY LIKE lcArray    && Display the form properties
```

Returns

Numeric

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

See Also

```
ADD CLASS Command
AINSTANCE( ) Function
CREATE CLASS Command
CREATE CLASSLIB Command
CREATEOBJECT( ) Function
DEFINE CLASS Command
```

AMEMORY()

AMEMORY(ArrayName, cTypes, lReturnRef)

WARNING

AMEMORY() is under development, and this page may undergo changes before Version 1.0 is released. Full support is slated for Version 2.

Remarks

Returns an array of memory objects found in the application.

Parameters

cTypes

A character expression defining what memory types to return. You may specify more than one by supplying a comma-delimited list.

Type	Description
ALL	Returns Arrays, Forms, Menus, Objects, and Variables
ARRAYS	Returns a list of all arrays in memory
FORMS	Returns a list of all instantiated forms
MENUS	Returns a list of all instantiated menus
OBJECTS	Returns a list of all objects
VARIABLES	Returns a list of all variables that are not arrays or objects

lReturnRef

If True (.T.), objects, menus, and forms will be returned, by reference, into column 8 of the array.

Example

Note

AMEMORY returns an array in the following format.

Column	Description
1	Name of object or variable
2	VARTYPE of memory object (C, D, N, L, O, T, X)
3	Scope,Level of object or variable (A local variable in level 3 returns L,3)
4	Procedure or method
5	Program or class
6	Rows (0 if not an array)
7	Columns (0 if not an array)
8	Object reference

Object references are read-only.

Items with a scope of L cannot be accessed directly. The highest level of a private object or memory variable takes precedence over other variables or objects of the same name at a lower level.

See Also

LIST MEMORY Command

AVARIABLE() Function

ASC()

ASC(cExpression)

Remarks

Returns the ASCII value of the first character of the character expression.

Parameters

cExpression

A character expression.

Example

```
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE

lcStr="FOUR SCORE"

FOR I=1 TO LEN(lcStr)
    ? ASC(SUBSTR(lcStr,I,1))
ENDFOR
```

Note

ASC() will only return the ASCII value of the first character if given more than one.

See Also

CHR() Function

SUBSTR() Function

JAXBase Character Functions

ASCAN()

ASCAN(ArrayName,eExpression [,nStartElement [,nElementsSearched [,nSearchColumn [,nFlags]]])

Remarks

Searches an array for an element matching the expression.

Parameters

Array Name

Name of the array to search.

eExpression

The expression to search for in the array.

nStartElement

Starting element for the search. If omitted, the entire array is searched. Specifying a 0 or negative number is the same as omitting the value.

nElementsSearched

Specifies how many elements to search. If you don't include nStartElement and nElementSearched, the search begins with the first array element and continues to the last array element if you do not specify nSearchColumn. Specifying 0 or a negative number is the same as omitting the value.

nSearchColumn

Specifies if only a column should be searched. Using 0 or a negative number will cause the whole array to be searched.

nFlags

nFlag	Bit	Description
0	0000	Case Sensitive; Exact OFF; Return element number
1	0001	Case Insensitive; Exact OFF; Return element number
2	0010	Case Sensitive; Exact ON; Return element number
3	0011	Case Insensitive; Exact ON; Return element number
4	0100	Case Sensitive; Exact OFF; Return row number
5	0101	Case Insensitive; Exact OFF; Return row number
6	0110	Case Sensitive; Exact ON; Return row number
7	0111	Case Insensitive; Exact ON; Return row number

Example

```

* Demonstrate ASCAN()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

USE (_JAXFolder+"data\counties")
COPY ALL TO ARRAY laCounties FOR STATE="CT"

* The following all return the
* array element for Hartfield, CT
? ASCAN(laCounties,"Hart")
? ASCAN(laCounties,"HART",0,0,1)
? ASCAN(laCounties,"Hartfield",0,0,2)
? ASCAN(laCounties,"HARTFIELD",0,0,3)

```

Return Value

Numeric

Note

If a match is found, ASCAN() returns the number of the element or row containing the expression. If a match cannot be found, ASCAN() returns 0.

Unless you specify a search column, nFlag 4 – 7 does not as useful as flags 0 - 3.

When searching, if using EXACT ON with nFlag, the match must be exact with all characters matching. With EXACT OFF, then as long as the element matches to the end of the expression, you have a match.

You must provide all parameters in or order to use the nFlag parameter, but nStartElement and nElementsSearched can be 0 or a negative number, which is the same as you omitted them.

See Also

ADEL() Function
 ADIR() Function
 AELEMENT() Function
 AFIELDS() Function
 AINS() Function
 ASORT() Function
 ASUBSCRIPT() Function
 DIMENSION Command
 SET EXACT Command

ASELOBJ() – Under consideration

ASELOBJ(ArrayName, [1 | 2 | 3])

Status

Under consideration

Remarks

Parameters

Example

Return Value

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

See Also

ASESSIONS()

ASESSIONS(ArrayName)

Remarks

Returns an array of active data sessions IDs.

Parameters

ArrayName

Name of the array into which ASESSIONS() will place the session IDs. Returns the number of elements created, which will always be at least 1 since Data Session 1 cannot be closed.

Example

```
* Demonstrate ASCAN()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE

SELECT DATASESSION 2
SELECT DATASESSION 3

ASESSIONS(laDataSessions)
DISP MEMO LIKE laDataSessions

?
CLOSE DATASESSION 3

ASESSIONS(laDataSessions)
DISP MEMO LIKE laDataSessions
```

Return Value

Numeric

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

See Also

SELECT DATASESSION Command
CLOSE DATASESSION Command
JAXBase Data Sessions

ASIN()

ASIN(nExpression)

Remarks

Returns in radians the arc sine of a numeric expression.

Parameters

nExpression

A numeric expression ranging from +1 to -1.

Example

```
* Demonstrate ASCAN()  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE
```

Return Value

Numeric

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

The value ASIN() returns can range from $-\pi/2$ through $+\pi/2$ (-1.57079 to 1.57079).

The number of decimal places in the display of the result can be specified with SET DECIMALS.

Use RTOD() to convert radians to degrees.

See Also

RTOD() Function

SET DECIMALS Command

SIN() Function

ASORT()

ASORT(ArrayName [, nStartElement [, nNumberSorted [, nSortOrder [, nFlags]]])

Status

Slated for Version 0.6

Remarks

Sorts elements in an array in ascending or descending order. A return value of 1 means a successful sort, while -1 means a problem was encountered.

Parameters

ArrayName

Name of the array that will be sorted.

nStartElement

Numeric expression indicating starting element. If omitted, 0, or negative in value, then the sort starts at the first element. If the array is one-dimensional then nStartElement indicates the starting element. If the array is two-dimensional, then the starting element indicates the row and column based on how JAXBase works with arrays (see How JAXBase Deals with Arrays).

nNumberSorted

Specifies the number of elements to sort in a one-dimensional array or the number of rows to sort in a two-dimensional array. If 0 or a negative number, all elements or rows are sorted starting at nStartElement.

nSortOrder

Numeric expression for sort order where omitted, 0 or negative is ascending and if any positive value the sort order is descending.

nFlags

A numeric expression indicating case sensitivity where if omitted, 0 or negative means sorting is case-sensitive while a positive value indicates that case insensitive sorting is used.

Example

```
* Demonstrate ASCAN()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DIMENSION laToSort[7]
FOR i = 1 TO 7
    laToSort[i]=INT(RAND()*1000)
ENDFOR
```

```
? "Starting values"
```

DISPLAY MEMORY LIKE laToSort

? "After sorting elements 2 through 6 in descending order"

ASORT(laToSort, 2, 5, 1)

DISPLAY MEMORY LIKE laToSort

Return Value

Numeric

Note

All elements involved in the sort must be of the same data type.

One-dimensional arrays are sorted by their elements. Two-dimensional arrays are sorted by their rows.

When a two-dimensional array is sorted, the rows are changed to align with the sorted column.

See Also

ACOPY() Function

ADEL() Function

ADIR() Function

AELEMENT() Function

AFIELDS() Function

AINS() Function

ALen() Function

ASCAN() Function

How JAXBase Deals with Arrays

ASTACKINFO()

ASTACKINFO(ArrayName)

Remarks

Creates an array that contains information on the current call stack. Returns the number of rows created.

Parameters

ArrayName

Name of the array that will contain the information in the following column order.

Column	Description
1	Call stack level (starts at 1)
2	Program name
3	Module or Object name
4	Module or Object source filename
5	Line number in the source file
6	Source line contents (if available)

Example

```
* Demonstrate ASTACKINFO()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE

DO PROC1
RETURN

PROCEDURE PROC1
DO PROC2
RETURN

PROCEDURE PROC3
ASTACKINFO(laStack)
DISPLAY MEMO LIKE laStack
RETURN
```

Return Value

Numeric

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

ASTACKINFO() only populates column six if the source is available.

Using the second and fourth column, you can find out if the current program is a child of a program or object. A program will have a Fully Qualified File Name (FQFN) in column four. If JAXBase is unable to retrieve this information, it will be blank.

See Also

LINENO() Function

PROCEDURE Command

PROGRAM() Function

ASUBSCRIPT()

ASUBSCRIPT(ArrayName, nElementNumber, nSubscript)

Remarks

Returns the row or column number of the indicated element.

Parameters

ArrayName

Name of the array in question.

nElementNumber

Numeric expression indicating which element to test.

nSubscript

Numeric expression where 1 is for row and 2 is for column.

If the array is one dimensional and nSubscript is 1, then 1 is always returned, while a value of 2 for nSubscript will return the nElement number as single-dimension arrays are treated as two-dimensional with one row.

Example

```
* Demonstrate ASUBSCRIPT()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE

CLEAR
DIMENSION laToSort[7]

? ASUBSCRIPT[laToSort,5,1]  && Returns 5
? ASUBSCRIPT[laToSort,5,2]  && Returns 0

DIMENSION laToSort[7,3]
? ASUBSCRIPT[laToSort,15,1] && Returns 3
? ASUBSCRIPT[laToSort,15,2] && Returns 1
```

Return Value

Numeric

Note

Two-dimensional arrays can be represented using [row,column] notation or [element] notation. Please see How JAXBase Deals with Arrays for more information.

If you give an nElementNumber that indicates a position outside of the array, an error will be generated.

See Also

ADEL() Function
ADIR() Function
AELEMENT() Function
AFIELDS() Function
AINS() Function
ALen() Function
ASCAN() Function
ASORT() Function
DIMENSION Command
DISPLAY MEMORY Command
How JAXBase Deals with Arrays

DRAFT

AT()

AT(cSearchExpression, cExpressionSearched [, nOccurrence])

Remarks

Search a character expression or memo field for the occurrence of another character expression.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nOccurrence

Which occurrence to return. If omitted, 1 is assumed. A value less than 1 always returns 0.

Example

*** Demonstrate ASCAN()**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

aStr="FOUR SCORE"

? AT("0",aStr) && Returns 2

? AT("0",aStr,2) && Returns 8

? AT("s",aStr) && Returns 0

Return Value

Numeric

Note

AT() is case sensitive. Use ATC to perform a case insensitive search.

If the indicated cSearchExpression or occurrence of same is not found then 0 is returned.

The value returned indicates the starting position of the first character of the matched cSearchExpression.

See Also

AT_C() Function
ATCC() Function
ATLINE() Function
LEFT() Function
RAT() Function
RATLINE() Function
RIGHT() Function
SUBSTR() Function
SUBSTRC() Function
STREXTRACT() Function
\$ Operator
OCCURS() Function
INLIST() Function

DRAFT

AT_C()

AT_C(cSearchExpression, cExpressionSearched [, nOccurrence])

Status

Double-byte string handling is slated for Version 2

DRAFT

ATC()

ATC(cSearchExpression, cExpressionToSearch, nFlags)

Status

Version 0.6

Remarks

Search a character expression or memo field for the occurrence of another character expression and returns how many occurrences were found.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nFlags

If omitted, the default is 0, which means a case sensitive partial-word match

Value (Additive)	Description
1	Case insensitive search
2	Whole word only

Example

```

* Demonstrate ASCAN()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE

```

aStr="FOUR SCORE"

? AT("O",aStr) && Returns 2

? AT("o",aStr,2) && Returns 8

? AT("s",aStr) && Returns 6

Return Value

Numeric

Note

ATC() is case insensitive. Use AT() to perform a case sensitive search.

If the indicated cSearchExpression or occurrence of same is not found, then 0 is returned.

The value returned indicates the starting position of the first character of the matched cSearchExpression.

See Also

AT() Function

RAT() Function

DRAFT

See Also

AT() Function
AT_C() Function
ATLINE() Function
LEFT() Function
RAT() Function
RATLINE() Function
RIGHT() Function
SUBSTR() Function
SUBSTRC() Function
STREXTRACT() Function
\$ Operator
OCCURS() Function
INLIST() Function

DRAFT

ATCC()

ATCC(cSearchExpression, cExpressionSearched [, nOccurrence])

Status

Double-byte string handling is slated for Version 2

DRAFT

ATCLINE()

ATCLINE(cSearchExpression, cExpressionToSearch, [,nOccurrence] [, cParseCharacter])

Status

Version 2.0

Remarks

Search a double-byte expression for a matching double-byte expression returning which line it was found.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nOccurrence

Which occurrence to return. If omitted, 1 is assumed. A value less than 1 always returns 0.

cParseCharacter

The character to use to terminate a line, such as CHR(13). If omitted, then the SET MEMOWIDTH value is used. If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

Example

```
* Demonstrate ATCLINE()  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE
```

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);  
      +"on this continent, a new nation, conceived in liberty, and"+CHR(13);  
+"dedicated to the proposition that all men are created equal."+CHR(13);  
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);  
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);  
+"long endure."
```

```
? ATCLINE("and", lcString,1,chr(13))      && Returns 1  
? ATCLINE("and", lcString,2,chr(13))      && Returns 2  
? ATCLINE("now", lcString,1,chr(13))      && Returns 4
```

Return Value

Numeric

Note

ATCLINE() is a case insensitive search.

If the search is successful, the line where the match was found is returned, otherwise 0 is returned.

If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

See Also

\$ Operator

AT() Function

ATC() Function

ATLINE() Function

INLIST() Function

LEFT() Function

MLINE() Function

OCCURS() Function

RAT() Function

RATLINE() Function

RIGHT() Function

SUBSTR() Function

SET MEMOWIDTH Command

ATLINE()

ATLINE(cSearchExpression, cExpressionToSearch, [,nOccurrence] [, cParseCharacter])

Status

Version 0.6

Remarks

Search an expression for a matching expression returning which line it was found.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nOccurrence

Which occurrence to return. If omitted, 1 is assumed. A value less than 1 always returns 0.

cParseCharacter

The character to use to terminate a line, such as CHR(13). If omitted, then the SET MEMOWIDTH value is used. If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

Example

```
* Demonstrate ATLINE()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
```

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
        +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

```
? ATLINE("and", lcString,1,chr(13))    && Returns 1
? ATLINE("and", lcString,2,chr(13))    && Returns 2
? ATLINE("now", lcString,1,chr(13))    && Returns 0
```

Return Value

Numeric

Note

ATLINE() is a case sensitive search.

If the search is successful, the line where the match was found is returned, otherwise 0 is returned.

If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

See Also

\$ Operator

AT() Function

ATC() Function

ATCLINE() Function

INLIST() Function

LEFT() Function

MLINE() Function

OCCURS() Function

RAT() Function

RATLINE() Function

RIGHT() Function

SUBSTR() Function

SET MEMOWIDTH Command

ATN2()

ATN2(nYCoordinate, nXCoordinate)

Remarks

Returns the arc tangent in all four quadrants from specified values.

Parameters

nYCoordinate

Numeric expression of the Y coordinate.

nXCoordinate

Numeric expression of the X coordinate.

Example

```

* Demonstrate ASCAN()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? PI()           && Displays 3.14
? ATN(0, -1)     && Displays 3.14

gnX=COS(PI())
gnY=SIN(PI())

? ATN2(gnY, gnX) && Displays 3.14
? ATN2(gnY, gnX)/PI() && Displays 1

```

Return Value

Numeric

Note

ATN2() returns the angle in radians between the line $y=0$ and the line connecting the given coordinates and the origin (0,0). Use RTOD() to convert to degrees.

ATN() returns a value between $\pi/2$ and $-\pi/2$.

See Also

ATAN() Function

RTOD() Function

SET DECIMALS Command

TAN() Function

ATOKENS()

ATOKENS(ArrayName, cStringToSearch [, cLeftBound, cRightBound [,lIncludeBounds]])

Status

Slated for Version 0.6

Remarks

Returns a list of tokens that appear between two different bounding character strings.

Parameters

ArrayName

Name of array to place the list of tokens.

cStringToSearch

Character based expression or field to search.

cLeftBound

Character expression containing the left boundary. Default is "<<".

cRightBound

Character expression containing the right boundary. Default is ">>".

lIncludeBounds

Logical flag indicating whether to include the bounding strings in the results. Default is true (.T.).

Example

*** Demonstrate ATOKENS**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

TEXT

Dear <<Name>>,

**I sent you a bill for services rendered on <<Date1>> for an
Amount of <<Balance>> and have yet to receive payment. It
Is now <<days>> days overdue.**

Please bring your account up to date by <<Date2>>.

Thank you,

John Smith's Services

ENDTEXT

? ATOKENS(aToks)

DISPLAY MEMORY LIKE aToks

Return Value

Numeric

Note

ATOKENS() returns the number of elements returned in the array.

This command is helpful for creating your own mail merge logic as a preprocessor or complete replacement to the built in JAXBase features.

If the array name does not exist, it will be created. If the array name exists, it will be overwritten with the corresponding number of elements. If there are no results, the array name is not created or modified.

The character expressions cLeftBound and cRightBound can be any number of characters, but they must be different and ATOKENS() uses them as case insensitive boundaries.

cStringToSerch is unaffected by ATOKENS().

See Also

STREXTRACT() Function

TEXT / ENDTEXT Command

Merging Text in JAXBase

AUSED()

AUSED(ArrayName [, cTableName] [, nDataSessionNumber] [, lSorted])

Status

Slated for Version 0.6

Remarks

Places table aliases, table names, and work areas for a data session into an array. Returns the number of rows created.

Parameters

ArrayName

Name of the array that will hold the returned results.

cTableName

Character expression for the table name to use in the search.

Valid expressions are file name stem, and a Fully Qualified File Name (FQFN).

nDataSessionNumber

Numeric expression for the data session to interrogate. 0 or a negative number represents the current data session.

nSorted

Value	Description
0	Lists work areas in order they were opened
1	Lists work areas in order (1 – n)
2	Lists work areas in order of last update

Example

```
* Demonstrate AUSED()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
SELECT FROM (_JAXFolder+"data\counties") WHERE State="CT" ALIAS CT
SELECT FROM (_JAXFolder+"data\counties") WHERE State="VA" ALIAS VA
SELECT FROM (_JAXFolder+"data\counties") WHERE State="CT" ALIAS SC
USE (_JAXFolder+"data\phonebook")
```

```
AUSED(laMatches,"counties")
DISPLAY MEMORY LIKE laMatches
```


Return Value

Numeric

Note

If the array name does not exist, it will be created. If the array name already exists, it will be overwritten and the corresponding number of rows created, or 0 if none were created. If no rows are created, the array will not be created, and if the array name already exists, it will not be changed.

AUSED returns an array with a row for each match where the first column is the table alias, and the second column is the work area it is in.

If a FQFN is given, then only tables that are a match to that exact FQFN are returned. If just a file stem, then any match for files or SQL table names are returned.

AUSED returns an array with the following columns.

Column	Description
1	Alias name
2	Work area number
3	Data session number
4	Database name or empty string
5	Table name (If a DBF, returns the fully qualified file name) or an empty string if not attached to a specific table.
6	Sortable encoded string indicating the date and time the table was opened*
7	Sortable encoded string indicating the date and time the table was last updated*

* The sortable encoded string representing date and time is unique for each run-time session across all threads in the application.

See Also

ALIAS() Function

JUSTSTEM() Function

SYSID() Function

SET DATASESSION Command

USE Command

AVARIABLE()

AVARIABLE(ArrayName, cSkeleton, tAllScopes)

WARNING

AVARIABLE() is under development, and this page may undergo changes before Version 1.0 is released. Full support is slated for Version 2.

Remarks

Returns an array listing memory objects in a fashion like the DISPLAY MEMORY Command

Parameters

ArrayName

cSkeleton

tAllScopes

If True (.T.), retrieves variables from all levels and all scopes. If False (.F.), (default) then returns only variables that are in scope.

Note

AVARIABLE() returns by value, so if a large amount of data is selected, then a large amount of memory is used. It is better to limit the number of variables returned.

The array returned by AVARIABLE() is as follows.

Column	Description
1	Variable name
2	Variable Type
3	Private, Local, Public
4	Row (0 if not an array)
5	Column (0 if not an array)
6	Value
7	Program
8	Level - 0 if tAllScopes is False (.F.)

BETWEEN()

BETWEEN(eTestValue, eLowValue, eHighValue)

Remarks

Tests whether eTestValue is between eLowValue and eHighValue, inclusive. Returns a logical true (.T.) or false (.F.).

Parameters

eTestValue

An expression representing the value to test.

eLowValue

The acceptable low end of the range to check.

eHighValue

The acceptable high end of the range to check.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** Returns .T.**

? BETWEEN(5,3,10)

? BETWEEN("C","A","Z")

? BETWEEN({01/15/2023},{10/01/2022},{5/16/2023 13:01:00})

*** Returns .F.**

? BETWEEN(5,6,10)

? BETWEEN("C","a","z")

Return Value

Logical true (.T.) if the test succeeds, otherwise logical false (.F.)

Note

All values must be of the same data type.

Dates and DateTimes can be intermingled as a DATE is actually stored as a DATETIME with the time portion set to 00:00:00.

See Also

INLIST() Function

MAX() Function

MIN() Function

BINDEVENT()

BINDEVENT(oEventSource, cEvent, oEventHandler, cDelegate [, nFlags])

BINDEVENT(hWnd | 0, nMessage, oEventHandler, cDelegate [, nFlags])

Remarks

Status

Slated for Version 2

DRAFT

BINTOC()

BINTOC(nExpression [, eFlags])

Status

Slated for Version 0.6

Remarks

Converts a numeric value to a binary character string.

Parameters

nExpression

Numeric expression to convert.

cFlags

Expressed as a 1 to 4 byte string the first byte is 1, 2, 4, or 8, indicating length of the character string returned. The second character is I indicating integer, D for decimal, and U for unsigned integer. An integer can be any length, but a Decimal is only 4 bytes (Float) or 8 bytes (Currency/Double). The optional third character will be R indicating the string is to be reversed.

If omitted, the result will be a 4 character, signed integer representation ("4I"). If the string is just a single character 1,2,4, or 8 then a signed integer value representation of nExprssion is returned.

If the value is larger than the specified results, an error is raised. The following table indicates expression ranges.

1	-127 to 127 or 0 to 255 if unsigned
2	-32,768 to 32,767 or 0 to 65,535 if unsigned
4	Approximately 1.5 E - 45 to 3.4 E + 38 for a decimal value or -2,147,486,648 to 2,187,483,647 for signed integer or 0 to 4,294,967,296 for unsigned integer
8	Approximately 2.2250738 E – 308 to 1.7976931 E + 308 for a decimal value or -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 for signed integer or 0 to 18,446,744,073,709,551,615 for unsigned integer

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? BINTOC(1, "1")
? BINTOC(255, "1S")
? BINTOC(124.06, "4F")
? BINTOC(124.06, "8F")
```

Return Value

Character

Note

When dealing with large numbers (4-byte Float, 8-byte Double, or Integers) you will lose precision when it returns scientific notation values.

JAXBase will protect integer values up to 72,057,594,037,927,936.

JAXBase will protect precision of 8-byte decimal values in the range of -549,755,813,999.9999 to 549,755,813,999.9999 and outside of that range it is subject to exponential notation and loss of information. Greater precision to the right of the decimal point reduces precision on the left.

See Also

CTOBIN() Function
 CREATEBINARY() Function
 Double Field Type
 Float Field Type
 Integer Field Type

BITAND()

BITAND(nExpression1, nExpression2)

Remarks

Performs a bitwise AND operation on two integer values and returns the result.

Parameters

nExpression1, nExpression2

Integer values used in the BITAND expression.

Example

```
* Demonstrate BITAND()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
A=43      && 00101011
C=226     && 11100010
```

```
? BITAND(A,C) && Returns 34
```

Return Value

Numeric

Note

If either numeric expression is not an integer, it will be converted.

The following table shows the result of a bitwise AND operation between two bits

Bit 1	Bit 2	AND Result
0	0	0
0	1	0
1	0	0
1	1	1

See Also

BITCLEAR() Function
 BITLSHIFT() Function
 BITNOT() Function
 BITOR() Function
 BITRSHIFT() Function
 BITSET() Function
 BITTEST() Function
 BITXOR() Function

BITCLEAR()

BITCLEAR(nExpression1, nExpression2)

Remarks

Clears the bit position nNumericExpression2 in the integer value of nNumericExpression1.

Parameters

nExpression1

Numeric expression that will have a bit cleared.

nExpression2

Numeric expression that indicates which bit is to be cleared.

Example

```
* Demonstrate BITCLEAR()  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
A=255
```

```
B=2      && 0 is first bit position
```

```
? BITCLEAR(A,B) && Returns 251
```

Return Value

Numeric

Note

BITCLEAR() operations will only support a nNumericExpression2 up to 31
If either numeric expression is not an integer, it will be converted.

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITLSHIFT()

BITLSHIFT(nExpression1, nExpression2)

Remarks

Moves the bits in nExpression1 left by nExpression2 positions.

Parameters

nExpression1

Numeric Expression to alter.

nExpression2

Numeric expression indicating how many bits to move.

Example

* Demonstrate BITLSHIFT()

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

A=255

B=2 && 0 is first bit position

? BITLSHIFT(A,B) && Returns 1020

Return Value

Numeric

Note

BITLSHIFT() operations will only support a nNumericExpression2 up to 31
If either numeric expression is not an integer, it will be converted.

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITNOT()

BITNOT(nExpression)

Remarks

Performs a bitwise NOT operation on the nNumericExpression.

Parameters

nNumericExpression

Numeric expression used in the NOT operation.

Example

*** Demonstrate BITNOT()**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

A=15

? BITNOT(A,B) && Returns -16

Return Value

Numeric

Note

If nNumericExpression is not an integer value, it will be converted.

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITOR()

BITOR(nExpression1, nExpression2)

Remarks

Perform a bitwise inclusive OR operation on two values.

Parameters

nExpression1, nExpression2

Numeric expressions to perform the OR operation.

Example

*** Demonstrate BITOR()**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

A=15

B=164

? BITNOT(A,B) && Returns 175

Return Value

Numeric

Note

If either numeric expressions are not integers, they will be converted.

The following table shows the result of a bitwise OR operation between two bits

Bit 1	Bit 2	OR Result
0	0	0
0	1	1
1	0	1
1	1	1

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITRSHIFT()

BITRSHIFT(nExpression1, nExpression2)

Remarks

Moves the bits in nExpression1 right by nExpression2 positions.

Parameters

nExpression1

Numeric Expression to alter.

nExpression2

Numeric expression indicating how many bits to move.

Example

* Demonstrate BITLSHIFT()

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

A=255

B=2 && 0 is first bit position

? BITRSHIFT(A,B) && Returns 63

Return Value

Numeric

Note

BITRSHIFT() operations will only support a nNumericExpression2 up to 31
If either numeric expression is not an integer, it will be converted.

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITSET()

BITSET(nExpression1, nExpression2)

Remarks

Sets a bit to 1 in a value.

Parameters

nExpression1

Value to alter.

nExpression2

Expression of bit to set to 1.

Example

```
* Demonstrate BITSET
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

A=15

B=4

? BITNOT(A,B) && Returns 31

Return Value

Numeric

Note

If either numeric expression is not an integer, it will be converted.

Valid bit values for nExpression2 are 0 to 31.

See Also

BITCLEAR() Function

BITLSHIFT() Function

BITNOT() Function

BITOR() Function

BITRSHIFT() Function

BITSET() Function

BITTEST() Function

BITXOR() Function

BITTEST()

BITTEST(nExpression1, nExpression2)

Remarks

Returns a logical True (.T.) value if the specified bit is set.

Parameters

nExpression1

Numeric expression to test.

nExpression2

Numeric expression of bit to test.

Example

```
* Demonstrate BITTEST()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
A=15
```

```
? BITTEST(A,0)    && Returns .T.
? BITTEST(A,1)    && Returns .T.
? BITTEST(A,2)    && Returns .T.
? BITTEST(A,3)    && Returns .T.
? BITTEST(A,4)    && Returns .F.
```

Return Value

Numeric

Note

If either numeric expression is not an integer, it will be converted.
The valid bit range for nExpression2 is 0 to 31.

See Also

BITCLEAR() Function
BITLSHIFT() Function
BITNOT() Function
BITOR() Function
BITRSHIFT() Function
BITSET() Function
BITTEST() Function
BITXOR() Function

BITXOR()

BITXOR(nExpression1, nExpression2)

Remarks

Perform a bit-wise exclusive OR operation on two values.

Parameters

nExpression1, nExpression2

Two values with which to perform the operation.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? BITXOR(255,15)    && Returns 240
? BITXOR(240,15)    && Returns 255
```

Return Value

Numeric

Note

If either numeric expression is not an integer, it will be converted.

The following table shows the result of a bitwise XOR operation between two bits

Bit 1	Bit 2	AND Result
0	0	0
0	1	1
1	0	1
1	1	0

See Also

BITCLEAR() Function
 BITLSHIFT() Function
 BITNOT() Function
 BITOR() Function
 BITRSHIFT() Function
 BITSET() Function
 BITTEST() Function
 BITXOR() Function

BOF()

BOF([nWorkArea | cTableAlias])

Remarks

Returns logical .T. when record pointer is at the beginning of the table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```

* Demonstrate BOF()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

```

```

USE (_JAXFolder+"data\counties")
INDEX ON NAME TO COUNTIES_NAME

```

```

GOTO BOTTOM
? RECNO(),BOF()

```

```

WHILE NOT BOF()
    Skip -1
ENDDO

```

```

? RECNO(),BOF()

```

Return Value

Logical

Note

BOF() returns true (.T.) if you try to move the record pointer above the top record. If an index is in effect, the top record may not be the first record in the table. BOF() accurately represents the new indexed top of file.

See Also

EOF() Function

FEOF() Function

CANDIDATE()

CANDIDATE(nIndexNumber [,nWorkArea] cAlias])

Status

Slated for Version 1.0

Remarks

Returns a value of true (.T.) if the index is a candidate index.

Parameters

nIndexNumber

Numeric expression indicating the index to use. Any index in JAXBase can be marked as candidate indexes.

nWorkArea

Numeric expression indicating the work area.

cAlias

Character expression indicating the table alias to query.

Example

```
* Demonstrate BOF()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Logical

Note

If workarea and table alias are omitted, the current workarea is used.

If nIndexNumber is less than 1, an error is generated. If nIndexNumber goes beyond the scope of open indexes, it will return false (.F.).

See Also

INDEX Command

CAPSLOCK()

CAPSLOCK(lExpression)

Status

Slated for Version 1.0

Remarks

Returns current CAPS LOCK mode and optionally sets it.

Parameters

lExpression

Logical expression that turns CAPS LOCK on when true (.T.) and off when false (.F.).

Example

```
* Demonstrate CAPSLOCK()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
glOldCapsLock=CAPSLOCK()
```

```
glNewCapsLock=CAPSLOCK(!glOldCapsLock)
```

```
? glOldCapsLock, glNewCapsLock
```

Return Value

Logical

Note

If you do not include an argument with CAPSLOCK(), it will simply return the CAPS LOCK state.

See Also

INSMODE() Function

NUMLOCK() Function

CDOW()

CDOW(dExpression | tExpression)

Remarks

Returns day of week from the provided Date or DateTime expression.

Parameters

dExpression

Date expression that is used to return the day of the week.

tExpression

DateTime expression that is used to return the day of the week.

Example

```
* Demonstrate DOW()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
SET STRICT DATE TO 0  
ACTIVATE CONSOLE  
CLEAR
```

```
? DOW({09/05/2025}) && Displays Friday
```

Return Value

Character

Note

CDOW() returns the proper name of the day of the week in question.

See Also

DATE() Function

DATETIME() Function

DAY() Function

DOW() Function

SET FDOW Command

SET FWEEK Command

CEILING()

CEILING(nExpression)

Remarks

Returns the next highest integer that is greater than or equal to nExpression.

Parameters

nExpression

Numeric expression used by CEILING() to get the next highest integer.

Example

*** Demonstrate CEILING()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? CEILING(10) && Displays 10

? CEILING(10.01) && Displays 11

? CEILING(-10.1) && Displays -10

Return Value

Numeric

Note

CEILING() rounds a number with a fractional decimal component to the next highest integer.

See Also

FLOOR() Function

ROUND() Function

Integer Field Type

CHR()

CHD(nExpression)

Remarks

Returns the character associated with the numeric expression.

Parameters

nExpression

Numeric expression in the range of 0 to 255.

Example

```
* Demonstrate CHR()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? CHR(49) && Displays 1  
? CHR(65) && Displays A  
? CHR(97) && Displays a
```

Return Value

Character

Note

CHR() returns a single character representing the corresponding code from the codepage table. Prior to Version 2.0, JAXBase will use the current default codepage.

See Also

ASC() Function

INKEY() Function

SET CODEPAGE Command

CHRTRAN()

CHRTRAN(cExpression, cSearchExpression, cReplacementExpression)

Remarks

Replaces letters in cExpression based on cSearchExpression and cReplacementExpression.

Parameters

cExpression

Character expression that will be modified by the CHRTRAN() function.

cSearchExpression

Character expression holding the characters that will be modified.

cReplacementExpression

Character expression containing the characters that will replace the cSearchExpression characters in the cExpression.

Example

*** Demonstrate CHRTRAN()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? CHRTRAN("Moby Dick","oi","0!") && Displays M0by D!ck

? CHRTRAN("> PASSWORD <"," A0","") && Displays >PSSWRD<

Return Value

Character

Note

cSearchExpression and cReplacementExpression are matched up character by character from left to right. If cReplacementExpression is not as long as cSearchExpression, then the characters in cSearchExpression that have no match are removed from cExpression.

See Also

CHRTRANC() Function

STRTRAN() Function

Creating Character Expressions

CHRTRANC()

CHRTRAN(cExpression, cSearchExpression, cReplacementExpression)

Status

Slated for Version 2.0

DRAFT

CMONTH()

CMONTH(dExpression | tExpression)

Remarks

Returns the name of the month for the specified date.

Parameters

dExpression

Date expression used to return the name of the month.

tExpression

DateTime expression used to return the name of the month.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET STRICTDATE TO 0

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? CMONTH({01/01/2025}) && Displays January

? CMONTH({07/01/2025}) && Displays July

Return Value

Chracter

Note

CMONTH() returns the name of the month as a string in proper noun format.

See Also

DATE() Function

DATETIME() Function

DMY() Function

MDY() Function

MONTH() Function

COMMIT()

COMMIT()

Status

Slated for Version 0.8

Remarks

Attempts to commit the current SQL database transaction. Returns a logical true (.T.) if successful.

Return Value

Logical

Note

If there is no open transaction in the current workarea, an error will be generated. COMMIT() will work against a SQL table that was opened under an OPEN DATABASE or using a SQL class.

COMMIT() returns logical true (.T.) if successful and false (.F.) if not.

COMMIT() may encounter an error during the process, but it is recorded rather than interrupting the program flow. You can use the AERROR() or the SQL class Error() function to discover the reason for the failure.

Under most circumstances, a failed commit can be tried again if the problem is fixed. Otherwise, you need to ROLLBACK() the transaction.

If an ENDTRANSACTION is encountered and the transaction is still open, a COMMIT() will be attempted and if it fails, an error is generated.

See Also

BEGIN TRANSACTION Command

ENDTRANSACTION Command

OPEN DATABASE Command

ROLLBACK()

SQL Class

COMPILE()

COMPILE(cCodeBlock)

Status

Slated for Version 0.6

Remarks

Compiles JAXBase source code into object code that can be placed into a class method or event using the MEUPDATE method in most user classes.

Parameters

cCodeBlock

A character string that contains valid JAXBase commands.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
aStr=COMPILE("DO TEST1")
```

```
FOR I=1 to len(aStr)  
  ?? ASC(SUBSTR(aStr,I,1))  
ENDFOR
```

Note

JAXBase allows on-the-fly updates to most events and methods in most user classes. This is a drastic departure from most xBase dialects and is one of the many powerful features of JAXBase.

See Also

COMPILE Command

MEUPDATE method

COS()

COS(nExpression)

Remarks

Returns the cosine of a numeric expression.

Parameters

nExpression

Numeric expression used by the COS() function.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? COS(0) && Displays 1.00

? COS(PI()) && Displays -1.00

? COS(45) && Displays 0.53

? COS(1024) && Displays 0.99

Return Value

Numeric

Note

COS() returns radians. Use DTOR() to convert an angle from degrees to radians and RTOD() to convert radians to degrees.

The number of decimal places returned is dictated by SET DECIMALS. The value COS() returns is between -1 and +1 inclusive.

See Also

ACOS() Function

DTOR() Function

RTOD() Function

SET DECIMALS Command

SIN() Function

TAN() Function

CPCONVERT()

Status

Slated for Version 2.0

DRAFT

CPCURRENT()

Status

Slated for Version 2.0

DRAFT

CPDBF()

Status

Slated for Version 2.0

DRAFT

CRC32()

CRC32(cExpression)

Status

Slated for Version 0.8

Remarks

Returns the CRC32 checksum for the provided expression.

DRAFT

CREATEBINARY()

Status

Slated for Version 2.0

DRAFT

CREATEOBJECT()

CREATEOBJECT(cClassName [,eParametersArray])

Remarks

Creates an object from a built in or user defined class definition.

Parameters

cClassName

Name of the built in JAXBase class or a user defined class.

eParametersArray

A two-dimensional array with the first column holding the property name and the second column holding the value to place into the property.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
aForm=CREATEOBJECT("FORM")
? aForm.Name      && Displays FORM1
```

Return Value

Class Object

Note

Properties names in the array that do not exist will attempt to be added to the class. If a property cannot be added or updated, an error will be generated.

The value of empty string will default to the empty value of that property.

See Also

DEFINE CLASS Command
GETOBJECT() Function
NEWOBJECT() Function
RELEASE Command
SET CLASSLIB Command

CTOBIN()

CTOBIN(cExpression [,cFlags])

Remarks

Converts a binary encoded number into a numeric value.

Parameters

cExpression

The character expression to convert.

cFlags

If the character expression is an 8 byte expression, you can control what is returned using the following:

cFlags is used to control what kind of value is returned where the first character is 1, 2, 4, or 8, indicating the type of value returned as shown in the table below. The second character is I indicating integer, D for decimal, and U for unsigned integer. An integer can be any length, but a Decimal is only 4 bytes (Float) or 8 bytes (Currency/Double). The optional third character will be R indicating the original string is to be reversed before conversion.

If omitted, the result will be a signed integer representation ("4I").

If the original string is not 8 bytes in length, then cFlags is ignored.

1	-127 to 127 or 0 to 255 if unsigned
2	-32,768 to 32,767 or 0 to 65,535 if unsigned
4	Approximately 1.5 E - 45 to 3.4 E + 38 for a decimal value or -2,147,486,648 to 2,187,483,647 for signed integer or 0 to 4,294,967,296 for unsigned integer
8	Approximately 2.2250738 E – 308 to 1.7976931 E + 308 for a decimal value or -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 for signed integer or 0 to 18,446,744,073,709,551,615 for unsigned integer

Example

```
* Demonstrate CTOBIN()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
aStr=BINTOC(1024)  
? CTOBIN(aStr)
```

```
aStr=BINTOC(8675309, "8I")  
? CTOBIN(aStr, "4I")
```

```
aStr=BINTOC(10, "8D")  
? CTOBIN(astr, "1I")
```

Return Value

Numeric

Note

If cExpression or cFlags have invalid information, an error is generated.

See Also

BINTOC() Function

CTOD()

CTOD(cExpression)

Remarks

Convert a character date expression to a Date data type.

Parameters

cExpression

Character expression in a recognized Date format. See How JAXBase Handles Dates.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

SET STRICTDATE TO 0

ACTIVATE CONSOLE

CLEAR

cDate="09/05/2025"

? CTOD(cDate)

? CMONTH(CTOD(cDate))

? CTOD("09-05-2025")

? CTOD("2025-09-05")

SET DATE TO BRITISH

? CMONTH(CTOD("05-09-2025"))

Return Value

Date

Note

If an invalid date expression is encountered, a blank date value { / / } is returned.

See Also

DATE() Function

DATETIME() Function

CMONTH() Function

SET DATE Command

SET STRICTDATE Command

CTOT()

CTOT(cExpression)

Remarks

Convert a character date expression to a DateTime data type.

Parameters

cExpression

Character expression in a recognized DateTime format. See How JAXBase Handles Dates.

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
SET STRICTDATE TO 0
SET HOURS TO 12
SET DATE TO AMERICAN
ACTIVATE CONSOLE
CLEAR

cDate="09/05/2025"
? CTOT(cDate)                && Displays 09-05-2025 12:00:00 AM
? CMONTH(CTOT(cDate))        && Displays September
? CTOT("09-05-2025 00:14:16") && Displays 09-05-2025 12:14:16 AM
? CTOT("2025-09-05 14:16:18") && Displays 09-05-2025 02:16:18 PM

SET DATE TO BRITISH
? CMONTH(CTOT("05-09-2025 12:05:00 AM")) && Displays September

```

Return Value

Date

Note

If an invalid date expression is encountered, a blank date value {/::} is returned.

Times are expressed in 12 or 24 hour format, depending on the SET TIME command. However, if a DateTime expression does not end in AM or PM, the time portion is automatically considered to be 24 hour (military) format.

See Also

DATE() Function

DATETIME() Function

CMONTH() Function

SET DATE Command

SET STRICTDATE Command

CURSORGETPROP()

Status

Slated for Version 2.0

DRAFT

CURSORSETPROP()

Status

Slated for Version 2.0

DRAFT

CURSORTOJSON()

CURSORTOJSON(nWorkArea | cAlias, cFile)

Status

Slated for Version 2.0

Remarks

Writes the records of a table or cursor to a JSON file.

Parameters

nWorkArea

Numeric expression that indicates which work area to use.

cAlias

Character expression that indicates which alias to use.

cFile

Character expression that holds the name of the file to create.

Return Value

Numeric

Note

CURSORTOJSON() returns the number of records written. If an error occurs during the process, the JSON file will not be created.

To limit the records sent to the JSON file, set a FILTER.

See Also

CURSORTOXML() Function

JSONTOCURSOR() Function

SET FILTER Command

XMLTOCURSOR() Function

CURSORTOXML()

CURSORTOXML(nWorkArea |cAlias, cFile[, nFlags])

Status

Slated for Version 2.0

Remarks

Writes the records of a table or cursor to XML.

Parameters

nWorkArea

Numeric expression that indicates which work area to use.

cAlias

Character expression that indicates which alias to use.

cFile

Character expression that holds the name of the file to create.

nFlags

nFlag	Description
0	Write the data to the XML file (default)
1	Write the schema to the XML file
2	Write the schema and data to the XML file

Return Value

Numeric

Note

CURSORTOXML() returns the number of records written. If an error occurs during the process, the XML file will not be created.

To limit the records sent to the XML file, set a FILTER.

See Also

CURSORTOJSON() Function

JSONTOCURSOR() Function

SET FILTER Command

XMLTOCURSOR() Function

CURVAL()

CURREC(nRecNo [, lRecStatus] [, nWorkArea | cAlias])

Status

Slated for Version 0.6

Remarks

Returns a record object directly from the current record in a JAXBase table.

Parameters

nRecNo

Numeric expression of the record to fetch. A record number of 0 indicates the current record.

lRecStatus

If set to a value of true (.T.), returns the following properties in the Record Object.

Name	Data Type	Description
_CURIDX	Integer	Value of the index in control. Use NDX() to get the name.
_DELETED	Logical	A value of true (.T.) indicates the record is marked for deletion
_RECNO	Integer	Absolute record number

nWorkArea

Numeric expression that indicates which work area to use.

cAlias

Character expression that indicates which alias to use.

Example

```
* Demonstrate CURVAL()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
GOTO TOP
```

```
SCATTER NAME oRec1          && Fetch the values of record 1
oRec2=CURVAL(100,.T.)       && Fetch the values of record 100
DISP MEMO LIKE orec*        && Display the two records
```

Return Value

JAXBase Record Object

Note

A JAXBase Record Object has properties named after the fields and reflect the current values based on the currently selected record. If the record is past the end of the file, an error is generated.

If lRecStatus is set to true (.T.) and there are fieldnames that match the property names, an error will be generated and an empty JAXBase Record Object will be returned.

If the work area or alias is omitted, the current workarea is used.

This command is typically used to quickly compare the current value of fields in a table to see if they have been altered in a multiuser environment.

nRecNo references is the absolute record number in the table and is unaffected by indexes. The value of 0 indicates the current record under the record pointer.

The CURVAL() command does not move the record pointer, so if the record pointer is set to record 10 and nRecNo is 100, it will fetch the values from record 100 without moving the record pointer.

See Also

EMPTY Class
EOF() Function
GOTO Command
SCATTER Command

DATE()

DATE([nYear, nMonth, nDay])

Remarks

Returns the operating system Date value or creates a Y2K compliant Date value.

Parameters

nYear

A numeric value between 1 and 9999.

nMonth

A numeric value between 1 and 12 for month.

nDay

A numeric value for the date between 1 and 31.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET DATE TO AMERICAN
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? DATE()      && Returns today's date
? Date(2005,12,31)  && Returns 12/31/2005
```

Return Value

Date value type

Note

If the combined year, month, and day values do not combine into a valid date (example: 02/31/2025) then a blank date ({ / / }) is returned.

See Also

CTOD() Function

DATETIME() Function

DTOC() Function

SET CENTURY Command

SET DATE Command

DATETIME()

DATETIME([nYear, nMonth, nDay [,nHours [,nMinutes , nSeconds]]]])

Remarks

Returns the operating system DateTime value or creates a Y2K compliant DateTime value.

Parameters

nYear

A numeric expression that evaluates to 1 and 9999 representing year.

nMonth

A numeric expression that evaluates to 1 and 12 representing month.

nDay

A numeric expression that evaluates to 1 and 31 representing day of month.

nHours

A numeric expression that evaluates to 0 through 23 representing the hour in military time.

nMinutes

A numeric expression that evaluates to 0 through 59 representing minutes.

nSeconds

A numeric expression that evaluates to 0 through 59 representing seconds.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET DATE TO AMERICAN
SET HOUR TO 12
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? DATETIME()           && Returns current DateTime from the operating system
? DATETIME(2005,12,31,23,59,59) && Returns 12/31/2005 11:59:59 PM
```

Return Value

DateTime value type

Note

If the combined values do not correspond to a valid DateTime value, then a blank DateTime value ({}:::}) is returned.

See Also

CTOT() Function

DATE() Function

DTOT() Function

HOUR() Function

SEC() Function

SECONDS() Function

SET DATE Command

SET HOUR Command

SET SECONDS Command

SET SYSFORMATS Command

TIME() Function

TTOC() Function

TTOD() Function

DRAFT

DAY()

DAY(dExpression | tExpression)

Remarks

Returns the numeric day of the month for a given Date or DateTime value.

Parameters

dExpression

Date expression used to return the name of the month.

tExpression

DateTime expression used to return the name of the month.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? DAY({09/05/2025})

&& Displays 5

? DOW({09/05/2025})

&& Displays 6

? DAY({11/22/1963})

&& Displays 22

? DOW({11/22/1963})

&& Displays 6

? DAY({01-02-2001 00:00:01})

&& Displays 2

? DOW({01-02-2001 00:00:01})

&& Displays 3

Return Value

Numeric

Note

Returns a number from 1 to 31 for a valid date or 0 for an invalid date.

See Also

CDOW() Function

DOW() Function

SET FDOW Command

SET FWEEK Command

DBC()

DBC()

Remarks

Returns the name of the current database.

Example

* Interrogate the JAXCorp database that you installed
* onto your MySQL or SQL Server database server using
* the CreatedSNFile program in the TOOLS folder.

```
CLEAR ALL
CLOSE ALL
CLEAR
```

```
SET NAMING TO LOWER
cConnect=DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx"))
OPEN DATABASE JAXCorp USING cConnect
```

```
? DBC()    && Returns JAXCORP
CLOSE DATABASE
? DBC()    && Return the empty string
```

Return Value

Character

Note

DBC() returns an empty string if there is no current database.

Use SET DATABASE to specify the current database.

See Also

CREATE DATABASE
CLOSE DATABASE
OPEN DATABASE
SET DATABASE
SQL Class

DBF()

DBF([nWorkArea | cAlias])

Remarks

Returns the name of a table open in a specified work area or opened with the specified alias.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

* Interrogate the JAXCorp database that you installed
* onto your MySQL or SQL Server database server using
* the CreatedSNFile program in the TOOLS folder.

```
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
SET NAMING TO LOWER
cConnect=DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx"))
OPEN DATABASE USING cConnect
SELECT 1
SELECT * FROM CUSTOMERS INTO CURSOR Cust
```

```
SELECT 15
USE (_JAXFolder+"data\counties")
```

```
? DBF("cust") && Returns jaxcorp!customers
? DBF(1) && Returns counties
```

Return Value

Character

Note

If no parameter is given, the current work area is assumed.

If no table is open in the specified work area, an empty string is returned.

See Also

FIELD() Function

NDX() Function

SET FULLPATH Command

USE Command

DBGETPROP()

Status

Slated for Version 2.0

DRAFT

DBSETPROP()

Status

Slated for Version 2.0

DRAFT

DBUSED()

DBUSED(cDatabaseName)

Status

Slated for Version 0.8

Remarks

Returns a value of true (.T.) if the specified database is open.

Parameters

cDatabaseName

Character expression of the database to query.

Example

* Interrogate the JAXCorp database that you installed
* onto your MySQL or SQL Server database server using
* the CreatedSNFile program in the TOOLS folder.

CLEAR ALL
CLOSE ALL
CLEAR

SET NAMING TO LOWER

cConnect=DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx"))

OPEN DATABASE USING cConnect

? DBUSED("JAXCORP") && Returns .T.

Return Value

Logical

Note

Returns the value true (.T.) if the database is open, otherwise returns false (.F.).

See Also

CLOSE DATABASE
CREATE DATABASE
OPEN DATABASE
SQL Class

DELETED()

DELETED([nWorkArea | cAlias])

Remarks

Returns whether the current record is deleted.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
GOTO TOP
DELETE
? DELETED()  && Returns .T.
RECALL
? DELETED()  && Returns .F.
```

Return Value

Logical

Note

Only local tables and cursors from tables or SQL tables can be queried with DELETED().

DELETED() returns a value of True (.T.) if the current record is marked for deletion.

See Also

DELETE Command

PACK Command

RECALL Command

SET DELETED Command

DESCENDING()

DESCENDING([cIndexName | nIndexNumber [,nWorkArea | cAlias]])

Status

Slated for Version 0.8

Remarks

Returns a value of true (.T.) if the indicated index or compound index tag is ordered in descending order.

Parameters

cIndexName

Character expression of an open index. 0 for the current selected index.

nIndexNumber

Numeric expression related to the list of open indexes.

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

Return Value

Logical

Note

DESCENDING() returns a value of true (.T) under two conditions.

The selected index was created with the DESCENDING keyword and opened normally.

The selected index with opened with the DESCENDING keyword in a USE, SET INDEX, or SET ORDER command.

If you do not include any parameters, then DESCENDING() looks at the controlling index in the current workarea.

If there is no index currently selected, DESCENDING() returns a value of false (.F.).

See Also

INDEX Command
SET INDEX Command
SET ORDER Command
USE Command

DRAFT

DIFFERENCE()

Status

Slated for Version 2.0

DRAFT

DIRECTORY()

DIRECTORY(cDirectoryName [,nFlags])

Status

Slated for Version 0.6

Remarks

Locates the specified directory.

Parameters

cDirectoryName

Character expression that specifies the name of the directory to check.

nFlags

Numeric expression where:

- 0 (default if omitted) returns false (.F.) if the directory exists but is Hidden or a System folder
- 1 causes DIRECTORY() to return true (.T.) if the directory exists regardless of its attributes

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET NAMING TO NATURAL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

IF "WINDOWS" $ OS
? DIRECTORY("C:\WINDOWS")  && Returns .T. in Windows
ELSE
? DIRECTORY("/root")      && Returns .T. in Linux
ENDIF
```

Return Value

Logical

Note

DIRECTORY() returns true (.T.) if the directory is found, otherwise false.

In Windows, if the directory does not have an absolute path, JAXBase searches the default directory for the provided directory expression.

Linux uses a unified filesystem tree where all drives and directories are listed under the root folder (/). If the folder in Linux does not start with the forward slash (/), then JAXBase searches the default directory for the provided directory expression.

See Also

ADIR() Function
CD Command
FILE() Function
GETDIR() Function
GETFILE() Function
MD Command
SET DEFAULT Command

DISKSPACE()

DISKSPACE(cVolumeName [,nType])

Status

Slated for Version 0.6

Remarks

Returns the number of bytes of the specified type that are available on the default or specified device or volume.

Parameters

cVolumeName

Character expression holding the specified device name or volume. If empty or omitted, returns the number of free bytes for the device or volume that holds the default directory.

nType

Numeric expression holding one of the following values which specify what to return:

Value	Description
1	Total bytes on the device or volume
2	Total free bytes on the device or volume (default)
3	Total amount of free space to the associated user

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? DISKSPACE("",1)
```

Return Value

Numeric

Note

DISKSPACE() number of bytes available. If an error occurs, it returns -1.

See Also

HEADER() Function

RECSIZE() Function

SET DEFAULT Command

DOY()

DOY(dDate | tDate)

Status

Slated for Version 0.6

Remarks

Returns the day of the year starting with 1 as January 1, and December 31 as either 365 or 366 if a leap year.

DRAFT

DYM()

DYM(dExpression | tExpression)

Remarks

Returns a date in the dd-Month-year format.

Parameters

dExpression

Date expression to convert.

tExpression

DateTime expression to convert.

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET CENTURY ON
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

```

```

? DYM(DATETIME())    && Returns current date in dd-Month-yyyy format

```

```

? DYM({09/06/2025}) && Returns 06-September-2025

```

Return Value

Character

Note

Returns the date portion of the expression in the dd-Month-yyyy format if CENTURY is set on.

Returns the date portion of the expression in the dd-Month-yy format if CENTURY is set OFF.

Returns an empty character value if an error occurs.

The month name is not abbreviated.

See Also

DATE() Function

DATETIME() Function

MDY() Function

SET CENTURY Command

SET DATE Command

DODEFAULT()

DODEFAULT([eParameter1 [, eParameter2] ...])

Status

Slated for Version 1.0

Remarks

Executes the parent class event or method of the same name from within a subclass.

Parameters

eParameter1 [,eParameter2] ...

Specifies parameters that are passed to the parent class event or method.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

T=CREATECLASS("TEST1")

? T.ReturnProperty("MYPROPERTY") && Displays 1

RETURN

DEFINE CLASS TEST0 AS COLLECTION

MyProperty=1

PROCEDURE ReturnProperty(cProp)

RETURN EVALUATE("this."+cProp)

ENDPROCEDURE

ENDDDEFINE

DEFINE CLASS TEST1 AS TEST0

MyProperty=2

PROCEDURE ReturnProperty(cProp)

RETURN DODEFAULT(cProp)

ENDPROCEDURE

ENDDDEFINE

Return Value

JAXBase supported data type or object

Note

DODEFAULT() returns the data type value that is sent from the called event or method.

If there is no parent class code to execute, DODEFAULT() returns true (.T.).

See Also

:: Scope Resolution Operator (Slated for Version 2)

ParentClass property

DRAFT

DOW()

DOW(dExpression | tExpression [, nFirstDayOfWeek])

Remarks

Returns a numeric day-of-the-week value from a Date or DateTime expression

Parameters

dExpression

Date expression used to get the DOW() value

tExpression

DateTime expression used to get the DOW() value

nFirstDayOfWeek

Accepted, but slated for Version 2. Currently, Sunday is the first day of the week.

Value	Description
0	Whatever day is set for first-day-of-the-week in the regional settings (default as of Ver 2)
1	Sunday (default prior to Version 2)
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

```

```

? DOW( DATE() )
? DOW( DATETIME() )

```

Return Value

Numeric

See Also

CDOW() Function

DAY() Function

DATE() Function

DATETIME() Function

SET FDOW Command

SET FWEEK Command

WEEK() Function

DRAFT

DRIVETYPE()

DRIVETYPE(cDevice)

Status

Slated for Version 0.6

Remarks

Returns the type of the specified device or volume.

Parameters

cDevice

In Windows, a device is a drive letter while in Linux, a device has a name that follows the convention sdX (example: sd0) or nvmeXnY (example: nvme0n1).

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ADRIVES(aDriveList)
? DRIVETYPE(aDriveList[1])
```

Return Value

Numeric

Note

The values returned by DRIVETYPE() are:

Value	Description
1	No type or unknown
2	Floppy disk
3	Hard disk
4	Removable drive or network driver
5	Optical drive (CD, DVD, Blu-Ray)
6	RAM drive

See Also

ADRIVES() Function

JUSTDRIVE() Function

DTOC()

DTOC(dExpression | tExpression)

Remarks

Returns the Date or DateTime in the regional format.

Parameters

dExpression

Date expression used to return the value.

tExpression

DateTime expression used to return the value.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CENTURY ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET DATE TO AMERICAN

? DTOC(DATE()) && Returns date in mm-dd-yyyy format

SET DATE TO BRITISH

? DTOC(DATE()) && Returns the date in dd-mm-yyyy format

Return Value

Character

Note

The year will be displayed as yy if CENTURY is OFF.

If a DateTime value is provided, only the date portion will be displayed.

See Also

CTOD() Function

SET CENTURY Command

SET DATE Command

TTOC() Function

DTOR()

DTOR(nExpression)

Remarks

Transforms degrees to radians.

Parameters

nExpression

Numeric expression specifying degrees.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? DTOR(40)    && Returns 0.70
? DTOR(400)   && Returns 6.98
```

Return Value

Numeric

Note

The number of decimal points returned is determined by the SET DECIMALS command.

Use RTOD() to turn radians to degrees.

See Also

ACOS() Function
ASIN() Function
ATAN() Function
ATN2() Function
COS() Function
RTOD() Function
SIN() Function
TAN() Function

DTOS()

DTOS(dExpression | tExpression)

Remarks

Returns a date as a string of numbers in year month date format.

Parameters

dExpression

Date expression used to create the return value.

tExpression

DateTime expression used to create the return value.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? DTOS(DATE())    && Returns date portion
? DTOS(DATETIME()) && Returns date portion
```

Return Value

Character

Note

This function is useful for indexing Date or DateTime fields.

The format of the returned string is `yyyymmdd` and is not affected by the SET CENTURY setting.

See Also

DATE() Function

DYM() Function

DATETIME() Function

CTOD() Function

DTOC() Function

DTOT() Function

DTOT()

DTOT(dExpression | tExpression)

Remarks

Turns a Date value into a DateTime value.

Parameters

dExpression

Date expression used to create the DateTime value.

tExpression

DateTime expression used to create the DateTime value.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ltDate=DATE()
? DTOT(ltDate)    && Returns DateTime with time portion set to midnight
```

Return Value

Character

Note

During conversion, the time portion is always set to midnight of the date.

The SET HOURS command determines how the time is displayed (24 for military, 12 for AM/PM) while the SET CENTURY and SET DATE command determines how the date portion is displayed.

See Also

CTOD() Function

DATE() Function

DATETIME() Function

SET DATE Command

SET HOURS Command

SET CENTURY Command

TIME() Function

EMPTY()

EMPTY(eExpression)

Remarks

Returns a logical value if eExpression evaluates to an empty value.

Parameters

eExpression
Expression to test.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? 0,EMPTY(0)
? 1,EMPTY(1)
? "",EMPTY("")
? "HI",EMPTY("HI")
? .F.,EMPTY(.F.)
? .T.,EMPTY(.T.)
```

Return Value

Logical

Note

All JAXBase data types have an empty value and EMPTY() will return true (.T.) if the expression evaluates to an empty value, as described in the following table.

Expression Type	Evaluates to
Binary or General field Varbinary data	0x0 or zero bytes
Character or Memo	Empty string
Currency	0
Date	Value equivalent to CTOD("")
DateTime	Value equivalent to CTOT("")
Double	0
Float	0
General	0x0 or zero bytes
Integer	0
Logical	.F.
Numeric	0

See Also

LEN() Function

TYPE() Function

ISNULL() Function

DRAFT

EOF()

EOF([nWorkArea | cAlias])

Remarks

Returns true if the specified table is at the end of file.

Parameters

Example

```

* Demonstrate EOF()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

```

```

USE (_JAXFolder+"data\counties")
? EOF()

```

```

GOTO BOTTOM
? EOF()
SKIP
? EOF()

```

```

SKIP -1
? EOF()

```

Return Value

Logical

Note

EOF() returns true when the record pointer attempts to go past the last record of the file or indexed order. For instance, when FIND(), LOCATE(), SEEK or SEEK() are unsuccessful (and SET NEAR is OFF).

See Also

BOF()
 GOTO Command
 SET NEAR Command
 SKIP Command

ERROR()

ERROR()

Remarks

Returns the last program error captured by JAXBase. An ON ERROR, TRY...CATCH, or Error method must be active to be able to use the ERROR() function.

Example

```
* Demonstrate ERROR()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR  
  
TRY  
    ERROR 10  
CATCH  
    ? ERROR()  
ENDTRY
```

Return Value

Numeric

Note

ERROR() returns the error number and MESSAGE() returns the error text related to the error.

ERROR() is reset by RETURN or RETRY.

See Also

ON ERROR Command
TRY/CATCH Command
ERROR Method

EVALUATE()

EVALUATE(cExpression)

Status

Slated for Version 0.6

Remarks

Evaluates a character expression.

Parameters

cExpression

Character expression representing an eExpression.

Example

```
* Demonstrate EVALUATE()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
A="2+3"  
? EVALUATE(A) && Returns 5
```

```
A="2>3"  
? EVALUATE(A) && Returns .F.
```

```
B=7  
A="B+2"  
? EVALUATE(A) && Returns 9
```

Return Value

JAXBase data type or object

Note

cExpression must represent a valid JAXBase expression or an error will be generated.

Maximum cExpression length is 255 characters.

EVALUATE() is usually faster than macro substitution, so use it whenever possible.

See Also

TYPE() Function

VARTYPE() Function

EVL()

EVL(eExpression1, eExpression2)

Status

Slated for Version 0.6

Remarks

Returns a non-empty value of two expressions.

Parameters

eExpression1

JAXBase expression to evaluate.

eExpression2

JAXBase expression to return if eExpression1 evaluates to an empty value.

Example

*** Demonstrate EVL()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? EVL("",1) && Returns 1

? EVL(1,7) && Returns 1

Return Value

JAXBase data type or object

Note

eExpression2 does not have to be of the same data type as eExpression1.

If eExpression1 evaluates to an empty value, then eExpression2 is returned.

See Also

EMPTY() Function

EXECSCRIPT()

EXECSCRIPT(cExpression [eParameter1 [,eParameter2, ...]])

Status

Slated for Version 1.0

Remarks

Enables you to run a character expression holding multiple lines of JAXBase code.

Parameters

cExpression

Character expression holding one or more lines of JAXBase source code.

Example

```
* Demonstrate EXECSCRIPT()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
aString="? [HELLO]" + chr(13) + "? [GOODBYE!]"
EXECSCRIPT(aString)
```

Return Value

If the script has a RETURN statement, the value passed back will be returned, otherwise a value of true (.T.) is returned.

Note

Code executed by EXECSCRIPT() runs in its own program level, so local variables in the program with EXECSCRIPT are not accessible by the script being run.

If you pass parameters using EXECSCRIPT() then the first line of executable code must be a parameter statement.

See Also

APARAMETERS Command
LPARAMETERS Command
PARAMETERS Command
ON ERROR Command
RETURN Command
Macro Substitution

EXP()

EXP(nExpression)

Status

Slated for Version 0.6

Remarks

Returns the value of e^x where x is a specified numeric expression.

Parameters

nExpression

Numeric expression specifying the exponent x , in the exponential expression e^x .

Example

*** Demonstrate EXP()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? EXP(0) && Displays 1.00

? EXP(1) && Displays 2.72

? EXP(2) && Displays 7.39

Return Value

Numeric

See Also

LOG() Fuction

LOG10() Function

FCOUNT()

FCOUNT(nWorkArea | cAlias)

Remarks

Returns the number of fields in the specified table or cursor.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate FCOUNT()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
USE (_JAXFolder+"data\counties")  
? FCOUNT()
```

Return Value

Numeric

Note

If you do not give it parameters, FCOUNT() uses the current work area. If a work area number was specified, or no parameter given, and a table is not open then FCOUNT() will return 0. If an alias is provided and does not exist, FCOUNT() will generate an error.

See Also

DBF() Function

FIELD() Function

FSIZE() Function

FDATE()

FDATE(cFileName)

Status

Slated for Version 0.6

Remarks

Returns the last modified DateTime of the specified file.

Parameters

cFileName

A character expression indicating the name of a file.

Example

```
* Demonstrate FDATE()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? FDATE(_JAXFolder+"data\counties.dbf")
```

Return Value

Numeric

Note

FDATE() returns the last modified date of a file. If a Fully Qualified File Name (FQFN) is not given, then JAXBase will search for the file in the default path list. If the file is not found then an error is generated.

See Also

DATETIME() Fuction

FTIME() Function

LUPDATE() Fuction

FULLPATH() Function

SET PATH Command

FIELD()

FIELD(nFieldName [,WorkArea | cAlias])

Status

Slated for Version 0.6

Remarks

Returns the specified field name.

Parameters

nFieldName

A numeric expression indicating which field name from the field list to return.

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

*** Demonstrate FIELD()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties")

? FIELD(1)

? FIELD(2)

Return Value

Character

Note

If there is no table open in the work area, an error is generated.

See Also

DISPLAY STRUCTURE Command

FCOUNT() Function

FSIZE() Function

FILE()

FILE(cFileName [,nFlags])

Remarks

Returns a value of true (.T.) if the specified file exists.

Parameters

cFileName

Character expression of the file name to search.

nFlags

Numeric expression where 0 (default) indicates that Hidden and System files should be ignored, while 1 indicates that all files are checked.

Example

```
* Demonstrate FILE()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? FILE(_JAXFolder+"data\counties.dbf")
```

Return Value

Logical

Note

If the cFileName is not a Fully Qualified File Name (FQFN), then JAXBase will search for the file in the path list.

Returns a value of true (.T.) if the file is found.

See Also

ADIR() Function

CD Command

FULLPATH() Function

GETFILE() Function

LOCFILE() Function

FILETOSTR()

FILETOSTR(cFileName)

Remarks

Returns the contents of the specified file.

Parameters

cFileName

Character expression of the file name to search.

Example

```
* Demonstrate FILETOSTR
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? FILETOSTR(_JAXFolder+"data\states.dbf")
```

Return Value

Character

Note

If the cFileName is not a Fully Qualified File Name (FQFN), then JAXBase will search for the file in the path list.

The contents of the file will be returned as a string.

If the file does not exist, an error will be generated.

See Also

STRTOFILE() Function

JAXBase System Capacities

FILTER()

FILTER(nWorkArea | cAlias)

Status

Slated for Version 0.8

Remarks

Returns the current filter to the specified work area or alias.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate FILTER()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties.dbf")
SET FILTER TO State="WI"
? FILTER()
COUNT TO A
? RECCOUNT(), A
```

Return Value

Character

Note

If an alias is given and it does not exist, an error is generated. If a work area is given or the parameter omitted, and there is no open table, then an empty character string is returned.

See Also

SET FILTER Command

Filter Property

FLOCK()

FLOCK(nWorkArea | cAlias)

Status

Slated for Version 0.8

Remarks

Attempts to lock the specified DBF table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

* Demonstrate FLOCK()

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties.dbf")

IF FLOCK()

 ? "FILE IS LOCKED"

ELSE

 ? "FAILED TO LOCK"

ENDIF

CLOSE ALL

Return Value

Logical

Note

When a table is locked, the table is available to the program that placed the lock. Other users on the network have read-only access to the table. To lock a table and prevent access by other users, see SET EXCLUSIVE Command and USE Command.

A table remains locked until it is unlocked. The table can be unlocked by UNLOCK, closing the table, or quitting JAXBase. Tables can be closed with USE, CLEAR ALL, or CLOSE DATABASES.

By default, FLOCK() attempts to lock a table once. Use SET REPROCESS to automatically retry a table lock when the first attempt fails. SET REPROCESS determines the number of lock attempts, or the length of time during which lock attempts are made when the initial lock attempt is unsuccessful.

A table is locked only when an explicit FLOCK() is placed against it.

Care must be taken when attempting to lock two or more tables as it is possible that someone on the network may be attempting to place locks on multiple tables at the same time and a death lock occurs. For an example of death locks and how to avoid them, look at the deathlock.prg program in SAMPLES.

See Also

CLEAR Commands
CLOSE DATABASES Command
CLOSE TABLES Command
ISFLOCKED() Function
ISRLOCKED() Function
LOCK() Function
RLOCK() Function
UNLOCK Command
USE

FLOOR()

FLOOR(nExpression)

Remarks

Returns the next lowest integer that is less than or equal to the specified value.

Parameters

nExpression

The numeric expression used to find the FLOOR() value.

Example

*** Demonstrate FLOOR()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? FLOOR(100.3) && Returns 100

? FLOOR(-50.7) && Returns -51

? FLOOR(10) && Returns 10

Return Value

Numeric

See Also

CEILING()

INT()

ROUND()

FONTMETRIC()

Status

Slated for Version 2

DRAFT

FOR()

FOR(nIndexNumber [, nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns the FOR filter expression for the specified index.

Parameters

nIndexNumber

Numeric expression indicating which index to interrogate from the index list.

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate FOR()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties.dbf")
SET INDEX TO COUNTIES_STATE
SET INDEX TO COUNTIES_NAME
```

```
? FOR(2)
```

Return Value

Charcter

Note

If the second parameter is not provided, the current work area is use.

If an alias is given and not found, an error is generated.

If there is no index open or the nIndexNumber goes beyond the limits of the index list, and empty character value is returned.

See Also

INDEX Command

DRAFT

FORCEEXT()

FORCEEXT(cFileName, cExtension)

Remarks

Returns a string with the specified filename with its old extension replaced by the specified extension.

Parameters

cFileName

String containing a file name.

cExtension

String containing a file extension without the leading period.

Example

```

* Demonstrate FORCEEXT()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

SET NAMING TO LOWER
? FORCEEXT("counties.dbf","BAK") && Returns counties.bak
? FORCEEXT("counties","BAK")    && Returns counties.bak
? FORCEExt("counties.dbf.", "BAK") && Returns counties.dbf.bak

```

Return Value

Character

Note

What FORCEEXT() returns is dictated by the SET NAMING command.

FORCEEXT() does not do any checking. It assumes that it is being given a legal PathFile name.

See Also

ADDBS() Function

DEFAULTTEXT() Function

FILE() Function

FORCEPATH() Function

JUSTDRIVE() Function

JUSTEXT() Function

JUSTFNAME() Function

JUSTFULLPATH() Function

JUSTPATH() Function

JUSTSTEM() Function

SET NAMING Command

FORCEPATH()

FORCEPATH(cFileName, cPath)

Remarks

Returns a string where the original file path is replaced with the specified file path.

Parameters

cFileName

String containing a file name.

cPath

String containing the new path.

Example

```
* Demonstrate FORCEPATH()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
SET NAMING TO LOWER
? FORCEPATH("counties.dbf","C:\DATA")    && Returns c:\data\counties.dbf
? FORCEPATH("", "C:\DATA")                && Returns c:\data\
? FORCEPATH("C:\TEST.TXT", "C:\DATA")    && Returns c:\data\counties.dbf
? FORCEPATH("C:\TEST", "C:\DATA")        && Returns c:\data\test
? FORCEPATH("C:\TEST\","C:\DATA")        && Returns c:\data\
```

Return Value

Character

Note

What FORCEPATH() returns is dictated by the SET NAMING command. Further, FORCEPATH() does not do any checking. It assumes that it is being given a legal Path and File information.

See Also

ADDDBS() Function

DEFAULTTEXT() Function

FILE() Function

FORCEPATH() Function

JUSTDRIVE() Function

JUSTEXT() Function

JUSTFNAME() Function

JUSTFULLPATH() Function

JUSTPATH() Function

JUSTSTEM() Function

SET NAMING Command

FORCESTEM()

Status

Slated for Version 0.6

DRAFT

FOUND()

FOUND([nWorkArea, cAlias])

Remarks

Returns a value of true (.T.) if the last LOCATE command, SEEK command, or SEEK() function successfully found an exact matching record.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate FOUND()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
USE (_JAXFolder+"data\counties")
SET INDEX TO COUNTIES_BYSTATE
LOCATE FOR STATE="FL"
? FOUND()
SEEK "CAVENTURA"
? FOUND()
```

Return Value

Logical

Note

Returns a value of true (.T.) if the most recent FIND, LOCATE, SEEK or CONTINUE command was successful. FOUND() is also affected by the SEEK() function.

If there is no table open in the work area you specify, FOUND() returns false (.F.). If you specify an alias and it does not exist, an error is generated.

See Also

CONTINUE Command

FIND Command

LOCATE Command

SEEK Command

SEEK() Function

EOF() Function

FSIZE()

FSIZE(cFieldName [, nWorkArea | cAlias])
FSIZE(cFilename)

Remarks

Returns the size in bytes of the specified file.

Parameters

cFieldName

Character expression containing a field name.

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

cFileName

Character expression containing a file name.

Example

```
* Demonstrate FSIZE()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? "File size ",FSIZE(_JAXFolder+"data\counties.dbf")
USE (_JAXFolder+"data\counties")
? "Field size",FSIZE("name")
```

Return Value

Numeric

Note

If the cFileName is not a Fully Qualified File Name (FQFN), then JAXBase will search for the file in the path list. If found, it will return the size in bytes. If not, it will return 0.

For field size, if there is no table open in the work area you specify, FSIZE() returns 0. If you specify an alias and it does not exist, an error is generated.

When a field size is returned, it should be noted that the following fields always return the following values.

Field Type	Size Returned
Currency	8
Date	8
DateTime	8
Double	8
Integer	4
Logical	1
Memo	4
Blob	4
General	4

Currency, DateTime, Double, and Integer fields are binary encoded character fields.

Dates are stored as text data in yyyyymmdd format.

Memo, Blob, and General fields are integer values that are pointers to the location of the data in the FPT file.

Logical fields hold the characters T or F which is translated to .T. and .F. values.

Field sizes can also be obtained using the DISPLAY STRUCTURE command or AFIELDS() function.

See Also

AFIELDS() Function

DISPLAY STRUCTURE Command

FCOUNT() Function

Table File Structure

JAXBase Data Types

DBF Files

FTIME()

FTIME(cFileName)

Status

Slated for Version 0.6

Remarks

Returns the last modified date of the specified file.

Parameters

cFileName

Character expression holding the file name to get the last modified date.

Example

```
* Demonstrate FTIME()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? FTIME(_JAXFolder+"data\counties.dbf")
```

Return Value

Character

Note

If the cFileName is not a Fully Qualified File Name (FQFN), then JAXBase will search for the file in the path list. If found, it will return the last modified time. If not, it will return an empty character string.

See Also

FDATE() Function
SET PATH Command
TOSEC() Function
TOTIME() Function

FULLPATH()

FULLPATH(cFileName)

Status

Slated for Version 0.6

Remarks

Returns a string representing a file with an absolute path.

Parameters

cFileName

Character expression holding a file or partial path and file name.

Example

```
* Demonstrate FULLPATH() assuming that
* the C:\TEMP folder exists
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
SET NAMING TO LOWER
ACTIVATE CONSOLE
CLEAR

SET DEFAULT TO C:\TEMP
=STRTOFILE("test", "TEST.txt")
? FULLPATH("test.txt")           && Returns c:\temp\test.txt
```

Return Value

Character

Note

FULLPATH() returns the file name with the absolute path, making it easier to address a file or pass a file with path to another module or program.

FULLPATH() is affected by the SET NAMING command.

If cFileName is not found, FULLPATH() will search the path list, and if found, return the file with an absolute path. If it is not found, an empty character string is returned.

See Also

DBF() Function

FILE() Function

GETFILE() Function

LOCFIL() Function

SET NAMING Command

FV()

FV(nPayment, nInterestRate, nPeriods)

Status

Slated for Version 0.6

Remarks

Returns the present value amount of each payment (positive or negative).

Parameters

nPayment

Numeric expression specifying the periodic payment amount.

nInterestRate

Numeric expression specifying the interest rate per period. If calculating monthly payments with annual interest, divide the interest rate by 12.

nPeriods

Numeric expression specifying the total number of payments to be made.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? FV(500, .075/12, 48)      && Displays 27887.93
```

Return Value

Numeric

Note

FV() assumes on-time payments earning compound interest providing the ability to create an investment plan.

See Also

CALCULATE Command

FV() Function

PV() Function

GETAUTOINCVALUE()

Status

Slated for Version 2.0

DRAFT

GETCP()

Status

Slated for Version 2.0

DRAFT

GETDATE()

GETDATE([lDateOnly, [dExpression | tExpression]])

Status

Slated for Version 0.6

Remarks

Opens the Date/Time dialog and return a Dates or DateTime value based lDateOnly setting.

Parameters

lDateOnly

Logical expression, if true (.T.) limits entry to a Date value, otherwise accepts Date and Time entry.

dExpression

Date expression for the starting date.

tExpression

DateTime expression for the starting date and time.

Example

```
* Demonstrate GETDATE()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? GETDATE()
```

Return Value

Date value if lDateOnly is true (.T.) and DateTime value if lDateTime is false (.F.) or omitted.

Note

If lDateOnly is true (.T.), the time controls are greyed out. Default is false (.F.) allowing the user to also set a time component.

If dExpression is provided and lDateOnly is false (.F.), the time component defaults to midnight.

If no Date or DateTime expression is provided, the current OS time is assumed. The time component is affected by SET HOURS command.

See Also

DATE() Function

DATETIME() Function

GETTIME() Function

DRAFT

GETDIR()

GETDIR([cDirectory [, cText [, cCaption [, nFlags, [lRootOnly]]]])

Status

Slated for Version 0.6

Remarks

Displays the Select Directory dialog box and returns the absolute directory path selected.

Parameters

cDirectory

A character expression indicating the starting point.

cText

Character expression to display under the caption.

cCaption

Character expression displayed in the caption.

nFlags

Numeric expression where the following values can be added together as needed to alter how the Select Directory dialog works. The default is to not display hidden and system folders, not include the edit box, not validate a path, and not create a path if it does not exist.

nFlag	Description
1	Display hidden and system folders
2	Include the edit box allowing the user to type in a directory path
4	Validate the path selected or entered
8	Create the path entered by the user if it does not exist

lRootOnly

If a value of true (.T) then the user cannot navigate to another drive nor to a directory above the current.

Example

```
* Demonstrate GETDIR()
```

```
CLEAR ALL
```

```
CLOSE ALL
```

```
SET EXCLUSIVE ON
```

```
SET CONSOLE ON
```

```
ACTIVATE CONSOLE
```

```
CLEAR
```

```
* Include the edit box and validate the directory path
```

```
? GETDIR(_JAXFolder, "Select", "Please select a directory", 2+4)
```


Return Value

Character

Note

If a directory is not selected or the user cancels the selection, an empty character string is returned.

Any error generated by the Select Directory dialog will produce an error dialog informing the user of the issue, but no JAXBase error is generated and the user will be allowed to make changes and try again.

See Also

DIR or DIRECTORY Command

DIRECTORY() Function

GETFILE() Command

DRAFT

GETENV()

GETENV(cVariableName)

Status

Slated for Version 0.6

Remarks

Returns the specified operating system environment variable as a string.

Parameters

cVariableName

Character expression specifying the environment variable's value to return.

Example

```
* Demonstrate GETENV()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

* JAXBase converts common environment
* variables between Windows and Linux.
? GETENV("$HOME")
? GETENV("%USERPROFILE%")
```

Return Value

Character

Note

JAXBase converts common Windows and Linux environment variables.

Windows	Linux	Description
%USERPROFILE%	\$HOME	User's home path
%TEMP%	\$TEMP or /tmp	Operating system's temporary folder
PATH	PATH	Operating path list

See Also

DISKSPACE() Function

OS() Fuction

VERSION() Function

GETFILE()

```
GETFILE([cFileExtensions] [,cText [,cOpenButtonCaption [,nButtonType [,cCaption]]]])
```

Status

Slated for Version 0.6

Remarks

Displays the Open File dialog box.

Parameters

cFileExtensions

Character expression with list of comma or semicolon delimited extensions.

cText

Character expression to display in the File Name label.

cOpenButtonCaption

Character expression to display in the Open Button's caption.

nButtonType

Value	Buttons Displayed
0 or omitted	OK, Cancel
1	OK, New, Cancel
2	OK, None, Cancel

cCaption

Character expression to display as the dialog caption.

Example

*** Demonstrate GETFILE()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? GETFILE("dbf,idx","Select",2,"Please select a file")

Return Value

Character

Note

GETFILE() returns the string Untitled is the New or None buttons are Clicked and returns an empty character string if the Cancel button is clicked.

See Also

FULLPATH() Function

GETDIR() Function

GETPICT() Function

LOCFILE() Function

PUTFILE() Function

DRAFT

GETFIELDSTATE()

Status

Slated for Version 2.0

DRAFT

GETFONT()

Status

Slated for Version 2.0

DRAFT

GETFORM()

DRAFT

GETFORMOBJECT()

DRAFT

GETMENU()

DRAFT

GETNEXTMODIFIED()

Status

Slated for Version 2.0

DRAFT

GETJSON() Function

GETJSON(cJSONString, cFieldToReturn)

Status

Slated for Version 0.6

Remarks

Returns a field value from a JSON string.

Parameters

cJSONString

Character expression holding the JSON string to parse.

cFieldToReturn

Character expression holding the name of the JSON field to return.

Example

```
* DEMONSTRATE GETJSON()
SET TALK OFF
CLEAR
JSONString=[{"Name":"Fred", "BDate":1979/07/05, "Balance":140.73, "LastIn":"2025-04-17 13:41:00"}]
Name=JSON(JSONString, "Name")
BirthDate=JSON(JSONString, "Bdate")
Balance=JSON(JSONString, "balance")
LastIn=JSON(JSONString, "lastin")
? VARTYPE(Name), Name
? VARTYPE(BirthDate), BirthDate
? VARTYPE(Balance), Balance
? VARTYPE(LastIn), LastIn
```

Return Value

Supported JAXBase data type

Note

The JSON will parse a JSON string and return the value of the JASON field from the JSON string. The data type returned depends on the JSON string, but will one of the following

JSON Conversion	
A	Array containing elements from the JSON field
C	Character
D	Date
L	Logical
N	Numeric (including integer, float, double, etc)
O	Returns an EMPTY object with fields/values of the JSON object
T	DateTime
U	Unknown or invalid field name
X	.NULL.

A JSON array will change the receiving variable into a JAXBase array variable, while a JSON object will change the receiving variable into a JAXBase object variable using the EMPTY class.

See Also

EMPTY Class

PUTJSON() Function

JAXBase Data Types

GETPICT()

GETPICT([cFileExtensions][, cFileNameCaption[, cOpenButtonCaption])

Status

Slated for Version 0.6

Remarks

Displays the Open Picture dialog box and returns the name of the picture chosen.

Parameters

cFileExtensions

Character expression with list of comma or semicolon delimited extensions.

cFileNameCaption

Character expression to display in the File Name label.

cOpenButtonCaption

Character expression to display in the Open Button's caption.

Example

*** Demonstrate GETPICT()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET DEFAULT TO (_JAXFolder+"GRAPHICS")

? GETPICT("BMP,JPG,PNG","Select","Select")

Return Value

Character

Note

Returns the absolute file path of the selected picture unless the CANCEL button is clicked or the dialog is exited with the close control, then an empty character string is returned.

cFileExtensions simply filters the directory listing, but can be overridden by the user if they enter a file skeleton in the edit box and hit the ENTER key.

No file type validation is performed by GETPICT().

See Also

GETFILE() Function

LOCFILE() Function

PUTFILE() Function

DRAFT

GETPRINTER()

GETPRINTER()

Status

Slated for Version 1.0

Remarks

Allows the user to select a printer from the available printers and returns a PRINTER class object for the selected printer.

Example

```
* Demonstrate GETPRINTER()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

oPrt=GETPRINTER()
IF VARTYPE(oPrt)="0"
    ? oPrt.Name
ELSE
    ? "User did not select a printer"
ENDIF
```

Return Value

JAXBase Printer object

Note

If a printer is not selected, a .NULL. is returned.

See Also

APRINTERS() Function
SET PRINTER Command

GETTIME()

GETTIME(cTime | nSeconds)

Status

Slated for Version 0.8

Remarks

Opens the Time dialog and returns a character Time value.

Parameters

cTime

Character expression in a valid 12 or 24 hour format.

nSeconds

A numeric expression from 0 to 86399 representing 00:00:00 to 23:59:59.

Example

```
* Demonstrate GETTIME()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
SET HOURS TO 12  
? GETTIME()
```

```
SET HOURS TO 24  
? GETTIME(TIME())
```

Return Value

Character

Note

If no Time expression is provided, the current OS time is assumed.

The time format is affected by SET HOURS command. The value returned will be in either the 12 or 24 hour format, based on the SET HOURS setting.

See Also

DATE() Function

DATETIME() Function

GETTIME() Function

SET DATE Command

SET HOURS command

TTOSEC()

DRAFT

GETWORDCOUNT()

GETWORDCOUNT(cExpression [,cDelimiters])

Status

Slated for Version 0.6

Remarks

Counts the words in a string

Parameters

cExpression

Specifies the character expression whose words will be counted.

cDelimiters

The character expression that contains one or more delimiting characters.

Example

* Demonstrate GETWORDCOUNT()

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
        +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

? GETWORDCOUNT(lcString) && Returns 53

Return Value

Numeric

Note

The default delimiting characters are space, tab, carriage return, and line feed.

GETWORDCOUNT uses each character in cDelimiters separately to get the word count.

If one or more delimiting characters are grouped together, that group will be counted as one delimiter.

See Also

GETWORDNUM()

GETWORDNUM()

GETWORDNUM(cExpression, nIndex, cDelimiters)

Status

Slated for Version 0.6

Remarks

Returns the indicated word from a string of words.

Parameters

cExpression

Character expression containing the string to search.

nIndex

Numeric expression indicating which word to return'

cDelimiters

Character string containing the individual characters use as delimiters.

Example

```
* Demonstrate GETWORDNUM()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
        +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

```
? GETWORDNUM(lcString,22)      && Returns "the"
```

Return Value

Character

Note

The default delimiting characters are space, tab, carriage return, and line feed.

GETWORDNUM uses each character in cDelimiters separately to get the word count.

If one or more delimiting characters are grouped together, that group will be counted as one delimiter.

See Also

GETWORDCOUNT()

DRAFT

GETCURSORADAPTER()

Status

Slated for Version 2.0

DRAFT

GOMONTH()

GOMONTH(dExpression | tExpression, nNumberOfMonths)

Status

Slated for Version 0.6

Remarks

Returns the date that is a specified number of months before or after a given Date or DateTime expression.

Parameters

dExpression

Starting Date expression.

tExpression

Starting DateTime expression.

nNumberOfMonths

Numeric expression, negative or positive, indicating how many months before or after the specified Date or DateTime.

Example

```
* Demonstrate GOMONTH()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

FOR I=-1 TO -10 STEP -1
? GOMONTH(DATE(),I)
ENDFOR
```

Return Value

Date

Note

GOMONTH() does not support dates earlier than 1752.

See Also

CMONTH() Function

DATETIME() Function

GETDATE() Function

GETDATETIME() Function

GETTIME() Function

GUID()

GUID()

Remarks

Returns a case sensitive 36-character GUID string in the format xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx.

Example

```
* Demonstrate GUID()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR  
  
? GUID()  
? GUID()  
? GUID()  
? GUID()
```

Return Value

Character

HEADER()

HEADER([nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns the HEADER class representing the header of the specified table or cursor.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate HEADER()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
USE (JAXFolder+"data\counties.dbf")  
? HEADER()
```

Return Value

Numeric

Note

If no parameter is given, the current work area is assumed. If an alias is provided and it is not found, an error is generated.

See Also

FSIZE() Function

RECSIZE() Function

HEXTOINT()

HEXTOINT (cHex)

Status

Slated for Version 0.6

Remarks

Returns a numeric value based on the provided character expression.

Parameters

cHex

Character expression containing a valid hexadecimal expression.

Example

```

* Demonstrate HEXTOINIT()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

```

```

? HEXTOINT("F")
? HEXTOINT("0x01FA")

```

Return Value

Numeric value

Note

The hex string may start with "0x" or "0X", which will be trimmed.

Hex strings start at 0x0 with a maximum value of 0xFFFFFFFFFFFFFFF.

See Also

INTTOHEX()

HOUR()

HOUR(dExpression | tExpression)

Remarks

Returns the hour from a DateTime expression.

Parameters

dExpression

Expression holding a Date value.

tExpression

Expression holding a DateTime value.

Example

```
* Demonstrate HOUR()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR  
  
? HOUR(DATETIME())
```

Return Value

Numeric

Note

If dExpression is provided, HOUR() always returns 0.

The return value is always based on the 24 hour clock.

See Also

DATE()

DATETIME()

GETDATE()

GETTIME()

TIME()

ICASE()

ICASE(lCondition1, eResult1 [, lCondition2, eResult2]..., eOtherwiseResult)

Remarks

Evaluates the result from a list of conditions.

Parameters

lCondition

Expression that evaluates to a logical true (.T.) or false (.F.)

eResult

An expression that is a valid JAXBase data type or object.

eOtherwiseResult

An expression that is a valid JAXBase data type or object.

Example

```
* Demonstrate ICASE()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? "Good "+ICASE(HOURS(SECONDS())<12,"morning!", HOURS(SECONDS())<18, "afternoon!", "Evening!")
```

Return Value

A valid JAXBase data type or object.

Note

ICASE() returns the eResult after the first lCondition that evaluates to true (.T.) or the eOtherwiseResult.

You can pass up to 100 pairs of parameters for ICASE().

See Also

IIF() Function

EVALUATE() Function

EVL() Function

IDXCOLLATE()

Status

Slated for Version 2.0

DRAFT

IIF()

IIF(lCondition, eExpression1, eExpression2)

Remarks

Returns one of two expressions based on the provided condition.

Parameters

lCondition

Logical condition expression.

eExpression1

Expression to return if lCondition is true (.T.)

eExpression2

Expression to return if lCondition is false (.F.)

Example

*** Demonstrate IIF()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

IIF(HOUR(SECONDS())<12, "AM", "PM")

Return Value

JAXBase data type or object

Note

Also known as an immediate IF, can replace simple IF / ENDIF statements and is especially useful in report and label expressions to conditionally specify field contents.

See Also

ICASE() Function

IF / ENDIF Command

IF Statements

INDEXSEEK()

INDEXSEEK(eExpression [, nWorkArea | cAlias [, nIndex | cIndexName [, nSession]])

Status

Slated for Version 0.8

Remarks

Performs a SEEK() function without moving the record pointer and returns the record number.

Parameters

eExpression

Expression that holds the value to search for in the index.

nWorkArea

Numeric expression indicating the work area to check. 0 or negative indicates the current work area.

cAlias

Character expression indicating the alias of the table to check.

nIndex

Index number from the open index list. 0 or negative means the current controlling index.

cIndexName

Index name from the open index list.

nSession

Session ID of work area or alias. 0 or negative means the current data session.

Example

* Demonstrate

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties")

GOTO TOP

? RECNO(),state,county

? INDEXSEEK("YORK",0,"counties_name")

? RECNO(),state,county

Return Value

Numeric

Notes

If a match is not found, -1 is returned.

See Also

SEEK Command

SEEK() Function

DRAFT

INKEY()**INKEY([nSeconds] [,cHideCursor])**Status

Slated for Version 1.0

Remarks

Returns a numeric value corresponding to the first key press or mouse.

Parameters**nSeconds**

Specifies the number of seconds to wait for a keystroke.

cHideCursor

Shows or hides the cursor or performs other tasks based on the following.

cHideCursor	Description
S	Show cursor (default)
H	Hide cursor
M	Check for a mouse click
E	Expand the keyboard macro if assigned. Read the entire macro using successive INKEYS with cHideCursor containing E.

Example*** Demonstrate INKEY()****CLEAR ALL****CLOSE ALL****SET EXCLUSIVE ON****SET CONSOLE ON****ACTIVATE CONSOLE****CLEAR****? "Press a key"****? "Key pressed =",INKEY(0)**Return Value

Numeric

Note

If you do not specify a value for nSeconds, INKEY() immediately returns a value for a keystroke. If nSeconds is 0, INKEY() waits indefinitely for a keystroke.

INKEY Values

KEY	Alone	SHIFT	CTRL	ALT
F1	28	84	94	104
F2	-1	85	95	105
F3	-2	86	96	106
F4	-3	87	97	107
F5	-4	88	98	108
F6	-5	89	99	109
F7	-6	90	100	110
F8	-7	91	101	111
F9	-8	92	102	112
F10	-9	93	103	113
F11	133	135	137	139
F12	134	136	138	140
0	48	41		19
1	49	33		120
2	50	64		121
3	51	35		122
4	52	36		123
5	53	37		124
6	54	94		125
7	55	38		126
8	56	42		127
9	57	40		128
a	97	65	1	30
b	98	66	2	48
c	99	67	3	46
d	100	68	4	32
e	101	69	5	18
f	102	70	6	33
g	103	71	7	34
h	104	72	127	35
i	105	73	9	23
j	106	74	10	36
k	107	75	11	37
l	108	76	12	38
m	109	77	13	50
n	110	78	14	49
o	111	79	15	24

KEY	Alone	SHIFT	CTRL	ALT
p	112	80	16	25
q	113	81	17	16
r	114	82	18	19
s	115	83	19	31
t	116	84	20	20
u	117	85	21	22
v	118	86	22	47
w	119	87	23	17
x	120	88	24	45
y	121	89	25	21
z	122	90	26	44
INS	22	22	146	162
HOME	1	55	29	151
DEL	7	7	147	163
END	6	49	23	159
PAGE UP	18	57	31	153
PAGE DOWN	3	51	30	161
UP ARROW	5	56	141	152
DONN ARROW	24	50	145	160
RIGHT ARROW	4	54	2	157
LEFT ARROW	19	52	26	155
ESC	27	27	27	1
ENTER	13	13	10	166
BACKSPACE	127	127	127	14
TAB	9	15	148	
SPACEBAR	32	32	32	57
` or ~	96	126		41
- or _	45	95		
+ or =	61	43		13
[or {	91	123	27	26
] or }	93	125	29	27
\ or	92	124	28	43
; or :	59	58		39
' or "	39	34		40
, or <	44	60		51
. or >	46	62		52
/ or ?	47	63		53

See Also

KEYBOARD Command

KeyPress Event

LASTKEY()

ON KEY LABEL Command

SET TYPEAHEAD Command

INLIST()

INLIST(eExpression1, eExpression2 [, eExpression3 ..])

Remarks

Indicates if an expression matches an of a set of expressions.

Parameters

eExpression1

Expression to test.

eExpression2 [,eExpresion3...]

List of expressions to test against.

Example

```
* Demonstrate INLIST()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? INLIST("C","A","B","C","D","E")
? INLIST("c","A","B","C","D","E")
```

Return Value

Logical

Note

Returns true (.T.) if the first expression matches one of included list elements.

All expressions must be of the same type.

INLIST() is case sensitive.

See Also

\$ Operator

BETWEEN() Function

AT() Function

ATLINE() Function

OCCURS() Function

RAT() Function

RATLINE() Function

INPUTBOX()

INPUTBOX(cInputPrompt [,cCaption [,cDefaultValue [,nTimeout [,cTimeoutValue [,cCancelValue]]]])

Status

Slated for Version 0.6

Remarks

Displays the modal dialog for input of a single string.

Parameters

cInputPrompt

Character expression to place above the input box as a prompt with 64 characters being the maximum length allowed.

cDialogCaption

Character expression to place in the caption of the dialog.

cDefaultValue

Character expression put into the input box.

nTimeout

Numeric expression given in milliseconds on how long to wait after the most recent keystroke before exiting the input box.

cTimeoutValue

Character expression to return if a timeout occurs (default is empty string).

cCancelValue

Character expression to return if the cancel button is pressed (default is empty string).

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? INPUTBOX("Your Text", "Enter Some Text", "Nothing Given", 15000, "Timed Out", "Canceled")
```

Return Value

Character

Note

The InputBox() dialog displays an edit box and the OK and CANCEL buttons. The OK button returns the TRIM() value of what's in the input box while the cancel button returns the cCancelValue.

If the cInputPrompt is longer than 64 characters, it will be trimmed.

See Also

GETDATE() Function

GETDIRECTORY() Function

GETFILE() Function

GETTIME() Function

MESSAGEBOX() Function

DRAFT

INSMODE()

INSMODE([lExpression])

Status

Slated for Version 1.0

Remarks

Returns the current insert mode and optionally turns the insert mode on or off.

Parameters

lExpression

Logical expression used to turn the insert mode on when true (.T.) or off when false (.F).

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

IF INSMODE()
    ? "Insert is ON"
ELSE
    ? "Insert is OFF"
ENDIF
```

Return Value

Logical

See Also

CAPSLOCK() Function

NUMLOCK() Function

<https://www.pinvoke.net/default.aspx/user32/GetKeyState.html>

INT()

INT(nExpression)

Remarks

Returns the integer portion of nExpression

Parameters

nExpression

Numeric expression to use with the INT() function.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? INT(PI())
? INT(123.114)
```

Return Value

Numeric

See Also

CEILING() Function

FLOOR() Function

ROUND() Function

INTTOHEX()

INTTOHEX(nExpression)

Status

Slated for Version 0.6

Remarks

Returns the a character representation of a hexadecimal number equal to the integer portion of the nExpression parameter.

Parameters

nExpression

Numeric expression between 0 and 72,057,594,037,927,936.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? INTTOHEX(256)  && Returns 00FF
? INTTOHEX(192038)  && Returns 0002EE26
```

Return Value

Character

Note

INTTOHEX() begins to lose validity above 72,057,594,037,927,936 as scientific notation will be used at some point above that value.

INTTOHEX() returns either 4, 8, 12, or 16 characters.

See Also

BINTOC() Function

CTOBIN() Function

HEXTOINT() Function

ISALPHA()

ISALPHA(cExpression [, lAll])

Remarks

Returns a logical true (.T.) if the specified character(s) are between A and Z.

Parameters

cExpression

Character expression to check.

lAll

If false (.F.) or omitted, check the first character, otherwise check all characters.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? ISALPHA("A") && Returns .T.

? ISALPHA("ALPHA3", .T.) && Returns .F.

Return Value

Logical

Note

Setting lAll to true (.T.) will force ISALPHA() to check all characters in the expression. Each character checked must be a letter between A and Z (case insensitive).

See Also

ISBLANK()

ISDIGIT()

ISBLANK()

ISBLANK(eExpression)

Remarks

Determines if an expression is blank or empty.

Parameters

eExpression

JAXBase data type expression.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? ISBLANK(0)
? ISBLANK("")
? ISBLANK(CHR(0))
? ISBLANK({/})
```

Return Value

Logical

Note

ISBLANK() returns a logical true (.T.) value based on the following.

Expression Type	Evaluates to
Binary or General field Varbinary data	0x0 or zero bytes
Character or Memo	Empty string or all spaces
Currency	0
Date	Value equivalent to CTOD("")
DateTime	Value equivalent to CTOT("")
Double	0
Float	0
Integer	0
Logical	.F.
Numeric	0

See Also

APPEND Command

BLANK Command

EMPTY() Function

ISNULL() Function

LEN() Function

DRAFT

ISDIGIT()

ISDIGIT(cExpression [,cIgnore])

Remarks

Returns a logical true (.T.) if the specified character string are all digits (0 – 9).

Parameters

cExpression

Character expression to check.

cIgnore

Character expression with characters to ignore (default is empty string).

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? ISALPHA("1234")	&& Returns .T.
? ISALPHA("-1234.17", ".")	&& Returns .F.
? ISALPHA("123-00-9876", "-")	&& Returns .T.
? ISALPHA("(809) 555-1230", "() -")	&& Returns .T.

Return Value

Logical

Note

Any characters found in the cIgnore expression (case sensitive) will be ignored. Each character checked must be a digit between 0 and 9.

ISDIGIT() will is not sufficient to check if the string can be converted to a numeric expression unless cIgnore is empty or omitted.

See Also

ISBLANK()

ISDIGIT()

ISEXCLUSIVE()

ISEXCLUSIVE([nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns a logical true (.T.) if the table is opened in exclusive mode.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties") EXCLUSIVE
? ISEXCLUSIVE()
```

Return Value

Logical

Note

A table must be opened using the EXCLUSIVE keyword or while SET EXCLUSIVE is set ON.

Cursors, by their nature, are always opened in EXCLUSIVE mode and ISEXCLUSIVE() will always return a true (.T.) for a cursor.

See Also

SET EXCLUSIVE Command

USE Command

ISFLOCKED()

ISFLOCKED([nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns a value of true (.T.) if the table header is locked, otherwise returns a value of false (.F.).

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties") EXCLUSIVE

? FLOCKED() && Returns .T.

SELECT * FROM COUNTIES FOR STATE="WY" INTO CURSOR WYNOT

? FLOCKED("WYNOT") && Cursor returns .T.

Return Value

Logical

Note

Tables opened with the EXCLUSIVE keyword, while EXCLUSIVE is set ON, all cursors, or a table with a lock on the header will all return a value of true (.T.) .

If the alias is not found, an error is generated.

See Also

FLOCK() Function

ISRLOCKED() Function

LOCK() Function

RLOCK() Function

SELECT SQL Command

USE Command

DRAFT

ISLEADBYTE()

Status

Slated for Version 2.0

DRAFT

ISLOWER()

ISLOWER(cExpression [, lAll [, cIgnore]])

Status

Slated for Version 0.6

Remarks

Returns a value of true if the checked characters in the string are alpha and lower case.

Parameters

cExpression

Character expression to check.

lAll

Logical expression indicating if all characters should be checked (default is .F.)

cIgnore

Character expression indicating which characters should be ignored (default is empty string).

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? ISLOWER("abcd") && Returns .T.

? ISLOWER("abcd", .T.) && Returns .F.

? ISLOWER("alpha-numeric", .T., "-") && Returns .T.

Return Value

Logical

Note

If a non-alpha character or an upper-case alpha character is found during the check, a value of false (.F.) is returned.

If lAll is true (.T.), then all characters are checked, otherwise just the first character.

If cIgnore is omitted, all characters are checked, otherwise if lAll is .F. then the first character that is not ignored will be checked and if lAll is .T. then all characters that are not in cIgnore are checked.

See Also

ISALPHA() Function

ISUPPER() Function

LOWER() Function

UPPER() Function

DRAFT

ISNULL()

ISNULL(eExpression)

Remarks

Returns a value of true (.T.) if the expression is .NULL.

Parameters

eExpression
Expression to check

Example

*** Demonstrate**

```
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
eNullValue=.NULL
? ISNULL(eNullValue)      && Returns .T.
? TYPE('eNullValue')    && Returns X
? VARTYPE(eNullValue)    && Returns X
? eNullValue = null      && Returns .NULL.
```

Return Value

Logical

Note

Most JAXBase data types can be set to a null value. If you set a variable holding an object to .NULL. then the object is released and the variable now holds a .NULL. value.

NULL, null, .null., and .NULL. are all equivalent values in an expression.

TYPE and VARTYPE return "X" for null values.

See Also

NVL() Function
SET NULL Command
TYPE() Function
VARTYPE() Function
Null Value Handling

ISODD()

ISODD(nExpression)

Status

Slated for Version 0.6

Remarks

Returns a value of true (.T.) if the integer portion of a numeric expression is not divided evenly by 2.

Parameters

nExpression

Numeric expression to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? ISODD(2.3) && Returns .T.
? ISODD(201) && Returns .F.
```

Return Value

Logical

Note

ISODD() checks the entire integer portion of the expression.

See Also

INT()

ROUND()

ISPEN()

Status

Slated for Version 2.0

DRAFT

ISREADONLY()

ISREADONLY([nWorkArea, cAlias])

Status

Slated for Version 0.8

Remarks

Returns a logical value to indicate if the table is read only.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cAlias

Character expression indicating the alias of the table to check.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties") READONLY

? ISREADONLY()

SELECT * FROM COUNTIES INTO CURSOR MyCurs READONLY

? ISREADONLY("MyCurs")

Return Value

Logical

Note

A table or cursor must be opened or created with the READONLY keyword.

See Also

ISEXCLUSIVE() Function

SELECT Command

USE Command

ISRLOCKED()

ISRLOCKED(nRecord [,nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns a true (.T.) value if the record is locked, otherwise returns false (.F.).

Parameters

nRecord

Numeric expression for physical record to check. If nRecord is 0, or if there are no parameters, then it returns the lock status for the current record.

nWorkArea

Numeric expression indicating the work area to check.

cAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties") EXCLUSIVE
```

```
IF RLOCK()
? ISRLOCKED()
ENDIF
```

Return Value

Logical

Note

If nRecord is 0, or if there are no parameters, then it returns the lock status for the current record.

A record lock remains until an UNLOCK is given against it, the table is closed, or JAXBase terminates.

See Also

RLOCK()

FLOCK()

ISFLOCKED()

DRAFT

ISUPPER()

ISUPPER(cExpression [, lAll [, cIgnore]])

Status

Slated for Version 0.6

Remarks

Returns a value of true if the checked characters in the string are alpha and upper case.

Parameters

cExpression

Character expression to check.

lAll

Logical expression indicating if all characters should be checked (default is .F.)

cIgnore

Character expression indicating which characters should be ignored (default is empty string).

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? ISUPPER("ABcd") && Returns .F.

? ISUPPER("ABCD", .T.) && Returns .T.

? ISLOWER("ALPHA-NUMERIC", .T., "-") && Returns .T.

Return Value

Logical

Note

A non-alpha character or lower-case alpha character found during the check, returns false (.F.).

If lAll is true (.T.), then all characters are checked, otherwise just the first character.

If clgnore is omitted, all characters are checked, otherwise if lAll is .F. then the first character that is not ignored will be checked and if lAll is .T. then all characters that are not in clgnore are checked.

See Also

ISALPHA() Function

ISUPPER() Function

LOWER() Function

UPPER() Function

DRAFT

JSONTOCURSOR()

JSONTOCURSOR(cJSONFile, cCursor)

Status

Slated for Version 2.0

Remarks

Writes JSON data to a cursor.

Parameters

cJSON

Character expression holding the name of the JSON file to convert.

cCursor

Character expression holding the name of the cursor to create or append to.

Return Value

Numeric

Note

CURSORTOJSON() returns the number of records written. If an error occurs during the process, the cursor will not be created, or the cursor being appended to may contain invalid or partial data.

See Also

CURSORTOJSON() Function

CURSORTOXML() Function

XMLTOCURSOR() Function

9

JSONT00BJ()

JSONT00BJ(cJSON)

Status

Slated for Version 0.8

Remarks

Creates a JAXBase EMPTY class object with properties specified in the JSON string.

Parameters

cJSON

Valid JSON character expression.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
JSONString=[{"Name":"Fred","BDate":1979/07/05,"Balance":140.73,"LastIn":"2025-04-17 13:41:00"}]
```

```
oTest=JSONT00BJ(JSONString)
```

```
DISPLAY MEMORY LIKE oTest
```

Return Value

JAXBase EMPTY Class object

Note

An invalid JSON string will cause an error to be generated.

The object will have property types based on the JSON string of type Array, Charcter, Date, DateTime, Logical, and Numeric.

See Also

GETJSON() Function
OBJTOJSON() Function
PUTJSON() Function

JUSTDRIVE()

JUSTDRIVE(cExpression)

Status

Slated for Version 0.8

Remarks

Returns the drive or share for Windows and device for Linux from a character expression.

Parameters

cExpression

Character expression with drive-path-file information.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? JUSTDRIVE("C:\TEMP") && Returns "C:" in Windows

Return Value

Character

Note

If the cExpression is invalid or does not contain drive information, an empty string is returned.

See Also

JUSTTEXT()

JUSTFNAME()

JUSTFULLPATH()

JUSTPATH()

JUSTSTEM()

JUSTEXT()

JUSTEXT(cExpression)

Remarks

Returns the extension from a file name in a character expression.

Parameters

cExpression

Character expression with drive-path-file information.

Example

```
* Demonstrate JUSTEXT()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

SET NAMING TO LOWER
? JUSTEXT("FILE.TXT")           && Returns "txt"
? JUSTEXT("MyObj.property")    && Returns "property"
? JUSTEXT("c:\TEMP.TEST\Test.jpg") && Returns "jpg"
```

Return Value

Character

Note

File name validation is not performed. JUSTEXT simply strips a file name from the expression and returns anything after the last period. If none is found, an empty string is returned.

SET NAME TO affects JUSTEXT().

See Also

JUSTDRIVE()
JUSTFNAME()
JUSTFULLPATH()
JUSTPATH()
JUSTSTEM()
SET NAMING Command

JUSTFNAME()

JUSTFNAME(cExpression)

Remarks

Parameters

cExpression

Character expression with drive-path-file information.

Example

```

* Demonstrate JUSTFNAME()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

SET NAMING TO LOWER
? JUSTFNAME("FILE.TXT")           && Returns "file.txt"
? JUSTFNAME("MyObj.property")    && Returns "myobj.property"
? JUSTFNAME("c:\TEMP.TEST\Test.jpg") && Returns "test.jpg"

```

Return Value

Character

Note

File name validation is not performed. JUSTFNAME simply strips a file name from the expression and returns it. If none is found, an empty string is returned.

SET NAMING TO affects JUSTFNAME.

See Also

JUSTDRIVE()
 JUSTEXT()
 JUSTFULLPATH()
 JUSTPATH()
 JUSTSTEM()
 SET NAMING Command

JUSTFULLPATH()

JUSTFULLPATH(cExpression)

Remarks

Returns the path of the file name expression ending with a backslash.

Parameters

cExpression

Character expression with drive-path-file information.

Example

*** Demonstrate JUSTFULLPATH()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET NAMING TO LOWER

? JUSTFULLPATH ("FILE.TXT")

&& Returns ""

? JUSTFULLPATH ("c:\TEMP.TEST\")

&& Returns "c:\temp.test\"

? JUSTFULLPATH ("c:\TEMP.TEST\Test.jpg")

&& Returns "c:\temp.test\"

Return Value

Character

Note

File name validation is not performed. JUSTFULLPATH simply strips a path from the expression and returns it. If none is found, an empty string is returned.

SET NAMING TO affects JUSTFULLPATH.

See Also

JUSTDRIVE()

JUSTEXT()

JUSTFNAME()

JUSTPATH()

JUSTSTEM()

SET NAMING Command

JUSTPATH()

JUSTPATH(cExpression)

Remarks

Returns the path of the file name expression.

Parameters

cExpression

Character expression with drive-path-file information.

Example

```
* Demonstrate JUSTPATH()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

SET NAMING TO LOWER
? JUSTPATH("FILE.TXT")           && Returns ""
? JUSTPATH ("c:\TEMP.TEST\")     && Returns "c:\temp.test"
? JUSTPATH ("c:\TEMP.TEST\Test.jpg") && Returns "c:\temp.test"
```

Return Value

Character

Note

File name validation is not performed. JUSTPATH simply strips a path from the expression and returns it. If none is found, an empty string is returned.

SET NAMING TO affects JUSTPATH().

See Also

JUSTDRIVE()

JUSTEXT()

JUSTFNAME()

JUSTFULLPATH()

JUSTSTEM()

SET NAMING Command

JUSTSTEM()

JUSTSTEM(cExpression)

Remarks

Returns the stem portion of a file name (filename minus extension).

Parameters

cExpression

Character expression with drive-path-file information.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET NAMING TO LOWER

? JUSTSTEM("FILE.TXT")

&& Returns "file"

? JUSTSTEM ("c:\TEMP.TEST\")

&& Returns ""

? JUSTSTEM ("c:\TEMP.TEST\Test.jpg")

&& Returns "test"

Return Value

Character

Note

File name validation is not performed. JUSTSTEM simply strips a file name from the expression, strips the extension from the file name and returns it. If none is found, an empty string is returned.

SET NAMING TO affects JUSTSTEM().

See Also

JUSTDRIVE()

JUSTEXT()

JUSTFNAME()

JUSTFULLPATH()

JUSTPATH()

SET NAMING Command

KBBUFFER()

Status

Slated for Version 1.0

Remarks

Returns the value of the keyboard buffer or allows you to set the keys in the keyboard buffer.

DRAFT

KEY()

KEY(nIndexNumber [, nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns the expression used to create the index order.

Parameters

nIndexNumber

Numeric expression indicating which index to interrogate from the index list.

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
SET INDEX TO Counties_ByState
? KEY()
```

Return Value

Character

Note

If nIndexNumber is less than 0, an error is generated, while 0 returns the index currently in control.

If nIndexNumber is greater than the number of indexes open, the empty string is returned.

If no index or alias parameter is given, the current work area is assumed. If an alias is provided and it is not found, an error is generated.

See Also

DESCENDING() Function

INDEX Command
REINDEX Command
SET INDEX Command
USE Command

DRAFT

LASTKEY()

LASTKEY()

Status

Slated for Version 1.0

Remarks

Returns an integer expression corresponding to the last key pressed.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? "Press a key"
? INKEY(0), LASTKEY()
```

Return Value

Numeric

Note

The values returned by LASTKEY() are identical to the values returned by INKEY().

See Also

INKEY() Function
READKEY() Function

LEFT()

LEFT(cExpression, nExpression)

Remarks

Returns a number of characters starting at the leftmost character.

Parameters

cExpression

Character expression to use to return the leftmost characters.

nExpression

Numeric expression indicating how many characters to return.

Example

```

* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? LEFT("ABCDEFGH",4)

```

Return Value

Character

Note

If there are fewer characters in cExpression than requested by nExpression, the entire expression is returned. If nExpression is 0, the empty string is returned and if nExpression is negative, an error is generated.

See Also

AT()

ATC() Function

ATCLINE() Function

ATLine() Function

LTRIM() Function

RAT() Function

RATLINE() Function

RIGHT() Function

RTRIM() Function

SUBSTR() Function

LEFTC()

Status

Slated for Version 2.0

DRAFT

LEN()

LEN(cExpression)

Remarks

Returns the number of characters in a character expression.

Parameters

cExpression

Character expression used to return the number of characters.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? LEN("ABC")
? LEN("  Abcdefg  ")
```

Return Value

Character

See Also

AT()

ATC() Function

ATCLINE() Function

ATLine() Function

LEFT() Function

LTRIM() Function

RAT() Function

RATLINE() Function

RIGHT() Function

RTRIM() Function

SUBSTR() Function

LENC()

Status

Slated for Version 2.0

DRAFT

LIKE()

LIKE(cExpression1, cExpression2)

Status

Slated for Version 0.6

Remarks

Returns true (.T.) if one character expression matches the other.

Parameters

cExpression1

Compare this expression with cExpression2. cExpression1 may contain wildcards * and ?.

cExpression2

The character expression to compare against.

Example

* Demonstrate

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? LIKE("abc*", "abcdef") && Returns .T.

? LIKE("abc*f", "abcdef") && Returns .T.

? LIKE("ab?def", "abcdef") && Returns .T.

? LIKE("ab?", "abcdef") && Returns .F.

Return Value

Logical

Note

LIKE() returns true (.T.) if cExpression1 matches cExpression2.

Trailing blanks in cExpression1 are used in the comparison.

The wildcard ? matches any single character while * matches any number of characters. You can mix wildcards together in any combination.

See Also

\$ Operator

AT() Function

ATC() Function

OCCURS() Function

RAT() Function

LIKEC() Function

LIKEC()

Status

Slated for Version 2

DRAFT

LINENO()

LINENO([1])

Status

Slated for Version 0.6

Remarks

Returns the line number of the currently executing program relative to the main program.

Parameters

1

Returns the line number of the currently executing program relative to the beginning of the current program or procedure.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
ON ERROR DO ErrHandler with LINENO()
ERROR 10
RETURN
```

```
PROCEDURE ErrHandler
PARAMETERS tnLineNo
? "Error in line",tnLineNo
return
```

Return Value

Numeric

Note

All source code files are limited to 4,096 lines.

See Also

[ERROR\(\) Function](#)

[MESSAGE\(\) Function](#)

[PROGRAM\(\) Function](#)

[Error Messages Listed Numerically](#)

LOADPICTURE()

LOADPICTURE(cFileName, oPictureObject)

Status

Slated for Version 0.6

Remarks

Loads a picture file into a PICTURE class object.

Parameters

cFileName

Absolute file name to load.

oPictureObject

Picture class object to load to which the file information is loaded.

Example

- * Demonstrate LOADPICTURE() in a GUI environment
- * A text only environment will generate an error

```
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
oPic=CREATEOBJECT("Picture")  
oPic.Left=100  
oPic.Right=100  
LOADPICTURE(_JAXFolder+"graphics\JAX.jpg", oPic)  
oPic.Show()  
INKEY(15)
```

Return Value

Logical

Note

LOADPICTURE() provides a cross-platform way to deal with pictures. The picture class object must already exist, and the picture and picture properties will be loaded to the object without modifying any of the other properties (such as position).

LOADPICTURE() returns a value of true (.T.) if the picture loads to the object, otherwise it returns a value of false. The ERRORNO property of LOADPICTURE() gives a JAXBase error result of an error occurs. The ERROR method can be used to create a custom error routine.

See Also

GETPICTURE() Function

SAVEPICTURE() Function

JAXBase Picture Class

DRAFT

LOCK()

LOCK([nWorkArea | cAlias [,nRecord | ArrayName])

Status

Slated for Version 0.8

Remarks

Attempts to lock one or more records in a table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

nRecord

Numeric expression of the record to lock

ArrayName

Array containing numeric values of records to lock.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\Counties")
IF LOCK(10)
    ? "Record 10 is locked"
ELSE
    ? "Failed to lock record 10"
ENDIF
```

```
INKEY(15000)
UNLOCK ALL
```

Return Value

Logical

Note

Locking even a small number of records will take more time than locking the entire file.

Locking is additive, so if there are records already locked, they will remain locked as you add more to the list.

When passing an array, if even one record fails to lock or a value is greater than the current number of records or less than 0, none of the records will be locked and LOCK() will return a value of false (.F.).

The quickest way to unlock a number of records is to use the UNLOCK ALL command.

SET MULTILOCKS limits locks to one record if set ON. If MULTLOCKS is OFF, then there is no limit.

See Also

CLEAR Commands
CLOSE Commands
ISFLOCKED() Function
ISRLOCKED() Function
FLOCK() Function
RLOCK() Function
SET MULTILOCKS Command
SET REPROCESS Command
UNLOCK Command
USE Command

LOG()

LOG(nExpression)

Status

Slated for Version 0.6

Remarks

Returns the natural logarithm of the specified numeric expression.

Parameters

nExpression

The numeric expression to which you want to get the logarithmic value.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? LOG(1)   && Displays 0.00
? LOG(100) && Displays 4.61
? LOG(1000) && Displays 6.91
```

Return Value

Numeric

Note

The base for the natural logarithm is the constant e.

The number of decimal places returned is specified by the SET DECIMALS command.

See Also

EXP() Function

LOG10() Function

SET DECIMALS Command

LOG10()

LOG10(nExpression)

Status

Slated for Version 0.6

Remarks

Returns the common logarithm value of the specified numeric expression.

Parameters

nExpression

The numeric expression to which you want to get the log10 value.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? LOG10(1)    && Displays 0.00  
? LOG10(100)  && Displays 2.00  
? LOG10(1000) && Displays 3.00
```

Return Value

Numeric

Note

The base for the common logarithm is 10.

The number of decimal places returned is specified by the SET DECIMALS command.

See Also

EXP() Function

LOG() Function

SET DECIMALS Command

LOWER()

LOWER(cExpression)

Remarks

Changes the characters in cExpression to all lower case.

Parameters

cExpression

Character expression to change to lower case.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? LOWER("ALTER EGO")

Return Value

Charcter

See Also

ISALPHA() Function

ISLOWER() Function

ISUPPER() Function

PROPER() Function

UPPER() Function

LTRIM()

LTRIM(cExpression)

Remarks

Remove the blank spaces from the left side of the expression.

Parameters

cExpression

Character expression to use in LTRIM()

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? LTRIM(" HELLO WORLD!")

Return Value

Character

Note

Very useful for removing the leading spaces from a numeric expression that was converted to a string by the STR() function.

See Also

ALLTRIM() Function

RTRIM() Function

STR() Function

TRIM() Function

MAX()

MAX(eExpression1, eExpression2, eExpression3...)

Remarks

MAX returns the maximum value from a list of values of the same type.

Parameters

eExpression1, eExpression2, eExpression3 ...

List of same data type values to compare.

Example

```
* Demonstrate MAX()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? MAX(1, 2, 3)           && Returns 3
? MAX(-4, 1, 1.5, 0.24, 2) ^& Returns 2
? MAX("A", "D4", "p", "Z") && Returns "p"
```

Return Value

Value of same data type as those compared

See Also

CALCULATE Command

MIN() Function

MD5()

MD5(cExpression)

Status

Slated for Version 0.8

Provides MD5 checksum for the provided character expression.

DRAFT

MD5Record()

MD5(cFieldList, lIncludeMemo]

Status

Slated for Version 0.8

Note

Using the record from the table in the current work area, returns a MD5 checksum for the list of fields indicated by the comma delimited cFieldList.

Note

Returns the checksum for the entire record if cFieldList is blank. Wild cards are allowed but may take extra processing time. The pointers (but not contents) for Blob, General, and Memo fields are included if the second parameter is True (.T.), otherwise they are ignored.

Use the MD5() function to generate separate checksums for Blob, General, and Memo fields.

MDY()

MDY(dExpression | tExpression)

Remarks

Returns the date expression in month-day-year format.

Parameters

dExpression

Date expression to return the specified format.

tExpression

DateTime expression to return the specified format

Example

```
* Demonstrate MDY()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
SET STRICTDATE TO 0  
ACTIVATE CONSOLE  
CLEAR
```

```
? MDY({1998-05-15}) && Displays May 05, 1998
```

Return Value

Charcter

Note

If SET CENTURY is OFF, the year portion will be two digits, otherwise four digits will be returned.

See Also

DMY() Function

SET CENTURY Command

SET DATE Command

MEMLINES()

MEMLINES(cExpression)

Remarks

Returns the number of lines in a character expression or memo field.

Parameters

cExpression

A character expression or memo field name.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET MEMOWIDTH TO 0

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
    +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

? MEMMLINES(lcString) && Returns 6

SET MEMOWIDTH TO 40

? MEMMLINES(lcString) && Returns 11

Return Value

Numeric

Note

SET MEMOWIDTH affects how lines in a large string or memo field are counted. If MEMOWIDTH is 0 then a CHR(13) is considered the end of a line. If MEMOWIDTH is set to a positive number, then if the number of characters in a line reach that value, the line is broken there and or at a CHR(13) character.

See Also

ALINES() Function

ATLINE() Function

COPY MEMO Command

MLINE() Function

MODIFY MEMO Command

SCATTER Command

SET MEMOWIDTH Command

MESSAGE()

MESSAGE([1])

Remarks

Returns the current JAXBase error message or source line.

Parameters

1

If 1 is given, the current line of source code is returned, otherwise the error message text.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

ON ERROR ? MESSAGE()
ERROR 10
```

Return Value

Character

Note

MESSAGE() is not reset by RETURN or RETRY.

See Also

ERROR Command
ERROR() Function
ON ERROR Command
AERROR() Function
Error Messages Listed Numerically

MESSAGEBOX()

MESSAGEBOX(cMessage [,nDialogBoxType [,cCaption [,nTimeout]]])

Status

Slated for Version 0.6

Remarks

Displays a user-defined dialog box.

Parameters

cMessage

nDialogBoxType

Based on the following tables, the value of nDialogBoxType specifies the following.

Value	Dialog box buttons
0	OK button only
1	OK and Cancel buttons
2	Abort, Retry, and Ignore buttons
3	Yes, No, and Cancel buttons
4	Yes and No buttons
5	Retry and Cancel buttons

Value	Icon to Display
16	Stop Sign
32	Question Mark
48	Exclamation Point
64	Information Icon

Value	Default Button
0	First button
256	Second button
512	Third button

cCaption

Character expression to place in the Dialog caption.

nTimeout

Numeric value of number of milliseconds to wait for a key before closing the dialog.

Example

```
* Demonstrate MESSAGEBOX()  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
MESSAGEBOX("Press the OK button!",0+48,"Example",15000)
```

Return Value

Numeric value based on the button selected. Escaping out or click the close icon (X) is the same as hitting the Cancel button.

Value	Button
1	OK
2	Cancel
3	Abort
4	Retry
5	Ignore
6	Yes
7	No

Note

MESSAGEBOX() requires you to use the parameters in order. If you want to set a timeout, you need to give a value for all the other parameters before it.

Setting nTimeout to 0 (default) means the dialog will wait until a button is pressed or the user performs an action that closes the dialog.

See Also

WAIT Command
INPUTBOX() Function

MIN()

MIN(eExpression1, eExpression2, eExpression3 ...)

Remarks

MAX returns the maximum value from a list of values of the same type.

Parameters

eExpression1, eExpression2, eExpression3 ...

List of same data type values to compare.

Example

*** Demonstrate MAX()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? MIN(1, 2, 3)	&& Returns 1
? MIN(-4, 1, 1.5, 0.24, 2)	^& Returns -4
? MIN("A", "D4", "p", "Z")	&& Returns "A"

Return Value

Value of same data type as those compared

See Also

CALCULATE Command

MAX() Function

MINUTE()

MINUTE(tExpression | dExpression)

Remarks

Returns the minute portion of the specified DateTime value.

Parameters

tExpression

DateTime value to get the MINUTE() value

dExpression

Date value to get the MINUTE() value

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? MINUTE(DATETIME())
```

Return Value

Numeric

Note

The minute value of a Date value is always 0.

See Also

CTOT() Function

DATE() Function

DATETIME() Function

DTOT() Function

HOUR() Function

SEC() Function

SET SECONDS Command

TIME() Function

MLINE()

MLINE(cExpression, nLine)

Remarks

Returns the indicated line of a character expression or memo field.

Parameters

cExpression

Character expression or memo field used extract the indicated line.

nLine

Numeric expression indicating which line to extract.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET MEMOWIDTH TO 0

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
    +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

? MLINE(2) && Displays line starting with "on this continent"

Return Value

Character

Note

SET MEMOWIDTH affects how lines are calculated. If MEMOWIDTH is set to 0 then a line terminates at a CHR(13) character. If MEMOWIDTH is greater than 0, then lines terminate either after that many characters or at a CHR(13) character.

See Also

ALINES() Function

ATLINE() Function

MEMLINES() Function

SET MEMOWIDTH Command

MOD()

MOD(nDividend, nDivisor)

Status

Slated for Version 0.6

Remarks

Returns the modulus; the remainder of the nDividend divided by the nDivisor.

Parameters

nDividend

The numeric expression to divide.

nDivisor

The numeric expression of how many times to divide nDividend.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? MOD(51,7)    && Returns 2
? MOD(11,10)   && Returns 1
? MOD(15.7/5)  && Returns 0.7
```

Return Value

Numeric

Note

The MOD() function and % operator produce identical results.

See Also

% Operator

MONTH()

MONTH(dExpression | tExpression)

Status

Slated for Version 0.6

Remarks

Returns the month number from a Date or DateTime expression.

Parameters

dExpression

Date expression used to extract the month.

tExpression

DateTime expression used to extract the month.

Example

* Demonstrate

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

SET STRICTDATE TO 0

SET DATE TO AMERICAN

ACTIVATE CONSOLE

CLEAR

? MONTH({12/14/2025})

? MONTH({2023/7/1})

Return Value

Numeric

Note

MONTH() returns a number between 1 and 12 with January is month 1 and December is the 12th month.

See Also

CMONTH() Function

DAY() Function

QUARTER() Function

YEAR() Function

NAMING()

NAMING(cExpr, nNaming)

Status

Slated for Version 0.6

Remarks

Returns cExpr using JAXBase file naming indicated by nNaming.

Parameters

cExpr

Character expression to change case.

nNaming

Integer value for which naming rule to use.

Value	Description
0	Leave as is. No case changes
1	Return string in upper case.
2	Return string in lower case.
3	Return string in lower case except for the following, if they are alpha characters: First character in the string First character after any special character or space

Example

*** Demonstrate Naming**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

cTest=_JAXFolder+"data\counties.dbf"

? NAMING(cTest,1) && Upper case

? NAMING(cTest,3) && Proper case

Return Value

Character

See Also

LOWER() Function

PROPER() Function

UPPER() Function

NDX()

NDX(nIndex [,nWorkArea | cAlias])

Status

Slated for Version 0.8

Remarks

Returns the name of the specified index in the current or specified work area or alias.

Parameters

nIndex

Numeric expression indicating what index name to return from the index list.

nWorkArea

Numeric expression that indicates which work area to use.

cAlias

Character expression that indicates which alias to use.

Example

*** Demonstrate NDX()**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties")

SET INDEX TO Counties_ByState

? NDX(1)

Return Value

Character

Note

If nIndex does not match an index entry, an empty character value is returned.

If nIndex references a structural index (Slated for Version 2.0) then the name will be returned in the format CDXNameStem + pipe character (|) + IndexName. If it is an IDX file, just the file name stem of the IDX file is returned.

See Also

INDEX Command

KEY() Function

SET INDEX Command

USE Command

DRAFT

NORMALIZE()

Status

Slated for Version 2.0

DRAFT

NUMLOCK()

NUMLOCK([lExpression])

Status

Slated for Version 1.0

Remarks

Returns the current state of the Num Lock key and optionally sets the state.

Parameters

lExpression

Logical expression that, if it is present, will set the state of the Num Lock key.

Example

```
* Demonstrate NUMLOCK()
```

```
CLEAR ALL
```

```
CLOSE ALL
```

```
SET EXCLUSIVE ON
```

```
SET CONSOLE ON
```

```
ACTIVATE CONSOLE
```

```
CLEAR
```

```
IF NUMLOCK()
```

```
    ? "NUMLOCK is set on"
```

```
ELSE
```

```
    ? "NUMLOCK is set off"
```

```
ENDIF
```

Return Value

Logical

See Also

CAPSLOCK() Function

INSMODE() Function

NVL()

NVL(eExpression1, eExpression2)

Remarks

Returns a non-null value from two expressions.

Parameters

eExpression1, eExpression2

NVL() returns eExpression2 if eExpression1 is null, otherwise returns eExpression1.

Example

```
* Demonstrate NVL
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
mNuL=.NULL.
? NVL(mNuL,"Not Null")
```

Return Value

JAXBase data type or object

Note

Use the NVL() function to prevent NULL values from being used where it is not supported or not relevant.

See Also

ISNULL() Function

SET NULL Command

OBJTOCLIENT()

AOBJTOCLIENT(ArrayName,oObject)

Status

Slated for Version 0.6

Remarks

Creates an array containing position information for the specified object.

Parameters

oObject

Object on which to return position information.

Example

```
* Demonstrate OBJTOCLIENT
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
Frm1=CREATEOBJECT("FORM")
Frm1.Height=100
Frm1.Width=200
```

```
Frm1.ADDOBJECT("lblTest1","label")
Frm1.Caption="Hello"
Frm1.lblTest.Left=30
Frm1.lblTest.Top=40
```

```
? CLIENTT0OBJ(aClient,Frm1.lblTest1)
LIST MEMORY LIKE aClient
```

Return Value

Numeric

Note

If ArrayName does not exist, it is created, otherwise it is overwritten with the correct number of corresponding elements.

Each array element contains an object with the following properties.

Property	Data Type	Description
BaseClass	Character	Baseclass of object
Height	Numeric	Height of object
Left	Numeric	Pixels oObject is from the left edge
Name	Character	Name of object
Object	Object	Object Reference
Top	Numeric	Pixels oObject is from top of object
Width	Numeric	Width of object

The first row will contain the oObject with zero as Left and Width. Subsequent rows will contain the hierarchy of objects that contain oObject, and the left and width values indicate where oObject is in relation to each object in the hierarchy.

As an example: If you create a form with a container that holds a pageframe and an object that is placed on a page, the OBJTOCLIENT() array will contain 5 rows. The first row will be for oObject, the second row will contain the page information, the third row will contain the pageframe information, the fourth row will contain the container information, and the fifth row will contain the form information. If the form is part of a form set, there will be an additional row for the formset, but all numeric properties for a formset are always zero as it is not a visual object.

See Also

Height Property
 Left Property
 Name Property
 Top Property
 Width Property

OBJTOJSON()

OBJTOJSON(oObject[, cControl])

Status

Slated for Version 0.6

Remarks

Creates a JSON string of an object's properties.

Parameters

oObject

JAXBase class object

cControl

Character expression providing control to what is saved.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties")

SCATTER NAME oRow

? OBJTOJSON(oRow)

? OBJTOJSON(oRow, "LIKE:name, stat*")

? OBJTOJSON(oRow, "EXCEPT:name, stat*")

Return Value

Character

Note

OBJTOJSON() creates a valid JSON string base on the contents of the object.

The optional cControl parameter is a character string that begins with the word LIKE: or EXCEPT: and is then followed by one or more field name skeletons, separated by commas, indicating whether to include or exclude properties with those names.

See Also

GETJSON()

JSONTOOBJ()

PUTJSON()

DRAFT

OCCURS()

OCCURS(cSearchExpression, cExpressionToSearch)

Status

Slated for Version 0.6

Remarks

Returns the number of times a string occurs in another string.

Parameters

cSearchExpression

Character expression of string to search for in cExpressionToSearch.

cExpressionToSearch

Character expression to use in searching for cSearchExpression.

Example

* Demonstrate

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

```

lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
        +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."

```

? OCCURS("f",lcString) && Returns 2

? OCCURS("F",UPPER(lcString)) && Returns 2

Return Value

Numeric

Note

OCCURS() is case sensitive.

OCCURS() returns 0 if cSearchExpression isn't found in cExpressionToSearch.

See Also

\$ Operator

AT() Function

ATLINE() Function

RAT() Function
RATLINE() Function
INLIST() Function

DRAFT

OLDVAL()

Status

Slated for Version 2.0

DRAFT

ON()

ON(cCommand [,LabelName])

Status

Slated for Version 0.6

Remarks

Returns the command assigned to the event-handling commands:

ON ESCAPE
ON KEY LABEL

Parameters

cCommand

Character expression that specifies one of the even-handling commands.

cCommand	ON Command
ERROR	ON ERROR
ESCAPE	ON ESCAPE
KEY	ON KEY

To return the command currently reserved to ON ERROR, you will use:

? ON("ERROR")

LabelName

Used with the ON KEY LABEL command to specify which key combination is assigned a command.

For a complete list of key combinations, see ON KEY LABEL.

To return the command assigned to the F1 key, use the following:

? ON("KEY","F7")

Example

```
* Demonstrate ON()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
* Assign to F12 key which will be
* available after the program ends
ON KEY LABEL F12 ? "HELLO WORLD!"
```

? ON("KEY", "F12")

Return Value

Character

Note

This function is most often used when a module needs to turn off or reassign an event command for a short period of time. It will save the current value of the event, assign it a new value, and when it's done, reassign the old value back.

This is especially useful when you want to have a local error handler designed specifically for the code that is about to be run.

If there is no command assigned to the specified event handler, ON() returns the empty string.

See Also

INKEY() Function

LASTKEY() Function

ON ERROR Command

ON ESCAPE Command

ON KEY Command

ON KEY LABEL Command

ORDER()

ORDER([nWorkArea | cAlias [, nSwitch]])

Status

Slated for Version 0.6

Remarks

Returns the name of the controlling index for the specified table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cAlias

Character expression indicating the alias of the table to check.

nSwitch

Numeric value that dictates what is returned in regards to the currently controlling index.

Value	Description
0 or omitted	Controlling index name
1	Controlling index file name
2	.T. if controlling index opened with keyword DESCENDING

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

USE (_JAXFolder+"data\counties")

SET INDEX TO Counties_ByState DESCENDING

? ORDER() && Return Name

? ORDER(0,1) && Return file name

? ORDER(0,2) && .T. if opened with DESCENDING keyword

Return Value

Character or logical

Note

Tables often have several indexes open at the same time. ODER() allows you to know which one is in control, where it's located, and if it's set for DESCENDING order. nSwitch value of 2 does not care if the index was created with a descending order, only if it was opened with the DESCENDING keyword.

A nWorkArea value of 0 means the current work area.

See Also

DESCENDING() Function

INDEX Command

SET INDEX Command

SET ORDER Command

USE Command

DRAFT

OS()

OS()

Status

Slated for Version 0.6

Remarks

Returns the operating system type

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? OS() && Returns WINDOWS or LINUX
```

Return Value

Character

See Also

GETENV()
_OS object

PADC() / PADL() / PADR()

```
PADC(eExpression, nResultSize [, cPadCharacter])
PADL(eExpression, nResultSize [, cPadCharacter])
PADR(eExpression, nResultSize [, cPadCharacter])
```

Remarks

Returns a string from an expression, padded with spaces or specified characters to the specified length, causing the eExpression to be centered, left justified, or right justified in the string.

Parameters

eExpression

Expression to convert to a string.

nResultSize

Length of the resulting string.

cPadCharacter

Character to use for padding. If omitted, space is used.

Example

```
* Demonstrate PADC(), PADL(), PADR()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? PADC("HELLO",20,".")
? PADL("HELLO",20,".")
? PADR("HELLO",20,".")
```

Return Value

Character

Note

PADC() centers the expression, PADL() right justifies the string by putting the pad characters on the left, and PADR() left justifies the expression by putting the pad characters on the right.

See Also

ALLTRIM() Function

LEFT() Function

LTRIM() Function

RIGHT() Function

STUFF() Function

TRIM() Function

PARAMETERS()

PARAMETERS()

Status

Slated for Version 0.6

Remarks

Returns the number of parameters most recently passed to a program, procedure, method, or user defined function.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

DO TEST1 WITH 1,5,.T., "HI"

RETURN

PROCEDURE TEST1

APARAMETERS aParms

? PARAMETERS()

RETURN

Return Value

Numeric

Note

PARAMETERS() is useful to know how many parameters were passed, allowing error checking and decisions on default values.

See Also

APARAMETERS Command

DO Command

FUNCTION Command

LPARAMETERS Command

PARAMETERS Command

PROCEDURE Command

PAYMENT()

PAYMENT(nPrincipal, nInterestRate, nPayments)

Status

Slated for Version 0.6

Remarks

Returns the amount of each payment on a fixed-interest loan.

Parameters

nPrincipal

Numeric expression specifying the original loan amount.

nInterestRate

Numeric expression specifying the interest rate per period. If calculating monthly payments with annual interest, divide the interest rate by 12.

nPayments

Numeric expression specifying the total number of payments to be made.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? PAYMENT(100000, .105/12, 240)  && Displays 998.38
```

Return Value

Numeric

Note

PAYMENT() assumes on-time payments and fixed interest providing the ability to create a payment plan.

See Also

CALCULATE Command

FV() Function

PV() Function

PEMSTATUS()

PEMSTATUS(oObject, cName)

Status

Slated for Version 2.0

Remarks

Returns information on a property, event, or method in an object.

Parameters

cObject

Object to use.

cName

Character expression containing the name of the property, event, or method on which to get information.

Return Value

Object

Note

PEMSTATUS returns an object with the following properties.

Property	Data Type	Description
Changed	Logical	Returns true (.T.) if a change occurred since instantiation
Inherited	Logical	Returns true (.T.) if inherited
Name	Character	Returns the name of the property, event, or method
Protected	Logical	Returns true (.T.) if protected
ReadOnly	Logical	Returns true (.T.) if read-only
Type	Character	Returns "P", "E", "M" indicating if a property, event, or method
UserDefined	Logical	Returns true (.T.) if user-defined

PI()

PI([nDecimals])

Remarks

Returns the value of PI

Parameters

nDecimals

Numeric expression 0 to 15 indicating how many decimal places to express.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? PI()
? PI(4)
? PI(10)
```

Return Value

Numeric

Note

The number of decimals shown using PI() is limited by the SET DECIMALS value. Using the nDecimals parameter, you can override the SET DECIMALS setting.

See Also

SET DECIMALS Command

PIXELPOS()

PIXELPOS(nTextPos, nPosType [, nText][, nGraphic])

Status

Slated for Version 0.6

Status

Basic support only, full support slated for Version 2.0

Remarks

Using the screen's metrics, converts a text row or column position into a graphic position.

Parameters

nTextPos

Numeric expression for the text position.

nPosType

Numeric expression for the text position type (row, col)

Value	Description
1	Returns the row conversion (default).
2	Returns the column conversion.

nText

If nText is omitted, it uses the full height or width of the screen in the conversion. Inserting a value for nText allows you to calculate for specific source text display.

nGraphic

If nGraphic is omitted, the value of the current graphic UI is used. If a graphic UI is not in use, an error is generated if nGraphic is omitted. Providing a value for nGraphic allows you to calculate for a specific target graphic display.

Example

```
* Demonstrate PIXELPOS()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
* Return the pixel row/col position coordinates
* matching on-screen location for a 80 x 26 text
* screen.
? PIXELPOS(104,2,80,1024)
? PIXELPOS(250,1,26,768)
```

Return Value

Numeric

Note

During startup, JAXBase determines the number of rows each screen has when in text mode, along with the graphical resolution of each. The PIXELPOS() function can be used to convert a text row or text column position into an equivalent pixel position so that objects written for text can be converted to the graphic UI.

During the conversion, the pixel returned will match the starting position of the upper left pixel of the text block.

Prior to Version 2.0, the utility of this functionality is limited in Windows and has basic support for Linux. Version 2 will improve upon the capabilities of converting visual elements between text and graphic user interfaces.

See Also

TEXTPOS() Function

PROGRAM()

Remarks

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Note

See Also

PRIMARY()

PRIMARY([1])

Status

Slated for Version 1

Remarks

Returns the name of the primary registered index in a JAXBase table.

Parameters

1

Returns the fully qualified name of the index registered to a JAXBase table.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

Return Value

Character

Note

If the 1 parameter is omitted, the index name part of the file is returned.

See Also

INDEX Command
REMOVE INDEX Command
SET ORDER Command
SET PRIMARY TO Command
USE Command

PROPER()

PROPER(cExpression)

Status

Slated for Version 0.6

Remarks

Changes the characters in cExpression to proper case

Parameters

cExpression

Character expression to change to proper case where the start of each word is capitalized, as in a person's name.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? PROPER("ALTER EGO")

Return Value

Charcter

See Also

ISALPHA() Function

ISLOWER() Function

ISUPPER() Function

LOWER() Function

UPPER() Function

PUTJSON()

PUTJSON(cJSON, cName, eExpression)

Status

Slated for Version 0.6

Remarks

Adds or updates a field name and value to a JSON string.

Parameters

cJSON

JSON string to update.

cName

Field name to add or update.

eExpression

JAXBase expression to place into the JSON string.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
SET STRICTDATE TO 0
ACTIVATE CONSOLE
CLEAR
```

```
cJSON=PUTJSON("", "Name", "JOHN SMITH")
cJSON=PUTJSON(cJSON, "BDate", {07/04/1999})
cJSON=PUTJSON(cJSON, "Balance", 114.37)
```

```
DIMENSION aTest[3]
aTest[1]=7
aTest[2]=14
aTest[3]=21
```

```
cJSON=PUTJSON(cJSON, "Misc", aTest)
```

```
? cJSON
```

Return Value

Character

Note

PUTJSON() will accept an empty string and create a valid JSON string with the property provided.

PUTJSON() can add any JAXBase data type or object, though care must be taken with objects and arrays, otherwise the JSON string can become quite long.

See Also

GETJSON()

JSONTOOBJ()

OBJTOJSON()

PUTJSON()

DRAFT

PUTFILE()

Status

Slated for Version 0.6

Remarks

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Note

See Also

PV()

PV(nPayment, nInterestRate, nPayments)

Status

Slated for Version 0.6

Remarks

Returns the present value amount of each payment (positive or negative).

Parameters

nPayment

Numeric expression specifying the periodic payment amount.

nInterestRate

Numeric expression specifying the interest rate per period. If calculating monthly payments with annual interest, divide the interest rate by 12.

nPayments

Numeric expression specifying the total number of payments to be made.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? PV(500, .075/12, 48)      && Displays 20679.19
```

Return Value

Numeric

Note

PV() assumes on-time payments and fixed interest providing the ability to create a investment plan.

See Also

CALCUALTE Command

FV() Function

PAYMENTS() Function

QUARTER()

QUARTER(dExpression | tExpression [, nMonth])

Remarks

Returns the quarter of the year in which a date or datetime expression occurs.

Parameters

dExpression

Date expression to get the quarter.

tExpression

DateTime expression to get the quarter.

nMonth

Starting month for the first quarter so you can base on Fiscal Year rather than Calendar Year.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? QUARTER(DATE())
```

Return Value

Numeric

Note

Returns 0 if given an empty value, otherwise returns 1 – 4 indicating quarter.

See Also

CMONTH() Function

DATE() Function

DATETIME() Function

RAND()

RAND(nSeedValue)

Remarks

Returns a random number between 0 and 1.

Parameters

nSeedValue

Numeric expression specifying the seed value which determines the RAND() sequence.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

* Get a random value from 1 to 100
? RAND(-1)
FOR I=1 TO 100
    ? INT(RAND()*10)+1
ENDFOR
```

Return Value

Numeric

Note

RAND() returns the same sequence based on the nSeedValue followed by subsequent RAND() calls without the seed.

If nSeedValue is negative, then a number based on the system clock is used.

The system initializes RAND() with the number 17,760,704 when the application initializes.

RAT()

RAT(cSearchExpression, cExpressionSearched [, nOccurrence])

Remarks

Search a character expression or memo field for the last occurrence of another character expression.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nOccurrence

Which occurrence to return. If omitted, 1 is assumed. A value less than 1 always returns 0.

Example

*** Demonstrate ASCAN()**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

aStr="FOUR SCORE"

? RAT("O",aStr) && Returns 8

? RAT("O",aStr,2) && Returns 2

? RAT("s",aStr) && Returns 0

Return Value

Numeric

Note

RAT() is case sensitive.

If the indicated cSearchExpression or occurrence of same is not found, then 0 is returned.

The value returned indicates the starting position of the first character of the matched cSearchExpression.

See Also

\$ Operator

AT() Function

AT_C() Function

ATCC() Function

ATLINE() Function

LEFT() Function

RATLINE() Function

RIGHT() Function

SUBSTR() Function

SUBSTRC() Function

STREXTRACT() Function

OCCURS() Function

INLIST() Function

DRAFT

RATC()

Status

Slated for Version 2.0

DRAFT

RATLINE()

RATLINE(cSearchExpression, cExpressionToSearch, [,nOccurence] [, cParseCharacter])

Remarks

Search an expression for the last matching expression returning which line it was found.

Parameters

cSearchExpression

The character expression to be used in the search.

cExpresionToSearch

The character expression to be used in the search. The expression may be a character field or memo field.

nOccurrence

Which occurrence to return. If omitted, 1 is assumed. A value less than 1 always returns 0.

cParseCharacter

The character to use to terminate a line, such as CHR(13). If omitted, then the SET MEMOWIDTH value is used. If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

Example

```
* Demonstrate ATLINE()
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
```

```
lcString="Fourscore and seven years ago our fathers brought forth,"+CHR(13);
        +"on this continent, a new nation, conceived in liberty, and"+CHR(13);
+"dedicated to the proposition that all men are created equal."+CHR(13);
+"Now we are engaged in a great civil war, testing whether that"+CHR(13);
+"nation, or any nation so conceived, and so dedicated, can "+CHR(13);
+"long endure."
```

```
? RATLINE("and", lcString,1,chr(13))    && Returns 2
? RATLINE("and", lcString,2,chr(13))    && Returns 1
? RATLINE("now", lcString,1,chr(13))    && Returns 0
```

Return Value

Numeric

Note

RATLINE() is a case sensitive search.

If the search is successful, the line where the match was found is returned, otherwise 0 is returned.

If using SET MEMOWIDTH and its value is set to 0 then 1 is always returned for a match.

See Also

\$ Operator

AT() Function

ATC() Function

ATCLINE() Function

ATLINE() Function

INLIST() Function

LEFT() Function

MLINE() Function

OCCURS() Function

RAT() Function

RIGHT() Function

SET MEMOWIDTH Command

SUBSTR() Function

RECCOUNT()

RECCOUNT([nWorkArea | cAlias])

Remarks

Returns the number of records in the specified table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
? RECCOUNT()
```

Return Value

Numeric

Note

If no parameter is provided, the current work area is used. If an alias is provided and not found, an error is generated.

See Also

FSIZE() Function
HEADER() Function
RECNO() Function
RECSIZE() Function

RECNO()

RECNO([nWorkArea | cAlias])

Remarks

Returns the physical record under the current record pointer.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
USE (_JAXFolder+"data\counties")  
LOCATE FOR STATE="MN"  
? RECNO()
```

Return Value

Numeric

Note

If no parameter is provided, the current work area is used. If an alias is provided and not found, an error is generated.

See Also

FSIZE() Function

HEADER() Function

RECCOUNT() Function

RECSIZE() Function

RELATION()

Status

Slated for Version 2.0

DRAFT

REMOVEPROPERTY()

REMOVEPROPERTY(oObject, cPropertyName)

Status

Slated for Version 0.6

Remarks

Removes a non-native property from an object.

Parameters

oObject

JAXBase object containing the property you wish to remove.

cPropertyName

Character expression containing the name of the property to remove.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
SCATTER NAME oRow
DISPLAY MEMO LIKE oRow
```

```
REMOVEPROPERTY(oRow, "name")
DISPLAY MEMO LIKE oRow
```

Return Value

Logical

Note

REMOVEPROPERTY() returns .T. if the property was successfully removed from the object.

You cannot remove native properties from an object, such as trying to remove the name or baseclass property from a Form object.

See Also

ADDPROPERTY() Function

AddProperty Method

REPLICATE()

REPLICATE(cExpression, nTimes)

Remarks

Returns a string of specified characters repeated a specified number of times.

Parameters

cExpression

Character expression containing one or more characters.

nTimes

Numeric expression containing the number of times to repeat cExpression.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? REPLICATE("+====", 10)
```

Return Value

Character

Note

The maximum length of the resulting character string is limited to available memory.

See Also

SPACE() Function
CLEAR Commands

REQUERY()

Status

Slated for Version 2

DRAFT

RGB()

RGB(nRed, nGreen, nBlue)

Status

Slated for Version 0.6

Remarks

Returns an integer value based on red/green/blue values.

Parameters

nRed

Numeric expression from 0 to 255 indicating intensity of the Red color.

nGreen

Numeric expression from 0 to 255 indicating intensity of the Green color.

nBlue

Numeric expression from 0 to 255 indicating intensity of the Blue color.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
oForm=CREATEOBJECT("FORM")
oForm.Show
=INKY(5)
oForm.BackColor=RGB(0,0,255) && set background to blue
=INKY(5)
oForm.BackColor=RGB(0,255,0) && set background to green
=INKY(5)
oForm.BackColor=RGB(255,255,255) && set background to white
=INKY(5)
RELEASE oForm
```

Return Value

Numeric

Note

The value created by RGB() can be used to set color properties in objects.

See Also

GETCOLOR() Function

BackColor Property

ForeColor property

DRAFT

RIGHT()

RIGHT(cExpression, nExpression)

Remarks

Returns the specified number of rightmost characters.

Parameters

cExpression

Character expression containing string which you want the rightmost characters.

nExpression

Numeric expression indicating how many characters to return.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? RIGHT("HELLO WORLD!",6) && Returns WORLD!
```

Return Value

Character

Note

If nExpression is 0, an empty string is returned. If nExpression is greater than the number of characters in the string, the whole string is returned.

See Also

AT() Function

ATLINE() Function

LEFT() Function

LTRIM() Function

RTRIM() Function

SUBSTR() Function

TRIM() Function

RIGHTC()

Status

Slated for Version 2.0

DRAFT

RLOCK()

RLOCK([nWorkArea | cAlias [, cRecordList | nRecord | Array]])

Status

Slated for Version 0.8

Remarks

Attempt to lock records in a table.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

cRecordList

Character expression holding one or more record numbers delimited with commas.

nRecord

Numeric expression holding the record number to lock.

Array

An array of numbers which correspond to the records to lock.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
IF RLOCK(0,"1,2,3")
  ? "Successful lock"
ELSE
  ? "Failed to lock"
ENDIF
```

Return Value

Logical

Note

If multiple records are being locked, then all must successfully lock, otherwise none will lock and a value of false (.F.) will be returned.

SET MULTILOCKS affects how many records you can lock. If set ON then you can only lock one record.

A lock stays in effect until released, the table is closed, or JAXBase terminates.

A lock can only be released by the user who sets it.

See Also

CLEAR Commands

CLOSE Commands

FLOCK() Function

ISFLOCKED() Function

ISRLOCKED() Function

LOCK() Function

SET MULTILOCKS Command

SET REPROCESS Command

ROLLBACK()

ROLLBACK()

Status

Slated for Version 0.8

Remarks

Rollback the current SQL transaction.

Example

```
* Interrogate the JAXCorp database that you installed
* onto your MySQL or SQL Server database server using
* the CreatedDSNFile program in the TOOLS folder.
CLEAR ALL
CLOSE ALL
CLEAR

SET NAMING TO LOWER
OPEN DATABASE JAXCorp USING DECRYPT(FILETOSTR(_JAXFolder+"\tools\dsn\JAXCorp.dsnx"))

BEGIN TRANSACTION
JAXCorp.Exec("Insert into log (event) values ("test")")

IF COMMIT()
    ? "Committed"
ELSE
    * Failed to commit, so rollback
    IF ROLLBACK()
        ? "Rolled back"
    ELSE
        ? "Failed to rollback!"
    ENDIF
ENDIF
ENDTRANSACTION
```

Return Value

Logical

Note

ROLLBACK() should only return a value of false (.F.) if there is no open transaction or the connection to the backend SQL engine is lost.

See Also

BEGIN TRANSACTION Command
 COMMIT() Function
 ENDTRANSACTION Command
 OPEN DATABASE Command

ROUND()

ROUND(nExpression, nDecimals)

Remarks

Rounds a numeric expression to a specified number of decimal places.

Parameters

nExpression

Numeric expression to round.

nDecimals

Numeric expression indicating number of decimal places to round nExpression.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

SET DECIMALS TO 4

SET FIXED ON

? ROUND(123.4567,3) && Displays 123.4570

? ROUND(123.4567,2) && Displays 123.4600

? ROUND(123.4567,0) && Displays 123.0000

Return Value

Numeric

See Also

CEILING() Function

FLOOR() Function

INT() Function

SET DECIMALS Command

SET FIXED Command

RTOD()

RTOD(nExpression)

Remarks

Changes radians to degrees.

Parameters

nExpression

Numeric expression containing radians value.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? RTOD(ACOS(0))  && Displays 90.00
? RTOD(0.7854)  && Displays 45.00
```

Return Value

Numeric

Note

Often times used in conjunction with COS(), TAN(), ACOS(), etc.

See Also

COS() Function
DTOR() Function
SIN() Function
TAN() Function

RTRIM()

RTRIM(cExpression, [cExpr2 [, cExpr3...]])

Remarks

Remove the blank spaces from the right side of the expression.

Parameters

cExpression

Character expression to use in RTRIM()

cExpr2 [, cExpr3...]

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? LEN(RTRIM("HELLO WORLD! ")))

Return Value

Character

See Also

ALLTRIM() Function

LTRIM() Function

STR() Function

TRIM() Function

SAVEPICTURE()

SAVEPICTURE(oPicture, cFileName [,nType])

Status

Slated for Version 0.6

Remarks

Saves a picture from a JAXBase picture class object.

Parameters

oPicture

JAXBase picture class object.

cFileName

Character expression holding a valid filename.

nType

Indicates in what format to save the picture based on the following table.

Value	Picture Type
0 or omitted	BMP
1	JPG
2	PNG

Return Value

Logical

Note

SAVEPICTURE() returns a logical true (.T.) value upon successful completion, otherwise an error is generated indicating the problem.

See Also

LOADPICTURE() Function

SET SAFETY Command

SEC()

SEC(tDate | cTime)

Remarks

Returns the seconds portion from a DateTime expression.

Parameters

tDate

Expression holding a DateTime value

cTime

Character expression containing a time value.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? SEC(DATETIME())
```

Return Value

Numeric

Note

SEC() will accept a Date expression, but the result will always be 0.

See Also

DATETIME() Function

HOUR() Function

MINUTE() Function

SET SECONDS Command

SECONDS()

SECONDS([tDate | cTime])

Remarks

Returns the number of seconds since midnight.

Parameters

tDate

Expression that contains a DateTime Value

cTime

Character expression that contains a Time value in 12 or 24 hour format.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? SECONDS()
? SECONDS({01/31/2025 01:00:45})  && Displays 3645
? SECONDS("12:14:00 PM")         && Displays 44040
? SECONDS("12:14 PM")            && Displays 44040
? SECONDS("13:45:18")           && Displays 49518
```

Return Value

Numeric

Note

If parameter is omitted, returns the current number of seconds since midnight. If a valid parameter is provided, returns the number of seconds from midnight represented by that value. SECONDS() will accept a Date value, but will always return 0.

See Also

DATE()

DATETIME()

HOURS()

MINUTES()

TIME()

Seconds Property

SEEK()

SEEK(eExpression [, nWorkArea | cAlias [, nIndex | cIndexName [, nSession]])

Status

Slated for Version 0.6

Remarks

Searches an indexed table for the first match of a record whose index key matches the provided eExpression.

Parameters

eExpression

Expression that holds the value to search for in the index.

nWorkArea

Numeric expression indicating the work area to check. 0 or negative number indicates current work area.

cAlias

Character expression indicating the alias of the table to check.

nIndex

Index number from the open index list. 0 or negative indicates current controlling index.

cIndexName

Index name from the open index list.

nSession

Session ID containing work area or alias. 0 or negative indicates current data session.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties")
SET INDEX TO COUNTY_NAME
```

```
* The following returns true and the record
* pointer now points at the first YORK county
* record in the table
? SEEK("YORK")
? RECNO(),state,county
```

Return Value

Logical

Note

You can only SEEK() on a table that has an open index.

eExpression must be the same type as the key expression in the searched index.

If SET NEAR is ON then if SEEK() fails to find the correct record, the next closest value is selected, otherwise the record pointer is placed after the end of file.

During the execution of the SEEK(), the data session and work area may change as needed, but will return to the original settings upon successful completion of the SEEK command.

See Also

EOF() Function
 FOUND() Function
 INDEXSEEK() Function
 LOCATE Command
 Seek Command
 SET NEAR Command
 SET KEY Command

SELECT()

SELECT ([0|1|2|3|cTableAlias])

Remarks

Returns the number of the currently selected work area, the highest unused under 36768, the lowest unused workarea, the next highest unused workarea over 36767, or the workarea of the indicated alias name.

Parameters

- 0 – Returns the number of the current workarea
- 1 – Returns the number of the highest unused work area under 36768
- 2 – Returns the number of the lowest unused work area
- 3 – Returns the number of the next highest unused work area over 36767

cTableAlias – character expression of table alias to search

Example

```
* Demonstrate SELECT()
CLEAR ALL
CLOSE ALL
SET COMPATIBLE OFF  && Default value
USE Company
? "Test 1=",SELECT(0)
? "Test 2=",SELECT(1)
? "Test 3=",SELECT(2)
? "Test 4=",SELECT(3)
? "Test 5=",SELECT("Company")
? "Test 6=",SELECT("Customer")
```

Return Value

Numeric

Note

SELECT() returns the number of the current work and is the same as SELECT(0).

An uninitialized work area is activated when a SELECT command or SELECT() function references it.

See Also

ALIAS() Function
SELECT Command

SET()

SET(cSetting [, 1|2|3|4])

Status

Full support by Version 1.0

Parameters

cSetting

Specifies a character expression of the SET command for which you want to return information. The current setting of the specified command is returned as a character or numeric string.

1|2|3|4

Specifies additional information to return about a SET command.

Setting	Description	Status
ALTERNATE	Returns the alternate output file setting	ON or OFF
ALTERNATE,1	Returns the alternate output file name	cFileName
ASSERTS		ON or OFF
AUTOINCERROR		ON or OFF
AUTOSAVE		ON or OFF
BELL,1		cWAVFileName
BLOCKSIZE		nBlockSize
CARRY		ON or OFF
CENTURY	Returns if the century is displayed as 4 digits (ON)	ON or OFF (default is ON)
CENTURY,1		nCentury (default is 19)
CENTURY,2		nRollOverYear (default is 75)
CLASSLIB	Returns list of open class libraries delimited with semi-colons	cClassLibName
COLLATE		ON or OFF
CONFIRM		ON or OFF
CONSOLE		ON or OFF
COVERAGE		ON or OFF
COVERAGE,1		cFileName
CPCOMPILE		nCodePage
CPDIALOG		ON or OFF
CURRENCY		LEFT or RIGHT
CURRENCY,1		cCurrencySymbol
CURSOR		ON or OFF
DATABASE		cDataBaseName
DATASESSION		nDataSession
DATE		AMERICAN, ANSI, BRITISH, FRENCH, GERMAN, ITALIAN, JAPAN, USA, MDY, DMY or YMD
DATE,1		Date Order: 0 – MDY1–MDY2–YMD
DEBUG		ON or OFF
DEBUGOUT		cFileName
DECIMALS		nDecimals

DEFAULT	Current default folder. JAXBase starts in the JAXBase folder found in Documents.	cDirectory
DEFINITION	Provides a list of definition files recognized by the system, separated by semi colons.	cFileName
DELETED		ON or OFF
DELIMITERS		ON or OFF
DELIMITERS,1		cDelimiters
DEVELOPMENT		ON or OFF
DEVICE		SCREEN, PRINTER, or FILE
DEVICE,1		cDeviceName
ESCAPE		ON or OFF
EVENTLIST		Comma delimited list of events
EVENTTRACKING		ON or OFF
EVENTTRACKING,1		cFileName
EXACT		ON or OFF
EXCLUSIVE		ON or OFF
FDOW		nDayOfWeek
FIELDS		ON or OFF
FIELDS,1		cFieldName1, cFieldName2,...
FIELDS,2		LOCAL or GLOBAL
FILTER		cFilterExpression
FIXED		ON or OFF
FULLPATH		ON or OFF
FWEEK		nWeekNumber
HEADINGS		ON or OFF
HELP		ON or OFF
HELP,1		cFileName
HELP,2		cCollectionURL
HELP,3		System
HOURS		12 or 24
INDEX		cIndexExpression
KEY		eExpression2, eExpression3
KEY,1		eExpression2
KEY,2		eExpression3
KEYCOMP		DOS or WINDOWS
LIBRARY		cLibraryName
LOCK		ON or OFF
LOGERRORS		ON or OFF
MACKEY		cKey
MARGIN		nMargin
MEMOWIDTH		nWidth (default is 0)
MESSAGE		nRow
MESSAGE,1		cMessageText
MOUSE		ON or OFF
MOUSE,1		nSensitivity
MULTILOCKS		ON or OFF
NEAR		ON or OFF
NOCPTRANS		comma delimited list of fields
NOTIFY		ON or OFF
NOTIFY,1		ON or OFF
NULL		ON or OFF
NULLDISPLAY		cDisplayString (default is .NULL.)

ODOMETER		nNumberOfRecords (default is 100)
ORDER		TAG cTagName OF cCDXFileName, cIDXFileName or blank
PALLETTE		ON or OFF
PATH		cPath
POINT		cDecimalPtChar (default is “.”)
PRINTER		ON or OFF
PRINTER,1		cFileName or cPortName
PRINTER,2		Default Windows printer name
PRINTER,3		Default JAXBase printer name
PROCEDURE		cPathAndName
READBORDER		ON or OFF
REFRESH		nSeconds1
REFRESH,1		nSeconds2
REPROCESS		Current session setting
REPROCESS,1		System session setting
REPROCESS,2		Current session setting type 0 is returned if REPROCESS is set to attempts. 1 is returned if REPROCESS is set to seconds.
REPROCESS,3		System session setting type 0 is returned if REPROCESS is set to attempts. 1 is returned if REPROCESS is set to seconds.
RESOURCE		ON or OFF
RESOURCE,1		cFileName
SAFETY		ON or OFF
SECONDS		ON or OFF
SEPARATOR		cSeparatorChar
SPACE		ON or OFF
SQLBUFFERING		ON or OFF
STATUS		ON or OFF
STATUSBAR		ON or OFF
STRICTDATE		ON or OFF
SYSFORMATS		ON or OFF
SYSTEMENU		ON, OFF, or AUTOMATIC
TABLEPROMPT		ON or OFF
TABLEVALIDATE		nLevel
TALK		ON or OFF
TALK,1		WINDOW, NOWINDOW or cWindowName
TEMPFOLDER	Folder where temp files are written by JAXBase	
TEXTMERGE		ON or OFF
TEXTMERGE,1		cLeftDelimiter and cRightDelimiter
TEXTMERGE,2		cFileName
TEXTMERGE,3		SHOW or NOSHOW

TEXTMERGE,4		Evaluate source of TEXT...ENDTEXT call and return nesting level
TOPIC		cHelpTopicName Expression
TOPIC,1		nContextID
UDFPARMS		VALUE or REFERENCE
UNIQUE		ON or OFF

Return Value

JAXBase data type

See Also

DISPLAY STATUS Command

LIST Commands

SET Command Overview

SETFLDSTATE()

Status

Slated for Version 2.0

DRAFT

SIGN()

SIGN(nExpression)

Remarks

Returns -1, 0, or 1 depending on if the number is negative, zero, or positive.

Parameters

nExpression

Numeric expression to check.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

```
? SIGN(PI()) && Displays 1  
? SIGN(-423) && Displays -1
```

Return Value

Numeric

See Also

ABS() Function

COS() Function

SIN() Function

SIN()

SIN(nExpression)

Status

Slated for Version 0.6

Remarks

Convert degrees (expressed in radians) to the sine value.

Parameters

nExpression

Numeric expression holding degree value expressed as radians

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? SIN(DTOR(45)) && Displays 0.71
```

Return Value

Numeric

See Also

ACOS() Function

COS() Function

DTOR() Function

RTOD() Function

TAN() Function

SET DECIMALS Command

SOUNDEX()

SOUNDEX(cExpression)

Status

Slated for Version 0.6

Remarks

Returns a phonetic representation of the specified character expression.

Parameters

cExpression

Character expression to use in the soundex calculation.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? SOUNDEX("JAXBase")
? SOUNDEX("Allen")
```

Return Value

Character

Note

The formula for creating a soundex representation is taken from
<https://physics.nist.gov/cuu/Reference/soundex.html>

SOUNDEX() returns a four-character string and is not case sensitive.

See Also

DIFFERENCE() Function

SPACE()

SPACE(nExpression)

Remarks

Returns a string containing nExpression number of spaces.

Parameters

nExpression

Numeric expression specifying how many spaces to return.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR

? SPACE(10)+"HELLO!"
? REPLICATE("-",8)
```

Return Value

Character

See Also

PADC() | PADL() | PADR() Functions

REPLICATE() Function

SQRT()

SQRT(nExpression)

Remarks

Returns the square root of a number.

Parameters

nExpression

Numeric value from which to get the square root.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? SQRT(4) && Returns 2

? SQRT(144) && Returns 12

? SQRT(42) && Returns "The Answer to Life, the Universe, and Everything" (kidding) 6.48

Return Value

Numeric

Note

The number of decimal places returned is subject to SET DECIMALS.

See Also

COS() Function

INT() Function

SIN() Function

TAN() Function

SET DECIMALS Command

STR()

STR(nExpression [,nLength [,nDecimals]])

Remarks

Returns a character expression from a numeric with formatting options.

Parameters

nExpression

Numeric expression to convert into a string.

nLength

Numeric expression indicating the length of the string.

nDecimals

Numeric expression indicating the number of decimal places.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? STR(10,6,2)    && Returns " 10.00"
? STR(99998473,6,2) && Returns "***.***"
```

Return Value

Character

Note

If the number will not fit into the space indicated with nLength, then an invalid number character string will appear with all asterisks.

STR() will round the value displayed if the number of decimals in nExpression exceeds nDecimals.

See Also

VAL() Function

STRCONV() Function

STRCONV()

Status

Slated for Version 2

DRAFT

STREXTRACT()

STREXTRACT(cSearchExpression, cBeginDelim [, cEndDelim [, nOccurrence [, nFlag]]])

Remarks

Extracts a string bounded by two delimiters.

Parameters

cSearchExpression

Character expression used to extract information bounded by delimiters.

cBeginDelim

Character expression holding the left delimiter.

cEndDelim

Character expression holding the right delimiter.

nOccurrence

Numeric expression indicating which occurrence to retrieve.

nFlag

Numeric expression indicating options on the retrieval. The values are additive.

Value	Description
1	Case-insensitive search.
2	End delimiter optional. If the end delimiter is not found, the rest of the string is returned.
4	Include the delimiters in the expression.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
lcStr="Title: Bourne Supremacy Length: 108m Gross: $291M"
? STREXTRACT(lcStr,"Title: "," Length:")      && Returns "Bourne Supremacy"
? STREXTRACT(lcStr,"Length: "," Gross:")      && Returns "108m"
? STREXTRACT(lcStr,"gross: ","",1,5)          && Returns "$291M"
```

```
lcStr="123|144|155"
? STREXTRACT(lcStr,"|","|",2,2)                && Returns "155"
```

Return Value

Character

Note

The default is to perform a case sensitive search, with both delimiters required (nFlag omitted or 0).

If cBeginDelim is empty, then the search starts at the beginning of the string. If cEndDelim is empty, the search ends at the end of the string.

See Also

ATOKENS() Function

STRTRAN() Function

AT() Function

RAT() Function

SUBSTR() Function

DRAFT

STRFORMAT()

STRFORMAT(cExpression [, eExpression1 [, eExpression2 ...]])

Status

Slated for Version 1.0

Remarks

Merges values into a string formatted use specifier codes.

Parameters

cExpression

Character expression containing codes that indicate how to merge the eExpressions into the string.

eExpression1, eExpression2 ...

Expressions that will be merged into cExpression.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

*** The following returns "Dear John, you own me \$12.43"**

? STRFORMAT("Dear {0}, you owe me {0:c}", "John", 12.43)

? STRFORMAT("{0:0.000}", 523.99437) && Returns "523.994"

*** Returns "one for the money and 2 for the show"**

? STRFORMAT("{%1} for the money and {%2} for the show", "one", 2)

? STRFORMAT("{0} + {0} = {1}", 1, 2) && Returns "1 + 1 = 2"

Return Value

Character

Note

STRFORMAT() uses codes like some popular languages where each {code} token accepts a value from the eExpression list. If there are 7 {code} tokens in the string, 7 eExpressions are expected. If there are not the right amount, an error is generated.

String format data {x} simply requires that you have the same number of eExpressions as the different {x} codes. Thus, if you have {0}, {1}, and {2} but the first two are repeated several times, it only requires you to have three eExpressions to convert.

The following table shows various codes to use in the {} brackets.

When you see x:, that stands for a number starting at zero and increasing with each {code} token. Such as the following example: "{0:c}, {1:c}, {2}"

Number Formats

Code	Data Type	Description
{x:c}		
{x:e}		
{x:f}		
{x:g}		
{x:n}		

Custom Number Formats

Code	Data Type	Description
{x:00.00}		
{x:(0).##}		
{x:0.000}		
{x:0,0}		
{x:0,0%}		

Date and Time Formats

Code	Data Type	Description
{x:d}		
{x:D}		
{x:t}		
{x:T}		
{x:f} or {x:F}		
{x:g} or {x:G}		
{x:M}		
{x:r}		
{x:s}		
{x:u}		
{x:U}		
{x:Y}		

String Format

Code	Data Type	Description
{x}		
{x,10}		
{x,-10}		

See Also

STR()

TOSTRING()

TRANSFORM()

DRAFT

STRTOFILE()

STRTOFILE(cExpression, cFileName [,nFlag])

Remarks

Write a file or append a string to a file.

Parameters

cExpression

Character expression holding a string to write to the file.

cFileName

Character expression holding file name to which the string is written.

nFlag

Numeric expression indicating how to write the data to the file.

The default is 0 for Overwrite with no terminator. Values are additive except for 4 & 8 which are mutually exclusive and will generate an error if both bits are set.

Value	Bit	Description
1	0001	Overwrite if 0, while 1 indicates to append to the file.
2	0010	Add text terminator if bit is set. Windows: character bytes 0x0A and 0x0D (10 and 13) Linux: character byte 0x0D (13)
4	0100	Unicode support slated for Version 2
8	1000	Unicode support slated for Version 2

Return Value

Numeric

Note

The return value is how many bytes were written to the file.

See Also

FILETOSTR() Function

SET SAFETY Command

FILE Class

STRTRAN()

STRTRAN(cExpression, cSearchFor, cReplaceWith)

Remarks

Replace one or more occurrences of a string found inside cExpression.

Parameters

cExpression

The character string expression to search.

cSearchFor

The character expression to search for.

cReplaceWith

The character expression to replace matches of cSearchFor.

nStartOccurence

Numeric expression indicating which occurrence to start with.

nNumberToReplace

Numeric expression indicating how many occurrences to replace.

nFlags

nFlags	Description
0 or omitted	Search is case-sensitive, and replacement is performed with exact cReplacement text (default).
1	Search is case-insensitive, and replacement is performed with exact cReplacement text.
2	Search is case-sensitive, and replacement is performed with cReplacement altered to match the case of the found.
3	Search is case-insensitive, and replacement is performed with cReplacement altered to match the case of the found.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

```
lcStr="The quick brown fox jumped over the lazy dog"
```

```
? STRTRAN(lcStr,"T","t",-1,-1,1)
```

```
? STRTRAN(lcStr,"dog","cat")
```

```
? STRTRAN(lcStr," ","_",3,2,0)
```

Return Value

Character

Note

Omitting everything after cReplaceWith or using -1 for nStartOccurence and nNumberToReplace will cause all occurrences to be replaced.

See Also

ATOKENS() Function

CHRTRAN() Function

STUFF() Function

DRAFT

STUFF()

STUFF(cExpression, nStart, nLength, cReplace)

Status

Slated for Version 0.6

Remarks

Creates a character string by replacing a number of characters in an expression with another expression.

Parameters

cExpression

Original character string that will be modified.

nStart

Numeric expression indicating which character to start the replacement.

nLength

Numeric expression indicating how many characters in the original string will be removed starting at the tnStart character.

cReplace

String to insert in that location.

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Character

Note

If nStart and nLength are greater than 0, then starting at the character indicated by nStart, that character and for a total of nLength, those characters will be removed and cReplace put in that location instead.

If nStart is greater than the length of cExpression, cReplace will be appended to cExpression.

If nStart is 0, then cReplace will be placed before the first character of cExpression and nStart will be ignored.

If nLength is 0, then cReplace will be placed before the character indicated by nStart and inserted. None of the cExpression characters will be removed.

If nStart or nLength are negative, an error will be generated.

See Also

LEFT() Function

PADC() | PADL() | PADR() Functions

RIGHT() Function

STREXTRACT() Function

STRTRAN() Function

SUBSTR() Function

STUFFC()

Status

Slated for Version 2.0

DRAFT

SUBSTR()

SUBSTR(cExpression, nStart [, nLength])

Remarks

Returns a character string by copying characters starting at nStart for nLength characters.

Parameters

cExpression

Expression to extract the sub-string from.

nStart

Numeric expression containing the starting position of the character to start the copy.

nLength

Numeric expression containing the number of characters to copy.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? SUBSTR("LeftSide",1,4)
? SUBSTR("Know your rights",11,5)
```

Return Value

Character

Note

If nLength is greater than the number of characters available after nStart, then only the remaining characters will be copied. If nStart is past the end of cExpression, an empty string will be returned; the same as if nLength is 0.

If nStart or nLength are negative, an error is generated.

See Also

LEFT() Function

RIGHT() Function

STREXTRACT() Function

STUFF() Function

SUBSTRC()

Status

Slated for Version 2.0

DRAFT

SYSID()

SYSID()

Status

Slated for Version 0.6

Remarks

Returns a unique sortable time-based character ID.

Example

```
* Demonstrate SYSID()  
CLEAR ALL  
CLOSE ALL  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR  
  
FOR I=1 to 20  
    ? SYSID()  
ENDFOR
```

Return Value

Character

Notes

The SYSID() is a base 36 representation of the number of 1/10,000 seconds that have passed since January 1, 0001.

SYSID() is composed of the upper case letters A – Z and the numbers 0 – 9.

SYSID() is intended only as a method for stamping when something happened relative to other similarly stamped events.

SYSID() is guaranteed to be unique for the run-time session, no matter how many threads may be opened by the application.

TAN()

TAN(nExpression)

Remarks

Returns the tangent of an angle.

Parameters

Returns the tangent of an angle expressed in radians.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? TAN(DTOR(0))    && Returns 0.00
? TAN(DTOR(45))   && Returns 1.00
? TAN(DTOR(90))   && Generates meaningless large number
```

Return Value

Numeric

See Also

COS() Function
DTOR() Function
RTOD() Function
SET DECIMALS Command
SINE() Function

TARGET()

Status

Slated for Version 2

DRAFT

TEXTMERGE()

Status

Slated for Version 0.6

Remarks

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Note

See Also

TEXTPOS()

TEXTPOS(nPixelPos, nPosType [, nGraphic] [, nText])

Status

Slated for Version 0.6

Status

Basic support only, full support slated for Version 2.0

Remarks

Using the screen's metrics, converts a text row or column position into a graphic position.

Parameters

nPixelPos

Numeric expression for the pixel position.

nPosType

Numeric expression for the text position type (row, col)

Value	Description
1	Returns the row conversion (default).
2	Returns the column conversion.

nGraphic

If nGraphic is omitted, the value of the current graphic UI is used. If a graphic UI is not in use, an error is generated if nGraphic is omitted. Providing a value for nGraphic allows you to calculate for a specific target graphic display.

nText

If nText is omitted, it uses the full height or width of the screen in the conversion. Inserting a value for nText allows you to calculate for specific source text display.

Example

```
* Demonstrate TEXTPOS()
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
* Return the text row/col position coordinates
* matching on-screen location for a 1024x768
* graphic area.
? TEXTPOS(104,2,1024,80)
? TESTPOS(250,1,768,26)
```

Return Value

Numeric

Note

During startup, JAXBase determines the number of rows each screen has when in text mode, along with the graphical resolution of each. The TEXTPOS() function can be used to convert a pixel position into an equivalent text row or text column position so that objects written for text can be converted to the TEXT only area.

During the conversion, the text position returned will match the starting position of the upper left pixel of the text block rounded down. A pixel position of 3 will return 0 for the text position.

Prior to Version 2.0, the utility of this functionality is limited in Windows and has basic support for Linux. Version 2 will improve upon the capabilities of converting visual elements between text and graphic user interfaces.

See Also

PIXELPOS() Function

TIME()

TIME([nExpression | tExpression])

Remarks

Returns the current time or converts a value to the current time.

Parameters

nExpression

Numeric expression containing a value of seconds to convert to a Time string.

tExpression

DateTime expression which will have its Time component returned.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET HOURS TO 12
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? TIME(60)
? DATETIME(), TIME(DATETIME())
```

Return Value

Character

Note

If the value of nExpression is less than 0 then an empty character value is returned. If the value is greater than 86399 then the nExpression % 86400 is used to return the time. Any digits to the right of the decimal point are rounded off to milliseconds (3 decimal places).

The TIME() command is affected by the SET HOURS command.

If no parameters are included, TIME() returns the current system time from the operating system.

See Also

% Operator

DATETIME() Function

MOD Command

SET HOURS Command

TOSEC() Function

DRAFT

TRANSFORM()

Status

Slated for Version 0.6

Remarks

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Note

See Also

TRIM()

TRIM(cExpression)

Remarks

Remove the blank spaces from the right side of the expression.

Parameters

cExpression

Character expression to use in TRIM()

Example

* Demonstrate TRIM()

CLEAR ALL

CLOSE ALL

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? LEN(TRIM("HELLO WORLD!"))

Return Value

Character

See Also

ALLTRIM() Function

LTRIM() Function

STR() Function

RTRIM() Function

TTOC()

TTOC(tDateTime [, nFlag])

Remarks

Converts the given DateTime value to a formatted string.

Parameters

tDateTime

Expression holding the DateTime value to convert.

nFlag

Numeric expression indicating format.

Value	Description
0 or omitted	Returns the expression in the current regional format. SET CENTURY, SET HOUR, and SET SECONDS affect the result.
1	Returns expression suitable for indexing in the format yyyyymmddhhmmss. Hours are expressed in military time. SET CENTURY, SET HOUR, SET SECONDS settings do not affect the result.
2	Returns the time expression in the format hh:mm:ss or hh:mm:ss AM/PM SET SECONDS will affect if the :ss portion is returned. SET HOUR will affect if it returns military time or standard AM/PM time.
3	Returns the DateTime expression in the format yyyy-mm-ddThh:mm:ss which is useful when setting a date in a SQL backend database. Hours are expressed in military time. SET CENTURY, SET HOUR, SET SECONDS settings do not affect the result.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? TTOC(DATETIME())
? TTOC(DATETIME(),1)
? TTOC(DATETIME(),2)
? TTOC(DATETIME(),3)
```

Return Value

Character

Note

An invalid parameter sent as either parameter returns an empty value.

TTOC() will also accept a Date value, but the time is always returned as midnight (00:00:00).

See Also

DATE() Function

DATETIME() Function

HOUR() Function

MINUTE() Function

SEC() Function

SECOND() Function

SET HOURS Command

SET DATE Command

SET SECONDS Command

TIME() Function

TOSEC() Function

TTOD() Function

TTOD()

TTOD(tDateTime)

Remarks

Returns a DateTime value as a Date value.

Parameters

tDateTime

Expression containing the DateTime value to convert.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
a=DATETIME()
? a
? TTOD(a)
```

Return Value

Date

See Also

DATE() Function
DATETIME() Function
HOUR() Function
MINUTE() Function
SEC() Function
SECONDS() Function
SET CENTURY Command
SET HOUR Command
SET SECOND Command
TIME() Function
TOSEC() Function
TTOC() Function

TOSEC()

TOSEC(cTime | tTime)

Status

Slated for Version 0.6

Remarks

Returns a numeric value indicating number of seconds since midnight for the specified value.

Parameters

cTime

Expression containing the time value as a string in either 12 or 24 hour format.

tTime

Expression containing the DateTime value to convert.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
ACTIVATE CONSOLE
CLEAR
```

```
? TOSEC(DATETIME())
? TTOD("01:14:15 AM")
? TTOD("13:14:15")
? TTOD("01:14")
? TTOD("13:14")
```

Return Value

Date

See Also

DATE() Function

DATETIME() Function

HOUR() Function

MINUTE() Function

SEC() Function

SECONDS() Function

SET CENTURY Command

SET HOUR Command

SET SECOND Command

TIME() Function

TTOC() Function

TXNLEVEL()

Status

Slated for Version 2.0

DRAFT

TXTWIDTH()

TXTWIDTH(cExpression, cFont, nSize [, cStyle])

Status

Slated for Version 0.6

Remarks

Returns the width of a string in pixels based on the current or specified font information.

Parameters

cExpression

Expression containing the character string to measure.

cFont

Required character expression holding the name of the font to use. The font name will match the first available font in a case-insensitive search. In Windows, "COURIER" will usually match "Courier New" while in Linux it will usually match "Courier".

nSize

Required numeric expression indicating font size.

cStyle

Optional character expression indicating font styles. Omitted or empty string means normal style.

Character	Font Style
B	Bold
I	Italic
O	Outline
Q	Opaque
S	Shadow
-	Strikeout
T	Transparent
U	Underline

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? TXTWIDTH("hello!", "Courier", 10)
```


Return Value

Numeric

Note

If the font and size combination is not available, TXTWIDTH will return 0.

See Also

GETFONT() Function

LEN() Function

DRAFT

TYPE()

TYPE(cExpression)

Remarks

Returns the type of the expression.

Parameters

cExpression

Character expression holding the expression whose type will be returned.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
DIMENSION aTest[2]
s="HELLO"
a = 4
b = 2
n=.NULL.
```

```
? TYPE('s')      && Returns C
? TYPE('a')      && Returns N
? TYPE('a>b')    && Returns L
? TYPE('aTest')  && Returns A
? TYPE('aTest[1]') && Returns L
? TYPE('Z')      && Returns U
? TYPE('n')      && Returns X
? TYPE('DATE()') && Returns D
? TYPE('{1/2/1996}') && Returns D
? TYPE('4+3')    && Returns N
```

Return Value

Characer

Note

If you pass an array without element information, the value returned is "A". To get the value of an array element, provide the element information.

cExpression must contain a valid expression. In the above example, TYPE(s) would be the same as TYPE('HELLO') and would generate a variable not found error.

TYPE() should not be used to evaluate UDFs (user defined functions) as an error can be generated by the UDF if an incorrect parameter is passed.

The possible values returned are as follows.

Value	Description
A	Array – must include element information to get type
C	Character, Varchar, Varchar (binary)
D	Date
G	General
L	Logical
M	Memo field
N	Numeric, Float, Double, or Integer
O	Object
Q	Varbinary field
T	DateTime
U	Unidentified
W	Blob field
X	Null
Y	Currency Field

See Also

DATE() Function

DATETIME() Function

EVALUATE() Function

VARTYPE() Function

ISNULL() Function

UNBINDEVENTS()

Status

Slated for Version 2.0

DRAFT

UPDATED()

Status

Slated for Version 0.8

Remarks

Parameters

Example

```
* Demonstrate  
CLEAR ALL  
CLOSE ALL  
SET EXCLUSIVE ON  
SET CONSOLE ON  
ACTIVATE CONSOLE  
CLEAR
```

Return Value

Note

See Also

UPPER()

UPPER(cExpression)

Remarks

Changes the characters in cExpression to upper case

Parameters

cExpression

Character expression to change to upper case.

Example

*** Demonstrate**

CLEAR ALL

CLOSE ALL

SET EXCLUSIVE ON

SET CONSOLE ON

ACTIVATE CONSOLE

CLEAR

? UPPER("alter ego")

Return Value

Charcter

See Also

ISALPHA() Function

ISLOWER() Function

ISUPPER() Function

LOWER() Function

PROPER() Function

USED()

USED([nWorkarea | cAlias])

Remarks

Returns if a table is in use in the specified work area, or if the specified alias name is in use.

Parameters

nWorkArea

Numeric expression indicating the work area to check.

cTableAlias

Character expression indicating the alias of the table to check.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET EXCLUSIVE ON
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
USE (_JAXFolder+"data\counties") ALIAS JC
```

```
? USED()
? USED(select())
? USED('JC')
```

Return Value

Logical

Note

If no parameter is provided, then USED() uses the current work area.

If the nWorkarea is uninitialized or has no open table, USED() returns false (.F.).

If the cAlias value is not found, USED() returns false (.F.).

See Also

ALIAS() Function

SELECT() Function

AUSED() Function

VAL()

VAL(cExpression)

Remarks

Convert a string into a numeric expression.

Parameters

cExpression

Character expression to convert into a numeric expression.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? VAL("1")
? VAL("33.14")
? VAL("ABC")
```

Return Value

Numeric

Note

If the character expression does not contain a numeric value, 0 is returned.

See Also

SET DECIMALS Command
STR() Function

VARTYPE()

VARTYPE(eExpression)

Remarks

Returns the data type of the expression

Parameters

eExpression

Expression for which a type is returned.

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
a=7
```

```
? VARTYPE(1+2)    && Returns N
? VARTYPE(a)      && Returns N
? VARTYPE("a")    && Returns C
```

Return Value

Character

Note

The possible values returned are as follows.

Value	Description
A	Array – must include element information to get type
C	Character, Varchar, Varchar (binary)
D	Date
G	General
L	Logical
M	Memo field
N	Numeric, Float, Double, or Integer
O	Object
Q	Varbinary field
T	DateTime
U	Unidentified
W	Blob field
X	Null
Y	Currency Field

See Also

DATE() Function

DATETIME() Function

EVALUATE() Function

VARTYPE() Function

ISNULL() Function

DRAFT

WEEK()

WEEK(dExpression | tExpression)

Status

Slated for Version 0.6

Remarks

Returns a numeric value indicating the week of the year.

Parameters

dExpression

Date expression to be used to return week number

tExpression

DateTime expression to be used to return week number

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? WEEK( DATE( ) )
```

Return Value

Numeric

Note

WEEK() is affected by SET FDOW and SET FWEEK Command.

See Also

CDOW() Function

DAY() Function

DOW() Function

SET FDOW Command

SET FWEEK Command

XMLTOCURSOR()

XMLTOCURSOR(cExpression[, cCursor [,nFlags]])

Status

Slated for Version 2.0

Remarks

Converts an XML expression to a cursor and returns number of records created.

Parameters

cExpression

Expression holding XML text.

cCursor

Name of cursor to create.

nFlags

Specifies how to manage the XML text.

nFlag (Additive)	Description
1	Preserves white space in the data
2	Appends data to cCursor
4	Map character fields to Varchar if set, otherwise character fields
8	Ignore text data over 254 characters
16	Ignore binary fields over 254 characters

Return Value

Numeric

Note

If an error occurs, the created cursor may be lost and the appended cursor may be partially updated.

See Also

CURSORTOXML()

CURSORTOJSON()

JSONTOCURSOR()

YEAR()

YEAR(dExpression | tExpression)

Remarks

Returns the year portion of a Date or DateTime expression.

Parameters

dExpression

Date expression to be used to return week number

tExpression

DateTime expression to be used to return week number

Example

```
* Demonstrate
CLEAR ALL
CLOSE ALL
SET CONSOLE ON
ACTIVATE CONSOLE
CLEAR
```

```
? YEAR(DATE())
```

Return Value

Numeric

Note

YEAR() returns the full year (1 – 9999) and is not affected by the SET CENTURY Command.

See Also

DATE() Function

DATETIME() Function

SET CENTURY Command

Notes

DRAFT

DRAFT

JAXBase Environment Objects and Variables

The JAXBase system has several objects and variables that provide information about the JAXBase environment, the operating system that it's running under, and the user interfaces. The following is a comprehensive description of each.

DRAFT

JAXBase Environment Variables

JAXBase environment variables are simple data types that hold a value related to the JAX environment. Typically, these variables hold something like a folder name or a value that reflects a setting in the application's configuration.

Most JAXBase environment variables cannot be altered and will generate an error if you attempt to alter them.

JAXFolder

The system folder that contains the TOOLS, GRAPHICS, EXAMPLES, and other folders that are useful for developing solutions. This variable is read-only and is set up during installation. It can only be changed by running the installer again and installing the system folder to a new location.

JAXHome

This points to the folder where the JAXBase system resides. Specifically, the IDE, runtime, and help files. These files should not be altered, but you may need to alter or append the configuration files stored here. This variable is read-only and set at startup.

JAXTemp

This points to the JAXBase temporary folder, which on Windows is normally C:\PROGRAMDATA\JAXBase\Temp. This variable is read-only and can be altered by changing the STARTUP.JBI file. Care should be taken when altering JBI files as it can cause unexpected issues if something is set incorrectly.

JAXBase Environment Objects and Arrays

All environment variables contain read-only data that can be used to find out more about the system, or provide information useful for initiating a process.

Environment variables are not JAXBase classes, but they do have similarities to class objects.

DRAFT

_Jax

The `_JAX` environment variable provides current information of the JAXBase system.

Property	Data Type	Description
Path	Character	Path where this application is located
Version	Numeric	Version is returned in Major.Minor format (ex: 2.14)
X64	Logical	.T. indicates the 64-bit version
Config	Character	Configuration file name
DefaultFolder	Character	Same as SET("Default") – returns current default folder
PathList	Character	Search paths delimited by semicolons
ProjectEditor	Character	Location and name of the editor
TableEditor	Character	Location and name of the editor
FileEditor	Character	Location and name of the editor
ImageEditor	Character	Location and name of the editor
ClassEditor	Character	Location and name of the editor
FormEditor	Character	Location and name of the editor
MenuEditor	Character	Location and name of the editor
ReportEditor	Character	Location and name of the editor
LabelEditor	Character	Location and name of the editor

Note

The `_JAX` environment object is read only. Changes can be made via SET commands.

_CPUS

The `_CPUS` array contains `_CPU` objects that provide information regarding the computer running JAXBase. This object is initialized at startup and remains static for the duration of the session.

Property	Data Type	Description
CPUID	Numeric	CPU number (1 – x)
CPUName	Character	Name of CPU
CoreCount	Numeric	Number of cores reported by the OS
LogicalCores	Numeric	Number of logical cores reported by the OS
ClockSpeed	Numeric	Clock speed reported by the OS

Note

The `_CPU` environment array objects are read only.

_DRIVES

Status

Full support slated for Version 2

_DRIVES is an object array that contains _DRIVE objects that report on drive/volume/share information received from the operating system. The array is updated automatically.

Property	Data Type	Description
Free	Numeric	Remaining free megabytes
DVSName	Character	Name of device, volume, or share
Online	Logical	Online status
Removable	Logical	Removable (network and removable drives list as .T.)
Size	Numeric	Physical Size in megabytes – 1 if less than 1 MB in size
Type	Numeric	Fixed, Removable, Share, etc.

Note

The _Drives environment array objects are read only

_Environment

Status

Full support slated for Version 1.0

Property	Data Type	Description
EOL	Character	End of line character(s)
MachID	Character	Machine network ID
MemoryFree	Numeric	Available free memory in bytes
MemoryRAM	Numeric	Total amount of physical RAM
MemoryTotal	Numeric	Total amount of RAM including virtual
UserID	Character	Logged in User ID

_Header

Status

Full support slated for Version 0.8

Note

The `_Header` environment object is read only

DRAFT

_Keyboard

Status

Full support slated for Version 1

The `_KEYBOARD` class is constantly updated by a background thread, providing information on the current keyboard status.

Property	Data Type	Description
KeyCount	Numeric	Number of keys on the keyboard
FKeyCount	Numeric	Number of function keys (F1 – Fx)
LastKey	Numeric	Value of last key pressed or 0
CurrentKey	Numeric	Current key that is down (0 if none are currently pressed down)
Shift	Logical	.T. if Shift key is down
LeftAlt	Logical	.T. if Left Alt key is down
RightAlt	Logical	.T. if Right Alt key is down
Ctrl	Logical	.T. if Control key is down
Fn	Logical	.T. if Fn (function) key is held down
CapsLock	Logical	.T. if caps lock is set
InsMode	Logical	.T. if insert mode is set
NumLock	Logical	.T. if number lock mode is set (arrows do not work)
ScrollLock	Logical	.T. if scroll lock is set
Queue	Numeric	Length of queue
Next	Numeric	Next key at top of queue – remove using INKEY() or CLEAR command

_MONITORS

Status

Full support slated for Version 2.0

_MONITORS is an array of MONITOR objects which provide information on connected monitors reported by the operating system. Each monitor is represented by a separate element in the array.

Each element is updated when a change to the monitor properties it represents is detected.

Each MONITOR object is as follows and all properties are read-only.

Property	Data Type	Description
ColorDepth	Numeric	Bits of color 1=monochrome, 4=16 colors, etc.
Height	Numeric	Number of pixel rows
ID	Character	Eight-character session ID
Order	Numeric	Based on the monitor virtual order chart below.
ScreenSaver	Logical	.T. if monitor appears to be in screen saver mode
Width		Number of pixel columns

Monitor Order Chart

13	14	15	16
9	10	11	12
5	6	7	8
1	2	3	4

Note

The _MONITORS environment objects are read only.

_Mouse

Status

Full support slated for Version 1.0

The _Mouse class is constantly updated so that you can get information quickly from one source.

Note

The _MOUSE environment object is read only.

DRAFT

_OS

Status

Full support slated for Version 1.0

The `_OS` object provides information on the operating system.

DRAFT

_PRINTERS

Status

Full support slated for Version 2.0

_Printers is an object array that holds _PRINTER class objects that provide information about all printers reported by the operating system.

Column	Data Type	Description
Connection	Character	Connection name, IP, or share
ConnectionType	Numeric	0 –Unknown, 1 – Local, 2 – Network Share, 3 – Client
IsColor	Logical	.T. if a color printer
Location	Character	Physical location
Name	Character	Printer name
Manufacturer	Character	Printer manufacturer
Model	Character	Model number
Online	Logical	.T. if currently online
Tray	Array	Single element array of logical values indicating if paper is out. False (.F.) means the tray has paper or is in an unknown state.
Width	Numeric	Width in millimeters

Each _PRINTERS element has the following methods.

Method	Description
EjectPage	Eject a page
SendTest	Send a test page
GetQueue	Returns an array of jobs in the printer queue
ManageQueue	Send a queue command (Cancel, Clear, Restart)

_Regex

Exposes the regular expression engine to JAXBase.

Property	Data Type	Description
IgnoreCase	Logical	Directs Regex methods to ignore case if true (.T.)
Pattern	Character	Optional pattern to use for Regex calls
Text	Caracter	Optional text to use for Regex calls

The _REGEX object has the following methods.

Method	Description
GetMatches	Returns an array of matches in the provided text, based on the Regex pattern. GETMATCHES(ArrayName) Returns an array of matches in the Text property using the Patterns property. GETMATCHES(ArrayName, cExpr) Returns an array of matches in cExpr using the Patterns property. GETMATCHES(ArrayName, cText, cPattern)) Returns an array of matches in cText using patterns in cPattern
Replace	Returns a string of the text with replacements from a string based on the Regex pattern. REPLACE(cReplace) Returns a string using Text with replacements of cReplace using the Patterns property. REPLACE(cText, cPattern, cReplace) Returns a string using cText with replacements of cReplace using cPattern.
Split	Returns an array containing text being split based on the Regex pattern with optional case sensitivity criteria. SPLIT(ArrayName) Return an array created using the Text property being split using the Pattern and IgnoreCase properties. SPLIT(ArrayName, cText, cPattern) Return an array by splitting cText, using cPattern and IgnoreCase property. SPLIT(ArrayName, cText, cPattern, lIgnoreCase) Return an array by splitting cText, using cPattern and lIgnoreCase parameters.

Note

Pattern and Text properties can come in handy when you wish to perform several different actions using the same pattern or text.

For more information, see [Using the Regular Expression Object](#).

_Table

Status

Full support slated for Version 0.8

Note

The `_TABLE` environment variable is read only and reflects the current state of the table in the current work area.

DRAFT

DRAFT

JAXBase Class Reference

Important Information about JAXBase Classes

All class objects created in JAXBase are assigned, depending on the SET ID setting, an ID string which is unique to the session or a 36-character GUID() which is unique globally.

DRAFT

_APPEND

Status

Basic support slated for Version 1.0

Full support slated for Version 2.0

DRAFT

BROWSE

Status

Basic support slated for Version 1.0

Full support slated for Version 2.0

Browse form used by the BROWSE command. You can have multiple active browse forms at one time.

Property	Data Type	Attributes	Default	Description
ActiveColumn	Numeric			
ActiveRow	Numeric			
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
AllowAddNew	Logical			
AllowAutoFit	Numeric			
Anchor	Numeric			
AllowCellSelection	Logical			
BackColor	Color			
BaseClass	Character	P	GRID	JAXBase class this control inherits from
Class	Character	P	BROWSER	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
ColumnCount	Numeric	P		
Columns	Collection			
DeleteMark	Logical			
Enabled	Logical			
FontBold	Logical			
FontItalic	Logical			
FontName	Character			
FontSize	Numeric			
FontStrikeThrough	Logical			
FontUnderline	Logical			
ForeColor	Color			
GridLineColor	Color			
GridLineWidth	Numeric			
GridLines	Numeric		3	0 – None, 1 – Horizontal, 2 – Vertical, 3 – Both
HeaderHeight	Numeric			
Height	Numeric			
Highlight	Logical			
HighlightBackColor	Color			
HighlightForeColor	Color			
Left	Numeric			
Name	Character			
Objects	Collection			
Parent	Object			
ParentClass	Character			
ReadOnly	Logical			

RecordMark	Logical			
RecordSource	Character			
RecordSourceType	Numeric			
RightToLeft	Logical			
RowColChange	Numeric			
RowHeight	Numeric			
ScrollBars	Numeric			
SelectedItemBackColor	Color			
SelectedItemForeColor	Color			
TabStop	Logical			
TabIndex	Numeric			
Tag	Character			Used to store extra information needed for the program. It is useful for storing a JSON string.
Top	Numeric			
Value				
Visible	Logical			
Width	Numeric			

Methods

Method	Description
AddColumn	
AddObject	
AddProperty	Adds a property to the control
AddMethod	Adds a user defined method to the control
DeleteColumn	
Refresh	
RemoveObject	
SetFocus	
ZOrder	

Events

Method	Description
AfterRowColChange	
BeforeRowColChange	
Click	
DblClick	
Deleted	
Destroy	Executed when class is released from memory
Error	If an unhandled error occurs, the error is written to the log and added to the AError array, if it is enabled, a JAXBase error will be generated.
Init	Executed when a class is created
KeyPress	
MiddleClick	
MouseDown	
MouseEnter	
MouseLeave	
MouseMove	
MouseUp	
MouseWheel	
Moved	
Resize	
RightClick	
Valid	
When	

Example

Notes

See Also

EDIT

Status

Basic support slated for Version 1.0

Full support slated for Version 2.0

DRAFT

_ERROR

The `_ERROR` object provides a unified error messaging system for returning error information. Many objects will have an `AERROR` array that provides information on all errors that occurred during the last transaction.

Properties

Property	Data Type	Attributes	Default	Description
BaseClass	Character	P	CUSTOM	JAXBase class this control inherits from
Class	Character	P	_ERROR	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
ErrorNo	Numeric	R	0	JAXBase Error Number
LineNo	Numeric	R	0	Line number where error occurred – 0 if not relevant
Message	Character	P		First message line from Messages
Messages	Character	R		Message text with all messages terminated with 0x0D
Procedure	Character	R		Procedure where error occurred
RefObject	Object	R	.NULL.	Pointer to object that is in error
RefObjectError	Numeric	R	0	Error number returned by driver (ODBC, network, etc)

Events

Method	Description
INIT	Sets the properties from the parameters passed in
ERROR	If an error occurs, the error is written to the log, if it is enabled, and a JAXBase error will be generated.

Example

* This is a model of how the **ERROR** event works in JAXBase objects

PROCEDURE Error

LPARAMETERS nErrNo, nErrLine, cMsg, cProcedure, oObj, nObjErrNo

LOCAL Idx

IF This.AError[1].ErrNo>0

 * Add to the error array

Idx= alen(this.AError,2)+1

DIMENSION This.AError[idx]

ELSE

 * Fill in the first element

 Idx=1

ENDIF

* Create the `_ERROR` object and assign it to the array

This.AError[idx]=CREATEOBJECT("_Error",nErrNo, nErrLine, cMsg, oObj, nObjErrNo, cProcedure)

ENDWITH

RETURN

Notes

You are encouraged to implement `_ERROR` object arrays when creating your custom error handlers to create a uniform method for error handling.

`_ERROR` object properties are assigned during initialization and cannot be altered after initialization.

The `AERROR` array will always have at least one element. If no error occurred, then `AERROR[1].ErrorNo` will equal zero.

See Also

`Error()` Function

`Line()` Function

`Message()` Function

`ON ERROR` Command

`Procedure()` Function

`TRY/CATCH/FINALLY/ENDTRY` Command

Error Method

Debugging and Error Handling

BARCODE

Status

Basic support slated for Version 1.0

Full support slated for Version 2.0

Properties

Property	Data Type	Attributes	Default	Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
BaseClass	Character	P	CUSTOM	JAXBase class this control inherits from
Class	Character	P	_ERROR	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
CalChecksum	Logical		.T.	If the value is set to true (.T.), then any required checksum is automatically calculated and added to the text. PlainText must be set to true (.T.) before the checksum will be calculated.
Checksum	Character			The WRITE method sets this to the calculated checksum if the check sum is calculated. The READ method extracts the check sum and places it into the CheckSum property. If no checksum was found, it is set to an empty string.
ErrorNo	Numeric	R	0	JAXBase Error Number (Default is 1525)
Image	Character			Encoded image file read by the Read method or created by the Write method.
PlainText	Logical		.T.	If value is true (.T.), any prefix and suffix information for the text required for the barcode is added.
Scale	Numeric			Values above 1 create a taller image while values less than 1 create a smaller image. If the resolution of the output device can be determined, a scale of 1 will produce an image approximately 1" tall, otherwise the resolution defaults to 72dpi and scales accordingly.
Symbology	Numeric			See Reading and Writing Barcodes
Tag	Character			Used to store extra information needed for the program. It is useful for storing a JSON string.
Text	Character			Barcode text to encode or decoded from image

Methods

Method	Description
Read	
Write	

Events

Method	Description
AfterRead	Called after a barcode image is decoded with no errors
AfterWrite	Called after a barcode image is created with no errors
INIT	Sets the properties from the parameters passed in
ERROR	If an error occurs, the error is written to the log, if it is enabled, and a JAXBase error will be generated.

Example

Notes

See Also

[GETJSON\(\) Function](#)
[GETJSON\(\) Function](#)
[Reading and Writing Barcodes](#)

CHECKBOX

Status

Slated for Version 0.6

DRAFT

COLLECTION

Status

Slated for Version 0.6

DRAFT

COMBOBOX

Status

Slated for Version 0.6

Property	Data Type	Attributes		Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
Anchor	Numeric		0	Defines how the edges of the control are bound to the edges of the container
BackColor	Numeric		240,240,240	Integer value representing the RGB background color
BaseClass	Character	P	LABEL	JAXBase class this control inherits from
Class	Character	P	LABEL	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the control can respond to user events
FontBold	Logical		.F.	.T. sets the control's font as bold
FontItalic	Logical		.F.	.T. sets the control's font as italics
FontName	Character		Ariel	Name of font in use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the control's font as striketrough
FontUnderline	Logical		.F.	.T. sets the control's font as <u>underline</u>
ForeColor	Numeric		0,0,0	Integer value representing the RGB foreground color
Height	Numeric		150	Height, in pixels
Left	Numeric		0	Leftmost position of control in the parent
MaxItems	Numeric		1 – 100	Number of items to display without forcing user to scroll
Name	Character	C	FORMx	Name of object
Parent	Object	R		Object to which the form is attached
ParentClass	Character	P		Class of parent object
RightToLeft	Logical		.F.	Displays text in right to left order
TabIndex	Numeric		0	This controls position in the tab order of controls
TabStop	Numeric		.T.	Indicates if this control can be reached via the TAB key
Top	Numeric		0	Topmost position of control in the parent
Visible	Logical		.T.	If .F. then this control is invisible
Width	Numeric		150	Width of control

COMMANDBUTTON

A command button provides a way to initiate an event by the user. When clicked, the CLICK event is executed.

Property	Data Type	Attributes		Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
Anchor	Numeric		0	Defines how the edges of the control are bound to the edges of the container
Autosize	Logical		.F.	Specifies if the control can automatically resize to fit the text in the caption.
BackColor	Numeric		240,240,240	Integer value representing the RGB background color
BaseClass	Character	P	LABEL	JAXBase class this control inherits from
Caption	Character		BUTTONx	Hold the text that is displayed by this control
Class	Character	P	LABEL	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the control can respond to user events
FontBold	Logical		.F.	.T. sets the control's font as bold
FontItalic	Logical		.F.	.T. sets the control's font as italics
FontName	Character		Ariel	Name of font in use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the control's font as striketrough
FontUnderline	Logical		.F.	.T. sets the control's font as <u>underline</u>
ForeColor	Numeric		0,0,0	Integer value representing the RGB foreground color
Height	Numeric		150	Height, in pixels
Left	Numeric		0	Leftmost position of control in the parent
Name	Character	C	FORMx	Name of object
Parent	Object	R		Object to which the form is attached
ParentClass	Character	P		Class of parent object
RightToLeft	Logical		.F.	Displays text in right to left order
TabIndex	Numeric		0	This controls position in the tab order of controls
TabStop	Numeric		.T.	Indicates if this control can be reached via the TAB key
Top	Numeric		0	Topmost position of control in the parent
Visible	Logical		.T.	If .F. then this control is invisible
Width	Numeric		150	Width of control

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
MEUpdate	Allows you to insert new code into most methods or events before initialization. Methods or events that contain code after initialization cannot be updated.
Refresh	Repaints the control and updates any values

Events

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

See Also

COMMANDGROUP

Status

Slated for Version 0.6

DRAFT

CONTAINER

Status

Slated for Version 0.6

DRAFT

CUSTOM

The Custom class is a minimal class that has the following properties, events, and methods.

Property	Data Type	Attributes	Default Value	Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
BaseClass	Character	P	CUSTOM	JAXBase class this control inherits from
Class	Character	P	CUSTOM	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the form can respond to user events
KeyPreview	Logical		.F.	.T. means the form will intercept keystrokes and send them to the KeyPress event
Left	Numeric		0	Leftmost position of form in the display area
Name	Character	D	FORMx	Name of form
Objects	Object[]	P		Array of child objects attached to form
Parent	Object	R		Object to which the form is attached
ParentClass	Character	P		Class of parent object

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
AddObject	Adds an object to an object
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Refresh	Repaints the control and updates any values

Events

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

See Also

DRAFT

EDITBOX

Status

Slated for Version 0.6

Property	Data Type	Attributes		Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
Anchor		C		
BackColor	Numeric		240,240,240	Integer representing the RGB color of the background
BaseClass	Character	P	LABEL	JAXBase class this control inherits from
BorderStyle	Numeric		3	Border style: 0 – None 1 – Fixed Single 2 – Fixed Dialog 3 – Sizable (Default)
Class	Character	P	LABEL	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the form can respond to user events
FontBold	Logical		.F.	.T. sets the controls font as bold
FontItalic	Logical		.F.	.T. sets the controls font as italics
FontName	Character		Ariel	Name of font to use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the controls font as striketrough
FontUnderline	Logical		.F.	.T. sets the controls font as <u>underline</u>
ForeColor	Numeric		0,0,0	Color of font
Height	Numeric		150	Height, in pixels
Left	Numeric		0	Leftmost position of form in the display area
MaxLength	Numeric			Maximum length of text allowed in the control. 0 = no limit
Name	Character	C	FORMx	Name of form
Parent	Object	P		Object to which the form is attached
ParentClass	Character	P		Class of parent object
PasswordChar	Character			Mask character to obfuscate the display of the text
ReadOnly	Logical		.F.	When .T., the user cannot modify its contents
RightToLeft	Logical		.F.	Scale Factor between Text and Graphic user interfaces
ScrollBars	Numeric		2	0 – None, 1 – Horizontal, 2 – Vertical, 3 – Both
TabIndex	Numeric		0	
TabStop	Numeric		.T.	
Text	Character			
TextAlignment	Numeric			0 – Left, 1 – Right, 2 – Center
TextLength	Numeric		0	Length of text in the control
Top	Numeric		0	Topmost position of form in display area
Visible	Logical		.T.	
Width	Numeric		150	Width of form
WordWrap	Logical		.F.	Value of true (.T.) turns on wordwrap

Methods

Method	Protected	Description
AddProperty	Y	
AddMethod	Y	
AddObject	Y	
Destroy	C	
Init	C	
MEUpdate	Y	
RemoveProperty	Y	
RemoveObject	Y	

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

See Also

EMPTY

The EMPTY class starts with no properties and does not support events or methods except for the ADDPROPERTY() and REMOVEPROPERTY() methods.

The EMPTY class is used as a basic object for storing information by various JAXBase commands.

See Also

GATHER NAME Command

SCATTER NAME Command

DRAFT

FILE

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

FORM

Remarks

All objects that are visible in JAXBase must be attached to a form class. A form provides the visible pallet area for displaying information and allowing user control.

Property	Data Type	Attributes	Default Value	Description
Activecontrol	Object	R		Direct reference to the current active control
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
AlwaysOnTop	Logical		.F.	Indicates if the form is always on top
AutoCenter	Logical		.F.	Is form centered on the viewing area when initialized
BackColor	Numeric		240,240,240	Integer representing the RGB color of the background
BaseClass			FORM	JAXBase Class this control inherits from
BorderStyle	Numeric		3	Border style: 0 – None, 1 – Fixed Single, 2 – Fixed Dialog, 3 – Sizable (Default)
Caption	Character		FORMx	Form's caption
Class	Character	P	FORM	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Closable	Logical		.T.	If .T. then the close icon is visible and active
ControlCount	Numeric	R	0	Number of objects on the form.
DataSession	Numeric	D,1	1	Data session ID: 1 – Default, 2 – Private data session
Enabled	Logical		.T.	Indicates if the form can respond to user events
FontBold	Logical		.F.	.T. sets the controls font as bold
FontItalic	Logical		.F.	.T. sets the controls font as italics
FontName	Character		Ariel	Name of font to use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the controls font as strike through
FontUnderline	Logical		.F.	.T. sets the controls font as <u>underline</u>
ForeColor	Numeric		0,0,0	Color of font
Height	Numeric		150	Height, in pixels
Hwnd	Numeric	R	0	OS Handle of the form
Icon	Character			File name of icon placed in upper left corner
KeyPreview	Logical		.F.	Indicates if keystrokes are sent to the KeyPress event
Left	Numeric		0	Leftmost position of form in the display area
LockScreen	Logical		.F.	.T. changes to the form are hidden until set back to .F.
MaxButton	Logical		.T.	.T. means the maximize button is visible and enabled
MaxHeight	Numeric		-1	Maximum allowed height of form
MaxWidth	Numeric		-1	Maximum allowed width of form
MinButton	Logical		.T.	.T. means the minimize button is visible and enabled
MinHeight	Numeric		-1	Minimum allowed height of form
MinWidth	Numeric		-1	Minimum allowed width of form
Name	Character	D	FORMx	Name of form
Objects	Object[]	P		Array of child objects attached to form

Property	Data Type	Attributes	Default Value	Description
Parent	Object	R		Object to which the form is attached
ParentClass	Character	P		Class of parent object
ScaleFactor	Numeric	D,2	0	Scale Factor between Text and Graphic user interfaces
ScrollBars	Numeric		0	Type of scroll bars displayed on the form 0 – None (default), 1 – Horizontal, 2 – Vertical, 3 – Both
Top	Numeric		0	Topmost position of form in display area
Visible	Logical		.T.	Indicates if control is visible
Width	Numeric		150	Width of form

D – Read/Write before initialization, read only once initialized

R – Read Only

P – Protected and cannot be modified

1 – The data session is read/write before initialization, read only once initialized. Once initialized, the value in DataSession is the actual data session ID.

2 – Scale factors (Under Consideration)

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
AddObject	Adds an object to an object
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Refresh	Repaints the control and updates any values

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

While the Objects collection cannot be modified directly, you can modify the properties of their elements.

When DataSession is set to 2, a private data session is set up for the form and will remain open until the form is released from memory. The CLOSE DATASESSION command will not close a Form's private data session.

When a form with a private data session calls an external program or method, data session 1 is selected before that external program or method is executed. When control passes back to the form, the form's private data session is selected.

See Also

CLOSE DATASESSION Command
SELECT Command
USE Command
ASESSIONS() Function

FORMSET

Status

Slated for Version 2.0

DRAFT

FTP

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

GRID

Status

Slated for Version 0.8

DRAFT

HTTP

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

IMAGE

Status

Basic support slated for Version 0.6

Full support slated for Version 2.0

DRAFT

LABEL

Property	Data Type	Attributes		Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
Anchor				
AutoSize				
BackColor	Numeric		240,240,240	Integer representing the RGB color of the background
BaseClass	Character	P	LABEL	JAXBase class this control inherits from
BorderStyle	Numeric		3	Border style: 0 – None 1 – Fixed Single 2 – Fixed Dialog 3 – Sizable (Default)
Caption	Character		LABELx	Form's caption
Class	Character	P	LABEL	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the form can respond to user events
FontBold	Logical		.F.	.T. sets the controls font as bold
FontItalic	Logical		.F.	.T. sets the controls font as italics
FontName	Character		Ariel	Name of font to use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the controls font as strike through
FontUnderline	Logical		.F.	.T. sets the controls font as <u>underline</u>
ForeColor	Numeric		0,0,0	Color of font
Height	Numeric		150	Height, in pixels
Left	Numeric		0	Leftmost position of form in the display area
Name	Character	D	FORMx	Name of form
Parent	Object	R		Object to which the form is attached
ParentClass	Character	P		Class of parent object
RightToLeft	Logical		.F.	Scale Factor between Text and Graphic user interfaces
TabIndex	Numeric		0	
Top	Numeric		0	Topmost position of form in display area
Visible	Logical		.T.	
Width	Numeric		150	Width of form

D – Read/Write before initialization, read only once initialized

R – Read Only

P – Protected and cannot be modified

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Refresh	Repaints the control and updates any values

Events

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

See Also

LISTBOX

Status

Slated for Version 0.6

DRAFT

MENU

Status

Slated for Version 0.8

DRAFT

OPTIONBUTTON

Status

Slated for Version 0.6

DRAFT

OPTIONGROUP

Status

Slated for Version 0.6

DRAFT

PAGEFRAME

Status

Slated for Version 0.6

DRAFT

PIPE

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

POP3

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

PRINTER

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

SHAPE

Status

Slated for Version 1.0

Full support slated for Version 2.0

Remarks

The SHAPE class is used to draw lines, circles, and polygons with optional text.

Property	Data Type	Attributes		Description
AError	Array			Array of _ERROR class
Anchor				
Autosize				
BackColor	Numeric		240,240,240	Integer representing the RBG color of the background
BackStyle				
BaseClass	Character		LABEL	JAXBase class this control inherits from
BorderColor				
BorderStyle	Numeric		3	Border style: 0 – None, 1 – Solid, 2 – Dotted, 3 – Dashed
BorderWidth				
Class	Character		LABEL	Name of the class control is based upon
ClassID	Character			JAXBase object ID assigned at initialization
ClassLibrary	Character			Indicates if the class is part of a class library
Curvature				
Enabled	Logical		.T.	Indicates if the form can respond to user events
FillColor				
FillStyle				
FontBold	Logical		.F.	.T. sets the controls font as bold
FontItalic	Logical		.F.	.T. sets the controls font as italics
FontName	Character		Ariel	Name of font to use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the controls font as striketrough
FontUnderline	Logical		.F.	.T. sets the controls font as <u>underline</u>
ForeColor	Numeric		0,0,0	Color of font
Height	Numeric		150	Height in pixels
Left	Numeric		0	Leftmost position of shape
Name	Character		FORMx	Name of form
Parent	Object			Object to which the form is attached
ParentClass	Character			Class of parent object
RightToLeft	Logical		.F.	Scale Factor between Text and Graphic user interfaces
Rotation	Numeric		0	
SpecialEffect				
Tag				
Text				Text to place with the shape
TextAlignment				0 – Right, 1 - Center, 2 - Left
TextPlacement				0 – Above, 1 – Inside, 2 - Below
Top	Numeric		0	Topmost position of shape in display area
Visible	Logical		.T.	
Width	Numeric		150	Width in pixels

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
CreateGraphic	Returns a graphic image based on parameters which can be placed onto the clipboard, added to a image control, or saved to a file.
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Refresh	Repaints the control and updates any values

Events

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Note

A shape can have text added to it, allowing the creation of flow charts, circuit diagrams, labels, and other forms or documents in a rapid manner. See the program FLOW.PRG in the Examples folder.

SMS

Status

Limited support slated for Version 1.0

Full support slated for Version 2

DRAFT

SMTP

Status

Basic Support by Version 1.0

Full support slated for Version 2

DRAFT

SOUND

Status

Basic support slated for Version 1.0

Full support slated for Version 2

Property	Data Type	Attributes	Default Value	Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
BaseClass	Character	P	CUSTOM	JAXBase class this control inherits from
Class	Character	P	CUSTOM	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
BitRate	Numeric			Gets or sets bit rate
Channels	Numeric			Gets or sets number of channels 1 – mono, 2 – stereo
Sound	Character			Name of sound file
Tag	Character			

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
AddObject	Adds an object to an object
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Play	Plays the sound file
Refresh	Repaints the control and updates any values
Stream	Allows a sound to be played from a memory variable

Events

Event	Description
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
Init	Executed when a control is created
Loaded	Executed when after a sound file is loaded

Example

Notes

DRAFT

SPINNER

Status

Slated for Version 0.6

DRAFT

SQL

Status

Slated for Version 0.6

Property	Data Type	Attributes	Default Value	Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
BaseClass	Character	P	SOUND	JAXBase class this control inherits from
Class	Character	P	SOUND	Name of the class control is based upon
ClassLibrary	Character	P		Indicates if the class is part of a class library
ConnectionString	Character			
Database	Character	C		
Driver	Character	C		
Encrypted	Logical			
Instance	Character	C		
IsConnected	Logical	R	.F.	
Port	Numeric	C		
Server	Character	C		
UserID	Character	C		
UserPW	Character	C		

Protected

C – Conditional. Cannot be changed after while connected.

P – Protected. Cannot be changed.

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
AddObject	Adds an object to an object
Connect	
Disconnect	
Exec	
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Refresh	Repaints the control and updates any values
Update	Pushes records from a local DBF into a SQL table

Events

Event	Description
Connected	
Destroy	Executed before a control is released from memory
Disconnected	
Error	Executed if an unhandled error occurs
Init	Executed when an control is created

Example

Notes

See Also

TEXTBOX

Property	Data Type	Attributes		Description
AError	Array	P		Array of _ERROR class for errors captured by the ERROR event, if active.
Anchor		C		
BackColor	Numeric		240,240,240	Integer representing the RGB color of the background
BaseClass	Character	P	LABEL	JAXBase class this control inherits from
BorderStyle	Numeric		3	Border style: 0 – None 1 – Fixed Single 2 – Fixed Dialog 3 – Sizable (Default)
Class	Character	P	LABEL	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Enabled	Logical		.T.	Indicates if the form can respond to user events
FontBold	Logical		.F.	.T. sets the controls font as bold
FontItalic	Logical		.F.	.T. sets the controls font as italics
FontName	Character		Ariel	Name of font to use
FontSize	Numeric		10	Size of font
FontStrikeThrough	Logical		.F.	.T. sets the controls font as strike through
FontUnderline	Logical		.F.	.T. sets the controls font as <u>underline</u>
ForeColor	Numeric		0,0,0	Color of font
Height	Numeric		150	Height, in pixels
Left	Numeric		0	Leftmost position of form in the display area
Name	Character	C	FORMx	Name of form
Parent	Object	P		Object to which the form is attached
ParentClass	Character	P		Class of parent object
PasswordChar	Character			Mask character to obfuscate the display of the text
ReadOnly	Logical		.F.	When .T., the user cannot modify its contents
RightToLeft	Logical		.F.	Scale Factor between Text and Graphic user interfaces
TabIndex	Numeric		0	
TabStop	Numeric		.T.	
Text				
Top	Numeric		0	Topmost position of form in display area
Visible	Logical		.T.	
Width	Numeric		150	Width of form

C – Read/Write before initialization, read only once initialized

R – Read Only once initialized.

P – Protected and cannot be modified

Methods

Method	Protected	Description
AddProperty	Y	
AddMethod	Y	
AddObject	Y	
Destroy	C	
Init	C	
MEUpdate	Y	
RemoveProperty	Y	
RemoveObject	Y	

Event	Description
Click	Executed when the user clicks on the control
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
GotFocus	Executed when a control receives focus
Init	Executed when an control is created
LostFocus	Executed when a control loses focus
MouseEnter	Executed when the user moves the mouse into the control
MouseLeave	Executed when the user moves the mouse out of the control
Valid	Executed before the control loses focus

Example

Notes

See Also

TCP

Status

Basic Support by Version 1.0

Full support slated for Version 2.0

DRAFT

TIMER

Status

Basic support slated for Version 1.0

Full support slated for Version 2.0

DRAFT

UDP

Status

Basic Support by Version 1.0

Full support slated for Version 2

DRAFT

VIDEO

Status

Basic support slated for Version 1.0

Full support slated for Version 2

Property	Data Type	Attributes	Default Value	Description
BaseClass	Character	P	VIDEO	JAXBase class this control inherits from
Class	Character	P	VIDEO	Name of the class control is based upon
ClassID	Character	P		JAXBase object ID assigned at initialization
ClassLibrary	Character	P		Indicates if the class is part of a class library
Codec	Character			
Duration	Numeric			Duration in seconds
FrameRate	Numeric			Number of frames per second
Height	Numeric			Name of sound file
Tag	Character			
Video	Character			Name of video file
Width	Numeric			

Methods

Method	Description
AddProperty	Adds a property to an object
AddMethod	Adds a user defined method to an object
AddObject	Adds an object to an object
MEUpdate	Allows you to insert new code into any method or event before initialization and any user defined method and event after initialization.
Pause	Logical parameter .T. cause video to pause, .F. to restart if paused
Play	Numeric parameter indicating at what second of the video to begin playing
Refresh	Repaints the control and updates any values
Stream	Allows a video to be loaded from a memory variable

Events

Event	Description
Destroy	Executed before a control is released from memory
Error	Executed if an unhandled error occurs
Init	Executed when a control is created
Loaded	Executed after a video file is loaded

Example

Notes

See Also

DRAFT

DRAFT

JAXBase Concepts

The following sections are designed to help you understand how JAXBase can be implemented for various use cases and attempt to clarify advanced topics.

DRAFT

Arrays, Variables, and Objects

DRAFT

Automated Testing

Automated testing is essential for today's modern developer. JAXBase contains commands that will allow you to create fully scripted test automation to make it easier to test your application as you build it. As with all automation platforms, there are limits, but the JAXBase system makes every effort to push those limits, allowing you to create a robust test platform.

One of the main components of automation testing is the SET KBMINPUT command which allows you to identify a Keyboard & Mouse input file that will be used in place of user keyboard and mouse input. The AUTOMATION.PRG program in the Examples folder provides a brief demonstration of that command.

A KBMINPUT file is a plain text file with keystrokes, mouse clicks/movements, and commands to create a rich set of tools for testing. An example of a KBMINPUT file would look like the following to fill in a user/password dialog.

Command	Description

The [JAXBase Automated Test System](#) document goes into great depth on creating automated tests for your application.

Code Page Support

Slated for Version 2.0

DRAFT

Data Sessions and Work Areas

Data Sessions

JAXBase expands upon the concept of Data Sessions. Data Session 1 is the default data session, but from there you can create and use other data sessions in your code, essentially creating data silos. Each data session is a separate entity and allows you greater flexibility, easily set up and you can even access another data sessions using the SESSION clause found in many commands.

Those of you familiar with Visual FoxPro may remember that forms can have private data sessions. JAXBase is identical in that respect, but you can also set up a private data session for a program. One such reason might be to have a routine that does something specific to a SQL database, and you have several SQL databases that you want to have open at the same time. If you like, open each SQL database in a separate data session and the data tables and result cursors for each will be contained in their specific data sessions.

Another use case for data sessions will be with threading in Version 2.0, where each thread can have its own data session and not interfere with the other threads.

The number of data sessions that you can define is dependent on available memory. Data session IDs are numeric, starting at 1 and can go up to the highest integer value, providing enough possible data sessions for even the most ambitious project.

Work Areas

Work areas, which exist in a data session, also start at 1 and go as high as the largest integer value. A work area holds one table or cursor and opening another will close the current one. Since a table or cursor is held in a work area, you can also access that table using its alias name. An alias name can be defined as a name different than the table name. Failing to provide an alias name sets the alias equal to the table name.

In a command like GOTO TOP, you can also include a work area or alias name using the IN clause. As an example, if the Customers table is open in work area one, you can go to the top record using the following commands.

```
GOTO TOP IN 1  
GOTO TOP IN Customers
```

Additionally, if the data you are looking for is in a different data session, you can include the SESSION clause, as well.

```
GOTO TOP IN 1 SESSION 3  
GOTO TOP IN Customers SESSION 3
```

Dates

In JAXBase, like most xBase dialects, there are Date and DateTime values. Date values hold just a month/day/year value while DateTime values also hold the time of day.

On the downside of the Date and DateTime issue is regional differences. While people in the United States look at 01/05/2024 and think January fifth, people in other countries will think May first. Version 1.0 will have some regional data handling, but it will exist and be carried into Version 2 which will provide a much richer set of tools.

Date and DateTime values are both stored in memory as DateTime, but the time value is set to 00:00:00 in a Date value and is discarded when the data part is used. Doing it this way makes it easier to use Date and DateTime values interchangeably.

Dates and Datetimes in JAXBase start at January 1, 0001 and go to December 31, 9999, based on the Gregorian Calendar. However, dates are a funny thing, and many people do not realize that the Gregorian Calendar has never been fully adopted, world-wide. While it is the de-facto standard for international purposes, some regions still use their calendars. Further, the Gregorian Calendar has only been in use since 1582 and Great Britain and its colonies (like what is now known as the United States) did not adopt the Gregorian Calendar until 1752. Thus, a Date prior to the adoption of the Gregorian calendar may not actually be the date that was used back then.

While it's possible to use a different calendar system, it might be easier to create a conversion routine to support the other calendar. For example, one lunar calendar involves months that are 29.5 days long, losing about 11 days per the Gregorian Calendar. There are books and online references that can help with this quest. Ideally, once you have a reference point, you can work forward and backward to get the correct conversion value.

DBF Fields

DRAFT

Debugging

DRAFT

Error Handling

Error handling is quite different in JAXBase as the ON ERROR command was removed from the system. This was done to force developers to be more structured in their error handling. The TRY/CATCH structure should be used in code while classes have an ERROR event and new AERROR array.

AError Array and the ERROR Method

Most classes have the new AError array with each element containing an object like the object created by the CATCH TO statement. Each element holds the following properties.

Property	Data Type	Description
ErrorNo	Numeric	The current error code - 0 means no error.
LineNo	Numeric*	The line number where the error occurred.
Message	Character*	The error message returned by the class.
Procedure	Character*	The procedure where the error occurred.

* When reset to its blank state, the LineNo, Message, and Procedure properties are set to False (.F.)

When a class method or event is executed, the AError array is reset to its blank state which has a single row and ErrorNo is zero (0). When control is returned from the class, checking the AError[1].ErrorNo value will tell you if an error occurred by checking if the value is greater than zero.

When the ERROR event in a class is left blank, errors in class execution are silently recorded to the AError array for post-mortem use. When the execution returns to your code, you have the option to interrogate the AError array to see if any errors occurred. Adding code to the ERROR event will allow you to catch and manage errors, giving you opportunities to correct and retry the action.

Here is an example of using the AError array to check for errors post-mortem.

```
* CREATE THE SQL OBJECT
MyDB=CREATEOBJECT("sql")

* Just fill in the connection data
WITH MyDB
  .Driver="SQL Server"
  .Server="MyServer"
  .Port=1433    && Default SQL Server connection
  .USERID="sqladmin"
  .PASSWORD="BobIsAtHD-3167"
  .DATABASE="JAXCorp"
  .CONNECT    && Try to connect
ENDWITH

* Look at the AError array
DO CASE
  CASE MyDB.AError[0].ErrorNo=0      && No errors!
  CASE MyDB.AError[0].ErrorNo=6000   && Invalid Connection String
  CASE MyDB.AError[0].ErrorNo=6002   && Not authorized
  OTHERWISE
ENDCASE
```

TRY/CATCH

While it is possible to put a single TRY/CATCH structure in the main procedure to catch all errors in your program, this is less than optimal. Errors, when they happen, should be caught and dealt with as soon as possible to create a more secure and polished application.

Ideally, your code would be written in a way to eliminate most run-time errors. However, it is difficult to prevent errors caused by external forces (think user input, corrupted data, file and network issues). Planning and preparing for error events will help prevent spending hours on the phone with frustrated users.

While it takes time to plan and write proper error catching and handling routines, the time savings will easily repay that effort once your application is in the hands of hundreds or even thousands of users.

JAXBase makes every effort to correctly identify errors and the chapter JAXBase: Error Codes will be invaluable for helping identify them. Ideally, your TRY/CATCH routine will look for and handle common errors and allow users to correct and retry the action to move forward. However, that is not always possible, so you should allow for the ability to gracefully exit the subroutine and go back to a spot that will allow them to continue to either correct the issue, or work on other tasks while someone looks at the problem.

CATCH / WHEN

Having the ability to write code for a specific error event can improve the performance and security of your system. While you may not need to be overly detailed in every instance, taking advantage of where it will help the user will improve the usability of your system.

The CATCH / WHEN clause allows you to create a code block that is only run when a specific error occurs in your code. This not only gives you greater flexibility, it also makes it easier to maintain your code, since you will have different actions occurring in separate code blocks.

The next section provides an example of a simplified, yet real life example of a form that reads in data, like an excel spreadsheet or formatted flat file. The user selects the file to process in the code block attempts to load the data to a temporary table or cursor and checks the validity of the data. Having separate CATCH/WHEN statements makes it easier to create and maintain the various issues that are easily anticipated, leaving the last CATCH statement to deal with unexpected issues, which may simply be where the user abandons the task and makes a phone call to someone about the issue.

TRY/CATCH/WHEN Example

A well-designed TRY/CATCH structure will have the ability to handle numerous types of errors and provide the ability to recover from as many as possible.

```

* The user selects a file which is loaded into a temporary
* table for further processing.
DO WHILE .T.
    TRY
        <program code>
        CATCH TO loErr WHEN loErr.ErrorNo=1 && File not found
            ans=MESSAGEBOX("File "+filename+" not found."+chr(13)+
                "Would you like to try selecting a different one?",4)

            IF ans=6
                CONTINUE && Loop around
            ENDIF

        CATCH TO loErr WHEN loErr.ErrorNo=3 && File in use
            ans=MESSAGEBOX("File "+filename+" is in use."+chr(13)+
                "Would you like to make another attempt?",4)

            IF ans=6
                CONTINUE && Loop around
            ENDIF

        CATCH TO loErr WHEN loErr.ErrorNo=4 && End of file encountered
        <code to log the error>
            ans=MESSAGEBOX("The file ended unexpectedly and may be corrupted."+chr(13)+
                "Would you like to use what you have?" +chr(13)+
                "(Selecting NO will throw away what was loaded.)",4)

            IF ans=7
                <program code to clear loaded data>
                CONTINUE && Allow them to try a different file
            ENDIF

        CATCH TO loErr && Catch-all for unhandled events
        <code to log the error>
            ans=MESSAGEBOX("An unexpected error occurred and data"+chr(13)+
                "may be corrupted. The error has been logged."+chr(13)+
                "Would you like to try another file?",4)

        <code to clear data file>

        IF ans=6
            CONTINUE && Loop around to try another file
        ENDIF
        FINALLY
        EXIT
    ENDTRY
ENDDO

* If the temporary table was not created or is empty, then nothing
* will be processed and control is returned to the menu.
<program code>

```

In the above example, there is different logic to handle each anticipated type of error along with a general catch-all to deal with unanticipated problems. In each, the user has a choice to try again, or unload the data and continue, where the program will simply skip any further steps as there is no data to process.

File Handling

File handling in JAXBase is done with the FILE class. Each FILE object represents one open file, making it easier to manage multiple open files at once. All file manipulations can be done using the FILE class with its various properties, methods, and events.

```
* Example 1 - Create hello.txt
oFile1=CREATEOBJECT('FILE')

WITH oFile1
    .FileName='hello.txt'

    .OPEN(2)  && Create, overwrite if it exists

    * Check for an error and try to write
    IF .AError[0].ErrorNo=0
        .PRINT('Hello World')
        .PRINT('The time is '+TIME())
    ELSE
        =MESSAGEBOX("Failed to open "+.FileName,0)
    ENDIF

    .Close  && Close file - does nothing if not open
ENDWITH

MODIFY FILE hello.txt NOWAIT  && Open file in edit window
```

* Example 2 – Copy ODD lines of file1 and append to file2

```
oFile1=CREATEOBJECT("FILE")
```

```
oFile2=CREATEOBJECT("FILE")
```

```
WITH oFile1
```

```
  .FileName="File1.txt"
```

```
  .OPEN()  && Open the file if it exists
```

```
  IF .AError[0]>0
```

```
    =MESSAGEBOX("Failed to open "+.FileName,0)
```

```
    RETURN
```

```
  ENDIF
```

```
ENDWITH
```

```
WITH oFile2
```

```
  .FileName="File2.txt"
```

```
  * Open the file if it exists, create if it does not
```

```
  * and set it for APPEND
```

```
  .OPEN(1,1)
```

```
  IF .AError[0]>0
```

```
    =MESSAGEBOX("Failed to open "+.FileName,0)
```

```
    RETURN
```

```
  ENDIF
```

```
ENDWITH
```

```
i=0
```

```
WHILE not oFile1.EOF
```

```
  F1Line=oFile1.READ()  && Read a line from the first text file
```

```
  IF AError[0]=0
```

```
    * Write the odd lines to the second text file
```

```
    i=i+1
```

```
    if i % 2=1
```

```
      oFile2.PRINT(F1Line)
```

```
    ENDIF
```

```
  ELSE
```

```
    =MESSAGEBOX(STRFORMAT("ERROR {0} reading line {1} of {2}",.AError.ErrorNo,i.FileName),0)
```

```
  EXIT
```

```
  ENDIF
```

```
ENDDO
```

Indexes

DRAFT

Keyboard Commands

DRAFT

Logging

Logging is the act of recording events that occur in a system for later review. Some systems create logs of just errors, some include warnings and unusual events, others make it possible to log nearly every action taken by the user. The degree of logging is completely up to you, but for a large system, logging can be a vital part of debugging and maintaining a system.

There are a few ways to perform logging, such as writing to a table, writing to a text file, or sending a message through the network to a log server. While some would not consider logging to be important, it is much easier to debug an issue when there is a record of the problem.

Some of the pros and cons of logging come down to the following.

PROS	CONS
Creates a record of events	May slow the system
Provides details of issues	Increases disk usage
Performance monitoring	Potential point of failure
Can enhance security	May log sensitive data
Compliance and auditing	Alert fatigue

Starting with the cons, slowing down the system can be a real concern with a high-volume system. Logging takes time and resources which may affect performance. You must be judicious in what you choose to write to a log file. The increased disk usage can be significant and creates a new point of failure. Running out of disk space or experiencing a corrupted log table can cause a system to completely fail. The danger of logging sensitive information is quite real, especially when working in the medical or financial industries where there are laws with significant penalties protecting customer privacy. Finally, a poorly designed log or log viewer can cause users to skip regular viewing of events because it takes too long and there is simply too much trivia to wade through to see if there is anything important.

On the plus side, creating a permanent record of events provides clues as to what happened when an error or unusual event occurred. Logs can also provide information on performance, allowing you to determine if an area of your system needs tweaking. Additionally, having a log of failed attempts of user access can point out potential hacking attempts. Finally, some customers need a permanent log to meet compliance and auditing requirements.

Types of Log Files

There are two main types of log files. The data table and the text file, and they provide different advantages and disadvantages. The data table typically takes up more space and is less flexible because of the fixed size of fields. It is also easier to corrupt a data table, which will prevent adding new records and may even crash the system. However, a data table will allow much faster speed of viewing and searching and can make it much easier to filter out unwanted events.

Text files are by nature quite easy to create, seldom cause problems with the program in terms of creation and updates, and usually take up less space and are faster to update as long as their size does not go beyond some reasonable limits. Converse to that is that a text file can take a lot longer to scan and search for specific events.

To get around some of these issues, you may decide to use both types of log files, where you create a text log file for speed and fewer risks of crashing and then create an application that loads select information to a database. The application can be set to parse and prioritize events, allowing you to filter out only what you want, making certain data quickly retrievable. Further, that separate application will have the time to send out alerts via email or other means without interrupting the flow of the main application.

The text files may hold a lot more data than is written into the log table, preventing clutter and fatigue for the person whose job is to check the logs. Since there really isn't a good reason to log information valuable to developers for debugging issues, and many audit logs are never looked at, you can simply compress the original text files and move them to folder or external long term storage device. Then the data is still available should someone need them.

Creating the Text Log File

One problem with a text-based log file is formatting the information so that it is quickly read and easy to identify an audit trail from a high priority concern. Using the JSON features in JAXBase is one way of solving that issue. For instance, an audit trail typically does not need to include program information that a developer might find useful, and an error log certainly does not need to include changes in prices or quantities. However, both need basic information like date, time, the operator, and other common information that can be used to understand where the record came from and how it fits into what happened that day.

A record written using JSON can contain common information used to detail what each log record needs and have different information written depending on the type of log it contains. Furthermore, a JSON string is formatted, which means if something goes wrong with writing that record, you'll know about it as soon as the application that converts the text file is run.

As an example, once customer had a workstation that seemed to be working perfectly, yet when the log files were inspected, the information written by that workstation was gibberish. Unfortunately, the data was written in a fixed field format, and the conversion application did very little error checking, thus days went by before someone realized there was a problem. Had JSON formatted data been used, it's very likely that problem would have been quickly found and corrected, which in this case was to reboot the workstation.

Converting the Text Files

Since many businesses use servers and even more have one or two older machines sitting around in a closet, it is usually a simple matter to set up a separate task on a machine nobody uses to convert the JSON files and send the desired information to a database.

The application should be created to allow users to decide what they want to convert and have the flexibility to write the different types of records to separate tables in the database. In that way, you can create separate viewing applications to provide specific users just the data they need quickly and efficiently, providing the end user with a polished look and feel, and more importantly not wasting their time with minutia.

Alerts and Notifications

The final piece to a well-designed logging system is getting important information out to those who need it in a timely manner. An inventory manager needs to know when stock quantities have dipped below reorder levels but typically does not need that information as quickly as the IT manager needs to know that there have been several failed logins over the past few minutes. Having the ability to set detection levels of specific events and the priority and means of sending alerts creates a very useful system for the customer. JAXBase includes classes and tools that will allow you to send SMS alerts, emails, or event notifications to a user's desktop.

In conclusion, a well-designed and implemented logging system can not only provide vital information to the user, but it can also make your system stand out from competitors and convince potential customers to take advantage of your experience.

Merging Text

DRAFT

Moving Data to and from a SQL Database

DRAFT

Printers

DRAFT

Reading and Writing Barcodes

JAXBase Version 1.0 will have support barcode generation of many common symbologies.

DRAFT

Scope Clauses

Just as in SQL, when working with a DBF table, you can define the scope of records to use. The JAXBase scope syntax is very similar to most database languages.

ALL

ALL

The ALL scope tells JAXBase to look at all records. If SET DELETED is ON, then only the records that are not marked for deletion are used. When completed, the record pointer is set past the end of the table and EOF() will return true (.T.).

NEXT

NEXT nRecords

NEXT tells JAXBase to start at the current record and use the next nRecords. If the value of nRecords takes it past the end of the table or cursor, the record pointer is set after the last record and EOF() will return true (.T.).

If the record pointer is already past the end of the file, an error is generated.

RECORD

RECORD nRecord

The RECORD clause specifies a particular physical record in the table, ignoring any index that may be in control of the table. The record pointer will be set to the selected record. If the selected record is past the end of the file, an error is generated.

New

RECORD nRecord, nCount

JAXBase Version 2.0 will include new syntax to the RECORD clause which allows you to specify the starting record and how many records to return. Unlike the standard RECORD clause which specifies an absolute record number, when nCount is included, the controlling index order is respected in relation to the starting nRecord. Record 1 is at the top of the indexed table if there is a controlling index, otherwise it will be the first record in the table.

If SET DELETED is ON, only the records not marked for deletion are used. If nRecord is past the end of the file, an error is generated. If nCount takes the record pointer past the end of the file, the record pointer is positioned after the last record and EOF() will return True (.T.).

REST

REST

REST causes JAXBase to start at the current record and return all records to the end of the file. If an index is in control, the records will be returned based on the index. When completed, the record pointer will be set after the end of the table and EOF() will return True (.T.).

TOP

TOP nRecords

The TOP clause tells JAXBase to go to the top of the file and return the number of records indicated by nRecords. If nRecords is omitted, 0, or 1, then just the first record is returned. The record pointer is set past the last record read in and if nRecords takes it past the end of the table, the record pointer is set after the last record and EOF() will return True (.T.).

Note

Omitting a scope, by default, causes JAXBase to only work with the current record.

Tables and Cursors

DRAFT

Using One or More a SQL Database

DRAFT

Using the Regular Expression Object

DRAFT

DRAFT

Appendixes

DRAFT

Appendix A – File Types

The recognized file types in JAXBase are as follows:

PRG	Source JAXBase code
JXP	Compiled JAXBase code
APP	One or more compiled JAXBase code files used to create an application
DEF	Class Definition source code
JXD	Compiled Class Definition code
H	Include file used during compiling of source code
DBF	Data table
FPT	Data table memo file
JST	Data table JSON file (64-bit tables only)
IDX	Simple data table index
VCX	Class Library
VCT	Class Library memo file
JXV	Compiled class library
JXVP	Intermediate JAX source file for class library
SCX	Form / Formset definition
SCT	Form / Formset memo file
JXS	Compiled Form/Formset
JXSP	Intermediate JAX source file for form/formset definition
FRX	Report definition
FRT	Report memo file
JXF	Compiled Report code
JXFP	Intermediate JAX source file for report definition
MNX	Menu Definition
MNT	Menu memo file
JXM	Compiled menu definition
JXMP	Intermediate JAX source file for menu definition
PJX	Project File
PJT	Project memo file
RPX	Report Definition
RPT	Report memo file
JXR	Compiled Report code
JXRP	Intermediate JAX source file for report definition
LBX	Label Definition
LBT	Label memo file
JXL	Compiled Library code
JXLP	Intermediate JAX source file for label definition

Appendix B – Table File Structure

32-bit Table Structure

Byte Offset	Description
0	File type: 0x01 FoxBASE: 0x02 FoxBASE+/Dbase III plus, no memo: 0x2F Visual FoxPro: 0x30 Visual FoxPro, autoincrement enabled: 0x31 Visual FoxPro, Varchar, Varbinary, or Blob-enabled: 0x42 JAXBase, VarChar, Varbinary, Blob-enabled: 0X?? dBASE IV SQL table files, no memo: 0x62 dBASE IV SQL system files, no memo: 0x82 FoxBASE+/dBASE III PLUS, with memo: 0x8A dBASE IV with memo: 0xCA dBASE IV SQL table files, with memo: 0xF4 FoxPro 2.x (or earlier) with memo: 0xFA
1 – 3	Last update (YYMMDD)
4 – 7	Number of records in file
8 – 9	Position of first data record
10 – 11	Length of one data record, including delete flag
12 – 27	Reserved
28	Table Flags 0x01 file has a structural .cdx 0x02 file has a Memo field 0x04 file is a database (.dbc) This byte can contain the sum of any of the above values. For example, the value 0x03 indicates the table has a structural .cdx and a Memo field.
29	Code Page Mark
30-31	Reserved, contains 0x00
32 – n	Field sub-records: The number of fields determines the number of field sub-records. One field sub-record exists for each field in the table.
n+1	Header record terminator (0x0D)
n+2 to n+264	If a VFP table, it may contain a backlink to a database container. JAXBase can read tables associated with VFP but will open the table as READONLY. If a JAXBase table, then these bytes are reserved.

32-bit Table Field Sub-Records

Byte Offset	Description
0 - 10	Field name padded with 0x00
11	Field type: W - Blob C - Character / Character binary Y - Currency B - Double D - Date T - DateTime F - Float G - General I - Integer L - Logical M - Memo / Memo binary N - Numeric P - Picture Q - Varbinary V - Varchar / Varchar binary 0 - System Field 1 - Null Tracking field
12 - 15	Displacement of field in record
16	Length of field in bytes
17	Number of decimal places
18	Field Flags: 0x01 System Column 0x02 Column can store null values 0x4 Binary column 0x0C Auto incrementing
19 - 22	Value of autoincrement Next value (Slated for Version 2)
23	Value of autoincrement Step value (Slated for Version 2)
24 - 31	Reserved

64-bit Table Structure

(Slated for Version 2)

Byte Offset	Description
0	JAXBase, VarChar, Varbinary, Blob-enabled: 0X??
1 – 3	Last update (YYMMDD)
4 – 11	Number of records in file
12 – 13	Position of first data record
14 – 15	Length of one data record, including delete flag
16 – 27	Reserved
28	Table Flags 0x01 file has a structural .cdx 0x02 file has a Memo field 0x04 file is a database (.dbc) This byte can contain the sum of any of the above values. For example, the value 0x03 indicates the table has a structural .cdx and a Memo field.
29	Code Page Mark
30-31	Reserved, contains 0x00
32 – n	Field sub-records: The number of fields determines the number of field sub-records. One field sub-record exists for each field in the table.
n+1	Header record terminator (0x0F)
n+2 to n+264	If a VFP table, it may contain a backlink to a database container. JAXBase can read tables associated with VFP but will open the table as READONLY. If a JAXBase table, then these bytes are reserved.

64-bit Table Field Sub-Records

(Slated for Version 2)

Byte Offset	Description
0 – 40	Field name padded with 0x00
41	Field type: W - Blob C - Character / Character binary Y - Currency B - Double D - Date T - DateTime F - Float G - General I - Integer J - JSON Text Field L - Logical O - Long Integer M - Memo / Memo binary N - Numeric P - Picture S - Timestamp Q - Varbinary V - Varchar / Varchar binary 0 - System Field 1 - Null Tracking field
42 - 45	Displacement of field in record
46	Length of field in bytes
47	Number of decimal places
48	Field Flags: 0x01 System Column 0x02 Column can store null values 0x4 Binary column 0x0C Auto incrementing
49 – 56	Value of autoincrement Next value
57	Value of autoincrement Step value
58 - 63	Reserved

DRAFT

Appendix C – JAXBase Field Types

Code	Data Type	Description
B	Double	
C	Character	
D	Date Only	
F	Float	
G	General	
I	Integer	
J	JAX Text Field*	Proposed fixed length text field made up of 1 to 254 blocks of 128 bytes or 128 to 32,512 bytes in length. (Slated for Version 2)
L	Logical	
M	Memo	
N	Numeric	
Q	Binary Varchar	Max of 254 bytes in the 32-bit version Proposed 8,192 bytes max in 64-bit tables*
T	DateTime	
V	Varchar	Max of 254 bytes in the 32-bit version Proposed 8,192 bytes max in 64-bit tables*
W	Blob	
Y	Currency	

* Not available in Version 1

APPENDIX D – JAXBase Data Types

DRAFT

Appendix E – System Variables

DRAFT

Appendix F – JAXBase Data Sessions

DRAFT

Appendix G – DEF Files

A DEF file is like an H file in that it only contains definitions. In this case, class definitions, written in pure JAXBase code, that will be used by JAXBase to create user defined class, such as forms and components during runtime.

A DEF file can contain any number of class definitions which can be found by JAXBase by referencing those files using the SET DEFINITION TO command.

Any code that is outside of a class definition will generate an error during compilation.

DRAFT

Appendix I – SYS() Function Support

Status

Slated for Version 1.0

The SYS() function in VFP plays a vital role in many applications and to help VFP users migrate over to JAXBase, SYS.PRG in the Examples folder can be added to a project to provide support for the following SYS() functions.

SYS() Function	Description	JAXBase Alternative
SYS(1)	Machine name and user ID	_Environment.MachID + " #" + _Environment.UserID
SYS(2)	Seconds since Midnight	SECONDS()
SYS(3)	Legal filename	SYSID()
SYS(5)	Default drive or volume	JUSTDRIVE(SET("default"))
SYS(6)	Current printer device	
SYS(12)	Available memory in bytes	_Environment.MemoryFree
SYS(13)	Printer status	
SYS(14)	Index Expression	
SYS(16)	Executing file name	
SYS(17)	Processor in use	_CPUS[0].CPUName
SYS(18)	Current control	
SYS(21)	Controlling index number	_Table.ControllingIDX
SYS(22)	Controlling tag or IDX Name	_Table.Indexes[_Table.ControllingIDX].Name
SYS(100)	Console Setting	SET("CONSOLE")
SYS(102)	Device setting	
SYS(103)	Talk Setting	SET("TALK")
SYS(1037)	Printer page setup	
SYS(1500)	Activate a menu item	Menu class Activate method
SYS(2000)	File name wildcard match	
SYS(2001)	Set command status	
SYS(2002)	Insertion point	
SYS(2003)	Current Directory	SET("DEFAULT") or _Jax.DefaultFolder
SYS(2004)	Starting directory	_Jax.Path or _JAXHome variable
SYS(2005)	Current resource file	
SYS(2007)	Checksum value	JAXBase supports three different checksum functions for values: CRC32() and MD5()
SYS(2015)	Unique procedure name	

SYS Function Table - Continued

SYS() Function	Description	JAXBase Alternative
SYS(2017)	Checksum of current record	Use MD5Rec() for the current record's checksum
SYS(2018)	Error message parameter	
SYS(2019)	Config file name and location	
SYS(2020)	Default disk free space	
SYS(2021)	Filtered index expression	
SYS(2023)	Temporary path	_JAXTemp variable
SYS(2024)	Detect Report Cancellation	Printer class Status property="Canceled"
SYS(2029)	Table Type	_TABLE.HeaderByte
SYS(2040)	Detect report status	Printer class Status property
SYS(2300)	Add or remove code page	Version 2.0
SYS(2450)	App path search order	_JAX.PathList
SYS(2910)	Set or return number of dropdown items to display	ComboBox MaxItems property

JAXBase FAQ

If you have questions about JAXBase, feel free to write to jlwalker_dev@gmail.com

Q: Why did you get rid of all those legacy commands? My application needs to be rewritten!

A: Most of the commands removed lent themselves to poor user experiences. I am sorry if you have been using a program developed in the last century, but much has changed since then. If you do not think JAXBase is right for you, then please feel free to continue using what you have. However, if being able to move to a powerful open source xBase system that works on both Linux and Windows, then is asking you to update your application to large of a price?

Q: Why don't you support structural indexes and database containers?

A: I would say, "see the last question", but it is a bit more complicated than that. The database container and structural index in Visual FoxPro, for instance, are incredibly powerful and have features that most SQL backends do not directly support. Further, the local database container was a band-aid to the larger problem of blown indexes and corrupted tables. Finally, DBF tables have a limit of 2-gigabytes and JAXBase is being designed to consider 2GB of data as a "good start". Since JAXBase supports both SQL Server and MySQL, and in the future will support PostgreSQL and other database back ends, it was decided to keep local DBF tables for applications that do not use large data sets.

Q: How am I supposed to convert my VFP project into JAXBase?

A: No promises, but JAXBase is being designed to be able to read a VFP project and convert it and various components using a set of conversion tools, all written in JAXBase. Unfortunately, those tools will not be fully developed until after Version 1.0 is released.

Q: Can I use JAXBase for production?

A: First, understand that JAXBase versions prior to Version 2.0 are written in C# using .NET and have little to no modern security features, putting it on par with many of the legacy systems. Version 2.0 will be written in C++ or Rust and will be designed from the ground up to address modern security concerns. If you want to create an application, feel free, but be aware of the security concerns inherent to many XBase systems. For small applications or systems not directly facing the web, JAXBase Version 1.0 may prove to be a valuable tool.

Q: Why were reports and labels made into classes?

A: Great question! The report system was originally developed under MS-DOS and support for Windows was done by creating wrapper applications or other... well... "kludges" is the word I am hesitating to use. Why should JAXBase build upon an MS-DOS-based reporting system and not take advantage of open-source solutions? Eliminating the in-language report preview rendering engine eliminates needless overhead and makes it easier to offer a cross-platform solution. Finally, interfacing with a cross-platform open-source office suite allows the creation of WYSIWYG reports that can be saved in different file formats.

Q: I notice there is a Barcode class. What is that about?

A: Barcodes are used extensively in industry and commerce and the lack of direct support in the xBase language is a stark reminder of its age and lack of advanced thinking by those who brought it into the new century. JAXBase will support reading and writing barcode images via the user's choice of various open-source or commercial solutions.

Q: What happened to the ON ERROR command?

A: Again, sloppy programming leads to sloppy results. It is time to learn about the TRY/CATCH command to capture errors or use the AError array and Error method if dealing with a class. Ad-hock error handling is so last century.

Q: Why did you get rid of the SYS() Functions?

A: The short answer is "I don't want to fight Microsoft". Granted, there is a good chance that nobody at Microsoft will ever speak the word "JAXBase", let alone care about what is happening in the XBase world. However, going out of the way to make sure that JAXBase does not give the nice Microsoft Lawyers a valid reason to start writing letters was considered prudent. Besides, in this writer's opinion, the SYS() function seems more like a kludge and less like a solution.

Q: Will JAXBase work on tablets or phones?

A: JAXBase is being designed to run on Linux and Windows desktops up to and including Version 1. While it is possible that tablets and phones, which are predominantly Android or IOS devices, will be supported in Version 2.0, that is too far down the road to give any kind of an answer.

Q: How advanced of a C# programmer are you?

A: I suspect I am at or slightly beyond the level of a second year C# developer for syntax and C# concepts, but I have been in the industry for over 40 years and have a decent idea how to look things up and get things working. Anyone with several of years of C# development will start any question about what I did with "Why did you", to which my answer is "I'm a VFP developer". If I were a developer with 40+ years of experience in any other language, I probably would have never thought about writing JAXBase.